

API docs

...

- 1. Create a New AI Assistant
- 2. List Embedding Models
- 3. List Reranker Models
- 4. List Large Language Models
- 5. List AI Assistants
- 6. Get a Specific AI Assistant
- 7. Update AI Assistant
- 8. Partially Update an AI Assistant
- 9. Delete AI Assistant
- 10. Duplicate AI Assistant
- 11. Enable or Disable Evaluation

Create a New AI Assistant

POST

/api/v1/chatbots/

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
largeLanguageModel	string (uuid)	Yes	Large language model used by the chatbot to generate responses (including organization-hosted AI Gateway models)
embeddingModel	string (uuid)	No	Embedding model used to vectorize text; optional
rerankerModel	string (uuid)	No	Model used for reranking; optional
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the chatbot
databases	array[IdName]	No	List of SQL databases accessible to the chatbot (used by the Text-to-SQL feature)

Field	Type	Required	Description
organization	string (uuid)	No	Organization the chatbot belongs to; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming reply <code>normal</code> : Normal ; <code>template</code> : Template ; <code>hybrid</code> : Hybrid ; <code>agent</code> : Agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format <code>text</code> : Text ; <code>json_schema</code> : JSON Schema;
rawOutputFormat	object	No	JSON schema definition for the custom output format
groups	array[IdName]	No	List of groups accessible to the chatbot
tools	array[ToolSummary]	No	List of tools available to the chatbot
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode <code>normal</code> : Normal ; <code>canvas</code> : Canvas ; <code>team</code> : Team;
numberOfRetrievedChunks	integer	No	Number of reference chunks retrieved. Defaults to 12; minimum: 1
defaultLabelRelations	object	No	Default knowledge base label filters at the assistant level, in the same format as <code>query_metadata.label_relations</code> ; null means no filtering. Applied as the lowest-priority fallback only when the caller does not provide <code>query_metadata</code> .
useFullRetrievalRange	boolean	No	When enabled, ignores the LLM top_k cutoff and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.

Field	Type	Required	Description
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output...
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; minimum: 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort configuration (JSON format)
enableToolSearching	boolean	No	Enables dynamic tool search, allowing the agent to discover and load tools dynamically through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)

Field	Type	Required	Description
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of memory nodes retrieved per long-term memory search (similarity_top_k for vector search). Defaults to 10; range: 1-50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.

Field	Type	Required	Description
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Request Schema Example

```
{
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate
responses (including organization-hosted AI Gateway models)
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional
(optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the chatbot (optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the chatbot (used by the Text-to-SQL
feature) (optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a
personal chatbot (optional)
  "replyMode"?: // Reply mode: normal or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
```

```

{
}
"rawOutputFormat"?: object // JSON schema definition for the custom output format (optional)
"groups"?: [ // List of groups accessible to the chatbot (optional)
  {
    "id": string (uuid)
  }
]
"tools"?: [ // List of tools available to the chatbot (optional)
  {
    "id": string (uuid)
  }
]
"skills"?: [ // List of skills associated with the chatbot (optional)
  {
    "id": string (uuid)
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (optional)
  {
    "id": string (uuid)
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved. Defaults to 12;
minimum: 1 (optional)
"defaultLabelRelations"?: object // Default knowledge base label filters at the assistant
level, in the same format as query_metadata.label_relations; null means no filtering. Applied
as the lowest-priority fallback only when the caller does not provide query_metadata.
(optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (optional)
"enableEvaluation"?: boolean // optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum:
512 (optional)
"voiceConfig"?: { // optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-

```

```
platform toggle. (optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type. (optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)
  "sttConfig"?: object // STT configuration parameters (JSON). (optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)
  "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)
  "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)
  "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (optional)
  "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(optional)
  "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (optional)
}
}
"thinkingConfig"?: object // Thinking effort configuration (JSON format) (optional)
"enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (optional)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (optional)
"longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory
search (similarity_top_k for vector search). Defaults to 10; range: 1-50 (optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (optional)
"gatewayFallbackGroup"?: string (uuid) // optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
```

```
for produced content. AGENT mode only. (optional)
}
```

Example Request Value

```
{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "example string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "example string",
  "unanswerableTemplate": "example string",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": true,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
}
```

```
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "example string",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "example string",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "example string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "example string",
  "unanswerableTemplate": "example string",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
}
```

```

"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "example string",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "example string",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
},'

```

Before running this command, replace YOUR_API_KEY and verify the request data.


```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example string",
  "unanswerableTemplate": "Example string",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
```

```

        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example string",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example string",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
  });

```

```
console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example string",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example string",
    "unanswerableTemplate": "Example string",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```

],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example string",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example string",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example string",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example string",
            "unanswerableTemplate": "Example string",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
},
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example string",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example string",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status code: 201

Response schema example

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // The large language model used by the chatbot to
generate responses (including custom AI Gateway models created by the organization)
  "llm": // The bound model ID and name, read directly from the foreign key and returned even
if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel?": string (uuid) // The embedding model used to vectorize text; optional
(Optional)
  "rerankerModel?": string (uuid) // The model used for reranking; optional (Optional)
  "instructions?": string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases?": [ // List of knowledge bases accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases?": [ // List of SQL databases accessible to the chatbot (used by the Text-to-SQL
feature) (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization?": string (uuid) // The organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode?": // Reply mode: standard or streaming response

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template?": string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate?": string // Template used by Template and Hybrid modes when the
```



```

question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for the custom output format (Optional)
  "groups"?: [ // List of groups accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools available to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; defaults to 12
with a minimum of 1 (Optional)
  "defaultLabelRelations"?: object // Default knowledge base label filter at the assistant
level, in the same format as query_metadata.label_relations; null means no filtering. Applied
only as the lowest-priority fallback when the caller does not provide query_metadata.
(Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k truncation and
returns the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run

```

```
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum
512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
  }
}
  "thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved for each long-term memory
search (similarity_top_k for vector retrieval); defaults to 10, range 1-50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
```

```

90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "updatedAt": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response string",
  "unanswerableTemplate": "Response string",
  "totalWordsCount": 456,

```

```
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "description": "Response string",
    "displayName": "Response string",
    "toolType": "Response string"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "description": "Response string"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
```

```
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

List Embedding Models

GET

</api/v1/embedding-models/>

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/embedding-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this example.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/embedding-models/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/embedding-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/embedding-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string
    "isDefault"?: boolean // Optional
    "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text + image)
    embeddings (Optional)
    "icon":
      {
        "id": string (uuid)
        "name": string
        "image": string (uri)
      }
  }
]
```

Example Response Value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "isDefault": false,
    "isMultimodal": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Reranker Models

GET

</api/v1/reranker-models/>

Code Examples

Shell/Bash

```
# Call the API (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/reranker-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this example.
```


JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/reranker-models/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/reranker-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/reranker-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string
    "isDefault?": boolean // Optional
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
  }
]
```

Example Response Value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "isDefault": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Large Language Models

GET

/api/v1/large-language-models/

Code Examples

Shell/Bash

```
# Call the API (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/large-language-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this example.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/large-language-models/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/large-language-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/large-language-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

[
  {
    "id": string (uuid)
    "name": string
    "contextWindow"?: integer // Used only by OpenAIIlike or Ollama models; other models ignore
    this value. (Optional)
    "isDefault"?: boolean // Optional
    "provider": string
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
    "isMultiModalSupport"?: boolean // Optional
    "isFunctionCallingAvailable"?: boolean // Optional
    "isToolCallingAvailable"?: boolean // Optional
  }
]

```

```

    "isTemplateAvailable": boolean
    "isOutputFormatAvailable": boolean
    "isThinkingAvailable"?: boolean // When checked, this model shows the thinking depth
    setting and requests carry thinking parameters (thinking_budget for Gemini, reasoning_effort
    for OpenAI-compatible hosts). Unchecking stops sending them; saved thinking settings are kept
    but have no effect. Check only for models known to support thinking parameters, otherwise the
    provider returns 4xx. (Optional)
    "thinkingConfigSchema": object
    "replyMode": [
        string
    ]
    "outputMode": [
        string
    ]
    "agentMode": [
        string
    ]
    "maxTokens"?: integer // Optional
    "minTokens": integer
    "isCustomModel": boolean // Whether the row is organization-owned (materialized from an AI
    Gateway custom model).
  }
]

```

Example Response Value

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "contextWindow": 456,
    "isDefault": false,
    "provider": "response string",
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "image": "https://example.com/file.jpg"
    },
    "isMultiModalSupport": false,
    "isFunctionCallingAvailable": false,
    "isToolCallingAvailable": false,
    "isTemplateAvailable": false,
    "isOutputFormatAvailable": false,
    "isThinkingAvailable": false,
    "thinkingConfigSchema": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "replyMode": [
      "response string"
    ],
    "outputMode": [
      "response string"
    ],
  },
]

```

```
"agentMode": [
  "response string"
],
"maxTokens": 456,
"minTokens": 456,
"isCustomModel": false
}
]
```

List AI Assistants

GET

/api/v1/chatbots/

Parameters

Parameter Name	Required	Type	Description
largeLanguageModel	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Whether to paginate (true/false)
query	X	string	
replyMode	X	string	
source	X	string	`self_built`: self_built; `built_in`: built_in; `all`: all;

Code Examples

Shell/Bash

```
# Call the API (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this example.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```


Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes
it only distinguishes chatbots.
      "groups": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
    }
  ]
  "largeLanguageModel": { // Base serializer for the LLM, containing only the basic fields
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel": {
    {
      "id": string (uuid)
      "name": string
      "isDefault"?: boolean // Optional
      "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text +
image) embeddings (Optional)
      "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
    }
  }
  "rerankerModel": {
    {
      "id": string (uuid)
      "name": string
      "isDefault"?: boolean // Optional
      "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
    }
  }
}
```

```

    }
    "knowledgeBases": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "tools": [
      {
        "id": string (uuid)
        "name": string
        "description": string
        "displayName": string
        "toolType": string
      }
    ]
    "skills": [
      {
        "id": string (uuid)
        "name": string
        "description": string
      }
    ]
    "agentMode"?: // Agent mode: Canvas or Normal. Canvas mode generates answers on a
canvas; Normal mode generates regular answers.

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "replyMode"?: // Reply mode: Normal, Template, Hybrid, or Agent.

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "updatedAt": string (timestamp)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, letting the agent
discover and load tools via tool_searching_tool. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
run code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached
knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
(Optional)
  "numberOfRetrievedChunks"?: integer // Optional
  "useFullRetrievalRange"?: boolean // When enabled, the retrieval tool ignores the top_k
set by the LLM and returns the full reranker result up to the maximum number of retrieved
chunks. Useful for knowledge bases with long tables that need a wider retrieval range.
(Optional)
  "isMultimodalLlm": boolean // Return whether the chatbot's LLM supports multimodal.

```

```

    "voiceConfig":
    {
        "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by
every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a
per-platform toggle. (Optional)
    }
    "createdBy":
    {
        "id": string (uuid)
        "name": string
    }
    "isLocked"?: boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified (Optional)
    "sourceBuildInAgent"?: string (uuid) // Points to the official Build-in Agent blueprint
when instantiated by a rental (Optional)
    "canUpdate": boolean // Return whether the current user can update this resource.
    "canDelete": boolean // Return whether the current user can delete this resource.
    }
}
}

```

Example Response Value

```

{
    "count": 123,
    "next": "http://api.example.org/accounts/?page=4",
    "previous": "http://api.example.org/accounts/?page=2",
    "results": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string",
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000",
                    "name": "response string"
                }
            ],
            "largeLanguageModel": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "response string"
            },
            "embeddingModel": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "response string",
                "isDefault": false,
                "isMultimodal": false,
                "icon": {
                    "id": "550e8400-e29b-41d4-a716-446655440000",
                    "name": "response string",
                    "image": "https://example.com/file.jpg"
                }
            },
            "rerankerModel": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "response string",
            }
        }
    ]
}

```

```
    "isDefault": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "image": "https://example.com/file.jpg"
    }
  },
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string",
      "displayName": "response string",
      "toolType": "response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string"
    }
  ],
  "agentMode": {},
  "replyMode": {},
  "updatedAt": "response string",
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableToolSearching": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "numberOfRetrievedChunks": 456,
  "useFullRetrievalRange": false,
  "isMultimodalLlm": false,
  "voiceConfig": {
    "enabled": false
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
]
```

Get a Specific AI Assistant

GET

/api/v1/chatbots/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# Call the API (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this example.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
  only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // The large language model used by the chatbot to
  generate responses (including organization-owned AI Gateway models)
  "llm": // The bound model id and name, read directly from the foreign key and returned even
  if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel"? : string (uuid) // The embedding model used to vectorize text; optional
  "rerankerModel"? : string (uuid) // The model used for reranking; optional
  "instructions"? : string // Role instructions describing the chatbot's role and behavior.
  (Optional)
  "knowledgeBases"? : [ // The list of knowledge bases accessible to the chatbot (Optional)

```



```

    {
        "id": string (uuid)
        "name": string
    }
]
"databases"?: [ // The list of SQL databases accessible to the chatbot (for Text-to-SQL)
(Optional)
    {
        "id": string (uuid)
        "name": string
    }
]
"updatedAt": string (timestamp)
"organization"?: string (uuid) // The organization that owns the chatbot; if empty, this is a
personal chatbot (Optional)
"replyMode"?: // Reply mode: standard or streaming responses

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
{
}
"template"?: string // Template used by Template and Hybrid modes. (Optional)
"unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
"totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for the custom output format (Optional)
"groups"?: [ // List of groups accessible to the chatbot (Optional)
    {
        "id": string (uuid)
        "name": string
    }
]
"tools"?: [ // List of tools available to the chatbot (Optional)
    {
        "id": string (uuid)
        "name": string
        "description": string
        "displayName": string
        "toolType": string
    }
]
"skills"?: [ // List of skills associated with the chatbot (Optional)
    {
        "id": string (uuid)
        "name": string
        "description": string
    }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth

```

```

authorization (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; the default is
12 and the minimum is 1 (Optional)
"defaultLabelRelations"?: object // Default assistant-level knowledge base label filter, in
the same format as query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not provide query_metadata. (Optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; the
minimum is 512 (Optional)
"voiceConfig"?: { // Optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
  "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
  "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
  "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
  "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
  "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
  "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
dynamically discover and load tools through tool_searching_tool (Optional)

```

```

    "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
    default: 20) (Optional)
    "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
    default: 600) (Optional)
    "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
    out (max 500 chars). Leave empty to use the system default. (Optional)
    "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
    "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
    character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
    whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
    web search results, etc.). (Optional)
    "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
    (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
    "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
    processed by fact extraction for semantic retrieval (existing behavior). When disabled,
    archived messages are discarded and only the active chat history window is used for multi-turn
    context. (Optional)
    "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory
    search (similarity_top_k for vector search); the default is 10, with a range of 1-50 (Optional)
    "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
    also reach this contact's other conversations in the same inbox, within the lookback window.
    When disabled, they only reach the current conversation. (Optional)
    "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
    reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
    90 days. Only effective when enable_cross_conversation_search is True. (Optional)
    "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
    LLM (Optional)
    "gatewayFallbackGroup"?: string (uuid) // Optional
    "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
    conversation. (Optional)
    "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
    specific message for context. (Optional)
    "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
    text attachments. AGENT mode only. (Optional)
    "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
    other attachments. AGENT mode only. (Optional)
    "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
    their capabilities. AGENT mode only. (Optional)
    "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
    for produced content. AGENT mode only. (Optional)
    "isLocked": boolean // Locked instance: config follows the official version; only the
    dedicated KB can be modified
    "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
    instantiated by a rental
    "canUpdate": boolean // Return whether the current user can update this resource.
    "canDelete": boolean // Return whether the current user can delete this resource.
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "updatedAt": "response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "response string",
  "unanswerableTemplate": "response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string",
      "displayName": "response string",
      "toolType": "response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
```

```

"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}

```

Update AI Assistant

PUT

/api/v1/chatbots/{id}

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
largeLanguageModel	string (uuid)	Yes	The large language model used by the chatbot to generate responses (including organization-hosted AI Gateway models)
embeddingModel	string (uuid)	No	The embedding model used to vectorize text › Optional
rerankerModel	string (uuid)	No	The model used for reranking › Optional
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the chatbot
databases	array[IdName]	No	List of SQL databases accessible to the chatbot (used for the Text-to-SQL feature)
organization	string (uuid)	No	The organization the chatbot belongs to; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming response <code>normal</code> : Normal; <code>template</code> : Template; <code>hybrid</code> : Hybrid; <code>agent</code> : Agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format <code>text</code> : Text; <code>json_schema</code> : JSON Schema;

Field	Type	Required	Description
rawOutputFormat	object	No	JSON schema definition for the custom output format
groups	array[IdName]	No	List of groups accessible to the chatbot
tools	array[ToolSummary]	No	List of tools available to the chatbot
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode normal : Normal ; canvas : Canvas ; team : Team;
numberOfRetrievedChunks	integer	No	Number of reference chunks retrieved; the default is 12 and the minimum is 1
defaultLabelRelations	object	No	Default assistant-level knowledge base label filter, in the same format as query_metadata.label_relations; null means no filtering. Applied as the lowest-priority fallback only when the caller does not provide query_metadata.
useFullRetrievalRange	boolean	No	When enabled, ignores the LLM top_k cutoff and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial outpu...
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; the minimum is 512
voiceConfig	object	No	

Field	Type	Required	Description
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort settings (JSON format)
enableToolSearching	boolean	No	Enables dynamic tool search, allowing the agent to dynamically discover and load tools through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.

Field	Type	Required	Description
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of memory nodes retrieved per long-term memory search (similarity_top_k for vector search); the default is 10, with a range of 1–50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Example Request Schema

```
{
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
  only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // The large language model used by the chatbot to
```

```

generate responses (including organization-hosted AI Gateway models)
  "embeddingModel"?: string (uuid) // The embedding model used to vectorize text, Optional
(Optional)
  "rerankerModel"?: string (uuid) // The model used for reranking, Optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the chatbot (used for the Text-to-SQL
feature) (Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // The organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode"?: // Reply mode: normal or streaming response

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for the custom output format (Optional)
  "groups"?: [ // List of groups accessible to the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "tools"?: [ // List of tools available to the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
    }
  ]

```

```

    }
  ]
  "agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
  "numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks; defaults to 12,
with a minimum of 1 (Optional)
  "defaultLabelRelations"?: object // Default knowledge base label filter at the assistant
level, using the same format as query_metadata.label_relations; null means no filtering. It is
applied only as the lowest-priority fallback when the caller does not provide query_metadata.
(Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and
returns the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens, with a
minimum of 512 (Optional)
  "voiceConfig"?: { // Optional
    {
      "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
      "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
      "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
      "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
      "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
      "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
      "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
      "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
      "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
      "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
      "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
    }
  }
  "thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
dynamically discover and load tools through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,

```

```

default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved for each long-term memory
search (similarity_top_k for vector search); defaults to 10, with a range of 1-50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}

```

Example Request Value

```

{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
],
"organization": "550e8400-e29b-41d4-a716-446655440000",
"replyMode": {},
"template": "Example String",
"unanswerableTemplate": "Example String",
"totalWordsCount": 123,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
```

```
"enableSizeBasedToolResultTruncation": true,  
"toolResultMaxChars": 123,  
"enableVectorMemory": true,  
"longTermMemoryTopK": 123,  
"enableCrossConversationSearch": true,  
"crossConversationLookbackDays": 123,  
"useGatewayFallback": true,  
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",  
"enableSearchMessagesTool": true,  
"enableGetMessageContextTool": true,  
"enableReadAttachmentFullTextTool": true,  
"enableParseAttachmentContentTool": true,  
"enableListAvailableParsersTool": true,  
"enableGetDownloadLinkTool": true  
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
```

```

"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
},'

```

Before running the request, replace YOUR_API_KEY and verify the request data.


```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
```

```

        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example String",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
  });

```

```
console.error(error.response?.data || error.message);  
});
```

```

import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

```

```

],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}

```

```

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example String",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example String",
            "unanswerableTemplate": "Example String",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
},
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate
responses (including organization-hosted AI Gateway models)
  "llm": // The bound model's id and name, read directly from the FK and returned even if the
organization has disabled the model
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel?": string (uuid) // Embedding model used to vectorize text; optional
(Optional)
  "rerankerModel?": string (uuid) // Model used for reranking; optional (Optional)
  "instructions?": string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases?": [ // List of knowledge bases accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases?": [ // List of SQL databases accessible to the chatbot (used for the Text-to-SQL
feature) (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization?": string (uuid) // Organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode?": // Reply mode: standard or streaming response

* `normal` - normal
* `template` - template
* `hybrid` - hybrid
* `agent` - agent (optional)
  {
  }
  "template?": string // Template used by Template and Hybrid modes. (optional)
  "unanswerableTemplate?": string // Template used by Template and Hybrid modes when the
```



```

question cannot be answered. (optional)
"totalWordsCount"?: integer (int64) // Accumulated total word count used. (optional)
"outputMode"?: // Output mode: text, table, or custom format

* `text` - text
* `json_schema` - JSON Schema (optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for the custom output format (optional)
"groups"?: [ // List of groups accessible to the chatbot (optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"tools"?: [ // List of tools available to the chatbot (optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
    "displayName": string
    "toolType": string
  }
]
"skills"?: [ // List of skills associated with the chatbot (optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - normal
* `canvas` - canvas
* `team` - team (optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; default: 12,
minimum: 1 (optional)
"defaultLabelRelations"?: object // Default knowledge base label filter conditions at the
assistant level, using the same format as query_metadata.label_relations; null means no
filtering. Applied as the lowest-priority fallback only when the caller does not provide
query_metadata. (optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (optional)
"enableEvaluation"?: boolean // optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run

```

```

code in an isolated container. (optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum:
512 (optional)
  "voiceConfig"?: { // optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (optional)
  }
}
  "thinkingConfig"?: object // Thinking effort settings (JSON format) (optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved from long-term memory per
search (vector search similarity_top_k), default: 10, range: 1-50 (optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to

```

```

90 days. Only effective when enable_cross_conversation_search is True. (optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (optional)
  "gatewayFallbackGroup"?: string (uuid) // optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "updatedAt": "response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "response string",
  "unanswerableTemplate": "response string",
  "totalWordsCount": 456,

```

```
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string",
    "displayName": "response string",
    "toolType": "response string"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
```

```
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

Partially Update an AI Assistant

PATCH `/api/v1/chatbots/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.

Field	Type	Required	Description
largeLanguageModel	string (uuid)	No	Large language model used by the chatbot to generate responses (including organization-provided AI Gateway models)
embeddingModel	string (uuid)	No	Embedding model used to vectorize text
rerankerModel	string (uuid)	No	Model used for reranking
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the chatbot
databases	array[IdName]	No	List of SQL databases accessible to the chatbot (used for the Text-to-SQL feature)
organization	string (uuid)	No	Organization that owns the chatbot; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming reply <code>normal</code> : normal ; <code>template</code> : template ; <code>hybrid</code> : hybrid ; <code>agent</code> : agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format <code>text</code> : text ; <code>json_schema</code> : JSON Schema;
rawOutputFormat	object	No	JSON schema definition for the custom output format
groups	array[IdName]	No	List of groups accessible to the chatbot
tools	array[ToolSummary]	No	List of tools available to the chatbot
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode <code>normal</code> : normal ; <code>canvas</code> : canvas ; <code>team</code> : team;
numberOfRetrievedChunks	integer	No	Number of reference chunks retrieved; default: 12, minimum: 1
defaultLabelRelations	object	No	Default knowledge base label filter conditions at the assistant level, using the same format as <code>query_metadata.label_relations</code> ; null

Field	Type	Required	Description
			means no filtering. Applied as the lowest-priority fallback only when the caller does not provide query_metadata.
useFullRetrievalRange	boolean	No	When enabled, ignores the LLM top_k cutoff and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output...
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; minimum: 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty

Field	Type	Required	Description
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort settings (JSON format)
enableToolSearching	boolean	No	Enables dynamic tool search, allowing the agent to discover and load tools dynamically through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of memory nodes retrieved from long-term memory per search (vector search similarity_top_k), default: 10, range: 1-50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only

Field	Type	Required	Description
			effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Request Schema Example

```
{
  "name"? : string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots. (optional)
  "largeLanguageModel"? : string (uuid) // Large language model used by the chatbot to generate
responses (including organization-provided AI Gateway models) (optional)
  "embeddingModel"? : string (uuid) // Embedding model used to vectorize text (optional)
  "rerankerModel"? : string (uuid) // Model used for reranking (optional)
  "instructions"? : string // Role instructions describing the chatbot's role and behavior.
(optional)
  "knowledgeBases"? : [ // List of knowledge bases accessible to the chatbot (optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"? : [ // List of SQL databases accessible to the chatbot (used for the Text-to-SQL
feature) (optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"? : string (uuid) // Organization that owns the chatbot; if empty, it is a
personal chatbot (optional)
  "replyMode"? : // Reply mode: normal or streaming reply

* `normal` - normal
* `template` - template
```

```

* `hybrid` - hybrid
* `agent` - agent (optional)
{
}
"template"?: string // Template used by Template and Hybrid modes. (optional)
"unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (optional)
"totalWordsCount"?: integer (int64) // Accumulated total word count used. (optional)
"outputMode"?: // Output mode: text, table, or custom format

* `text` - text
* `json_schema` - JSON Schema (optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for the custom output format (optional)
"groups"?: [ // List of groups accessible to the chatbot (optional)
{
  "id": string (uuid)
}
]
"tools"?: [ // List of tools available to the chatbot (optional)
{
  "id": string (uuid)
}
]
"skills"?: [ // List of skills associated with the chatbot (optional)
{
  "id": string (uuid)
}
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (optional)
{
  "id": string (uuid)
}
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve. Defaults to
12; minimum: 1 (Optional)
"defaultLabelRelations"?: object // Default knowledge base label filter conditions at the
assistant level. Uses the same format as query_metadata.label_relations; null means no
filtering. Applied as the lowest-priority fallback only when the caller does not provide
query_metadata. (Optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)

```

```

"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum:
512 (Optional)
"voiceConfig"?: { // Optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
  "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
  "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
  "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
  "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
  "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
  "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"longTermMemoryTopK"?: integer // Number of memory nodes retrieved from long-term memory per
search (similarity_top_k for vector search). Defaults to 10; range: 1-50 (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)

```

```

"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}

```

Example Request Value

```

{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
}

```

```

"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}

```

Code Examples

```
# API request example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
}
```

```

    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example String",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  },

```

Before running this command, replace YOUR_API_KEY and verify the request data.

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
```



```

        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example String",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
  });

```

```
console.error(error.response?.data || error.message);  
});
```

```

import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

```

```

],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example String",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example String",
            "unanswerableTemplate": "Example String",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
},
"agentMode": {},
"numberOfRetrievedChunks": 123,
"defaultLabelRelations": null,
"useFullRetrievalRange": true,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"longTermMemoryTopK": 123,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate
responses (including organization-managed AI Gateway models)
  "llm": // The bound model id and name, read directly from the foreign key and returned even
if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel?": string (uuid) // Embedding model used to vectorize text; optional
(Optional)
  "rerankerModel?": string (uuid) // Model used for reranking; optional (Optional)
  "instructions?": string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases?": [ // List of knowledge bases accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases?": [ // List of SQL databases accessible to the chatbot (used by Text-to-SQL)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization?": string (uuid) // Organization that owns the chatbot. If empty, this is a
personal chatbot (Optional)
  "replyMode?": // Reply mode: standard or streaming response

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template?": string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate?": string // Template used by Template and Hybrid modes when the
```

```

question cannot be answered. (Optional)
"totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for the custom output format (Optional)
"groups"?: [ // List of groups accessible to the chatbot (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"tools"?: [ // List of tools available to the chatbot (Optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
    "displayName": string
    "toolType": string
  }
]
"skills"?: [ // List of skills associated with the chatbot (Optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks; defaults to 12,
minimum is 1 (Optional)
"defaultLabelRelations"?: object // Default assistant-level knowledge base label filter. Uses
the same format as query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when query_metadata is omitted by the caller. (Optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run

```



```
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum
is 512 (Optional)
  "voiceConfig"?: { // Optional
    {
      "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
      "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
      "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
      "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
      "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
      "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
      "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
      "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
      "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
      "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
      "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
    }
  }
  "thinkingConfig"?: object // Thinking effort configuration (JSON format) (Optional)
  "enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to discover
and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory query
(similarity_top_k for vector retrieval); defaults to 10, range 1-50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
```

```

90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example String"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example String"
    }
  ],
  "updatedAt": "Example String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 456,

```

```
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example String"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example String",
    "description": "Example String",
    "displayName": "Example String",
    "toolType": "Example String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example String",
    "description": "Example String"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example String"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
```

```
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Example String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

Delete AI Assistant

DELETE

`/api/v1/chatbots/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X DELETE "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Duplicate AI Assistant

POST

/api/v1/chatbots/{id}/duplicate/

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Example Request Schema

```
{
  "name": string
}
```

Example Request Value

```
{
  "name": "Example Name"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name"
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Example Response Schema

```

{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
  only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate
  responses (including organization-managed AI Gateway models)
  "llm": // The bound model id and name, read directly from the foreign key and returned even
  if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel"? : string (uuid) // Embedding model used to vectorize text; optional

```

```

(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the chatbot (used by Text-to-SQL)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // Organization that owns the chatbot. If empty, this is a
personal chatbot (Optional)
  "replyMode"?: // Reply mode: standard or streaming response

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for the custom output format (Optional)
  "groups"?: [ // List of groups accessible to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools available to the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)

```

```

    "name": string
    "description": string
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode
* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
  {
  }
"numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; defaults to 12,
minimum 1 (optional)
"defaultLabelRelations"?: object // Default knowledge base label filter at the assistant
level, in the same format as query_metadata.label_relations; null means no filtering. Applied
as the lowest-priority fallback only when the caller does not provide query_metadata.
(optional)
"useFullRetrievalRange"?: boolean // When enabled, ignores the LLM top_k cutoff and returns
the complete reranker results (optional)
"enableEvaluation"?: boolean // optional
"enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum
512 (optional)
"voiceConfig"?: { // optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (optional)
  }
}

```

```

}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (optional)
"enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (optional)
"longTermMemoryTopK"?: integer // Number of memory nodes retrieved from long-term memory each
time (vector retrieval similarity_top_k); defaults to 10, range 1-50 (optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (optional)
"gatewayFallbackGroup"?: string (uuid) // optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (optional)
"isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
"canUpdate": boolean // Return whether the current user can update this resource.
"canDelete": boolean // Return whether the current user can delete this resource.
}

```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "updatedAt": "response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "response string",
  "unanswerableTemplate": "response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string",
      "displayName": "response string",
      "toolType": "response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "response string"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```


Enable or Disable Evaluation

PATCH

/api/v1/chatbots/{id}/enable-evaluation/

Parameters

Parameter	Required	Type	Description
id	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
enableEvaluation	boolean	No	

Example Request Schema

```
{
  "enableEvaluation"?: boolean // optional
}
```

Example Request Value

```
{
  "enableEvaluation": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "enableEvaluation": true
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "enableEvaluation": true
};

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "enableEvaluation": true
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "enableEvaluation": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "enableEvaluation"?: boolean // optional
}
```

Example Response Value

```
{  
  "enableEvaluation": false  
}
```

API call example (Shell)

...

目錄

1. Create Web Chat
2. List Web Chats
3. Get a Specific Web Chat
4. Update Web Chat
5. Partially Update Web Chat
6. Set Contact Credentials
7. Get Voice Agent Token
8. Upload Batch QA File
9. Export Batch QA to Excel
10. Get LLM Usage Statistics
11. Get AI Assistant Statistics

API call example (Shell)

Create Web Chat

POST `/api/v1/web-chats/`

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	

Field	Type	Required	Description
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a floating row of buttons above the input box (which remains available); when disabled, the current overlay panel is used. This is a feature flag and can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	

Field	Type	Required	Description
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration; range: 3–60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI response before the survey appears; range: 1–60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave blank to use the default localized text from the frontend
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	

Field	Type	Required	Description
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. <code>narrative</code> shows per-stage copy; <code>brand_animation</code> plays the uploaded asset and hides every system narration line. Defaults to <code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // optional
  "logo"?: string (uri) // optional
  "description"?: string // optional
  "cover"?: string (uri) // optional
```

```

"backUrl"?: string // optional
"isActive"?: boolean // optional
"theme": {
  {
    "primaryColor"?: string // optional
    "navbarTextColor"?: string // optional
    "conversationBackgroundColor"?: string // optional
    "chatbotMessageTextColor"?: string // optional
    "userMessageTextColor"?: string // optional
    "chatbotMessageBackgroundColor"?: string // optional
    "userMessageBackgroundColor"?: string // optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // optional
    "userMessageBackgroundShadowEnabled"?: boolean // optional
    "chatbotMessageGradientEnabled"?: boolean // optional
    "userMessageGradientEnabled"?: boolean // optional
  }
}
"darkTheme": {
  {
    "primaryColor"?: string // optional
    "navbarTextColor"?: string // optional
    "conversationBackgroundColor"?: string // optional
    "chatbotMessageTextColor"?: string // optional
    "userMessageTextColor"?: string // optional
    "chatbotMessageBackgroundColor"?: string // optional
    "userMessageBackgroundColor"?: string // optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // optional
    "userMessageBackgroundShadowEnabled"?: boolean // optional
    "chatbotMessageGradientEnabled"?: boolean // optional
    "userMessageGradientEnabled"?: boolean // optional
  }
}
"enableFileUpload"?: boolean // optional
"enableDisplayCitations"?: boolean // optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (optional)
"enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a floating
row of buttons above the input box (which remains available); when disabled, the current overlay
panel is used. This is a feature flag and can be disabled at any time to roll back. (optional)
"enableDisplayThinking"?: boolean // optional
"enableUserThinkingControl"?: boolean // optional
"enableDisplayQueryMetadata"?: boolean // optional
"enableDisplayHookMarkers"?: boolean // optional
"hookMarkerTextInputModify"?: string // optional
"hookMarkerTextOutputModify"?: string // optional
"hookMarkerTextInputBlock"?: string // optional
"hookMarkerTextOutputBlock"?: string // optional
"enableShareConversation"?: boolean // optional
"enableLocation"?: boolean // optional
"enableShowPoweredBy"?: boolean // optional
"poweredByLogo"?: string (uri) // optional
"enableAnonymous"?: boolean // optional
"enableDisplayPreviousConversation"?: boolean // optional
"enableDisplayConversationHistory"?: boolean // optional
"enableConversationTimer"?: boolean // optional
"conversationTimerDuration"?: integer // Timer timeout duration; range: 3-60 minutes (optional)
"enableAutoNewConversationOnTimeout"?: boolean // optional
"enableSatisfactionSurvey"?: boolean // optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before

```

```

the survey appears; range: 1–60 minutes (optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the default
localized text from the frontend (optional)
"enableThemeModeToggle"?: boolean // optional
"enableDisplayFontSizeSwitch"?: boolean // optional
"defaultThemeMode"?: // optional
{
}
"enableDownloadCitations"?: boolean // optional
"customDomain"?: // optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // optional
"enableDisplayChatbotName"?: boolean // optional
"buttonIcon"?: string (uri) // optional
"conversationStartersDisplayCount"?: integer // optional
"preChatFormEnabled"?: boolean // optional
"preChatFormFields"?: object // optional
"preChatFormTitle"?: object // optional
"preChatFormSubtitle"?: object // optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(optional)
"toolCardRunningIconUrl"?: string (uri) // optional
"toolCardCompletedIconUrl"?: string (uri) // optional
}

```

Example Request Value

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",

```

```
"cover": "https://example.com/file.jpg",
"backUrl": "example string",
"isActive": true,
"theme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"darkTheme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"enableFileUpload": true,
"enableDisplayCitations": true,
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "example string",
"hookMarkerTextOutputModify": "example string",
"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
```

```
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
  "enableLocation": true,
```

```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Before running this command, replace YOUR_API_KEY and verify the request data.


```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```

```

    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.post("https://api.maiagent.ai/api/v1/web-chats/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "example string",
    "isActive": true,
    "theme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "example string",
    "hookMarkerTextOutputModify": "example string",
    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",

```

```

"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
    "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while sending the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "example string",
            "isActive": true,
            "theme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "example string",
            "hookMarkerTextOutputModify": "example string",

```

```

        "hookMarkerTextInputBlock": "example string",
        "hookMarkerTextOutputBlock": "example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while sending the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input field (which remains usable); when disabled, the current overlay panel is
  retained. This is a feature flag that can be disabled at any time to roll back. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
}
```

```

"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, from 3 to 60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // The number of minutes after the last AI response
before the survey appears, from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend
localization default (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to

```



```

`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}

"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid and never blocks a reply. (Optional)

"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)

"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the long-wait copy appears. (Optional)

"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays `loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is empty. (Optional)

"loadingAnimationSharedAssetUrl"?: string (uri) // Optional

"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset. (Optional)

"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected. (Optional)

"toolCardRunningIconUrl"?: string (uri) // Optional

"toolCardCompletedIconUrl"?: string (uri) // Optional

"inbox": object

"quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when absent (read-only)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,

```

```
    "userMessageGradientEnabled": false
  },
  "darkTheme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "enableFileUpload": false,
  "enableDisplayCitations": false,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": false,
  "enableDisplayThinking": false,
  "enableUserThinkingControl": false,
  "enableDisplayQueryMetadata": false,
  "enableDisplayHookMarkers": false,
  "hookMarkerTextInputModify": "response string",
  "hookMarkerTextOutputModify": "response string",
  "hookMarkerTextInputBlock": "response string",
  "hookMarkerTextOutputBlock": "response string",
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": false,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": false,
  "enableDisplayPreviousConversation": false,
  "enableDisplayConversationHistory": false,
  "enableConversationTimer": false,
  "conversationTimerDuration": 456,
  "enableAutoNewConversationOnTimeout": false,
  "enableSatisfactionSurvey": false,
  "satisfactionSurveyDelayMinutes": 456,
  "satisfactionSurveyPrompt": "response string",
  "accessType": {},
  "enableThemeModeToggle": false,
  "enableDisplayFontSizeSwitch": false,
  "defaultThemeMode": {},
  "enableDownloadCitations": false,
  "customDomain": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "domain": "response string",
    "status": {}
  },
  "enableDisplayChatbotAvatar": false,
  "enableDisplayChatbotName": false,
  "buttonIcon": "https://example.com/file.jpg",
  "enableCodeInterpreter": false,
  "voiceAgentEnabled": false,
  "conversationStartersDisplayCount": 456,
  "includeGreetingInContext": false,
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "preChatFormEnabled": false,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 456,
  "loadingAnimationUseSharedAsset": false,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": false,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quickMenu": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```

List Web Chats

GET

</api/v1/web-chats/>

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while sending the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while sending the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while sending the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "avatar"?: string (uri) // Optional
    "name": string
  }
]
```

Example Response Value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "avatar": "https://example.com/file.jpg",
    "name": "response string"
  }
]
```

Get a Specific Web Chat

GET

/api/v1/web-chats/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while sending the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options float above the input
  box as a row of buttons (the input box remains available); when disabled, the current overlay panel
  is retained. This is a feature flag that can be disabled at any time to roll back. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
}
```

```

"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3 to 60 minutes
(Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before
displaying the survey, ranging from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the default
localized frontend text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers `display_name` (set on MCP tools, where
`name` is null) and falls back to `name` for API tools where
`display_name` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;

```

```

`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}

"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
"quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}

```

Example Response Value

```

{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "name": "Test doc",
  "avatar": "https://example.com/static/images/default-chatbot-avatar.png",
  "logo": null,
  "description": "",
  "cover": null,
  "backUrl": "",
  "isActive": true,
  "enableSpeech": false,
  "displayGreeting": null,
  "displayConversationStarters": [],
  "theme": {
    "primaryColor": "#3854d8",
    "dialogColor": "#ddeaf6",
    "navbarTextColor": "#ffffff"
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": true,
  "poweredByLogo": null,

```

```
"accessType": "web",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Test doc",
  "signAuth": {
    "id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "signSource": {
      "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "accessType": "keycloak"
    },
  },
  "signParams": {
    "keycloak": {
      "url": "example.com",
      "realm": "example",
      "clientId": "example"
    }
  }
}
```

Update Web Chat

PUT

/api/v1/web-chats/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	

Field	Type	Required	Description
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options float above the input box as a row of buttons (the input box remains available); when disabled, the current overlay panel is retained. This is a feature

Field	Type	Required	Description
			flag that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, ranging from 3 to 60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI response before displaying the survey, ranging from 1 to 60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave blank to use the default localized frontend text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	

Field	Type	Required	Description
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. <code>narrative</code> shows per-stage copy; <code>brand_animation</code> plays the uploaded asset and hides every system narration line. Defaults to <code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....

Field	Type	Required	Description
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Example Request Schema

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options float above the input
  box as a row of buttons (the input box remains available); when disabled, the current overlay panel
  is retained. This is a feature flag that can be disabled at any time to roll back. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
}
```



```

"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3 to 60 minutes
(Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before
displaying the survey, ranging from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the default
localized frontend text (Optional)
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-

```

existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.

(Optional)

```
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}
```

Example Request Value

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
  "enableShowPoweredBy": true,
  "poweredByLogo": "https://example.com/file.jpg",
```

```
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# Call API example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
```

```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Replace YOUR_API_KEY and verify the request data before running.

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
```

```

    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",

```



```

"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
    "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "Example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "Example string",
            "isActive": true,
            "theme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "Example string",

```

```

        "hookMarkerTextOutputModify": "Example string",
        "hookMarkerTextInputBlock": "Example string",
        "hookMarkerTextOutputBlock": "Example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "Example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "Example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options float above the input
  box as a row of buttons (the input box remains available); when disabled, the current overlay panel
  is retained. This is a feature flag and can be disabled at any time to roll back. (Optional)
```

```

"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3 to 60 minutes
(Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before
the survey appears, ranging from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text. Leave blank to use the default
localized text from the frontend (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
  object
]
"preChatFormEnabled"?: boolean // Optional

```

```

"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
"quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
  }
}

```

```
"chatbotMessageBackgroundColor": "response string",
"userMessageBackgroundColor": "response string",
"chatbotMessageBackgroundShadowEnabled": false,
"userMessageBackgroundShadowEnabled": false,
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "response string",
  "navbarTextColor": "response string",
  "conversationBackgroundColor": "response string",
  "chatbotMessageTextColor": "response string",
  "userMessageTextColor": "response string",
  "chatbotMessageBackgroundColor": "response string",
  "userMessageBackgroundColor": "response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
```

```
"conversationStartersDisplayCount": 456,
"includeGreetingInContext": false,
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example Response Description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Response Name",
  "description": "Example Response Description"
},
"quickMenu": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Response Name",
  "description": "Example Response Description"
}
}
```

Partially Update Web Chat

PATCH `/api/v1/web-chats/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	No	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	No	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	

Field	Type	Required	Description
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options float above the input box as a row of buttons (the input box remains available); when disabled, the current overlay panel is retained. This is a feature flag and can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, ranging from 3 to 60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI response before the survey appears, ranging from 1 to 60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text. Leave blank to use the default localized text from the frontend
enableThemeModeToggle	boolean	No	

Field	Type	Required	Description
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. narrative shows per-stage copy; brand_animation plays the uploaded asset and hides every system narration line. Defaults to narrative , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds loading_long_wait_threshold_seconds , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays loading_animation_shared_asset_url , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...

Field	Type	Required	Description
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Example Request Structure

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme"?: { // Optional
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme"?: { // Optional
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
}
```

```

"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
"enableAskOptionButtons"?: boolean // When enabled, ask_user_question options float above the input
box as a row of buttons (the input box remains available); when disabled, the current overlay panel
is retained. This is a feature flag and can be disabled at any time to roll back. (Optional)
"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3 to 60 minutes
(Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before
the survey appears, ranging from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text. Leave blank to use the default
localized text from the frontend (Optional)
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,

```

```

so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}

```

Example Request Value

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,

```

```
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "Example string",
"hookMarkerTextOutputModify": "Example string",
"hookMarkerTextInputBlock": "Example string",
"hookMarkerTextOutputBlock": "Example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# Call API example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
```



```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Make sure to replace YOUR_API_KEY and verify the request data before running the command.

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
```

```

    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",

```

```

"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "Example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
    "domain": "Example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "Example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "Example string",
            "isActive": true,
            "theme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "Example string",

```

```

        "hookMarkerTextOutputModify": "Example string",
        "hookMarkerTextInputBlock": "Example string",
        "hookMarkerTextOutputBlock": "Example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "Example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "Example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // optional
  "logo"?: string (uri) // optional
  "description"?: string // optional
  "cover"?: string (uri) // optional
  "backUrl"?: string // optional
  "isActive"?: boolean // optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // optional
      "navbarTextColor"?: string // optional
      "conversationBackgroundColor"?: string // optional
      "chatbotMessageTextColor"?: string // optional
      "userMessageTextColor"?: string // optional
      "chatbotMessageBackgroundColor"?: string // optional
      "userMessageBackgroundColor"?: string // optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // optional
      "userMessageBackgroundShadowEnabled"?: boolean // optional
      "chatbotMessageGradientEnabled"?: boolean // optional
      "userMessageGradientEnabled"?: boolean // optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // optional
      "navbarTextColor"?: string // optional
      "conversationBackgroundColor"?: string // optional
      "chatbotMessageTextColor"?: string // optional
      "userMessageTextColor"?: string // optional
      "chatbotMessageBackgroundColor"?: string // optional
      "userMessageBackgroundColor"?: string // optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // optional
      "userMessageBackgroundShadowEnabled"?: boolean // optional
      "chatbotMessageGradientEnabled"?: boolean // optional
      "userMessageGradientEnabled"?: boolean // optional
    }
  }
  "enableFileUpload"?: boolean // optional
  "enableDisplayCitations"?: boolean // optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a floating
  row of buttons above the input box (which remains usable); when disabled, the current overlay panel
  remains in use. This is a feature flag that can be disabled at any time to roll back. (optional)
```



```

"enableDisplayThinking"?: boolean // optional
"enableUserThinkingControl"?: boolean // optional
"enableDisplayQueryMetadata"?: boolean // optional
"enableDisplayHookMarkers"?: boolean // optional
"hookMarkerTextInputModify"?: string // optional
"hookMarkerTextOutputModify"?: string // optional
"hookMarkerTextInputBlock"?: string // optional
"hookMarkerTextOutputBlock"?: string // optional
"enableShareConversation"?: boolean // optional
"enableLocation"?: boolean // optional
"enableShowPoweredBy"?: boolean // optional
"poweredByLogo"?: string (uri) // optional
"enableAnonymous"?: boolean // optional
"enableDisplayPreviousConversation"?: boolean // optional
"enableDisplayConversationHistory"?: boolean // optional
"enableConversationTimer"?: boolean // optional
"conversationTimerDuration"?: integer // Timer timeout duration, from 3 to 60 minutes (optional)
"enableAutoNewConversationOnTimeout"?: boolean // optional
"enableSatisfactionSurvey"?: boolean // optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the final AI response before
the survey appears, from 1 to 60 minutes (optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend
default localized text (optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // optional
"enableDisplayFontSizeSwitch"?: boolean // optional
"defaultThemeMode"?: // optional
{
}
"enableDownloadCitations"?: boolean // optional
"customDomain"?: // optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // optional
"enableDisplayChatbotName"?: boolean // optional
"buttonIcon"?: string (uri) // optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
  object
]
"preChatFormEnabled"?: boolean // optional
"preChatFormFields"?: object // optional

```

```

"preChatFormTitle"?: object // optional
"preChatFormSubtitle"?: object // optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(optional)
"toolCardRunningIconUrl"?: string (uri) // optional
"toolCardCompletedIconUrl"?: string (uri) // optional
"inbox": object
"quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}

```

Response Example Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
  }
}

```

```
"userMessageBackgroundColor": "response string",
"chatbotMessageBackgroundShadowEnabled": false,
"userMessageBackgroundShadowEnabled": false,
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "response string",
  "navbarTextColor": "response string",
  "conversationBackgroundColor": "response string",
  "chatbotMessageTextColor": "response string",
  "userMessageTextColor": "response string",
  "chatbotMessageBackgroundColor": "response string",
  "userMessageBackgroundColor": "response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
```

```
"includeGreetingInContext": false,
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response Example Name",
    "description": "Response Example Description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response Example Name",
  "description": "Response Example Description"
},
"quickMenu": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response Example Name",
  "description": "Response Example Description"
}
}
```

Set Contact Credentials

POST

</api/v1/web-chats/{id}/setup-contact-credentials/>

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
sourceId	string	Yes	Unique identifier for the contact
name	string	No	Contact display name
mcpCredentials	object (with 2 properties: toolId, headers)	No	MCP credentials configuration
mcpCredentials.toolId	string (uuid)	Yes	MCP Tool ID
mcpCredentials.headers	object	No	Authentication headers
embedOrigin	string	No	Origin of the embedding page (forwarded by the web chat frontend) for allowlist validation
ts	integer	No	Unix timestamp (seconds, UTC) the signature was computed with. Required when the inbox login settings enable Source ID access with signature verification.
sig	string	No	Hex HMAC-SHA256 of "{webChatId}.{sourceId}.{ts}" computed by the embedding site backend

Request Schema Example

```
{
  "sourceId": string // Unique identifier for the contact
  "name"? : string // Contact display name (optional)
  "mcpCredentials"? : // MCP credentials configuration (optional)
  {
    "toolId": string (uuid) // MCP Tool ID
    "headers"? : object // Authentication headers (optional)
  }
  "embedOrigin"? : string // Origin of the embedding page (forwarded by the web chat frontend) for
allowlist validation (optional)
  "ts"? : integer // Unix timestamp (seconds, UTC) the signature was computed with. Required when the
inbox login settings enable Source ID access with signature verification. (optional)
  "sig"? : string // Hex HMAC-SHA256 of "{webChatId}.{sourceId}.{ts}" computed by the embedding site
backend (optional)
}
```

Request Example Value

```
{
  "sourceId": "example string",
  "name": "Example Name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  },
  "embedOrigin": "example string",
  "ts": 123,
  "sig": "example string"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "sourceId": "example string",
  "name": "Example Name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  },
  "embedOrigin": "example string",
  "ts": 123,
  "sig": "example string"
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "sourceId": "example string",
  "name": "Example Name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  },
  "embedOrigin": "example string",
  "ts": 123,
  "sig": "example string"
};

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-  
contact-credentials/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceId": "example string",
    "name": "Example Name",
    "mcpCredentials": {
        "toolId": "550e8400-e29b-41d4-a716-446655440000",
        "headers": {}
    },
    "embedOrigin": "example string",
    "ts": 123,
    "sig": "example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "sourceId": "example string",
            "name": "Example Name",
            "mcpCredentials": {
                "toolId": "550e8400-e29b-41d4-a716-446655440000",
                "headers": {}
            },
            "embedOrigin": "example string",
            "ts": 123,
            "sig": "example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "contactId": string (uuid) // The created or existing contact ID
  "created": boolean // Whether the contact was created by this call. False means an existing contact
  matched the source_id; in that case the request name is not applied (update names via PATCH
  /contacts/{id}).
}

```

Response Example Value

```
{
  "contactId": "550e8400-e29b-41d4-a716-446655440000",
  "created": false
}
```

Status Code: **400** - No response body

Get Voice Agent Token

POST /api/v1/web-chats/{id}/voice-agent/token/

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	

Field	Type	Required	Description
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a floating row of buttons above the input box (which remains available); when disabled, the current overlay panel is retained. This is a feature flag that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	

Field	Type	Required	Description
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, from 3 to 60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI response before the survey appears, from 1 to 60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text. Leave blank to use the frontend localization default.
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	

Field	Type	Required	Description
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. <code>narrative</code> shows per-stage copy; <code>brand_animation</code> plays the uploaded asset and hides every system narration line. Defaults to <code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
```

```

"backUrl"?: string // Optional
"isActive"?: boolean // Optional
"theme": {
{
"primaryColor"?: string // Optional
"navbarTextColor"?: string // Optional
"conversationBackgroundColor"?: string // Optional
"chatbotMessageTextColor"?: string // Optional
"userMessageTextColor"?: string // Optional
"chatbotMessageBackgroundColor"?: string // Optional
"userMessageBackgroundColor"?: string // Optional
"chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
"userMessageBackgroundShadowEnabled"?: boolean // Optional
"chatbotMessageGradientEnabled"?: boolean // Optional
"userMessageGradientEnabled"?: boolean // Optional
}
}
"darkTheme": {
{
"primaryColor"?: string // Optional
"navbarTextColor"?: string // Optional
"conversationBackgroundColor"?: string // Optional
"chatbotMessageTextColor"?: string // Optional
"userMessageTextColor"?: string // Optional
"chatbotMessageBackgroundColor"?: string // Optional
"userMessageBackgroundColor"?: string // Optional
"chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
"userMessageBackgroundShadowEnabled"?: boolean // Optional
"chatbotMessageGradientEnabled"?: boolean // Optional
"userMessageGradientEnabled"?: boolean // Optional
}
}
"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
"enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a floating
row of buttons above the input box (which remains available); when disabled, the current overlay
panel is retained. This is a feature flag that can be disabled at any time to roll back. (Optional)
"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, from 3 to 60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before

```

```

the survey appears, from 1 to 60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text. Leave blank to use the frontend
localization default. (Optional)
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}

```

Request Example Value

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",

```

```
"cover": "https://example.com/file.jpg",
"backUrl": "example string",
"isActive": true,
"theme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"darkTheme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"enableFileUpload": true,
"enableDisplayCitations": true,
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "example string",
"hookMarkerTextOutputModify": "example string",
"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
```



```
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
```

```
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```

```

"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
};

```

```

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",

```

```

    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
        "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while sending the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "Example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "Example string",
            "isActive": true,
            "theme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "Example string",
                "navbarTextColor": "Example string",
                "conversationBackgroundColor": "Example string",
                "chatbotMessageTextColor": "Example string",
                "userMessageTextColor": "Example string",
                "chatbotMessageBackgroundColor": "Example string",
                "userMessageBackgroundColor": "Example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "Example string",

```



```

        "hookMarkerTextOutputModify": "Example string",
        "hookMarkerTextInputBlock": "Example string",
        "hookMarkerTextOutputBlock": "Example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "Example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "Example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while sending the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options float in a button row
  above the input box (which remains available); when disabled, the current overlay panel is retained.
  This is a feature flag that can be disabled at any time to roll back. (Optional)
```

```

"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration; range: 3–60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI response before
the survey appears; range: 1–60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the default
localized frontend text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional

```

```

"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
"quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "Response string"
  ],
  "theme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
  }
}

```

```
"userMessageBackgroundColor": "Response string",
"chatbotMessageBackgroundShadowEnabled": false,
"userMessageBackgroundShadowEnabled": false,
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "Response string",
  "navbarTextColor": "Response string",
  "conversationBackgroundColor": "Response string",
  "chatbotMessageTextColor": "Response string",
  "userMessageTextColor": "Response string",
  "chatbotMessageBackgroundColor": "Response string",
  "userMessageBackgroundColor": "Response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "Response string",
"hookMarkerTextOutputModify": "Response string",
"hookMarkerTextInputBlock": "Response string",
"hookMarkerTextOutputBlock": "Response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "Response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "Response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
```

```
"includeGreetingInContext": false,
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"quickMenu": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Upload Batch QA File

POST

/api/v1/web-chats/{webchatPk}/batch-qas/

Parameters

Parameter Name	Required	Type	Description
webchatPk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
file	string (uri)	Yes	

Example Request Structure

```
{
  "file": string (uri)
}
```

Example Request Value

```
{
  "file": "https://example.com/file.jpg"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "file": "https://example.com/file.jpg"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "file": "https://example.com/file.jpg"
};

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while sending the request:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while sending the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "file": "https://example.com/file.jpg"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while sending the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "file": string (uri)
  "createdAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "file": "https://example.com/file.jpg",
```

```
"createdAt": "Response string"
}
```

Export Batch QA to Excel

GET

/api/v1/web-chats/{webchatPk}/batch-qas/{id}/export-excel/

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Batch QA File.
webchatPk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this command.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
200	Excel file download

Get LLM Usage Statistics

GET

/api/v1/chatbots/{chatbotPk}/llm-usage-statistics/

Parameters

Parameter Name	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
endDate	X	string	End date in YYYY-MM-DD format

startDate	V	string	Start date in YYYY-MM-DD format
timeGranularity	V	string	Time granularity (day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this command.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
[
  {
    "date": string (date)
  }
]
```

Example Response Value

```
[
  [
    {
      "totalWordsCount": 511,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",
          "llmModelProvider": "openai"
        }
      ]
    }
  ]
]
```

Status Code: 400

Example Response Schema

```
{
  "detail"?: string // optional
  "valid_types"?: [ // optional
    string
  ]
}
```

Example Response Value

```
[
  [
    {
      "totalWordsCount": 511,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",

```



```
      "llmModelProvider": "openai"
    }
  ]
}
]
```

Get AI Assistant Statistics

GET

`/api/v1/chatbots/{chatbotPk}/statistics/`

Parameters

Parameter Name	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>datetimeFrom</code>	X	string	Start time
<code>datetimeTo</code>	X	string	End time
<code>timeGranularity</code>	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this command.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
[
  {
    "datetime": integer // Timestamp (milliseconds)
    "messagesCount": integer // Number of messages
    "conversationsCount": integer // Number of conversations
    "likesCount": integer // Number of likes
    "dislikesCount": integer // Number of dislikes
  }
]
```

Example Response Value

```
[
  {
    "datetime": 456,
    "messagesCount": 456,
    "conversationsCount": 456,
    "likesCount": 456,
```

```
    "dislikesCount": 456  
  }  
]
```

API call example (Shell)

...

目錄

- 1. Send Message (Streaming)
- 2. Send Message (Create)
- 3. Send an Outgoing Message
- 4. Create a new conversation
- 5. List messages
- 6. Get a specific message
- 7. List conversations
- 8. Get a specific conversation
- 9. Get a specific conversation
- 10. Rename Conversation
- 11. Share a conversation
- 12. Quickly share a conversation
- 13. List shared conversations
- 14. Retrieve a shared conversation
- 15. Update a shared conversation
- 16. Delete Shared Conversation
- 17. Fork Shared Conversation
- 18. List My Shared Conversations
- 19. Create Message Feedback
- 20. Update Message Feedback
- 21. Delete Message Feedback
- 22. List conversation records
- 23. Get a conversation record
- 24. Get a specific conversation record
- 25. Create a conversation record export request
- 26. Get the conversation record export status

API call example (Shell)

Send Message (Streaming)

POST

/api/v1/chatbots/{chatbotId}/completions/

Parameters

Parameter Name	Required	Type	Description
chatbotId	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	No	Unique conversation identifier. If empty, a new conversation is created (optional)
message	object (with 8 properties: content, contentPayload, queryMetadata...)	Yes	Message content to send
message.content	string	Yes	Text content of the message; it may be an empty string when attachments are provided
message.contentPayload	object	No	Additional message content payload in JSON format (optional)
message.queryMetadata	object	No	Query metadata in JSON format (optional)
message.metadata	object	No	Message environment metadata, such as time zone information, in JSON format (optional)
message.attachments	array[AttachmentInput]	No	List of message attachments (optional)
message.toolIds	array[string]	No	List of tool IDs enabled for this message (optional)
message.clientTools	array[any]	No	List of tool definitions provided by the client (such as a Chrome extension) for this message (optional). Each item must include name (str), description (str), and input_schema (dict). The backend does not execute these tools during the LLM call; instead, it returns the clientToolCall content and ends the current turn.
message.sender	string (uuid)	No	Sender Contact ID (optional)
isStreaming	boolean	No	Whether to respond in streaming mode. Defaults to false (optional)
waitForAttachments	boolean	No	Whether to wait for attachment processing to complete before entering the agent workflow. Defaults to true (optional)

Request Schema Example

```
{
  "conversation"?: string (uuid) // Unique conversation identifier. If empty, a new
  conversation is created (optional) (optional)
```

```

"message": // Message content to send
{
  "content": string // Text content of the message; it may be an empty string when
attachments are provided
  "contentPayload"?: object // Additional message content payload in JSON format (optional)
(optional)
  "queryMetadata"?: object // Query metadata in JSON format (optional) (optional)
  "metadata"?: object // Message environment metadata, such as time zone information, in JSON
format (optional) (optional)
  "attachments"?: [ // List of message attachments (optional) (optional)
    {
      "id": string (uuid) // Unique attachment identifier
      "type": // Attachment type. Available values: image, video (under development and not
yet supported), audio, sticker (under development and not yet supported), and other

* `image` - Image
* `video` - Video
* `audio` - Audio
* `sticker` - Sticker
* `other` - Other
      {
      }
      "filename": string // Attachment filename
      "file": string (uri) // URL of the attachment file
    }
  ]
  "toolIds"?: [ // List of tool IDs enabled for this message (optional) (optional)
    string (uuid)
  ]
  "clientTools"?: [ // List of tool definitions provided by the client (such as a Chrome
extension) for this message (optional). Each item must include name (str), description (str),
and input_schema (dict). The backend does not execute these tools during the LLM call; instead,
it returns the clientToolCall content and ends the current turn. (optional)
    object
  ]
  "sender"?: string (uuid) // Sender Contact ID (optional) (optional)
}
  "isStreaming"?: boolean // Whether to respond in streaming mode. Defaults to false (optional)
(optional)
  "waitForAttachments"?: boolean // Whether to wait for attachment processing to complete
before entering the agent workflow. Defaults to true (optional) (optional)
}

```

Request Example Value

```

{
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
}

```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "message": {
        "content": "Hello, please introduce yourself"
    },
    "is_streaming": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "message": {
                "content": "Hello, please introduce yourself"
            },
            "is_streaming": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Conversation response content; an event stream when streaming
400	Invalid request parameters. Possible causes include the Contact ID not belonging to the organization

Send Message (Create)

POST

</api/v1/messages/>

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the scope of queryable knowledge; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of queryable knowledge bases; an empty array or null means none are queryable (no permission), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filtering criteria; an empty conditions array means no label filtering (not no permission). This must be provided together with knowledgeBases —when provided

Field	Type	Required	Description
			alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label criteria are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label criteria; supports nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Request Schema Example

```
{
  "conversation": string (uuid)
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "attachments"?: [ // optional
    {
      "type"?: // optional
      {
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // optional
    }
  ]
}
```

```

    }
  ]
  "canvas"?: { // optional
    {
      "name": string
      "canvasType":
        {
        }
      "title": string
      "content": string
    }
  }
  "slideTemplate"?: // optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of queryable knowledge; see
each endpoint description for details (optional)
    {
      "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none are queryable (no permission), while omission means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the listed items; false = select only the listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filtering criteria; an empty conditions array means no label
filtering (not no permission). This must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
        {
          "operator": string (enum: AND, OR) // How label criteria are combined
          "conditions"?: [ // List of label criteria; supports nested operator/conditions
combinations (optional)
            object
          ]
        }
      }
    }
  }
}

```

```

}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"suggestedForId"?: string (uuid) // optional
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
    string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
    string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
}

```

Request Example Value

```

{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiaagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ]
  },
  "metadata": null
},
"feedback": "like",
"createdAt": "1748314926000",
"attachments": [],
"citations": [],
"citationNodes": [],
"canvas": {
  "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
  "name": "system-architecture-diagram",
  "canvasType": "markdown",
  "title": "System Architecture Diagram",
  "content": "<!-- MERMAID_PLACEHOLDER_START -->
<div class='mermaid-diagram'>
  <div class='mermaid'>
    \ngraph TD\n      A[User] --> B[Frontend Interface]\n      B --> C[API Layer]\n      C -->

```



```
D[Database]\n
  </div>
  <p class="mermaid-caption">Mermaid 圖表</p>
</div>
<!-- MERMAID_PLACEHOLDER_END --> ",
  "createdAt": "1748314926000"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ]
  },
  "metadata": null
},
"feedback": "like",
"createdAt": "1748314926000",
"attachments": [],
"citations": [],
"citationNodes": [],
"canvas": {
  "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
  "name": "system-architecture-diagram",
  "canvasType": "markdown",
  "title": "System Architecture Diagram",
  "content": "
```

```
\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n
```

Mermaid 圖表

```
",
  "createdAt": "1748314926000"
```

```
}  
}'
```

```
# Before running, replace YOUR_API_KEY and verify the request data.
```

```

const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
    "title": "System Architecture Diagram",
    "content": "

```

```

\ng
graph TD
  A[User] --> B[Frontend Interface]
  B --> C[API Layer]
  C --> D[Database]

```

```
    ",  
    "createdAt": "1748314926000"  
  }  
};  
  
axios.post("https://api.maiagent.ai/api/v1/messages/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('An error occurred while making the request:');  
    console.error(error.response?.data || error.message);  
  });
```

```

import requests

url = "https://api.maiagent.ai/api/v1/messages/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
    "sender": {
        "id": 2.992461161002561e+38,
        "name": "jessiekuo",
        "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
    },
    "type": "incoming",
    "content": "This is a test message",
    "contentPayload": {
        "content": "This is a test message",
        "items": [
            {
                "type": "text",
                "text": "This is a test message",
                "startTimestamp": null,
                "citations": []
            }
        ],
        "metadata": null
    },
    "feedback": "like",
    "createdAt": "1748314926000",
    "attachments": [],
    "citations": [],
    "citationNodes": [],
    "canvas": {
        "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
        "name": "system-architecture-diagram",
        "canvasType": "markdown",
        "title": "System Architecture Diagram",
        "content": "

```

```

\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n
```

Mermaid 圖表

```
"",  
    "createdAt": "1748314926000"  
  }  
}
```

```
response = requests.post(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
            "sender": {
                "id": 2.992461161002561e+38,
                "name": "jessiekuo",
                "avatar": "https://media-dev.maiagent.ai/static/images/default-user-
avatar.png"
            },
            "type": "incoming",
            "content": "This is a test message",
            "contentPayload": {
                "content": "This is a test message",
                "items": [
                    {
                        "type": "text",
                        "text": "This is a test message",
                        "startTimestamp": null,
                        "citations": []
                    }
                ]
            },
            "metadata": null
        },
        "feedback": "like",
        "createdAt": "1748314926000",
        "attachments": [],
        "citations": [],
        "citationNodes": [],
        "canvas": {
            "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
            "name": "system-architecture-diagram",
            "canvasType": "markdown",
            "title": "System Architecture Diagram",
            "content": "

```

```

\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C -->
D[Database]\n

```


Mermaid 圖表

```
    ",
        "createdAt": "1748314926000"
    }
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email"?: string (email) // optional
    "phoneNumber"?: string // optional
  }
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // optional
    "updatedAt": string (timestamp)
  }
  "createdAt": string (timestamp)
```

```
"attachments"?: [ // optional
```

```
{
  "id": string (uuid)
  "type"?: // optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.
```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```
  "conversation"?: string (uuid) // optional
}
]
```

```
"citations": [
```

```
{
  "id": string (uuid)
  "filename": string // Filename
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // optional
```

```
{
  "id": string (uuid)
  "name": string
}
```

```
"size": integer
"status":
```

```
{
}
```

```
"parser":
```

```
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "isTimestampSttProvider": boolean
}
```

```
"labels"?: [ // optional
```

```
{
  "id": string (uuid)
  "name": string
}
```

```
]
```

```
"rawUserDefineMetadata"?: object // optional
```

```
"speakerLabels": [
```

```
{
  "id": string (uuid)
  "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
  "customName"?: string // User-defined speaker name (e.g., John) (optional)
  "displayName": string
}
```

```
]
```

```
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
```

```
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived,
previewable PDF file (displayed in the frontend PDF viewer); None otherwise.
```

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a consistent format (including each storage provider's presign/host rules). Do not use `get_absolute_url()`, as it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
]
"citationNodes": [
{
  "chatbotTextNode": {
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support frontend image previews
      "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
      "pageNumber": integer // Backward-compatible alias of ``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title displayed in the inline citation hover card
      "citationDescription": string // Source description displayed in the inline citation
hover card
      "citationQuote": string // Quoted text excerpt displayed in the inline citation hover
card
      "hasImage": boolean // Indicates whether an image is included (checks the file type or
Markdown images in text)
      "imageUrl": string // Extracts the image URL (uses the file URL first; otherwise
extracts it from Markdown)
      "displayText": string // Returns the full text with Markdown image markup removed
      "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
      "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.
```

Priority:

1. ES/OpenSearch native highlight (set by `retrieve_api` via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [
{
  "id": string (uuid)
  "name": string
}
]
"rawUserDefineMetadata?": object // Users can define their own metadata keys and values
(optional)
```

```
    "metadataEnabledForSearch": object // Map each user-defined metadata key on this node to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and* the node carries a value for it. Since this maps over ``raw\_user\_define\_metadata`` (the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
    "startCharIdx": integer
    "endCharIdx": integer
  }
}
"score"?: number (double) // optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply content (optional)
"highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (optional)
}
]
"canvas"?: { // optional
{
  "id": string (uuid)
  "name": string
  "canvasType":
  {
  }
  "title": string
  "content": string
  "createdAt": string (timestamp)
}
}
"slideTemplate"?: // optional
{
  "slug": string
  "displayName": string
  "thumbnailUrls"?: [ // optional
    string
  ]
}
}
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each endpoint description for details (optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted (optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
      string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this field takes precedence when both are provided) (optional)
      string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
```

```

        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude listed items; false = select only listed items (optional)
}
]
"labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
{
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
        object
    ]
}
}
}
}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"recordId": string // Returns the ChatbotRecord ID corresponding to the Message

```

Note:

```

    For unsaved Messages (such as virtual Messages during streaming), return None directly
    to avoid triggering a DB query on the OneToOneField reverse relation (N+1 issue).
    "broadcast"?: boolean // optional
    "skipCopilotTrigger"?: boolean // optional
    "activeRevisionNumber": integer
    "revisions": [
        {
            "id": string (uuid)
            "revisionNumber": integer
            "content": string
            "contentPayload": object
            "isActive": boolean
            "createdAt": string (timestamp)
        }
    ]
    "suggestedForId"?: string (uuid) // optional
    "suggestionStatus": string
    "suggestionTriggerSource": string
    "suggestionError": string
    "retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
    "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
        string (uuid)
    ]
    "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
        string (uuid)
    ]
    "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation

```

```
(camelCase: templateId). (optional)
  "videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
  ``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}
```

Example Response Value

```
{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
    "title": "System Architecture Diagram",
    "content": "<!-- MERMAID_PLACEHOLDER_START -->
<div class='mermaid-diagram'>
  <div class='mermaid'>
    \ngraph TD\n  A[User] --> B[Frontend Interface]\n  B --> C[API Layer]\n  C -->
    D[Database]\n
  </div>
  <p class='mermaid-caption'>Mermaid 圖表</p>
</div>
<!-- MERMAID_PLACEHOLDER_END -->",
    "createdAt": "1748314926000"
  }
}
```

Send an Outgoing Message

POST

/api/v1/messages/outgoing/

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the searchable knowledge scope; see each endpoint description for details

Field	Type	Required	Description
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filter criteria; an empty conditions array means no label filtering (not no permission). Must be provided together with knowledgeBases—when provided alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).

Field	Type	Required	Description
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Example Request Schema

```
{
  "conversation": string (uuid)
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "attachments"?: [ // optional
    {
      "type"?: // optional
      {
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // optional
    }
  ]
  "canvas"?: { // optional
    {
      "name": string
      "canvasType":
      {
      }
      "title": string
      "content": string
    }
  }
  "slideTemplate"?: // optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each endpoint description for details (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
```

```

field takes precedence when both are provided) (optional)
    string (uuid)
  ]
  "faqIds"?: [ // List of FAQ IDs (excluded or selected exclusively depending on
hasUserSelectedAll) (optional)
    string (uuid)
  ]
  "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude listed items; false = select only listed items (optional)
}
]
"labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
  {
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
      object
    ]
  }
}
}
}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"suggestedForId"?: string (uuid) // optional
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
}

```

Example Request Value

```

{
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
}
],
"canvas": {
  "name": "Example Name",
  "canvasType": {},
  "title": "Example Name",
  "content": "Hello! I would like to learn more about the product."
},
"slideTemplate": {
  "slug": "example string",
  "displayName": "example string",
  "thumbnailUrls": [
    "example string"
  ]
},
"queryMetadata": {
  "knowledgeBases": [
    {
      "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "faqIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "hasUserSelectedAll": true
    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "canvas": {
    "name": "Example Name",
    "canvasType": {},
    "title": "Example Name",
    "content": "Hello! I would like to learn more about the product."
  },
  "slideTemplate": {
    "slug": "example string",
    "displayName": "example string",
    "thumbnailUrls": [
      "example string"
    ]
  },
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
```

```
        "conditions": [  
            null  
        ]  
    }  
},  
"metadata": null,  
"broadcast": true,  
"skipCopilotTrigger": true,  
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",  
"skillIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
],  
"toolIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
],  
"templateId": "550e8400-e29b-41d4-a716-446655440000"  
}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```

const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "canvas": {
    "name": "Example Name",
    "canvasType": {},
    "title": "Example Name",
    "content": "Hello! I would like to learn more about the product."
  },
  "slideTemplate": {
    "slug": "example string",
    "displayName": "example string",
    "thumbnailUrls": [
      "example string"
    ]
  },
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ]
  }
};

```

```

    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/messages/outgoing/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```



```

import requests

url = "https://api.maiagent.ai/api/v1/messages/outgoing/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "type": "example string",
    "content": "Hello! I would like to learn more about the product.",
    "contentPayload": null,
    "attachments": [
        {
            "type": {},
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "conversation": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "canvas": {
        "name": "Example Name",
        "canvasType": {},
        "title": "Example Name",
        "content": "Hello! I would like to learn more about the product."
    },
    "slideTemplate": {
        "slug": "example string",
        "displayName": "example string",
        "thumbnailUrls": [
            "example string"
        ]
    },
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ]
    },
    ],

```

```
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        },
    },
    "metadata": null,
    "broadcast": true,
    "skipCopilotTrigger": true,
    "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
    "skillIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "toolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "templateId": "550e8400-e29b-41d4-a716-446655440000"
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/outgoing/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "conversation": "550e8400-e29b-41d4-a716-446655440000",
            "type": "example string",
            "content": "Hello! I would like to learn more about the product.",
            "contentPayload": null,
            "attachments": [
                {
                    "type": {},
                    "filename": "document.pdf",
                    "file": "https://example.com/file.jpg",
                    "conversation": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "canvas": {
                "name": "Example Name",
                "canvasType": {},
                "title": "Example Name",
                "content": "Hello! I would like to learn more about the product."
            },
            "slideTemplate": {
                "slug": "example string",
                "displayName": "example string",
                "thumbnailUrls": [
                    "example string"
                ]
            },
            "queryMetadata": {
                "knowledgeBases": [
                    {
                        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                        "chatbotFileIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "knowledgeBaseFileIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "faqIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "hasUserSelectedAll": true
                    }
                ]
            }
        }
    ]);
} catch (Exception $e) {
    // Handle exception
}

```

```

        }
    ],
    "labelRelations": {
        "operator": "AND",
        "conditions": [
            null
        ]
    }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
}
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 201

Example Response Schema

```

{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email?": string (email) // optional
    "phoneNumber?": string // optional
  }
}

```

```

"type"?: string // optional
"content"?: string // optional
"contentPayload"?: object // optional
"feedback":
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // optional
  "updatedAt": string (timestamp)
}
"createdAt": string (timestamp)
"attachments"?: [ // optional
{
  "id": string (uuid)
  "type"?: // optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```

  "conversation"?: string (uuid) // optional
}
]
"quotations": [
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // optional
  {
    "id": string (uuid)

```

```

    "name": string
  }
]
"rawUserDefineMetadata"?: object // optional
"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John) (optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // Presentations (pptx/ppt) return a URL for a previewable PDF
derivative (shown in the frontend PDF viewer); otherwise None.

```

Generate the URL with the same CustomizedFileFieldSerializer used by the `file` field to keep the format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()`—it produces an incorrect URL format.

```

    "createdAt": string (timestamp)
  }
]
"citationNodes": [
  {
    "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
        "filename": string
        "chatbotFile": object // Return complete ChatbotFile details, including the file URL
and type, to support image previews in the frontend
        "knowledgeBaseFile": object // Same as get_chatbot_file, for backward compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title shown on the inline citation hover card.
        "citationDescription": string // Source description shown on the inline citation hover
card.
        "citationQuote": string // Quoted text snippet shown on the inline citation hover card.
        "hasImage": boolean // Whether an image is present (checks the file type or Markdown
images in text)
        "imageUrl": string // Extract the image URL (prefer the file URL; otherwise extract it
from Markdown)
        "displayText": string // Return the full text with image Markdown removed
        "displayTitle": string // Return the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [  
  {  
    "id": string (uuid)  
    "name": string  
  }  
]  
"rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.  
(optional)  
"metadataEnabledForSearch": object // Map each user-defined metadata key on this node  
to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and* the node carries a value for it. Since this maps over ``raw\_user\_define\_metadata`` (the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
"startCharIdx": integer  
"endCharIdx": integer  
}  
}  
"score"?: number (double) // optional  
"displayScore": integer  
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply  
content (optional)  
"highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (optional)  
}  
]  
"canvas"?: { // optional  
{  
  "id": string (uuid)  
  "name": string  
  "canvasType":  
  {  
  }  
  "title": string  
  "content": string  
  "createdAt": string (timestamp)  
}  
}  
"slideTemplate"?: // optional  
{  
  "slug": string  
  "displayName": string  
  "thumbnailUrls"?: [ // optional  
    string  
  ]  
}  
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each  
endpoint for details (optional)  
{  
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means  
none are accessible (no permission), while omission means no restriction (optional)  
  {
```

```

    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"? : [ // List of file IDs (excluded or selected only, depending on
hasUserSelectedAll) (optional)
        string (uuid)
    ]
    "knowledgeBaseFileIds"? : [ // List of file IDs (new name for chatbotFileIds; takes
precedence when both are provided) (optional)
        string (uuid)
    ]
    "faqIds"? : [ // List of FAQ IDs (excluded or selected only, depending on
hasUserSelectedAll) (optional)
        string (uuid)
    ]
    "hasUserSelectedAll"? : boolean // true selects all knowledge base content except listed
items; false selects only listed items (optional)
}
]
"labelRelations"? : { // Label filter; an empty conditions array disables label filtering
(it does not mean no permission). Provide it with knowledgeBases; when provided alone, the
label filter is not applied to knowledge base retrieval (equivalent to no restriction)
(optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"? : [ // List of label conditions; supports nested operator/conditions
combinations (optional)
            object
        ]
    }
}
}
}
}
"metadata"? : object // Message metadata, such as timezone and other environment information
(optional)
"recordId": string // Return the ChatbotRecord ID for this Message

```

Note:

For an unsaved Message (such as a virtual Message during streaming), return None directly to avoid a DB query through the OneToOneField reverse relation (the N+1 problem).

```

"broadcast"? : boolean // optional
"skipCopilotTrigger"? : boolean // optional
"activeRevisionNumber": integer
"revisions": [
    {
        "id": string (uuid)
        "revisionNumber": integer
        "content": string
        "contentPayload": object
        "isActive": boolean
        "createdAt": string (timestamp)
    }
]
"suggestedForId"? : string (uuid) // optional
"suggestionStatus": string
"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message

```



```

generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-attempt records live in metadata["retry_attempts"].

"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase: skillIds). (optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds). (optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation (camelCase: templateId). (optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery; ``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string"
  },
  "type": "response string",
  "content": "response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "response string",
    "updatedAt": "response string"
  },
  "createdAt": "response string",
  "attachments": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "expiresAt": "response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "citations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "response string",
      "updatedAt": "response string",
      "filename": "response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    },
    "knowledgeBaseFile": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  },
  "pageNumber": 456,
  "pageStart": 456,
```

```
    "pageEnd": 456,
    "citationTitle": "response string",
    "citationDescription": "response string",
    "citationQuote": "response string",
    "hasImage": false,
    "imageUrl": "response string",
    "displayText": "response string",
    "displayTitle": "response string",
    "highlightedText": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
      "key2": "value2",
      "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
  },
  "score": 456,
  "displayScore": 456,
  "citationNumber": 456,
  "highlightedText": "response string"
}
],
"canvas": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "canvasType": {},
  "title": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"slideTemplate": {
  "slug": "response string",
  "displayName": "response string",
  "thumbnailUrls": [
    "response string"
  ]
},
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"recordId": "response string",
"broadcast": false,
"skipCopilotTrigger": false,
"activeRevisionNumber": 456,
"revisions": [
```

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "revisionNumber": 456,
  "content": "response string",
  "contentPayload": null,
  "isActive": false,
  "createdAt": "response string"
},
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"suggestionStatus": "response string",
"suggestionTriggerSource": "response string",
"suggestionError": "response string",
"retryCount": 456,
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000",
"videoGeneration": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Create a new conversation

POST

</api/v1/conversations/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{}'
```

Before running, replace YOUR_API_KEY and verify the request data.

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/conversations/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/conversations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {}
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 201

Example response schema

```

{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId?": string // optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": object
  "effectiveThinkingConfig": object // Return the effective thinking config after applying the
  priority chain:
  conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
"handoff": object // Handoff details: enabled is the platform switch; the continuation
conversation has source_conversation_id/source_title; the original conversation has
continued_to_conversation_id/continued_to_title and latest_summary_message_id
"title": string
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "sourceId": "response string"
  },
  "lastMessageCreatedAt": "response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
```



```
    "description": "Example response description"
  },
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "callerOrganizationId": "response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "response string"
}
```

List messages

GET

/api/v1/messages/

Parameters

Parameter	Required	Type	Description
conversation	V	string	Conversation ID
cursor	X	string	The pagination cursor value.
pageSize	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "conversation": string (uuid)
      "sender":
        {
          "id": string (uuid)
          "name": string
          "avatar": string
          "email"?: string (email) // optional
          "phoneNumber"?: string // optional
        }
      "type"?: string // optional
      "content"?: string // optional
      "contentPayload"?: object // optional
      "feedback":
        {
          "id": string (uuid)
          "type":
            {
            }
          "suggestion"?: string // optional
          "updatedAt": string (timestamp)
        }
      "createdAt": string (timestamp)
      "attachments"?: [ // optional
        {
          "id": string (uuid)
          "type"?: // optional
          {
          }
          "filename": string
          "file": string (uri)
          "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

          For conversation-bound attachments: conversation.last_message_created_at + retention_days.
          For attachments without a conversation: created_at + retention_days.
          Returns None when cleanup is disabled (effective_from not set).
          "conversation"?: string (uuid) // optional
        }
      ]
      "citations": [
        {
```

```

    "id": string (uuid)
    "filename": string // File name
    "file": string (uri) // File to upload
    "fileType": string
    "knowledgeBase"?: // optional
    {
        "id": string (uuid)
        "name": string
    }
    "size": integer
    "status":
    {
    }
    "parser":
    {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
    }
    "labels"?: [ // optional
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "rawUserDefineMetadata"?: object // optional
    "speakerLabels": [
        {
            "id": string (uuid)
            "originalLabel": string // Original speaker label from diarization (e.g.,
SPEAKER_00)
            "customName"?: string // User-defined speaker name (e.g., John) (optional)
            "displayName": string
        }
    ]
    "vectorStorageSize": integer // Size of vectors for this file in Elasticsearch
(bytes)
    "chunksCount": integer // Number of chunks/nodes generated from this file
    "waitingTime": number (double)
    "processingTime": number (double)
    "processingTimeDetails": object
    "previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived
previewable PDF (displayed by the frontend PDF viewer); otherwise None.

```

Generate the URL with the same CustomizedFileFieldSerializer used for the `file` field to keep its format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()`; it produces an incorrectly formatted URL.

```

    "createdAt": string (timestamp)
}
]
"citationNodes": [
{

```

```

"chatbotTextNode": {
{
  "id": string (uuid)
  "charactersCount": integer
  "hitsCount": integer
  "text": string
  "updatedAt": string (timestamp)
  "filename": string
  "chatbotFile": object // Return the full ChatbotFile details, including file URL
and type, to support frontend image previews
  "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
  "pageNumber": integer // Backward-compatible alias of ``page_start``.
  "pageStart": integer
  "pageEnd": integer
  "citationTitle": string // Source title shown on the inline citation hover card.
  "citationDescription": string // Source description shown on the inline citation
hover card.
  "citationQuote": string // Quoted text snippet shown on the inline citation hover
card.
  "hasImage": boolean // Whether it contains an image (check the file type or
Markdown images in text)
  "imageUrl": string // Extract the image URL (prefer the file URL; otherwise extract
it from Markdown)
  "displayText": string // Return the full text with Markdown image markup removed
  "displayTitle": string // Return the display title (fallback: citation_title ->
filename)
  "highlightedText": string // Return text with matched terms wrapped in ``<mark>``
tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(optional)
  "metadataEnabledForSearch": object // Map each user-defined metadata key on this
node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
  "startCharIdx": integer
  "endCharIdx": integer
}
}
"score"?: number (double) // optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the

```

```

reply content (optional)
    "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags
(optional)
    }
  ]
  "canvas"?: { // optional
    {
      "id": string (uuid)
      "name": string
      "canvasType":
        {
        }
      "title": string
      "content": string
      "createdAt": string (timestamp)
    }
  }
  "slideTemplate"?: // optional
    {
      "slug": string
      "displayName": string
      "thumbnailUrls"?: [ // optional
        string
      ]
    }
  }
  "queryMetadata"?: { // Query metadata controlling the searchable knowledge scope; see the
endpoint documentation (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null
means none are searchable (no permission); omitted means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or selected according to
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; takes
precedence when both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true selects all knowledge base content except
listed items; false selects only listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filters; an empty conditions array means no label
filtering (not lack of permission). Provide together with knowledgeBases; on its own, label
filtering is not applied to knowledge base retrieval (equivalent to unrestricted) (optional)
        {
          "operator": string (enum: AND, OR) // How label conditions are combined
          "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
            object

```

```

    ]
  }
}
}
"metadata"?: object // Message metadata, such as timezone and other environment
information (optional)
"recordId": string // Return the ChatbotRecord ID associated with the Message

Note:
For an unsaved Message (such as a virtual Message during streaming), return None directly
to avoid a DB query through the OneToOneField reverse relation (N+1 problem).
"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"activeRevisionNumber": integer
"revisions": [
  {
    "id": string (uuid)
    "revisionNumber": integer
    "content": string
    "contentPayload": object
    "isActive": boolean
    "createdAt": string (timestamp)
  }
]
"suggestedForId"?: string (uuid) // optional
"suggestionStatus": string
"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase:
toolIds). (optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload
recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",

```



```
"results": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "sender": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string",
      "avatar": "Example string",
      "email": "response@example.com",
      "phoneNumber": "Example string"
    },
    "type": "Example string",
    "content": "Example string",
    "contentPayload": null,
    "feedback": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "suggestion": "Example string",
      "updatedAt": "Example string"
    },
    "createdAt": "Example string",
    "attachments": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "filename": "Example string",
        "file": "https://example.com/file.jpg",
        "expiresAt": "Example string",
        "conversation": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "citations": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "filename": "Example string",
        "file": "https://example.com/file.jpg",
        "fileType": "Example string",
        "knowledgeBase": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example string"
        },
        "size": 456,
        "status": {},
        "parser": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example string",
          "provider": {},
          "isTimestampSttProvider": false
        },
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example string"
          }
        ],
        "rawUserDefineMetadata": null,

```

```
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "Example string",
    "customName": "Example string",
    "displayName": "Example string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "Example string",
"createdAt": "Example string"
}
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Example string",
      "updatedAt": "Example string",
      "filename": "Example string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
    },
    "knowledgeBaseFile": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "pageNumber": 456,
    "pageStart": 456,
    "pageEnd": 456,
    "citationTitle": "Example string",
    "citationDescription": "Example string",
    "citationQuote": "Example string",
    "hasImage": false,
    "imageUrl": "Example string",
    "displayText": "Example string",
    "displayTitle": "Example string",
    "highlightedText": "Example string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
```

```
        "key2": "value2",
        "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
},
"score": 456,
"displayScore": 456,
"citationNumber": 456,
"highlightedText": "Example string"
}
],
"canvas": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example string",
    "canvasType": {},
    "title": "Example string",
    "content": "Example string",
    "createdAt": "Example string"
},
"slideTemplate": {
    "slug": "Example string",
    "displayName": "Example string",
    "thumbnailUrls": [
        "Example string"
    ]
},
"queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
},
"metadata": null,
"recordId": "Example string",
"broadcast": false,
"skipCopilotTrigger": false,
"activeRevisionNumber": 456,
"revisions": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "revisionNumber": 456,
        "content": "Example string",
        "contentPayload": null,
        "isActive": false,
        "createdAt": "Example string"
    }
],
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"suggestionStatus": "Example string",
"suggestionTriggerSource": "Example string",
"suggestionError": "Example string",
"retryCount": 456,
"skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
```

```
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "videoGeneration": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
]
```

Get a specific message

GET

`/api/v1/messages/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Message.
<code>conversation</code>	V	string	Conversation ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?conversation=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example"
headers = {
  "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-
a716-446655440000/?conversation=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email?": string (email) // optional
    "phoneNumber?": string // optional
  }
  "type?": string // optional
  "content?": string // optional
  "contentPayload?": object // optional
  "feedback":
```

```

{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // optional
  "updatedAt": string (timestamp)
}
"createdAt": string (timestamp)
"attachments"?: [ // optional
  {
    "id": string (uuid)
    "type"?: // optional
    {
    }
    "filename": string
    "file": string (uri)
    "expiresAt": string // ISO datetime when this attachment will be auto-deleted.
  }
]

```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```

  "conversation"?: string (uuid) // optional
}
]
" Citations": [
  {
    "id": string (uuid)
    "filename": string // File name
    "file": string (uri) // File to upload
    "fileType": string
    "knowledgeBase"?: // optional
    {
      "id": string (uuid)
      "name": string
    }
    "size": integer
    "status":
    {
    }
    "parser":
    {
      "id": string (uuid)
      "name": string
      "provider":
      {
      }
      "isTimestampSttProvider": boolean
    }
    "labels"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // optional
  }
]

```

```

"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
    "customName?": string // User-defined speaker name (e.g., John) (optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived
previewable PDF (displayed by the frontend PDF viewer); otherwise None.

```

Generate the URL with the same CustomizedFileFieldSerializer used for the `file` field to keep its format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()`; it produces an incorrectly formatted URL.

```

    "createdAt": string (timestamp)
  }
]
"citationNodes": [
  {
    "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
        "filename": string
        "chatbotFile": object // Return the full ChatbotFile details, including file URL and
type, to support frontend image previews
        "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title shown on the inline citation hover card.
        "citationDescription": string // Source description shown on the inline citation hover
card.
        "citationQuote": string // Quoted text snippet shown on the inline citation hover card.
        "hasImage": boolean // Whether it contains an image (check the file type or Markdown
images in text)
        "imageUrl": string // Extract the image URL (prefer the file URL; otherwise extract it
from Markdown)
        "displayText": string // Return the full text with Markdown image markup removed
        "displayTitle": string // Return the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by `retrieve_api` via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)


```

    "labels": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(optional)
    "metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
    "startCharIdx": integer
    "endCharIdx": integer
  }
}
"score"?: number (double) // Optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (Optional)
"highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (Optional)
}
]
"canvas"?: { // Optional
{
  "id": string (uuid)
  "name": string
  "canvasType":
  {
  }
  "title": string
  "content": string
  "createdAt": string (timestamp)
}
}
"slideTemplate"?: // Optional
{
  "slug": string
  "displayName": string
  "thumbnailUrls"?: [ // Optional
    string
  ]
}
}
"queryMetadata"?: { // Query metadata controlling the scope of searchable knowledge; see each
endpoint description (Optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are accessible (no permission); omitted means unrestricted (Optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or selected according to
hasUserSelectedAll) (Optional)

```

```

        string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; takes
precedence when both are provided) (Optional)
        string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or selected according to hasUserSelectedAll)
(Optional)
        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true=Select all content in the knowledge base except
listed items; false means select only listed items (Optional)
    }
}
    "labelRelations"?: { // Label filter; an empty conditions array means no label filtering
(not lack of permission). Must be provided together with knowledgeBases; when provided alone,
label filtering is not applied to knowledge-base retrieval (equivalent to unrestricted)
(Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
            object
        ]
    }
}
}
}
    "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
    "recordId": string // Return the ChatbotRecord ID associated with the Message

```

Note:

For an unsaved Message (such as a virtual Message during streaming), return None directly to avoid a OneToOneField reverse relation database query (the N+1 problem).

```

"broadcast"?: boolean // Optional
"skipCopilotTrigger"?: boolean // Optional
"activeRevisionNumber": integer
"revisions": [
    {
        "id": string (uuid)
        "revisionNumber": integer
        "content": string
        "contentPayload": object
        "isActive": boolean
        "createdAt": string (timestamp)
    }
]
"suggestedForId"?: string (uuid) // Optional
"suggestionStatus": string
"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:

```

```

skillIds). (Optional)
    string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string"
  },
  "type": "response string",
  "content": "response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "response string",
    "updatedAt": "response string"
  },
  "createdAt": "response string",
  "attachments": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "expiresAt": "response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "citations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    }
  ],

```

```
"size": 456,
"status": {},
"parser": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "provider": {},
  "isTimestampSttProvider": false
},
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "response string",
    "customName": "response string",
    "displayName": "response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "response string",
"createdAt": "response string"
}
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "response string",
      "updatedAt": "response string",
      "filename": "response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "pageNumber": 456,
      "pageStart": 456,
      "pageEnd": 456,
      "citationTitle": "response string",
      "citationDescription": "response string",
```

```
    "citationQuote": "response string",
    "hasImage": false,
    "imageUrl": "response string",
    "displayText": "response string",
    "displayTitle": "response string",
    "highlightedText": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
      "key2": "value2",
      "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
  },
  "score": 456,
  "displayScore": 456,
  "citationNumber": 456,
  "highlightedText": "response string"
}
],
"canvas": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "canvasType": {},
  "title": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"slideTemplate": {
  "slug": "response string",
  "displayName": "response string",
  "thumbnailUrls": [
    "response string"
  ]
},
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"recordId": "response string",
"broadcast": false,
"skipCopilotTrigger": false,
"activeRevisionNumber": 456,
"revisions": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "revisionNumber": 456,
```

```
    "content": "response string",
    "contentPayload": null,
    "isActive": false,
    "createdAt": "response string"
  }
],
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"suggestionStatus": "response string",
"suggestionTriggerSource": "response string",
"suggestionError": "response string",
"retryCount": 456,
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000",
"videoGeneration": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

List conversations

GET

/api/v1/conversations/

Parameters

Parameter name	Required	Type	Description
assignee	X	string	
chatbot	X	string	
contact	X	string	
contactTags	X	array	Multiple values may be separated by commas.
cursor	X	string	The pagination cursor value.
endDate	X	string	
externalConversationId	X	string	

externalSource	X	string	
inbox	X	string	
isPinned	X	string	
mine	X	boolean	
mode	X	string	`chat`: Chat; `cowork`: Cowork;
pageSize	X	integer	Number of results to return per page.
startDate	X	string	
status	X	array	`open`: Open; `resolved`: Resolved; `queued`: Queued;
tags	X	array	Multiple values may be separated by commas.
unassigned	X	boolean	
updatedAfter	X	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAfter=2025-01-01T00:00:00.000Z"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/?
assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-
446655440000&contact=550e8400-e29b-41d4-a716-
446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=ex
ample&externalSource=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=exa
mple&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Response schema example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "contact": { // Lightweight Contact serializer for the Conversation List view

Includes only the fields needed by the List view, omitting inboxes, mcp_credentials, and

```

api_credentials

to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself; the frontend conversation sidebar needs to display them, with no N+1 cost.

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string // Stores our own media as a relative storage key, or an external CDN
URL as-is. (Optional)
  "email"?: // May have different types (Optional)
string (email) // Optional
  "phoneNumber"?: string // Optional
  "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
  "sourceId"?: string // Optional
  "tags": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
}
}
"inbox": { // Inbox summary embedded in a conversation payload.
```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
```

```

    "realm": string
  }
  "line":
  {
    "liffId": string
  }
  "ad":
  {
    "saml": string
  }
  "google"?: // Optional
  {
    "enabled": boolean
  }
}
}

"sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a
Source ID (verified per the verify mode below) may chat without going through the login flow.
Disabled keeps every existing behavior unchanged. (Optional)
"sourceIdVerifyMode"?: // "Require signature verification": the embedding site must
send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID
can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
{
}

"sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
{
}

"keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a
visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API
tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak
sign source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
"keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com"
wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
  string
]
}
}

"enableAnalysis"?: boolean // Optional
}
}

"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already
eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
"title": string

```

```

    "lastMessage"?: { // Optional
    {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
    }
    }
    "callRecord":
    {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
        "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
        "isActive": boolean
        "canHangup": boolean
    }
    "lastMessageCreatedAt": string (timestamp)
    "unreadMessagesCount": integer
    "autoReplyEnabled": boolean
    "isAutoReplyNow": boolean
    "lastReadAt": string (timestamp)
    "createdAt": string (timestamp)
    "isGroupChat": boolean
    "enableGroupMention"?: boolean // Optional
    "queryMetadata"?: { // Query metadata controlling the scope of searchable knowledge; see
each endpoint description (Optional)
    {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null
means none are accessible (no permission); omitted means unrestricted (Optional)
        {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
                string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; takes
precedence when both are provided) (Optional)
                string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
                string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true=Select all content in the knowledge base
except listed items; false means select only listed items (Optional)
        }
    }
}

```

```

    "labelRelations"?: { // Label filter; an empty conditions array means no label
filtering (not lack of permission). Must be provided together with knowledgeBases; when
provided alone, label filtering is not applied to knowledge-base retrieval (equivalent to
unrestricted) (Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
            object
        ]
    }
}
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly
off, {...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying
the priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

    "llm": string (uuid)
    "deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
    {
    }
    "deepResearchOutputFormat": // May have different types
    string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set
only when the user clicks a format in the pre-research choice card; reset to NULL each time
Deep Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and
falls back to Canvas at render time, but stays distinct from an explicit canvas pick so the
frontend can lock the card only after a real selection. To roll back to fixed-Canvas behavior,
force this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
    "ipAddress": string
    "ipCountryCode": string
    "ipRecordedAt": string (timestamp)
    "assignee":

```

```

    {
        "id": string (uuid)
        "name": string
        "userId": string (uuid)
    }
    "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "memberIds"?: [ // Optional
                string (uuid)
            ]
        }
    ]
    "progressStatus": string
    "firstResponseAt"?: string (timestamp) // Optional
    "queuedAt"?: string (timestamp) // Optional
    "assignedAt"?: string (timestamp) // Optional
    "transferReason"?: string // Optional
    "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale
    (1=very dissatisfied, 5=very satisfied)
    "satisfactionComment": string
    "satisfactionSubmittedAt": string (timestamp)
    "latestAnalysisSummary": string // The conversation's current analysis summary (its
    active summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```

    "clientPlatform": // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
    {
    }
    "workingDirectory": string // Absolute path of the local folder bound to a desktop
    conversation. Only set for desktop conversations; null for web conversations.
    "slideProjectId": string
    "matchedField": string // Cached per instance: ``get_matched_message_id`` /
    ``get_matched_snippet``

```

both read this, so resolving it once avoids recomputing the keyword and title-match logic three times per serialized conversation.

```

    "matchedSnippet": string // Keyword-centred excerpt of the matched message, with
    ellipses.

```

Reads the ``matched\_message\_content`` annotation added by ``ConversationViewSet.annotate\_keyword\_match``; no extra query.

```

    "matchedMessageId": string
    "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
    "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped
    into. Null means unclassified (shown under the time groups). Deleting the folder resets this to
    null without deleting the conversation.

```

```
"hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to resume.
```

Lets the frontend know it should proactively re-request the browser panel (emit ``browser:start``) after a page reload, instead of only ever showing the panel when a live ``browser:frame`` push happens to arrive first.

```
"callerOrganizationId": string // Bound caller organization (stamped at conversation start); null when unbound
```

```
"handoff": object // Handoff details: enabled is the platform switch; the continuing conversation includes source_conversation_id/source_title; the original conversation being handed off includes continued_to_conversation_id/continued_to_title and latest_summary_message_id
```

```
    }  
  ]  
}
```

Example response value

```
{  
  "count": 123,  
  "next": "http://api.example.org/accounts/?page=4",  
  "previous": "http://api.example.org/accounts/?page=2",  
  "results": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000",  
      "contact": {  
        "id": "550e8400-e29b-41d4-a716-446655440000",  
        "name": "Response string",  
        "avatar": "Response string",  
        "email": "response@example.com",  
        "phoneNumber": "Response string",  
        "metadata": null,  
        "sourceId": "Response string",  
        "tags": [  
          {  
            "id": "550e8400-e29b-41d4-a716-446655440000",  
            "name": "Response string"  
          }  
        ]  
      },  
    },  
    {  
      "inbox": {  
        "id": "550e8400-e29b-41d4-a716-446655440000",  
        "name": "Response string",  
        "channelType": "line",  
        "unreadConversationsCount": 456,  
        "signAuth": {  
          "id": "550e8400-e29b-41d4-a716-446655440000",  
          "signSource": "550e8400-e29b-41d4-a716-446655440000",  
          "signParams": {  
            "keycloak": {  
              "clientId": "Response string",  
              "url": "Response string",  
              "realm": "Response string"  
            }  
          }  
        }  
      },  
    }  
  ],  
}
```



```
    "line": {
      "liffId": "Response string"
    },
    "ad": {
      "saml": "Response string"
    },
    "google": {
      "enabled": false
    }
  },
  "sourceIdAccessEnabled": false,
  "sourceIdVerifyMode": {},
  "sourceIdSignatureTtlMinutes": {},
  "keycloakTokenPassthrough": false,
  "keycloakTokenPassthroughAllowedHosts": [
    "Response string"
  ]
},
"enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```

```
]
}
```

Get a specific conversation

GET

`/api/v1/conversations/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Conversation.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId?": string // Optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": object
  "effectiveThinkingConfig": object // Return the effective thinking config after applying the
  priority chain:
  conversation override > chatbot config > LLM metadata.
}

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
"handoff": object // Handoff details: enabled is the platform switch; the continuing
conversation includes source_conversation_id/source_title; the original conversation being
handed off includes continued_to_conversation_id/continued_to_title and
latest_summary_message_id
"title": string
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "sourceId": "Response string"
  },
  "lastMessageCreatedAt": "Response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
```

```
  "description": "Example response description"
},
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"callerOrganizationId": "Response string",
"handoff": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Response string"
}
```

Get a specific conversation

GET

[/api/v1/conversations/tab-counts/](#)

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/tab-counts/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/tab-counts/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/tab-counts/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/tab-counts/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]));

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "contact": { // Lightweight Contact serializer for the Conversation List view

    Contains only fields needed by the List view; omits inboxes, mcp_credentials, and
    api_credentials

    to avoid N+1 queries. email / phone_number / metadata are scalar Contact fields
    needed in the frontend conversation sidebar, with no N+1 cost.

    {
      "id": string (uuid)
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
      as-is. (Optional)
      "email"?: // May have different types (Optional)
      string (email) // Optional
    }
  }
}

```

```

    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
}
"inbox": { // Inbox summary embedded in a conversation payload.

```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```

  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?: // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
        {
          "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
          "line":
          {
            "liffId": string
          }
          "ad":
          {
            "saml": string
          }
        }
      }
    }
  }

```

```

    "google"?: // Optional
    {
        "enabled": boolean
    }
}
}

"sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (Optional)

"sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }

    "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }

    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)

    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
    ]
}
"enableAnalysis"?: boolean // Optional
}
}

"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.

"title": string
"lastMessage"?: { // Optional
    {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
    }
}
"callRecord":
{

```

```

    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
"isGroupChat": boolean
"enableGroupMention"?: boolean // Optional
"queryMetadata"?: { // Query metadata that controls which knowledge can be queried; see each
endpoint description (Optional)
{
    "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none are accessible (no permission); omitted means unrestricted (Optional)
{
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
string (uuid)
]
    "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; this field
takes precedence when both are provided) (Optional)
string (uuid)
]
    "faqIds"?: [ // List of FAQ IDs (excluded or selected according to hasUserSelectedAll)
(Optional)
string (uuid)
]
    "hasUserSelectedAll"?: boolean // true selects all knowledge base content except listed
items; false selects only listed items (Optional)
}
]
    "labelRelations"?: { // Tag filter; an empty conditions array means no tag filtering (not
lack of permission). Provide it with knowledgeBases; alone, it is not applied to knowledge base
retrieval (equivalent to unrestricted) (Optional)
{
    "operator": string (enum: AND, OR) // How tag conditions are combined
    "conditions"?: [ // List of tag conditions; supports nested operator/conditions
combinations (Optional)
object
]
}
}

```

```

    }
  }
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying the
priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

  "llm": string (uuid)
  "deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"ipAddress": string
"ipCountryCode": string
"ipRecordedAt": string (timestamp)
"assignee":
{
  "id": string (uuid)
  "name": string
  "userId": string (uuid)
}
"status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
"tags": [
  {
    "id": string (uuid)

```

```

    "name": string
    "memberIds"?: [ // Optional
        string (uuid)
    ]
}
]
"progressStatus": string
"firstResponseAt"?: string (timestamp) // Optional
"queuedAt"?: string (timestamp) // Optional
"assignedAt"?: string (timestamp) // Optional
"transferReason"?: string // Optional
"satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
"satisfactionComment": string
"satisfactionSubmittedAt": string (timestamp)
"latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```
"clientPlatform": // Client platform that created this conversation (web, desktop)
```

```
* `web` - Web
```

```
* `desktop` - Desktop
```

```
{
}
```

```
"workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
```

```
"slideProjectId": string
```

```
"matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
```

both read this, so resolving it once avoids recomputing the keyword and title-match logic three times per serialized conversation.

```
"matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.
```

Reads the ``matched\_message\_content`` annotation added by

```
``ConversationViewSet._annotate_keyword_match``; no extra query.
```

```
"matchedMessageId": string
```

```
"pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
```

```
"folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
```

```
"hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.
```

Lets the frontend know it should proactively re-request the browser panel (emit ``browser:start``) after a page reload, instead of only ever showing the panel when a live ``browser:frame`` push happens to arrive first.

```
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
```

```
"handoff": object // Handoff information: enabled is the platform switch; the continued
conversation carries source_conversation_id/source_title; the original conversation being
handed off carries continued_to_conversation_id/continued_to_title and
```

```
latest_summary_message_id
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string",
    "metadata": null,
    "sourceId": "response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "response string",
          "url": "response string",
          "realm": "response string"
        },
        "line": {
          "liffId": "response string"
        },
        "ad": {
          "saml": "response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "response string"
      ]
    },
    "enableAnalysis": false
  }
}
```

```
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "response string",
"createdAt": "response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"effectiveThinkingConfig": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"llm": "550e8400-e29b-41d4-a716-446655440000",
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"ipAddress": "response string",
"ipCountryCode": "response string",
"ipRecordedAt": "response string",
"assignee": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "userId": "550e8400-e29b-41d4-a716-446655440000"
```



```
},
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "response string",
"firstResponseAt": "response string",
"queuedAt": "response string",
"assignedAt": "response string",
"transferReason": "response string",
"satisfactionScore": 456,
"satisfactionComment": "response string",
"satisfactionSubmittedAt": "response string",
"latestAnalysisSummary": "response string",
"clientPlatform": {},
"workingDirectory": "response string",
"slideProjectId": "response string",
"matchedField": "response string",
"matchedSnippet": "response string",
"matchedMessageId": "response string",
"pinnedAt": "response string",
"folder": "550e8400-e29b-41d4-a716-446655440000",
"hasActiveBrowserSession": false,
"callerOrganizationId": "response string",
"handoff": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Rename Conversation

PATCH </api/v1/conversations/{id}/rename/>

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer for the Conversation List view. It includes only the fields required by the list view and omits inboxes, mcp_credentials, and api_credentials to avoid N+1 queries. email / phone_number / metadata are scalar Contact fields needed by the conversation sidebar, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. unread_conversations_count stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus,	Yes	line : LINE; telegram : Telegram; teams : Teams; web : Web; messenger : Messenger; instagram : Instagram; email : Email; whatsapp : WhatsApp; slack : Slack; team_plus : Team+; `lin...

Field	Type	Required	Description
	line_works, viber)		
inbox.unreadConversationsCount	integer	No	
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 property: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 property: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 property: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. 1 : 1 ; 5 : 5 ; 15 : 15;

Field	Type	Required	Description
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of searchable knowledge; see each endpoint description
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means no restriction
queryMetadata.labelRelations	object	No	Label filter criteria; an empty conditions array means no label filtering (not a lack of permission). Must be supplied with knowledgeBases—if supplied alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id:

Field	Type	Required	Description
			bool}
thinkingConfig	object	No	None = unset (inherit the chatbot setting), {} = explicitly off, {...} = custom override
status	string (enum: open, resolved, queued)	No	open : Open ; resolved : Resolved ; queued : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Example Request Schema

```
{
  "contact"?: { // Lightweight Contact Serializer for the Conversation List view

    Includes only the fields required by the List view; omits inboxes, mcp_credentials, and
    api_credentials
    to avoid N+1 queries. email / phone_number / metadata are scalar fields of Contact itself,
    required in the frontend conversation sidebar, with no N+1 cost. (Optional)
    {
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
      as-is. (Optional)
      "email"?: // May have different types (Optional)
      string (email) // Optional
      "phoneNumber"?: string // Optional
      "metadata"?: object // Custom customer information shown in the customer panel of the
      conversation page (Optional)
      "sourceId"?: string // Optional
    }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

    ``unread_conversations_count`` stays here because it is part of the
    documented REST conversation API. It is an org-wide denormalized
    counter, so WS emit paths must run the payload through
    :func:`strip_org_wide_unread_count` before broadcasting it to rooms
    that include members without ``GroupInbox.can_view_conversations``. (Optional)
    {
      "name": string
      "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
      whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
      * `telegram` - Telegram
    }
  }
}
```

```

* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
        "google"?: // Optional
        {
          "enabled": boolean
        }
      }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
    ID (verified per the verify mode below) may chat without going through the login flow. Disabled
    keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
    an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
    as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
  {
  }
  "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
  contact identity. Not a session lifetime: once exchanged, the session follows the existing
  WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
  {
  }

```

"keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)

"keycloakTokenPassthroughAllowedHosts"?: [// Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)

```
    string
  ]
}
"enableAnalysis"?: boolean // Optional
}
}
"lastMessage"?: { // Optional
{
  "type"?: string // Optional
  "content"?: string // Optional
}
}
"enableGroupMention"?: boolean // Optional
"queryMetadata"?: { // Query metadata that controls the scope of searchable knowledge; see
each endpoint description (Optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission), while omission means no restriction (Optional)
    {
      "knowledgeBaseId": string (uuid) // Knowledge base ID
      "chatbotFileIds"?: [ // List of file IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
        string (uuid)
      ]
      "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; this field
takes precedence when both are supplied) (Optional)
        string (uuid)
      ]
      "faqIds"?: [ // List of FAQ IDs (excluded or selected according to hasUserSelectedAll)
(Optional)
        string (uuid)
      ]
      "hasUserSelectedAll"?: boolean // true selects all knowledge base content except listed
items; false selects only listed items (Optional)
    }
  ]
  "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not a lack of permission). Must be supplied with knowledgeBases—if supplied alone,
label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
(Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
  }
}
```

```

}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (Optional)
"status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
"firstResponseAt"?: string (timestamp) // Optional
"queuedAt"?: string (timestamp) // Optional
"assignedAt"?: string (timestamp) // Optional
"transferReason"?: string // Optional
}

```

Example Request Value

```

{
  "contact": {
    "name": "Example name",
    "avatar": "example string",
    "email": null,
    "phoneNumber": "example string",
    "metadata": null,
    "sourceId": "example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "example string",
          "url": "example string",
          "realm": "example string"
        },
        "line": {
          "liffId": "example string"
        },
        "ad": {
          "saml": "example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "example string"
      ]
    }
  }
}

```



```

    ]
  },
  "enableAnalysis": true
},
"lastMessage": {
  "type": "example string",
  "content": "Hello! I would like to learn about your products."
},
"enableGroupMention": true,
"queryMetadata": {
  "knowledgeBases": [
    {
      "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "faqIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "hasUserSelectedAll": true
    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"status": "open",
"firstResponseAt": "example string",
"queuedAt": "example string",
"assignedAt": "example string",
"transferReason": "example string"
}

```

Code Examples

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "contact": {
    "name": "Example name",
    "avatar": "example string",
    "email": null,
    "phoneNumber": "example string",
    "metadata": null,
    "sourceId": "example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "example string",
          "url": "example string",
          "realm": "example string"
        },
        "line": {
          "liffId": "example string"
        },
        "ad": {
          "saml": "example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "example string"
      ]
    },
    "enableAnalysis": true
  },
  "lastMessage": {
    "type": "example string",
```

```
    "content": "Hello! I would like to learn about your products."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "example string",
  "queuedAt": "example string",
  "assignedAt": "example string",
  "transferReason": "example string"
}'
```

Replace YOUR_API_KEY and verify the request data before running.

```

const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "contact": {
    "name": "Sample name",
    "avatar": "Sample string",
    "email": null,
    "phoneNumber": "Sample string",
    "metadata": null,
    "sourceId": "Sample string"
  },
  "inbox": {
    "name": "Sample name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Sample string",
          "url": "Sample string",
          "realm": "Sample string"
        },
        "line": {
          "liffId": "Sample string"
        },
        "ad": {
          "saml": "Sample string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Sample string"
      ]
    },
    "enableAnalysis": true
  }
};

```

```

    },
    "lastMessage": {
      "type": "Sample string",
      "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Sample string",
    "queuedAt": "Sample string",
    "assignedAt": "Sample string",
    "transferReason": "Sample string"
  };

```

```

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contact": {
        "name": "Sample name",
        "avatar": "Sample string",
        "email": null,
        "phoneNumber": "Sample string",
        "metadata": null,
        "sourceId": "Sample string"
    },
    "inbox": {
        "name": "Sample name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Sample string",
                    "url": "Sample string",
                    "realm": "Sample string"
                },
                "line": {
                    "liffId": "Sample string"
                },
                "ad": {
                    "saml": "Sample string"
                },
                "google": {
                    "enabled": true
                }
            },
            "sourceIdAccessEnabled": true,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": true,
            "keycloakTokenPassthroughAllowedHosts": [
                "Sample string"
            ]
        },
        "enableAnalysis": true
    },
}

```

```

    "lastMessage": {
        "type": "Sample string",
        "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Sample string",
    "queuedAt": "Sample string",
    "assignedAt": "Sample string",
    "transferReason": "Sample string"
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/rename/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "contact": {
                "name": "Sample name",
                "avatar": "Sample string",
                "email": null,
                "phoneNumber": "Sample string",
                "metadata": null,
                "sourceId": "Sample string"
            },
            "inbox": {
                "name": "Sample name",
                "channelType": "line",
                "unreadConversationsCount": 123,
                "signAuth": {
                    "signSource": "550e8400-e29b-41d4-a716-446655440000",
                    "signParams": {
                        "keycloak": {
                            "clientId": "Sample string",
                            "url": "Sample string",
                            "realm": "Sample string"
                        },
                        "line": {
                            "liffId": "Sample string"
                        },
                        "ad": {
                            "saml": "Sample string"
                        },
                        "google": {
                            "enabled": true
                        }
                    },
                    "sourceIdAccessEnabled": true,
                    "sourceIdVerifyMode": {},
                    "sourceIdSignatureTtlMinutes": {},
                    "keycloakTokenPassthrough": true,
                    "keycloakTokenPassthroughAllowedHosts": [
                        "Sample string"
                    ]
                }
            },
            "sourceIdAccessEnabled": true,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": true,
            "keycloakTokenPassthroughAllowedHosts": [
                "Sample string"
            ]
        }
    ]
}

```



```

        "enableAnalysis": true
    },
    "lastMessage": {
        "type": "Sample string",
        "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Sample string",
    "queuedAt": "Sample string",
    "assignedAt": "Sample string",
    "transferReason": "Sample string"
    }
    });

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only fields needed by the List view; excludes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact,
required by the conversation sidebar, with no N+1 cost.
    {
      "id": string (uuid)
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (optional)
      "email"?: // May have different types (optional)
string (email) // optional
      "phoneNumber"?: string // optional
      "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (optional)
      "sourceId"?: string // optional
      "tags": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
    }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
    {
      "id": string (uuid)
      "name": string
      "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```

* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // optional
  "signAuth"?: // optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
        "google"?: // optional
          {
            "enabled": boolean
          }
      }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (optional)
    "sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
  {
  }
  "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (optional)
  {
  }
  "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (optional)

```

```

    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (optional)
        string
    ]
}
"enableAnalysis"?: boolean // optional
}
}
"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaigPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
"title": string
"lastMessage"?: { // optional
{
    "id": string (uuid)
    "type"?: string // optional
    "content"?: string // optional
    "createdAt": string (timestamp)
}
}
"callRecord":
{
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
"isGroupChat": boolean
"enableGroupMention"?: boolean // optional
"queryMetadata"?: { // Query metadata controlling the searchable knowledge scope; see each
endpoint for details (optional)
{
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission); omitted means unrestricted (optional)
        {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // File ID list (excluded or selected depending on
hasUserSelectedAll) (optional)

```

```

        string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // File ID list (new name for chatbotFileIds; takes
precedence if both are provided) (optional)
        string (uuid)
    ]
    "faqIds"?: [ // FAQ ID list (excluded or selected depending on hasUserSelectedAll)
(optional)
        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true selects all content in the knowledge base except
listed items; false selects only listed items (optional)
    }
]
"labelRelations"?: { // Tag filter; an empty conditions array means no tag filtering (not a
lack of permission). Provide it together with knowledgeBases; by itself, the filter is not
applied to knowledge base searches (equivalent to unrestricted) (optional)
    {
        "operator": string (enum: AND, OR) // How tag conditions are combined
        "conditions"?: [ // List of tag conditions; supports nested operator/conditions groups
(optional)
            object
        ]
    }
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying the
priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

    "llm": string (uuid)
    "deepResearchStatus": // Status of deep research for this conversation

```

```

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed

```

```

{
}

```

```

"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only

```

when the user clicks a format in the pre-research choice card; reset to NULL each time Deep Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force this to NULL/canvas – no redeploy needed.

```
* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"? : string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"? : [ // optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"? : string (timestamp) // optional
  "queuedAt"? : string (timestamp) // optional
  "assignedAt"? : string (timestamp) // optional
  "transferReason"? : string // optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).
```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```
  "clientPlatform": // Client platform that created this conversation (web, desktop)
```

```
* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
```

```

`get_matched_snippet`
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the `matched_message_content` annotation added by
`ConversationViewSet._annotate_keyword_match`; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
  Null means unclassified (shown under the time groups). Deleting the folder resets this to null
  without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
  resume.

Lets the frontend know it should proactively re-request the browser
panel (emit `browser:start`) after a page reload, instead of only
ever showing the panel when a live `browser:frame` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
  null when unbound
  "handoff": object // Handoff information: enabled is the platform switch; the continued
  conversation carries source_conversation_id/source_title; the original conversation carries
  continued_to_conversation_id/continued_to_title and latest_summary_message_id
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {

```

```
    "clientId": "Response string",
    "url": "Response string",
    "realm": "Response string"
  },
  "line": {
    "liffId": "Response string"
  },
  "ad": {
    "saml": "Response string"
  },
  "google": {
    "enabled": false
  }
},
"sourceIdAccessEnabled": false,
"sourceIdVerifyMode": {},
"sourceIdSignatureTtlMinutes": {},
"keycloakTokenPassthrough": false,
"keycloakTokenPassthroughAllowedHosts": [
  "Response string"
]
},
"enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},

```



```
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"effectiveThinkingConfig": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"llm": "550e8400-e29b-41d4-a716-446655440000",
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"ipAddress": "Response string",
"ipCountryCode": "Response string",
"ipRecordedAt": "Response string",
"assignee": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "userId": "550e8400-e29b-41d4-a716-446655440000"
},
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "Response string",
"firstResponseAt": "Response string",
"queuedAt": "Response string",
"assignedAt": "Response string",
"transferReason": "Response string",
"satisfactionScore": 456,
"satisfactionComment": "Response string",
"satisfactionSubmittedAt": "Response string",
"latestAnalysisSummary": "Response string",
"clientPlatform": {},
"workingDirectory": "Response string",
"slideProjectId": "Response string",
"matchedField": "Response string",
"matchedSnippet": "Response string",
"matchedMessageId": "Response string",
"pinnedAt": "Response string",
"folder": "550e8400-e29b-41d4-a716-446655440000",
"hasActiveBrowserSession": false,
"callerOrganizationId": "Response string",
"handoff": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
```

```
"description": "Example response description"
}
```

Share a conversation

POST

/api/v1/conversations/{conversationPk}/share/

Parameters

Parameter name	Required	Type	Description
conversationPk	V	string	

Request body

Request parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	organization : Organization ; group : Group ; member : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Example request schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
  * `group` - Group
  * `member` - Member
  "permission"? : // optional
  {
  }
  "title"? : string // optional
  "description"? : string // optional
}
```

```
"groupIds"?: [ // optional
  string (uuid)
]
"memberIds"?: [ // optional
  string (uuid)
]
}
```

Example request value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running the example °
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example name",
    "description": "Example string",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "shareScope": "organization",
            "permission": {},
            "title": "Example name",
            "description": "Example string",
            "groupIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "memberIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 201

Example response schema

```

{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization

```

```
* `group` - Group
* `member` - Member
"permission"?: // optional
{
}
"title"?: string // optional
"description"?: string // optional
"groupIds"?: [ // optional
  string (uuid)
]
"memberIds"?: [ // optional
  string (uuid)
]
}
```

Example response value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response string",
  "description": "Response string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Quickly share a conversation

POST `/api/v1/conversations/{conversationPk}/share/quick/`

Parameters

Parameter name	Required	Type	Description
<code>conversationPk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	<code>organization</code> : Organization ; <code>group</code> : Group ; <code>member</code> : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Example request schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
  * `group` - Group
  * `member` - Member
  "permission"? : // optional
  {
  }
  "title"? : string // optional
  "description"? : string // optional
  "groupIds"? : [ // optional
    string (uuid)
  ]
  "memberIds"? : [ // optional
    string (uuid)
  ]
}
```

Example request value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running the example °
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example name",
    "description": "Example string",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "shareScope": "organization",
            "permission": {},
            "title": "Example name",
            "description": "Example string",
            "groupIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "memberIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Example response schema

```

{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization

```

```
* `group` - Group
* `member` - Member
"permission"?: // optional
{
}
"title"?: string // optional
"description"?: string // optional
"groupIds"?: [ // optional
  string (uuid)
]
"memberIds"?: [ // optional
  string (uuid)
]
}
```

Example response value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response string",
  "description": "Response string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

List shared conversations

GET

</api/v1/shared-conversations/>

Parameters

Parameter name	Required	Type	Description
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running the example °
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```


Response body

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "title": string // Custom title for the shared conversation
      "description": string // Custom description for the shared conversation
      "shareScope":
        {
        }
      "permission":
        {
        }
      "sharedBy":
        {
          "id": string (uuid)
          "name": string
        }
      "messageCount": integer
      "createdAt": string (timestamp)
    }
  ]
}
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "title": "Response string",
      "description": "Response string",
      "shareScope": {},
      "permission": {},
      "sharedBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "messageCount": 456,
      "createdAt": "Response string"
    }
  ]
}
```

```
]
}
```

Retrieve a shared conversation

GET

/api/v1/shared-conversations/{id}

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this Shared Conversation.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running the example °
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
```

```
"conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Response string",
  "description": "Response string",
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdAt": "Response string",
  "conversationSnapshot": null
}
```

Update a shared conversation

PATCH `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Shared Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
title	string	No	
description	string	No	
permission	string (enum: readonly, copy)	No	<code>readonly</code> : Read Only ; <code>copy</code> : Copyable;

Request Schema Example

```
{
  "title"?: string // Optional
  "description"?: string // Optional
  "permission"?: string (enum: readonly, copy) // * `readonly` - Read Only
  * `copy` - Copyable (Optional)
}
```

Request Example Value

```
{
  "title": "Example name",
  "description": "Example string",
  "permission": "readonly"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "title": "Example name",
  "description": "Example string",
  "permission": "readonly"
}'

# Replace YOUR_API_KEY before running, and verify the request data°
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "title": "Example name",
  "description": "Example string",
  "permission": "readonly"
};

axios.patch("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "title": "Example name",
    "description": "Example string",
    "permission": "readonly"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "title": "Example name",
            "description": "Example string",
            "permission": "readonly"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "title"?: string // Optional
  "description"?: string // Optional
  "permission"?: string (enum: readonly, copy) // * `readonly` - Read Only
  * `copy` - Copyable (Optional)
}

```

Response Example Value

```
{
  "title": "Response string",
  "description": "Response string",
  "permission": "readonly"
}
```

Delete Shared Conversation

DELETE `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY before running, and verify the request data °
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Fork Shared Conversation

POST </api/v1/shared-conversations/{id}/fork/>

Parameters

Parameter Name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Shared Conversation.
----	---	--------	---

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{}'
```

Replace YOUR_API_KEY before running, and verify the request data °

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/fork/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {}
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
```



```
"conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}  
}
```

Response Example Value

```
{  
  "id": "550e8400-e29b-41d4-a716-446655440000",  
  "title": "Response string",  
  "description": "Response string",  
  "permission": {},  
  "sharedBy": {  
    "id": "550e8400-e29b-41d4-a716-446655440000",  
    "name": "Response string"  
  },  
  "createdAt": "Response string",  
  "conversationSnapshot": null  
}
```

List My Shared Conversations

GET

/api/v1/shared-conversations/my-shares/

Code Examples

Shell/Bash

```
# Call API Example (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Replace YOUR_API_KEY before running, and verify the request data°
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/my-shares/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/my-shares/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}

?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "shareScope":
  {
  }
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
}
```

```
"messageCount": integer
"createdAt": string (timestamp)
}
```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Response string",
  "description": "Response string",
  "shareScope": {},
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "messageCount": 456,
  "createdAt": "Response string"
}
```

Create Message Feedback

POST

/api/v1/messages/{messagePk}/feedback/

Parameters

Parameter Name	Required	Type	Description
messagePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Request Schema Example

```
{
  "type":
  {
  }
  "suggestion"?: string // Optional
}
```

Request Example Value

```
{
  "type": {},
  "suggestion": "Example string"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "suggestion": "Example string"
}'

# Replace YOUR_API_KEY before running, and verify the request data °
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "type": {},
  "suggestion": "Example string"
};

axios.post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "type": {},
    "suggestion": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", [
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY',
        'Content-Type' => 'application/json'
    ],
    'json' => {
        "type": {},
        "suggestion": "Example string"
    }
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
```


Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "Response string",
  "updatedAt": "Response string"
}
```

Status Code: **400** - This message already has feedback , use the PATCH method to update it

Update Message Feedback

PUT

`/api/v1/messages/{messagePk}/feedback/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Message Feedback.
<code>messagePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Request Schema Example

```
{
  "type":
  {
  }
```

```
"suggestion"?: string // Optional
}
```

Request Example Value

```
{
  "type": {},
  "suggestion": "Example string"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "suggestion": "Example string"
}'

# Replace YOUR_API_KEY before running, and verify the request data °
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "type": {},
  "suggestion": "Example string"
};

axios.put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "type": {},
    "suggestion": "Example string"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "type": {},
            "suggestion": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion?": string // Optional
  "updatedAt": string (timestamp)
}

```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "Response string",
  "updatedAt": "Response string"
}
```

Delete Message Feedback

DELETE`/api/v1/messages/{messagePk}/feedback/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Message Feedback.
<code>messagePk</code>	V	string	

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X DELETE "https://api.maiaagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY before running, and verify the request data °
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
>delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	Feedback deleted successfully
400	This feedback does not belong to the specified message

List conversation records

GET

/api/v1/records/

Parameters

Parameter	Required	Type	Description
callerMember	X	array	Multiple values may be separated by commas.
chatbot	X	string	Filter by a specific AI assistant (UUID)
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
endDateetime	X	string	
errorType	X	array	Multiple values may be separated by commas. `system`: System; `llm`: LLM; `embedding`: Embedding; `reranker`: Reranker; `vector_db`: Vector DB; `workflow`: Workflow; `tool`: Tool; `timeout`: Ti...
feedbackType	X	array	Multiple values may be separated by commas. `like`: Like; `dislike`: Dislike;
hasError	X	boolean	
hasFeedback	X	boolean	
keyword	X	string	Keyword search (searches both user messages and assistant responses)
largeLanguageModel	X	string	Filter by a specific large language model (UUID)
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
source	X	string	Filter record sources, separated by commas; omit to return both sources. interactive = user conversations; scheduled = scheduled runs.
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)
startDateetime	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "senderName": string // Returns the sender name of the user message
      "callerMemberId": string
      "callerMemberName": string
      "feedback":
      {

```

```

    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "inputMessage": string
  "condenseMessage": string // User message rewritten from the chat history and context.
  "outputMessage": string
  "faithfulnessScore": number (double) // Whether the chatbot reply is grounded in the
knowledge base rather than made up.
  "displayFaithfulnessScore": integer
  "answerRelevancyScore": number (double) // Whether the chatbot reply is relevant to the
user's question.
  "displayAnswerRelevancyScore": integer
  "contextPrecisionScore": number (double) // Whether the chatbot reply is relevant to the
reference context.
  "displayContextPrecisionScore": integer
  "answerCorrectnessScore": number (double) // Whether the chatbot reply is correct
(requires a ground-truth answer).
  "displayAnswerCorrectnessScore": integer
  "answerSimilarityScore": number (double) // Whether the chatbot reply is similar to the
ground-truth answer.
  "displayAnswerSimilarityScore": integer
  "contextRecallScore": number (double) // Whether the chatbot reply recalls the relevant
information from the reference context.
  "displayContextRecallScore": integer
  "replyTime": string
  "createdAt": string (timestamp)
  "error": string // Error message raised during execution.
  "errorType": // May be one of several types
    string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other

  "processingTime": object // Returns timing statistics for each processing stage,
including the streaming_metrics waterfall
  "usage": object // Returns usage statistics, including character count, token count, and
token details for each LLM call
  "citationNodes": [
    {
      "id": string
      "text": string

```

```
    "chatbotFileId": string
    "fileName": string
    "url": string
  }
]
"inputHookLogs": [
  {
    "id": string (uuid)
    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"outputHookLogs": [
  {
    "id": string (uuid)
    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"instructionId": string
"llm":
{
  "id": string (uuid)
  "name": string
}
"effectiveMaxLlmOutputTokens": integer // max_tokens actually used for this reply, from
Chatbot.custom_max_llm_output_tokens or LLM.max_tokens.
"chatbot":
{
  "id": string (uuid)
  "name": string
}
"context": string
"conversationId": string (uuid)
```

```

    "inboxId": string (uuid)
    "botMessageId": string (uuid)
    "userMessageId": string (uuid)
    "creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
    object
  ]
  "cacheSavedCredits": number (double) // Net credits saved by prompt caching for this
reply; ``None`` outside credit mode.

Reuses the usage-statistics saving semantics (``compute_cache_saved_credits``)
on the reply's own billing batch, so the per-reply figure and the hourly
aggregate are defined by the same formula. The net value may be negative
(cache-creation premium exceeding read savings); the frontend hides
non-positive values by design.

  "trace": object // Return the round trace timeline for the detail view; ``None`` for
legacy records.
  "llmRawRequest": object // Raw messages array sent to the LLM on the final round,
including system prompt and memory
  "source": string // Origin discriminator for the unified /records/ response.
  "schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
}
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "senderName": "Example response string",
      "callerMemberId": "Example response string",
      "callerMemberName": "Example response string",
      "feedback": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "suggestion": "Example response string",
        "updatedAt": "Example response string"
      },
      "inputMessage": "Example response string",
      "condenseMessage": "Example response string",
      "outputMessage": "Example response string",
      "faithfulnessScore": 456,
      "displayFaithfulnessScore": 456,
      "answerRelevancyScore": 456,
      "displayAnswerRelevancyScore": 456,
      "contextPrecisionScore": 456,
      "displayContextPrecisionScore": 456,
      "answerCorrectnessScore": 456,
      "displayAnswerCorrectnessScore": 456,
    }
  ]
}

```

```
"answerSimilarityScore": 456,
"displayAnswerSimilarityScore": 456,
"contextRecallScore": 456,
"displayContextRecallScore": 456,
"replyTime": "Example response string",
"createdAt": "Example response string",
"error": "Example response string",
"errorType": "system",
"processingTime": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"usage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"citationNodes": [
  {
    "id": "Example response string",
    "text": "Example response string",
    "chatbotFileId": "Example response string",
    "fileName": "Example response string",
    "url": "Example response string"
  }
],
"inputHookLogs": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "hook": "550e8400-e29b-41d4-a716-446655440000",
    "hookName": "Example response string",
    "hookTypeLabel": "Example response string",
    "hookAction": "Example response string",
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "message": "550e8400-e29b-41d4-a716-446655440000",
    "messageContent": "Example response string",
    "chatbotId": "Example response string",
    "chatbotName": "Example response string",
    "metadata": null,
    "triggerPoint": "Example response string",
    "createdAt": "Example response string",
    "updatedAt": "Example response string"
  }
],
"outputHookLogs": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "hook": "550e8400-e29b-41d4-a716-446655440000",
    "hookName": "Example response string",
    "hookTypeLabel": "Example response string",
    "hookAction": "Example response string",
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "message": "550e8400-e29b-41d4-a716-446655440000",
    "messageContent": "Example response string",
    "chatbotId": "Example response string",
```



```

        "chatbotName": "Example response string",
        "metadata": null,
        "triggerPoint": "Example response string",
        "createdAt": "Example response string",
        "updatedAt": "Example response string"
    }
],
"instructionId": "Example response string",
"llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
},
"effectiveMaxLlmOutputTokens": 456,
"chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
},
"context": "Example response string",
"conversationId": "550e8400-e29b-41d4-a716-446655440000",
"inboxId": "550e8400-e29b-41d4-a716-446655440000",
"botMessageId": "550e8400-e29b-41d4-a716-446655440000",
"userMessageId": "550e8400-e29b-41d4-a716-446655440000",
"creditCosts": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
    }
],
"cacheSavedCredits": 456,
"trace": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
},
"llmRawRequest": null,
"source": "Example response string",
"schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
}
}
]
}

```

Get a conversation record

GET

/api/v1/records/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Chatbot Record.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "senderName": string // Returns the sender name of the user message
  "callerMemberId": string
  "callerMemberName": string
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion?": string // Optional
    "updatedAt": string (timestamp)
  }
  "inputMessage": string
}

```

```

"condenseMessage": string // User message rewritten from the chat history and context.
"outputMessage": string
"faithfulnessScore": number (double) // Whether the chatbot reply is grounded in the
knowledge base rather than made up.
"displayFaithfulnessScore": integer
"answerRelevancyScore": number (double) // Whether the chatbot reply is relevant to the
user's question.
"displayAnswerRelevancyScore": integer
"contextPrecisionScore": number (double) // Whether the chatbot reply is relevant to the
reference context.
"displayContextPrecisionScore": integer
"answerCorrectnessScore": number (double) // Whether the chatbot reply is correct (requires a
ground-truth answer).
"displayAnswerCorrectnessScore": integer
"answerSimilarityScore": number (double) // Whether the chatbot reply is similar to the
ground-truth answer.
"displayAnswerSimilarityScore": integer
"contextRecallScore": number (double) // Whether the chatbot reply recalls the relevant
information from the reference context.
"displayContextRecallScore": integer
"replyTime": string
"createdAt": string (timestamp)
"error": string // Error message raised during execution.
"errorType": // May be one of several types
string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other

"processingTime": object // Returns timing statistics for each processing stage, including
the streaming_metrics waterfall
"usage": object // Returns usage statistics, including character count, token count, and
token details for each LLM call
"citationNodes": [
  {
    "id": string
    "text": string
    "chatbotFileId": string
    "fileName": string
    "url": string
  }
]
"inputHookLogs": [
  {
    "id": string (uuid)

```

```

    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"outputHookLogs": [
  {
    "id": string (uuid)
    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"instructionId": string
"llm":
{
  "id": string (uuid)
  "name": string
}
"effectiveMaxLlmOutputTokens": integer // max_tokens actually used for this reply, from
Chatbot.custom_max_llm_output_tokens or LLM.max_tokens.
"chatbot":
{
  "id": string (uuid)
  "name": string
}
"context": string
"conversationId": string (uuid)
"inboxId": string (uuid)
"botMessageId": string (uuid)
"userMessageId": string (uuid)
"creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
  object
]
"cacheSavedCredits": number (double) // Net credits saved by prompt caching for this reply;

```

```
``None`` outside credit mode.
```

Reuses the usage-statistics saving semantics (``compute\_cache\_saved\_credits``) on the reply's own billing batch, so the per-reply figure and the hourly aggregate are defined by the same formula. The net value may be negative (cache-creation premium exceeding read savings); the frontend hides non-positive values by design.

```
"trace": object // Return the round trace timeline for the detail view; ``None`` for legacy records.
```

```
"llmRawRequest": object // Raw messages array sent to the LLM on the final round, including system prompt and memory
```

```
"source": string // Origin discriminator for the unified /records/ response.
```

```
"schedule": object // Shape parity with scheduled rows, which link their owning ChatbotSchedule here.
```

```
}
```

Example response

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "senderName": "Response string",
  "callerMemberId": "Response string",
  "callerMemberName": "Response string",
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "Response string",
    "updatedAt": "Response string"
  },
  "inputMessage": "Response string",
  "condenseMessage": "Response string",
  "outputMessage": "Response string",
  "faithfulnessScore": 456,
  "displayFaithfulnessScore": 456,
  "answerRelevancyScore": 456,
  "displayAnswerRelevancyScore": 456,
  "contextPrecisionScore": 456,
  "displayContextPrecisionScore": 456,
  "answerCorrectnessScore": 456,
  "displayAnswerCorrectnessScore": 456,
  "answerSimilarityScore": 456,
  "displayAnswerSimilarityScore": 456,
  "contextRecallScore": 456,
  "displayContextRecallScore": 456,
  "replyTime": "Response string",
  "createdAt": "Response string",
  "error": "Response string",
  "errorType": "system",
  "processingTime": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "usage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Example response name",
    "description": "Example response description"
  },
  "citationNodes": [
    {
      "id": "Response string",
      "text": "Response string",
      "chatbotFileId": "Response string",
      "fileName": "Response string",
      "url": "Response string"
    }
  ],
  "inputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "Response string",
      "hookTypeLabel": "Response string",
      "hookAction": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "Response string",
      "chatbotId": "Response string",
      "chatbotName": "Response string",
      "metadata": null,
      "triggerPoint": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "outputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "Response string",
      "hookTypeLabel": "Response string",
      "hookAction": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "Response string",
      "chatbotId": "Response string",
      "chatbotName": "Response string",
      "metadata": null,
      "triggerPoint": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "instructionId": "Response string",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "effectiveMaxLlmOutputTokens": 456,
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```



```
    "name": "Response string"
  },
  "context": "Response string",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "botMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "userMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "creditCosts": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "cacheSavedCredits": 456,
  "trace": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llmRawRequest": null,
  "source": "Response string",
  "schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```

Status code: **404** - The specified conversation record was not found

Get a specific conversation record

GET

</api/v1/records/export-excel-requests/>

Parameters

Parameter name	Required	Type	Description
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
keyword	X	string	Filter conversation records containing a specific keyword (searches both user messages and bot replies)
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/export-excel-
requests/?endDate=example&keyword=example&startDate=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	Excel file download

Create a conversation record export request

POST

/api/v1/records/export-excel-requests/

Parameters

Parameter name	Required	Type	Description
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
keyword	X	string	Filter conversation records containing a specific keyword (searches both user messages and bot replies)
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)

Request body

Request parameters

Field	Type	Required	Description
chatbot	string	No	Specify the Chatbot UUIDs to export, separated by commas. Leave blank to export all Chatbots in the organization that you can access.
chatbotId	string (uuid)	No	Specify the Chatbot UUID to export (an alias for chatbot; supports only one UUID).
largeLanguageModel	string	No	Specify the UUIDs of the large language models to export, separated by commas.
startDatetime	string (timestamp)	No	Precise start time (ISO 8601, inclusive), with the same semantics as the list drill-down filter.
endDatetime	string (timestamp)	No	Precise end time (ISO 8601, exclusive), with the same semantics as the list drill-down filter.

Field	Type	Required	Description
hasError	boolean	No	Filter records by error presence (true: only records with errors; false: only records without errors).
errorType	string	No	Filter by error type; supports comma-separated values (OR logic). Allowed values match the conversation record list.
hasFeedback	boolean	No	Filter by user feedback (true: only liked/disliked records; false: only records without feedback).
feedbackType	string	No	Filter by user feedback type (like / dislike); supports comma-separated values (OR logic).
callerMember	string	No	Filter by speaking member UUID; supports comma-separated values (OR logic).

Example request schema

```
{
  "chatbot?": string // Specify the Chatbot UUIDs to export, separated by commas. Leave blank
to export all Chatbots in the organization that you can access. (Optional)
  "chatbotId?": string (uuid) // Specify the Chatbot UUID to export (an alias for chatbot;
supports only one UUID). (Optional)
  "largeLanguageModel?": string // Specify the UUIDs of the large language models to export,
separated by commas. (Optional)
  "startDatetime?": string (timestamp) // Precise start time (ISO 8601, inclusive), with the
same semantics as the list drill-down filter. (Optional)
  "endDatetime?": string (timestamp) // Precise end time (ISO 8601, exclusive), with the same
semantics as the list drill-down filter. (Optional)
  "hasError?": boolean // Filter records by error presence (true: only records with errors;
false: only records without errors). (Optional)
  "errorType?": string // Filter by error type; supports comma-separated values (OR logic).
Allowed values match the conversation record list. (Optional)
  "hasFeedback?": boolean // Filter by user feedback (true: only liked/disliked records; false:
only records without feedback). (Optional)
  "feedbackType?": string // Filter by user feedback type (like / dislike); supports comma-
separated values (OR logic). (Optional)
  "callerMember?": string // Filter by speaking member UUID; supports comma-separated values
(OR logic). (Optional)
}
```

Example request

```
{
  "chatbot": "Example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": "Example string",
  "startDatetime": "Example string",
  "endDatetime": "Example string",
  "hasError": true,
  "errorType": "Example string",
  "hasFeedback": true,
  "feedbackType": "Example string",
}
```

```
"callerMember": "Example string"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "chatbot": "Example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "keyword": "Example string",
  "largeLanguageModel": "Example string",
  "startDate": "Example string",
  "endDate": "Example string",
  "startDatetime": "Example string",
  "endDatetime": "Example string",
  "hasError": true,
  "errorType": "Example string",
  "hasFeedback": true,
  "feedbackType": "Example string",
  "callerMember": "Example string"
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "chatbot": "Example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "keyword": "Example string",
  "largeLanguageModel": "Example string",
  "startDate": "Example string",
  "endDate": "Example string",
  "startDatetime": "Example string",
  "endDatetime": "Example string",
  "hasError": true,
  "errorType": "Example string",
  "hasFeedback": true,
  "feedbackType": "Example string",
  "callerMember": "Example string"
};

axios.post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/?\nendDate=example&keyword=example&startDate=example"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY",\n    "Content-Type": "application/json"\n}\n\n# Request body (payload)\ndata = {\n    "chatbot": "Example string",\n    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",\n    "keyword": "Example string",\n    "largeLanguageModel": "Example string",\n    "startDate": "Example string",\n    "endDate": "Example string",\n    "startDatetime": "Example string",\n    "endDatetime": "Example string",\n    "hasError": true,\n    "errorType": "Example string",\n    "hasFeedback": true,\n    "feedbackType": "Example string",\n    "callerMember": "Example string"\n}\n\nresponse = requests.post(url, json=data, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?endDate=example&keyword=example&startDate=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "chatbot": "Example string",
            "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
            "keyword": "Example string",
            "largeLanguageModel": "Example string",
            "startDate": "Example string",
            "endDate": "Example string",
            "startDatetime": "Example string",
            "endDatetime": "Example string",
            "hasError": true,
            "errorType": "Example string",
            "hasFeedback": true,
            "feedbackType": "Example string",
            "callerMember": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	Excel file download

Get the conversation record export status

GET

/api/v1/records/export-excel-requests/{exportId}

Parameters

Parameter name	Required	Type	Description
exportId	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}"

headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "senderName": string // Return the name of the user message sender
  "callerMemberId": string
  "callerMemberName": string
  "inputMessage": string
  "outputMessage": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "condenseMessage": string // User message rewritten from the chat history and context.
  "feedback":

```

```

{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
"displayFaithfulnessScore": integer
"displayAnswerRelevancyScore": integer
"displayContextPrecisionScore": integer
"displayAnswerCorrectnessScore": integer
"displayAnswerSimilarityScore": integer
"displayContextRecallScore": integer
"replyTime": string
"processingTime": object // Return processing-time statistics for the list view, calculated
the same way as the detail view but without the streaming_metrics waterfall
"usage": object // Return usage statistics, including character count, token count, and token
details for each LLM call
"llm":
{
  "id": string (uuid)
  "name": string
}
"createdAt": string (timestamp)
"conversationId": string (uuid)
"errorType": // May have different types
string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other
"creditCosts": [
  object
]
"source": string // Origin discriminator for the unified /records/ response.
"schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
}

```

Example response

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
"senderName": "Response string",
"callerMemberId": "Response string",
"callerMemberName": "Response string",
"inputMessage": "Response string",
"outputMessage": "Response string",
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
"condenseMessage": "Response string",
"feedback": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "Response string",
  "updatedAt": "Response string"
},
"displayFaithfulnessScore": 456,
"displayAnswerRelevancyScore": 456,
"displayContextPrecisionScore": 456,
"displayAnswerCorrectnessScore": 456,
"displayAnswerSimilarityScore": 456,
"displayContextRecallScore": 456,
"replyTime": "Response string",
"processingTime": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"usage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"llm": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
"createdAt": "Response string",
"conversationId": "550e8400-e29b-41d4-a716-446655440000",
"errorType": "system",
"creditCosts": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
],
"source": "Response string",
"schedule": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```


Example API call (Shell)

...

目錄

1. Create tool
2. List tools
3. Get a specific tool
4. Get a specific tool
5. Retrieve a specific tool
6. Update a tool
7. Update tool
8. Partially update a tool
9. Delete tool
10. List available tools
11. Create connector
12. List connectors
13. Get a specific connector
14. Get a specific connector
15. Update a connector
16. Partially update a connector
17. Delete connector
18. List authorized connector members
19. Remove authorized connector member
20. List MCP Tool Registrations
21. Get a Specific MCP Tool Registration
22. List Tool Execution Records
23. Get a Specific Tool Execution Record
24. Get AI Assistant Filter Options for Execution Records
25. Get tool filter options for execution records
26. List execution records for a specific tool
27. Get a specific execution record for a specific tool
28. Get AI assistant filter options for a specific tool execution record
29. Get tool filter options for a specific tool execution record

Example API call (Shell)

Create tool

POST

/api/v1/tools/

Request body

Request parameters

Field	Type	Required	Description
name	object	No	Tool names may contain only letters, numbers, underscores (_) and hyphens (-). MCP tools do not need this field; API tools use it when calling the LLM.
displayName	string	Yes	A name that may contain any characters, used by users to configure and manage the tool.
description	string	No	The tool description shown to users. MCP tools do not need this field.
prompt	string	No	The tool description/prompt for the LLM. If empty, the system uses the description field. MCP tools do not need this field.
toolType	object	No	
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-

Field	Type	Required	Description
			generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Use this default header when the Contact has no corresponding MCP Credential.
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...

Field	Type	Required	Description
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example request schema

```

{
  "name"?: // May have different types (optional)
    string // Tool names may contain only letters, numbers, underscores (_) and
    hyphens (-). MCP tools do not need this field; API tools use it when calling the
    LLM. (optional)
    "displayName": string // A name that may contain any characters, used by users
    to configure and manage the tool.
    "description"?: string // The tool description shown to users. MCP tools do not
    need this field. (optional)
    "prompt"?: string // The tool description/prompt for the LLM. If empty, the
    system uses the description field. MCP tools do not need this field. (optional)
    "toolType"?: // optional
    {
    }
    "apiUrl"?: string // optional
    "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
    "rawHeaders"?: object // optional
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (optional)
    "rawParametersSchema"?: object // optional
    "functionCode"?: string // optional
    "mcpUrl"?: string // optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (optional)
    "mcpAllowedTools"?: object // optional
    "rawMcpHeader"?: object // Use this default header when the Contact has no
    corresponding MCP Credential. (optional)
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
  FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
  carousel / box / text / image / button etc.). (optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
  whose keys are field paths parsed from flex_template_json (e.g. text / image url
  / button label / button uri / button message). Each value may be either a plain
  string (the AI prompt) or an object carrying additional metadata such as a field
  tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the

```

```

template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
  Structured JSON describing the chip buttons: static list or dynamic (LLM-
  generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
  JSON describing the persistent input-bar menu rendered by the web chat only:
  layout (grid / list), default open state, and 2-6 items each carrying a display
  label plus an action payload (send prompt / prefill text / open http(s) link).
  (optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
  code_execution, file_processing, image_video_generation, database_query,
  third_party_integration, browser_operation, agent_collaboration, other) // *
  `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Example request value

```

{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,

```

```
"functionCode": "Example string",
"mcpUrl": "Example string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code examples

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/tools/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/tools/", data, config)
```

```
.then(response => {  
  console.log('Response received:');  
  console.log(response.data);  
})  
.catch(error => {  
  console.error('Request error:');  
  console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": {},
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
```

```
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/tools/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "Example string",
            "description": "Example string",
            "prompt": "Example string",
            "toolType": {},
            "apiUrl": "Example string",
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "Example string",
            "mcpUrl": "Example string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "quickMenuConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
}
```

```

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response body

Status code: 201

Example response schema

```

{
  "id": string (uuid)
  "name"?: // May have different types (optional)
    string // Tool names may contain only letters, numbers, underscores (_) and
    hyphens (-). MCP tools do not need this field; API tools use it when calling the
    LLM. (optional)
  "displayName": string // A name that may contain any characters, used by users
  to configure and manage the tool.
  "description"?: string // The tool description shown to users. MCP tools do not
  need this field. (optional)
  "prompt"?: string // The tool description/prompt for the LLM. If empty, the
  system uses the description field. MCP tools do not need this field. (optional)
  "toolType"?: // optional
    {
    }
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
}

```

```

"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
"mcpAllowedTools"?: object // optional
"rawMcpHeader"?: object // Use this default header when the Contact has no
corresponding MCP Credential. (optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration

```

```

* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": {},
  "apiUrl": "Response string",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
}

```



```
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
"canDelete": false
}
```

List tools

GET

</api/v1/tools/>

Parameters

Parameter name	Required	Type	Description
<code>clientPlatform</code>	X	string	Caller platform. Pass "desktop" to include desktop tools; other values or an omitted value exclude desktop tools by default.
<code>isGlobal</code>	X	boolean	Set to "true" to retrieve global tools; otherwise retrieve organization tools.
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>toolType</code>	X	string	Filter by tool type: api, function, mcp, desktop.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api",
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name"?: // May have different types (Optional)
    }
  ]
}
```

```

    string // Tool names may contain only letters, numbers, underscores (_) and
    hyphens (-). MCP tools do not need this field; API tools use it to expose the
    tool to the LLM. (Optional)
    "displayName": string // A name that may contain any characters and is used
    for user configuration and management.
    "description"?: string // Tool description shown to users. MCP tools do not
    need this field. (Optional)
    "prompt"?: string // Tool description/prompt for the LLM. If empty, the
    system uses the description field. MCP tools do not need this field. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: string // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every
    request of this API tool. Values support the same {{variable}} template rendering
    as raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use
    this tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default header used when the Contact has no
    matching MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
    carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON

```

object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be either a plain string (the AI prompt) or an object carrying additional metadata such as a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the template default. (Optional)

"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt. (Optional)

"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a display label plus an action payload (send prompt / prefill text / open http(s) link). (Optional)

"category"?: string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge\_base` - Knowledge base retrieval

* `web\_search` - Web search

* `web\_fetch` - Web page fetch

* `code\_execution` - Code execution

* `file\_processing` - File processing

* `image\_video\_generation` - Image/video generation

* `database\_query` - Database query

* `third\_party\_integration` - Third-party integration

* `browser\_operation` - Browser operation

* `agent\_collaboration` - Agent collaboration

* `other` - Other (catch-all) (Optional)

"displayCopyByLocale"?: object // Optional

"outputRenderTemplateByLocale"?: object // Optional

"toolDisplayMode"?: // May have different types (Optional)

string (enum: full, compact, hidden) // Optional

"groups"?: [// Groups this tool is assigned to. Non-owner members must pick at least one group they belong to. (Optional)

```
{
  "id": string (uuid)
  "name": string
```

```
}
```

```
]
```

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

"canUpdate": boolean // Return whether the current user can update this resource.

"canDelete": boolean // Return whether the current user can delete this resource.

```
}
```

```
]
```

```
}
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "displayName": "Response string",
      "description": "Response string",
      "prompt": "Response string",
      "toolType": {},
      "apiUrl": "Response string",
      "httpMethod": "get",
      "rawHeaders": null,
      "rawHeadersMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "rawQueryParams": null,
      "rawQueryParamsMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "rawParametersSchema": null,
      "functionCode": "Response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "createdBy": "550e8400-e29b-41d4-a716-446655440000",
      "isGlobal": false,
      "mcpUrl": "Response string",
      "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
      "mcpAllowedTools": null,
      "rawMcpHeader": null,
      "rawMcpHeaderMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "authSubject": {},
      "flexTemplateJson": null,
      "flexSchema": null,
      "quickReplyConfig": null,
      "quickMenuConfig": null,
      "category": "knowledge_base",
    }
  ]
}
```



```
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
"canDelete": false
}
]
```

Get a specific tool

GET

`/api/v1/tools/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool names may contain only letters, numbers, underscores (_) and
    hyphens (-). MCP tools do not need this field; API tools use it to expose the
    tool to the LLM. (Optional)
  "displayName": string // A name that may contain any characters and is used for
  user configuration and management.
```

```

    "description"?: string // Tool description shown to users. MCP tools do not
    need this field. (Optional)
    "prompt"?: string // Tool description/prompt for the LLM. If empty, the system
    uses the description field. MCP tools do not need this field. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: string // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
    tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default header used when the Contact has no matching
    MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

    * `shared` - Shared
    * `contact` - Contact
    * `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
    carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
    whose keys are field paths parsed from flex_template_json (e.g. text / image url
    / button label / button uri / button message). Each value may be either a plain
    string (the AI prompt) or an object carrying additional metadata such as a field
    tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
    template default. (Optional)

```

```

    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
    Structured JSON describing the chip buttons: static list or dynamic (LLM-
    generated) mode, each field either a fixed value or an LLM prompt. (Optional)
    "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
    JSON describing the persistent input-bar menu rendered by the web chat only:
    layout (grid / list), default open state, and 2-6 items each carrying a display
    label plus an action payload (send prompt / prefill text / open http(s) link).
    (Optional)
    "category"?: string (enum: knowledge_base, web_search, web_fetch,
    code_execution, file_processing, image_video_generation, database_query,
    third_party_integration, browser_operation, agent_collaboration, other) // *
    `knowledge_base` - Knowledge base retrieval
    * `web_search` - Web search
    * `web_fetch` - Web page fetch
    * `code_execution` - Code execution
    * `file_processing` - File processing
    * `image_video_generation` - Image/video generation
    * `database_query` - Database query
    * `third_party_integration` - Third-party integration
    * `browser_operation` - Browser operation
    * `agent_collaboration` - Agent collaboration
    * `other` - Other (catch-all) (Optional)
    "displayCopyByLocale"?: object // Optional
    "outputRenderTemplateByLocale"?: object // Optional
    "toolDisplayMode"?: // May have different types (Optional)
    string (enum: full, compact, hidden) // Optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
    least one group they belong to. (Optional)
    {
        "id": string (uuid)
        "name": string
    }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update this
    resource.
    "canDelete": boolean // Return whether the current user can delete this
    resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",

```

```
"prompt": "Response string",
"toolType": {},
"apiUrl": "Response string",
"httpMethod": "get",
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawParametersSchema": null,
"functionCode": "Response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
```

```
"canDelete": false
}
```

Get a specific tool

GET

</api/v1/tools/category-copy-overrides/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/category-copy-overrides/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/category-
copy-overrides/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool names may contain only letters, numbers, underscores (_) and
    hyphens (-). MCP tools do not need this field; API tools use it to expose the
    tool to the LLM. (Optional)
  "displayName": string // A name that may contain any characters and is used for
  user configuration and management.
```

```

    "description"?: string // Tool description shown to users. MCP tools do not
    need this field. (Optional)
    "prompt"?: string // Tool description/prompt for the LLM. If empty, the system
    uses the description field. MCP tools do not need this field. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: string // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
    tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default header used when the Contact has no matching
    MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
    carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
    whose keys are field paths parsed from flex_template_json (e.g. text / image url
    / button label / button uri / button message). Each value may be either a plain
    string (the AI prompt) or an object carrying additional metadata such as a field
    tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
    template default. (Optional)

```

```

    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
    Structured JSON describing the chip buttons: static list or dynamic (LLM-
    generated) mode, each field either a fixed value or an LLM prompt. (Optional)
    "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
    JSON describing the persistent input-bar menu rendered by the web chat only:
    layout (grid / list), default open state, and 2-6 items each carrying a display
    label plus an action payload (send prompt / prefill text / open http(s) link).
    (Optional)
    "category"?: string (enum: knowledge_base, web_search, web_fetch,
    code_execution, file_processing, image_video_generation, database_query,
    third_party_integration, browser_operation, agent_collaboration, other) // *
    `knowledge_base` - Knowledge base retrieval
    * `web_search` - Web search
    * `web_fetch` - Web page fetch
    * `code_execution` - Code execution
    * `file_processing` - File processing
    * `image_video_generation` - Image/video generation
    * `database_query` - Database query
    * `third_party_integration` - Third-party integration
    * `browser_operation` - Browser operation
    * `agent_collaboration` - Agent collaboration
    * `other` - Other (catch-all) (Optional)
    "displayCopyByLocale"?: object // Optional
    "outputRenderTemplateByLocale"?: object // Optional
    "toolDisplayMode"?: // May have different types (Optional)
    string (enum: full, compact, hidden) // Optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
    least one group they belong to. (Optional)
    {
        "id": string (uuid)
        "name": string
    }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update this
    resource.
    "canDelete": boolean // Return whether the current user can delete this
    resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",

```

```
"prompt": "Response string",
"toolType": {},
"apiUrl": "Response string",
"httpMethod": "get",
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawParametersSchema": null,
"functionCode": "Response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
```

```
"canDelete": false
}
```

Retrieve a specific tool

GET

</api/v1/tools/category-default-copies/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/category-default-copies/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/category-default-copies/",
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/tools/category-default-copies/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/category-
default-copies/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name"?: // May have different types (optional)
  string // The tool name may contain only letters, numbers, underscores (_), and
hyphens (-). MCP tools do not need this field; API tools use it for the LLM.
(optional)
  "displayName": string // A name that may contain any characters and is used for
user configuration and management.
```

```

    "description"?: string // A tool description shown to users. MCP tools do not
    need this field. (optional)
    "prompt"?: string // The tool description/prompt for the LLM. If empty, the
    system uses the description field. MCP tools do not need this field. (optional)
    "toolType"?: // optional
    {
    }
    "apiUrl"?: string // optional
    "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
    "rawHeaders"?: object // optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // optional
    "functionCode"?: string // optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
    tool
    "mcpUrl"?: string // optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (optional)
    "mcpAllowedTools"?: object // optional
    "rawMcpHeader"?: object // This default header is used when the Contact has no
    matching MCP Credential. (optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

    * `shared` - Shared
    * `contact` - Contact
    * `caller` - Caller (optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
    carousel / box / text / image / button etc.). (optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
    whose keys are field paths parsed from flex_template_json (e.g. text / image url
    / button label / button uri / button message). Each value may be either a plain
    string (the AI prompt) or an object carrying additional metadata such as a field
    tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
    template default. (optional)

```

```

    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
    Structured JSON describing the chip buttons: static list or dynamic (LLM-
    generated) mode, each field either a fixed value or an LLM prompt. (optional)
    "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
    JSON describing the persistent input-bar menu rendered by the web chat only:
    layout (grid / list), default open state, and 2-6 items each carrying a display
    label plus an action payload (send prompt / prefill text / open http(s) link).
    (optional)
    "category"?: string (enum: knowledge_base, web_search, web_fetch,
    code_execution, file_processing, image_video_generation, database_query,
    third_party_integration, browser_operation, agent_collaboration, other) // *
    `knowledge_base` - Knowledge base retrieval
    * `web_search` - Web search
    * `web_fetch` - Web page fetch
    * `code_execution` - Code execution
    * `file_processing` - File processing
    * `image_video_generation` - Image/video generation
    * `database_query` - Database query
    * `third_party_integration` - Third-party integration
    * `browser_operation` - Browser operation
    * `agent_collaboration` - Agent collaboration
    * `other` - Other (catch-all) (optional)
    "displayCopyByLocale"?: object // optional
    "outputRenderTemplateByLocale"?: object // optional
    "toolDisplayMode"?: // May have different types (optional)
    string (enum: full, compact, hidden) // optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
    least one group they belong to. (optional)
    {
        "id": string (uuid)
        "name": string
    }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update this
    resource.
    "canDelete": boolean // Return whether the current user can delete this
    resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",

```

```
"prompt": "Response string",
"toolType": {},
"apiUrl": "Response string",
"httpMethod": "get",
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"rawParametersSchema": null,
"functionCode": "Response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
```

```
"canDelete": false
}
```

Update a tool

PUT`/api/v1/tools/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Request body

Request parameters

Field	Type	Required	Description
name	object	No	The tool name may contain only letters, numbers, underscores (_), and hyphens (-). MCP tools do not need this field; API tools use it for the LLM.
displayName	string	Yes	A name that may contain any characters and is used for user configuration and management.
description	string	No	A tool description shown to users. MCP tools do not need this field.
prompt	string	No	The tool description/prompt for the

Field	Type	Required	Description
			LLM. If empty, the system uses the description field. MCP tools do not need this field.
toolType	object	No	
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	This default header is used when the Contact has no matching MCP Credential.
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for

Field	Type	Required	Description
			FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...

Field	Type	Required	Description
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example request schema

```
{
  "name"?: // May have different types (optional)
    string // The tool name may contain only letters, numbers, underscores (_), and
    hyphens (-). MCP tools do not need this field; API tools use it for the LLM.
    (optional)
  "displayName": string // A name that may contain any characters and is used for
  user configuration and management.
  "description"?: string // A tool description shown to users. MCP tools do not
  need this field. (optional)
  "prompt"?: string // The tool description/prompt for the LLM. If empty, the
  system uses the description field. MCP tools do not need this field. (optional)
  "toolType"?: // optional
  {
  }
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
  string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (optional)
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
  "mcpUrl"?: string // optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
  manual-URL MCP tools. (optional)
  "mcpAllowedTools"?: object // optional
  "rawMcpHeader"?: object // This default header is used when the Contact has no
  matching MCP Credential. (optional)
  "authSubject"?: // Shared uses the tool level token plus per contact
  credentials; contact requires per contact authentication only; caller mints a
  short-lived token for the conversation-bound member (falls back to per contact
```

credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared

* `contact` - Contact

* `caller` - Caller (optional)

```
{  
}
```

"flexTemplateJson"?: object // FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.). (optional)

"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be either a plain string (the AI prompt) or an object carrying additional metadata such as a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the template default. (optional)

"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt. (optional)

"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a display label plus an action payload (send prompt / prefill text / open http(s) link). (optional)

"category"?: string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other) // *

* `knowledge\_base` - Knowledge base retrieval

* `web\_search` - Web search

* `web\_fetch` - Web page fetch

* `code\_execution` - Code execution

* `file\_processing` - File processing

* `image\_video\_generation` - Image/video generation

* `database\_query` - Database query

* `third\_party\_integration` - Third-party integration

* `browser\_operation` - Browser operation

* `agent\_collaboration` - Agent collaboration

* `other` - Other (catch-all) (optional)

"displayCopyByLocale"?: object // optional

"outputRenderTemplateByLocale"?: object // optional

"toolDisplayMode"?: // May have different types (optional)

string (enum: full, compact, hidden) // optional

"groups"?: [// Groups this tool is assigned to. Non-owner members must pick at least one group they belong to. (optional)

```
{  
  "id": string (uuid)  
}
```

```
]
}
```

Example request value

```
{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
```

```
446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": {},
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
```



```
print("Response received:")  
print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "Example string",
            "description": "Example string",
            "prompt": "Example string",
            "toolType": {},
            "apiUrl": "Example string",
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "Example string",
            "mcpUrl": "Example string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "quickMenuConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
}

```

```
$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name"?: // may be one of several types (optional)
    string // Tool names may contain only letters, digits, underscores (_), and
    hyphens (-). MCP tools do not require this field; API tools use it as the name
    exposed to the LLM. (optional)
  "displayName": string // A name that may contain any characters, used to
  configure and manage the tool.
  "description"?: string // Tool description shown to users. MCP tools do not
  require this field. (optional)
  "prompt"?: string // Tool description or prompt used by the LLM. If empty, the
  system uses the description field. MCP tools do not require this field.
  (optional)
  "toolType"?: // optional
    {
    }
  "apiUrl"?: string // optional
  "httpMethod"?: // may be one of several types (optional)
    string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object
```

```

"rawParametersSchema"?: object // optional
"functionCode"?: string // optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
"mcpAllowedTools"?: object // optional
"rawMcpHeader"?: object // This default header is used when the Contact has no
corresponding MCP Credential (optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation

```

```

* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may be one of several types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": {},
  "apiUrl": "Response string",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
}

```

```
"functionCode": "Response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
"canDelete": false
}
```

Update tool

PUT

</api/v1/tools/category-copy-overrides/>

Request

Request parameters

Field	Type	Required	Description
name	object	No	Tool names may contain only letters, digits, underscores (_), and hyphens (-). MCP tools do not require this field; API tools use it as the name exposed to the LLM.
displayName	string	Yes	A name that may contain any characters, used to configure and manage the tool.
description	string	No	Tool description shown to users. MCP tools do not require this field.
prompt	string	No	Tool description or prompt used by the LLM. If empty, the system uses the description field. MCP tools do not require this field.
toolType	object	No	
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	

Field	Type	Required	Description
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	This default header is used when the Contact has no corresponding MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a

Field	Type	Required	Description
			fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	<code>knowledge_base</code> : Knowledge base retrieval ; <code>web_search</code> : Web search ; <code>web_fetch</code> : Web page fetch ; <code>code_execution</code> : Code execution ; <code>file_processing</code> : File processing ; <code>image_video_generation</code> : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Request schema example

```
{
  "name"? : // may be one of several types (optional)
    string // Tool names may contain only letters, digits, underscores (_), and
    hyphens (-). MCP tools do not require this field; API tools use it as the name
    exposed to the LLM. (optional)
  "displayName": string // A name that may contain any characters, used to
  configure and manage the tool.
  "description"? : string // Tool description shown to users. MCP tools do not
  require this field. (optional)
```

```

    "prompt"?: string // Tool description or prompt used by the LLM. If empty, the
system uses the description field. MCP tools do not require this field.
(optional)
    "toolType"?: // optional
    {
    }
    "apiUrl"?: string // optional
    "httpMethod"?: // may be one of several types (optional)
    string (enum: get, post, put, delete, patch) // optional
    "rawHeaders"?: object // optional
    "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (optional)
    "rawParametersSchema"?: object // optional
    "functionCode"?: string // optional
    "mcpUrl"?: string // optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
    "mcpAllowedTools"?: object // optional
    "rawMcpHeader"?: object // This default header is used when the Contact has no
corresponding MCP Credential (optional)
    "authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
    "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)

```

```

    "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
    "displayCopyByLocale"?: object // optional
    "outputRenderTemplateByLocale"?: object // optional
    "toolDisplayMode"?: // may be one of several types (optional)
    string (enum: full, compact, hidden) // optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
        {
            "id": string (uuid)
        }
    ]
}

```

Example request value

```

{
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": {},
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
}

```

```
"quickMenuConfig": null,  
"category": "knowledge_base",  
"displayCopyByLocale": null,  
"outputRenderTemplateByLocale": null,  
"toolDisplayMode": null,  
"groups": [  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
]  
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and check the request data before running this
example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/tools/category-copy-overrides/",
```

```
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```

import requests

url = "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": {},
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")

```

```
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/tools/category-
copy-overrides/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "Example string",
            "description": "Example string",
            "prompt": "Example string",
            "toolType": {},
            "apiUrl": "Example string",
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "Example string",
            "mcpUrl": "Example string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "quickMenuConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
}

```

```

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Response schema example

```

{
  "id": string (uuid)
  "name"?: // May have different types (optional)
    string // Tool names may contain only letters, digits, underscores (_) and
    hyphens (-). MCP tools do not need this field. API tools use it when calling the
    LLM. (optional)
  "displayName": string // A name that may contain any characters, used when
  users configure and manage the tool.
  "description"?: string // Tool description shown to users. MCP tools do not
  need this field. (optional)
  "prompt"?: string // Tool description/prompt for the LLM. If empty, the system
  uses the description field. MCP tools do not need this field. (optional)
  "toolType"?: // optional
    {
    }
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // optional
}

```

```

"functionCode"?: string // optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
"mcpAllowedTools"?: object // optional
"rawMcpHeader"?: object // This default header is used when the Contact has no
matching MCP Credential (optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query

```

```

* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": {},
  "apiUrl": "Response string",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response string",
}

```

```
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
"canDelete": false
}
```

Partially update a tool

PATCH

/api/v1/tools/{id}

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Request body

Request parameters

Field	Type	Required	Description
name	object	No	Tool names may contain only letters, digits, underscores (_) and hyphens (-). MCP tools do not need this field. API tools use it when calling the LLM.
displayName	string	No	A name that may contain any characters, used when users configure and manage the tool.
description	string	No	Tool description shown to users. MCP tools do not need this field.
prompt	string	No	Tool description/prompt for the LLM. If empty, the system uses the description field. MCP tools do not need this field.
toolType	object	No	
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template

Field	Type	Required	Description
			rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	This default header is used when the Contact has no matching MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...

Field	Type	Required	Description
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example request schema

```

{
  "name"?: // May have different types (optional)
    string // Tool names may contain only letters, digits, underscores (_) and
    hyphens (-). MCP tools do not need this field. API tools use it when calling the
    LLM. (optional)
    "displayName"?: string // A name that may contain any characters, used when
    users configure and manage the tool. (optional)
    "description"?: string // Tool description shown to users. MCP tools do not
    need this field. (optional)
    "prompt"?: string // Tool description/prompt for the LLM. If empty, the system
    uses the description field. MCP tools do not need this field. (optional)
    "toolType"?: // optional
    {
    }
    "apiUrl"?: string // optional
    "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
    "rawHeaders"?: object // optional
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (optional)
    "rawParametersSchema"?: object // optional
    "functionCode"?: string // optional
    "mcpUrl"?: string // optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (optional)
    "mcpAllowedTools"?: object // optional
    "rawMcpHeader"?: object // This default header is used when the Contact has no
    matching MCP Credential (optional)
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

    * `shared` - Shared
    * `contact` - Contact
    * `caller` - Caller (optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
    carousel / box / text / image / button etc.). (optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
    whose keys are field paths parsed from flex_template_json (e.g. text / image url
    / button label / button uri / button message). Each value may be either a plain
    string (the AI prompt) or an object carrying additional metadata such as a field
    tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the

```

```

template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
  Structured JSON describing the chip buttons: static list or dynamic (LLM-
  generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
  JSON describing the persistent input-bar menu rendered by the web chat only:
  layout (grid / list), default open state, and 2-6 items each carrying a display
  label plus an action payload (send prompt / prefill text / open http(s) link).
  (optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
  code_execution, file_processing, image_video_generation, database_query,
  third_party_integration, browser_operation, agent_collaboration, other) // *
  `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Example request value

```

{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,

```

```
"functionCode": "Example string",
"mcpUrl": "Example string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code examples

```
# Call API example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and check the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": {},
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
```



```
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": {},
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
```

```
        print("Response received successfully:")
        print(response.json())
    except Exception as e:
        print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "Example string",
            "description": "Example string",
            "prompt": "Example string",
            "toolType": {},
            "apiUrl": "Example string",
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "Example string",
            "mcpUrl": "Example string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "quickMenuConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]
}

```

```

    });

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "name"?: // May have different types (optional)
    string // Tool names may contain only letters, digits, underscores (_) and
    hyphens (-). MCP tools do not need this field. API tools use it when calling the
    LLM. (optional)
  "displayName": string // A name that may contain any characters, used when
  users configure and manage the tool.
  "description"?: string // Tool description shown to users. MCP tools do not
  need this field. (optional)
  "prompt"?: string // Tool description/prompt for the LLM. If empty, the system
  uses the description field. MCP tools do not need this field. (optional)
  "toolType"?: // optional
    {
    }
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
    string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object

```

```

"rawParametersSchema"?: object // optional
"functionCode"?: string // optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
"mcpAllowedTools"?: object // optional
"rawMcpHeader"?: object // This default header is used when the Contact has no
matching MCP Credential (optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation

```

```

* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // Optional
  "toolDisplayMode"?: // May have different types (Optional)
  string (enum: full, compact, hidden) // Optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": {},
  "apiUrl": "Response string",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
}

```

```
"functionCode": "Response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "Response string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"quickMenuConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string",
"canUpdate": false,
"canDelete": false
}
```

Delete tool

DELETE

`/api/v1/tools/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Code examples

Shell/Bash

```
# Call the API (Shell example)
curl -X DELETE "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running the
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

List available tools

GET

`/api/v1/tools/available-tools/`

Parameters

Parameter name	Required	Type	Description
<code>availableTools</code>	V	array	List of all available MCP tools
<code>mcpAllowedTools</code>	X	array	List of allowed MCP tools
<code>mcpUrl</code>	V	string	MCP server URL (required)
<code>rawMcpHeader</code>	X		MCP header (JSON format)
<code>toolId</code>	X	string	Tool ID for per-member OAuth token lookup

Code examples

Shell/Bash

```
# Call the API (Shell example)
curl -X GET "https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running the
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/available-tools/?\n\navailableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=example&toolId=550e8400-e29b-41d4-a716-446655440000"\n\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "mcpUrl": string // MCP server URL (required)
  "rawMcpHeader"?: object // MCP header (JSON format) (Optional)
  "mcpAllowedTools"?: [ // List of allowed MCP tools (Optional)
    string
  ]
}
```



```
"toolId"?: string (uuid) // Tool ID for per-member OAuth token lookup
(Optional)
"availableTools": [ // List of all available MCP tools
  string
]
}
```

Example response value

```
{
  "mcpUrl": "Response string",
  "rawMcpHeader": null,
  "mcpAllowedTools": [
    "Response string"
  ],
  "toolId": "550e8400-e29b-41d4-a716-446655440000",
  "availableTools": [
    "Response string"
  ]
}
```

Status code: **400**

Example response schema

```
{
  "detail"?: string // Optional
}
```

Example response value

```
{
  "detail": "Response string"
}
```

Create connector

POST

/api/v1/connectors/

Request body

Request parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	Yes	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. knowledge_base : Knowledge base retrieval ; web_search : We...
defaultDisplayNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list

Field	Type	Required	Description
			disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Example request schema

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayName. Empty = frontend falls back to its name-based heuristic. An
```

```

individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
}

```

Example request value

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}

```

Code examples

```
# Call the API (Shell example)
curl -X POST "https://api.maiagent.ai/api/v1/connectors/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Replace YOUR_API_KEY and verify the request data before running the
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.post("https://api.maiagent.ai/api/v1/connectors/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/connectors/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "Example string",
            "description": "Example string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "Example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?:// May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (Optional)
    "logoUrl"?:// May have different types (Optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType?": // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (Optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (Optional)
  "defaultCategory?": // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // Optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "Response string",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List connectors

GET

/api/v1/connectors/

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/connectors/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
[
  {
    "id": string (uuid)
    "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "mcpRegistryName": string
    "mcpRegistryId": string
    "mcpRegistryIsGlobal": boolean
    "logoUrl": string
```

```

    "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
    "isEnabled"?: boolean // Whether this connector is enabled for contacts in
the organization (Optional)
    "authHeadersMasked": object
    "allowedContextDataKeys": object // Allowlist of webchat contextData keys
this connector may render into auth_headers through {{context_data.<key>}}. An
empty list disables the feature for this connector: the placeholder is left
verbatim, exactly as before the feature existed. Keys are matched literally and
are case-sensitive; at most 20 keys of letters, digits and underscores.
    "authorizedMembersCount": integer
    "createdBy":
    {
        "id": string (uuid)
        "name": string
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
]

```

Example response value

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Response string",
    "mcpRegistryName": "Response string",
    "mcpRegistryId": "Response string",
    "mcpRegistryIsGlobal": false,
    "logoUrl": "Response string",
    "defaultCategory": "knowledge_base",
  }
]

```



```
"isEnabled": false,
"authHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"allowedContextDataKeys": null,
"authorizedMembersCount": 456,
"createdBy": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
"createdAt": "Response string",
"updatedAt": "Response string"
}
]
```

Get a specific connector

GET

`/api/v1/connectors/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
```

```

    "displayName?": string // Human-readable display name (defaults to name if
empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"? : // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl"? : // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
(Optional)
    "isActive"? : boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"? : boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"? : // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    }
    "mcpRegistryId": string (uuid)
    "name": string
    "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "description?": string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
    "defaultCategory"? : // May have different types (Optional)
        string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation

```

```

* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (Optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (Optional)

"effectiveDescription": string
"remoteMcpUrl": string
"oauthServerUrl": string
"logoUrl": string
"authType": string
"authHeaders"?: object // Optional
"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
"organization": string (uuid)
"createdBy": string (uuid)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
  }
}

```

```

    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "Response string",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}

```

Get a specific connector

GET

</api/v1/connectors/enablement-requests/>

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/enablement-requests/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/enablement-requests/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/enablement-requests/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/enablement-requests/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
```

```
empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (Optional)
  "logoUrl"?: // May have different types (Optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
  "isActive"?: boolean // Whether this MCP service is currently available for
  use (Optional)
  "isGlobal"?: boolean // Global registries are maintained by the platform and
  available to all organizations. (Optional)
  "authType"?: // How to authenticate with this MCP server.
```

```
* `oauth` - OAuth 2.0
```

```
* `service_account` - Service Account
```

```
* `none` - No Authentication (Optional)
```

```
{
}
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
"mcpRegistryId": string (uuid)
"name": string
"display_name"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
"description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
"defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
  Registry default category for this connector's dynamic tools. Empty = use the MCP
  Registry default_category.
```

```
* `knowledge_base` - Knowledge base retrieval
```

```
* `web_search` - Web search
```

```
* `web_fetch` - Web page fetch
```

```
* `code_execution` - Code execution
```

```
* `file_processing` - File processing
```

```
* `image_video_generation` - Image/video generation
```

```
* `database_query` - Database query
```

```
* `third_party_integration` - Third-party integration
```

```
* `browser_operation` - Browser operation
```

```
* `agent_collaboration` - Agent collaboration
```

```

* `other` - Other (catch-all) (optional)
  "defaultDisplayNameByLocale"?: object // Locale-keyed server-level default
  group headline shown in chat for this connector's dynamic MCP tools (those
  without an individual Tool row). Shape: {"<locale>": str}. The headline for the
  conversation locale is resolved server-side (see
  whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
  > settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
  displayName. Empty = frontend falls back to its name-based heuristic. An
  individual Tool row overrides this. (optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
  icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
  frontend uses its default MCP icon. Streamed as toolIcon. (optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string
  "authType": string
  "authHeaders"?: object // optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
  connector may render into auth_headers through {{context_data.<key>}}. An empty
  list disables the feature for this connector: the placeholder is left verbatim,
  exactly as before the feature existed. Keys are matched literally and are case-
  sensitive; at most 20 keys of letters, digits and underscores. (optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
  exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
  server tools. Use this to limit a connector to a safe subset (e.g. read-only
  tools). (optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
  organization (optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
  }
}

```

```
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "Response string",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update a connector

PUT

[/api/v1/connectors/{id}](#)

Parameters

Parameter name	Required	Type	Description
id	V	string	

Request body

Request parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	Yes	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. <code>knowledge_base</code> : Knowledge base retrieval ; <code>web_search</code> : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: <code>{"": str}</code> . The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	

Field	Type	Required	Description
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request schema example

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to MCP Registry name) (optional)
  "description"?: string // Custom description for this connector (defaults to MCP Registry description) (optional)
  "defaultCategory"?: // May have different types (optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other) // Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

  * `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (optional)

  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those
```

```

without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)
  "authHeaders"?: object // optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (optional)
}

```

Example request value

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}

```

Code examples


```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "Example string",
            "description": "Example string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "Example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (optional)
    "description?": string // Brief description of the MCP service (optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?:// May have different types (optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (optional)
    "logoUrl"?:// May have different types (optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (optional)
    "authType?": // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (optional)
  "defaultCategory?": // May have different types (optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "Response string",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially update a connector

PATCH

/api/v1/connectors/{id}

Parameters

Parameter name	Required	Type	Description
id	V	string	

Request body

Request parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	No	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. knowledge_base : Knowledge base retrieval ; web_search : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual

Field	Type	Required	Description
			Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request schema example

```
{
  "mcpRegistryId?": string (uuid) // optional
  "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (optional)
  "description?": string // Custom description for this connector (defaults to
MCP Registry description) (optional)
  "defaultCategory"?: // May have different types (optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
```

```

* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"authHeaders"?: object // optional

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (optional)
}

```

Example request value

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example string",
  "description": "example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,

```

```
"isEnabled": true
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiaagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example string",
  "description": "example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Replace YOUR_API_KEY and verify the request data before running this
command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example string",
  "description": "example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "example string",
    "description": "example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "example string",
            "description": "example string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (optional)
    "description?": string // Brief description of the MCP service (optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?:// may have different types (optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (optional)
    "logoUrl"?:// may have different types (optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (optional)
    "authType?": // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (optional)
  "defaultCategory?": // may have different types (optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "displayName": "response string",
    "description": "response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response string",
    "updatedAt": "response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "displayName": "response string",
  "description": "response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "response string",
  "effectiveDescription": "response string",
  "remoteMcpUrl": "response string",
  "oauthServerUrl": "response string",
  "logoUrl": "response string",
  "authType": "response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

Delete connector

DELETE

/api/v1/connectors/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

List authorized connector members

GET**/api/v1/connectors/{id}/authorized-members/**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
```

```

    "displayName?": string // Human-readable display name (defaults to name if
empty) (optional)
    "description?": string // Brief description of the MCP service (optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // may have different types (optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (optional)
    "logoUrl"?: // may have different types (optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
(optional)
    "isActive?": boolean // Whether this MCP service is currently available for
use (optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
available to all organizations. (optional)
    "authType"?: // How to authenticate with this MCP server.

```

```

* `oauth` - OAuth 2.0

```

```

* `service_account` - Service Account

```

```

* `none` - No Authentication (optional)

```

```

    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    }
    "mcpRegistryId": string (uuid)
    "name": string
    "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (optional)
    "description?": string // Custom description for this connector (defaults to
MCP Registry description) (optional)
    "defaultCategory"?: // may have different types (optional)
        string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

```

```

* `knowledge_base` - Knowledge base retrieval

```

```

* `web_search` - Web search

```

```

* `web_fetch` - Web page fetch

```

```

* `code_execution` - Code execution

```

```

* `file_processing` - File processing

```

```

* `image_video_generation` - Image/video generation

```

```

* `database_query` - Database query

```

```

* `third_party_integration` - Third-party integration

```

```

* `browser_operation` - Browser operation

```

```

* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string
"remoteMcpUrl": string
"oauthServerUrl": string
"logoUrl": string
"authType": string
"authHeaders"?: object // optional
"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (optional)
"organization": string (uuid)
"createdBy": string (uuid)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "displayName": "response string",
    "description": "response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
  }
}

```

```

    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response string",
    "updatedAt": "response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "displayName": "response string",
  "description": "response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "response string",
  "effectiveDescription": "response string",
  "remoteMcpUrl": "response string",
  "oauthServerUrl": "response string",
  "logoUrl": "response string",
  "authType": "response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response string",
  "updatedAt": "response string"
}

```

Remove authorized connector member

DELETE

/api/v1/connectors/{id}/authorized-members/{memberId}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>memberId</code>	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}"

headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/{memberId}
", [
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

List MCP Tool Registrations

GET

/api/v1/mcp/registry/

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/mcp/registry/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (optional)
    "description?": string // Brief description of the MCP service (optional)
```

```

    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
        empty to auto-discover from the MCP server (RFC 9728 protected resource
        metadata). Only fill in when auto-discovery fails or the server advertises
        incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
        api.supabase.com) (optional)
    "logoUrl"?: // May have different types (optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
        (optional)
    "isActive"?: boolean // Whether this MCP service is currently available for
    use (optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
    available to all organizations. (optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]

```

Example Response

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]

```

Get a Specific MCP Tool Registration

GET

/api/v1/mcp/registry/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric,
underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if
empty) (optional)
  "description"?: string // Brief description of the MCP service (optional)
```

```

    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
        empty to auto-discover from the MCP server (RFC 9728 protected resource
        metadata). Only fill in when auto-discovery fails or the server advertises
        incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
        api.supabase.com) (optional)
    "logoUrl"?: // May have different types (optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (optional)
    "isActive"?: boolean // Whether this MCP service is currently available for use
    (optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
    available to all organizations. (optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}

```

Example Response

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "logoUrl": "https://example.com/file.jpg",
  "isActive": false,
  "isGlobal": false,
  "authType": {},
  "createdAt": "Response string",
  "updatedAt": "Response string"
}

```

List Tool Execution Records

GET

/api/v1/tool-execution-records/

Parameters

Parameter	Required	Type	Description
chatbot	X	string	Filter by AI assistant ID
endDate	X	string	End date (inclusive), format: YYYY-MM-DD
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search AI assistant names, tool names, and message content
startDate	X	string	Start date (inclusive), format: YYYY-MM-DD
status	X	string	Filter by execution status: success, failure, pending, timeout
tool	X	string	Filter by tool ID
toolType	X	string	Filter by tool type: api, function, mcp

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/?  
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex  
ample&status=failure&tool=example&toolType=api"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"? : string (uri) // optional
  "previous"? : string (uri) // optional
  "results": [
    {
```

```

    "id": string (uuid)
    "tool": object // Get tool information
    "chatbot": object // Get AI assistant information
    "messageContent": string // Get message content
    "status"?: // optional
    {
    }
    "executionTime": integer // Get execution time (milliseconds)
    "errorMessage"?: string // optional
    "retryCount"?: integer // How many times the platform auto-retried this
    tool call before the final outcome (0 = the first attempt was the final one). API
    tools count tenacity retries inside APIRequestService; MCP tools count transient-
    error retries (read-classified tools only). Function tools are never auto-
    retried. (optional)
    "createdAt": string (timestamp)
    "source": string // Scheduled runs' tool calls are attributed via
    ChatbotScheduleExecution.user_message.
  }
]
}

```

Example Response

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "messageContent": "Response string",
      "status": {},
      "executionTime": 456,
      "errorMessage": "Response string",
      "retryCount": 456,
      "createdAt": "Response string",
      "source": "Response string"
    }
  ]
}

```

```
]
}
```

Get a Specific Tool Execution Record

GET

`/api/v1/tool-execution-records/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	Y	string	A UUID string identifying this tool execution record.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "tool": object // Get tool information, including operated_tool_name
  "chatbot": object // Get AI assistant information
  "message":
  {
    "id": string (uuid)
    "content": string
  }
}
```

```

}
"inputParameters": object
"outputResult"?: object // optional
"errorMessage"?: string // optional
"status"?: // optional
{
}
"executionTime": integer // Get execution time (milliseconds)
"retryCount"?: integer // How many times the platform auto-retried this tool
call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (optional)
"requestUrl"?: // May have different types (optional)
string (uri) // Final rendered URL used when calling an API tool. Null for non-
API tools. (optional)
"requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
tool. Null for non-API tools. (optional)
"requestHeaders": object // Return rendered request headers with sensitive
values masked.

APICredential keys for the (contact, tool) pair are masked automatically
in addition to ``settings.TOOL_API_SENSITIVE_HEADERS``.
"requestBody"?: object // Final rendered body sent to the external API. Null
for non-API tools or read-only methods. (optional)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"source": string // Scheduled runs' tool calls are attributed via
ChatbotScheduleExecution.user_message.
}

```

Example Response

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "content": "Response string"
  }
}

```

```
{,
  "inputParameters": null,
  "outputResult": null,
  "errorMessage": "Response string",
  "status": {},
  "executionTime": 456,
  "retryCount": 456,
  "requestUrl": "https://example.com/file.jpg",
  "requestMethod": "Response string",
  "requestHeaders": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "requestBody": null,
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "source": "Response string"
}
```

Get AI Assistant Filter Options for Execution Records

GET

</api/v1/tool-execution-records/chatbots/>

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/",
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response structure

```
[
  {
    "id?": string (uuid) // Optional
    "name?": string // Optional
  }
]
```


Example response value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
]
```

Get tool filter options for execution records

GET

/api/v1/tool-execution-records/tools/

Parameters

Parameter name	Required	Type	Description
chatbot	X	string	
endDate	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
startDate	X	string	
status	X	string	`success`: success; `failure`: failure; `pending`: pending; `timeout`: timeout;
tool	X	string	

toolType

X

string

```
`api`: API; `function`: Function; `mcp`: MCP;  
`desktop`: Desktop; `flex`: Flex Message;  
`quick_reply`: Quick Reply; `quick_menu`: Quick  
Menu;
```

Code examples

Shell/Bash

```
# Example API call (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?  
chatbot=550e8400-e29b-41d4-a716-  
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=exam  
ple&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
```

```
    "id": string (uuid)
    "displayName": string
  }
]
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "response string"
    }
  ]
}
```

List execution records for a specific tool

GET

`/api/v1/tools/{toolPk}/tool-execution-records/`

Parameters

Parameter name	Required	Type	Description
<code>toolPk</code>	V	string	
<code>chatbot</code>	X	string	Filter by AI assistant ID
<code>endDate</code>	X	string	End date (inclusive), format: YYYY-MM-DD
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

query	X	string	Search AI assistant names, tool names, and message content
startDate	X	string	Start date (inclusive), format: YYYY-MM-DD
status	X	string	Filter by execution status: success, failure, pending, timeout
tool	X	string	Filter by tool ID
toolType	X	string	Filter by tool type: api, function, mcp

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
```

```

    "id": string (uuid)
    "tool": object // Get tool information
    "chatbot": object // Get AI assistant information
    "messageContent": string // Get message content
    "status"?: // Optional
    {
    }
    "executionTime": integer // Get execution time (milliseconds)
    "errorMessage"?: string // Optional
    "retryCount"?: integer // How many times the platform auto-retried this
    tool call before the final outcome (0 = the first attempt was the final one). API
    tools count tenacity retries inside APIRequestService; MCP tools count transient-
    error retries (read-classified tools only). Function tools are never auto-
    retried. (Optional)
    "createdAt": string (timestamp)
    "source": string // Scheduled runs' tool calls are attributed via
    ChatbotScheduleExecution.user_message.
  }
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "messageContent": "response string",
      "status": {},
      "executionTime": 456,
      "errorMessage": "response string",
      "retryCount": 456,
      "createdAt": "response string",
      "source": "response string"
    }
  ]
}

```

```
]
}
```

Get a specific execution record for a specific tool

GET

`/api/v1/tools/{toolPk}/tool-execution-records/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool execution record.
<code>toolPk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "tool": object // Get tool information, including operated_tool_name
  "chatbot": object // Get AI assistant information
  "message":
  {
    "id": string (uuid)
```

```

    "content": string
  }
  "inputParameters": object
  "outputResult"?: object // Optional
  "errorMessage"?: string // Optional
  "status"?: // Optional
  {
  }
  "executionTime": integer // Get execution time (milliseconds)
  "retryCount"?: integer // How many times the platform auto-retried this tool
  call before the final outcome (0 = the first attempt was the final one). API
  tools count tenacity retries inside APIRequestService; MCP tools count transient-
  error retries (read-classified tools only). Function tools are never auto-
  retried. (Optional)
  "requestUrl"?: // May have different types (Optional)
    string (uri) // Final rendered URL used when calling an API tool. Null for non-
  API tools. (Optional)
  "requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
  tool. Null for non-API tools. (Optional)
  "requestHeaders": object // Return rendered request headers with sensitive
  values masked.

  APICredential keys for the (contact, tool) pair are masked automatically
  in addition to ``settings.TOOL_API_SENSITIVE_HEADERS``.
  "requestBody"?: object // Final rendered body sent to the external API. Null
  for non-API tools or read-only methods. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "source": string // Scheduled runs' tool calls are attributed via
  ChatbotScheduleExecution.user_message.
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",

```



```
    "content": "response string"
  },
  "inputParameters": null,
  "outputResult": null,
  "errorMessage": "response string",
  "status": {},
  "executionTime": 456,
  "retryCount": 456,
  "requestUrl": "https://example.com/file.jpg",
  "requestMethod": "response string",
  "requestHeaders": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "requestBody": null,
  "createdAt": "response string",
  "updatedAt": "response string",
  "source": "response string"
}
```

Get AI assistant filter options for a specific tool execution record

GET

`/api/v1/tools/{toolPk}/tool-execution-records/chatbots/`

Parameters

Parameter name	Required	Type	Description
<code>toolPk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response structure

```
[
  {
    "id"?: string (uuid) // Optional
    "name"?: string // Optional
  }
]
```

Example response value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
]
```

Get tool filter options for a specific tool execution record

GET `/api/v1/tools/{toolPk}/tool-execution-records/tools/`

Parameters

Parameter name	Required	Type	Description
toolPk	V	string	
chatbot	X	string	
endDate	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
startDate	X	string	
status	X	string	`success`: success; `failure`: failure; `pending`: pending; `timeout`: timeout;
tool	X	string	
toolType	X	string	`api`: API; `function`: Function; `mcp`: MCP; `desktop`: Desktop; `flex`: Flex Message;

`quick\_reply`: Quick Reply; `quick\_menu`: Quick Menu;

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-
e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=exam
ple&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```
{
  "id": string (uuid)
  "displayName": string
}
]
```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "Response string"
    }
  ]
}
```

Call API example (Shell)

...

目錄

1. Upload Skill
2. List Skills
3. Get Specific Skill
4. Update Skill
5. Partially Update Skill
6. Delete Skill
7. Export Skill
8. Re-upload Skill
9. List Skill Resources
10. Delete Skill Resource
11. Download Skill Resource

Call API example (Shell)

Upload Skill

POST

`/api/v1/skills/upload/`

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/upload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

-F "attached_tool_ids=Example String"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');
formData.append('attached_tool_ids', 'Example String');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/upload/",
formData, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
    'application/pdf'),
    'attached_tool_ids': (None, 'Example String')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'attached_tool_ids',
                'contents' => 'Example String'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```


Response

Status Code: 201

Response Schema Example

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "attached_tools"?: [ // Tool names automatically bound from the skill
package front matter (optional)
    string
  ]
  "missing_tools"?: [ // Tool names declared by the skill package but not
found in this organization and requiring manual binding (optional)
    string
  ]
  "message"?: string // Optional
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "attached_tools": [
    "Response string"
  ],
  "missing_tools": [
    "Response string"
  ],
  "name": "Response String",
  "description": "Response String",
  "message": "Response String"
}
```

Status Code: 400

Response Schema Example

```
{
  "detail"?: string // Error message (Optional)
}
```

Response Example

```
{
  "detail": "Response String"
}
```

List Skills

GET

/api/v1/skills/

Parameters

Parameter	Required	Type	Description
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
search	X	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "name": string
    "description": string
    "whenToUse": string // Describe when the LLM should trigger this
skill. Max 200 characters recommended.
    "source": // How the skill was created: upload (.skill/.zip) or
manual

* `upload` - Upload
* `manual` - Manual
    {
    }
    "attachedTools": [
      {
        "id": string (uuid)
        "name": string
        "description": string
        "displayName": string
        "toolType": string
      }
    ]
    "groups": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "createdBy": object // Show 'System' as the creator for global
skills (uploaded by platform staff).
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "lastUsedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update
this resource.
    "canDelete": boolean // Return whether the current user can delete
this resource.
  }
]
}

```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "whenToUse": "Response String",
      "source": {},
      "attachedTools": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String",
          "description": "Response String",
          "displayName": "Response String",
          "toolType": "Response String"
        }
      ],
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response Example Name",
        "description": "Response Example Description"
      },
      "createdAt": "Response String",
      "updatedAt": "Response String",
      "lastUsedAt": "Response String",
      "canUpdate": false,
      "canDelete": false
    }
  ]
}
```


Get Specific Skill

GET

`/api/v1/skills/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"description": string
"whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
"instructions": string // Markdown text with instructions for the skill
"compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
"metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
"allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
"source": // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
{
}
"skillPackagePath": string // S3 path for the uploaded .skill/.zip file
"skillPackageName": string // Original filename of the uploaded skill
package
"skillPackageSize": integer // Size of the uploaded skill package in
bytes
"skillPackageUploadedAt": string (timestamp)
"attachedTools": [
  {
    "id": string (uuid)
    "name": string
    "description": string
    "displayName": string
    "toolType": string
  }
]
"resources": [
  {
    "id": string (uuid)
    "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
    "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
    "fileSize": integer // File size in bytes
    "mimeType": string
    "createdAt": string (timestamp)
  }
]
"groups": [
  {
```

```
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"lastUsedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "displayName": "Response String",
      "toolType": "Response String"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response String",
      "filePath": "Response String",
      "fileSize": 456,
```

```
    "mimeType": "Response String",
    "createdAt": "Response String"
  },
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Update Skill

PUT

/api/v1/skills/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Skill.

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	Yes	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name": string
  "description": string
  "whenToUse"? : string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"? : string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"? : [ // Optional
    string (uuid)
  ]
  "groups"? : [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Request Example

```
{
  "name": "Example Name",
  "description": "Example String",
```



```
"whenToUse": "Example String",
"instructions": "Example String",
"attachedToolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code Examples

```
# Call API example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "description": "Example String",
    "whenToUse": "Example String",
    "instructions": "Example String",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "description": "Example String",
            "whenToUse": "Example String",
            "instructions": "Example String",
            "attachedToolIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
  Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
  Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
  Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
  Agent Skills Spec frontmatter (experimental).
  "source": // How the skill was created: upload (.skill/.zip) or manual

  * `upload` - Upload
  * `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
  package
  "skillPackageSize": integer // Size of the uploaded skill package in
  bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```

```

    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"resources": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "path": "Response String",
    "filePath": "Response String",
    "fileSize": 456,
    "mimeType": "Response String",
    "createdAt": "Response String"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Partially Update Skill

PATCH

</api/v1/skills/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Request

Request Parameters

Field	Type	Required	Description
<code>name</code>	string	No	
<code>description</code>	string	No	
<code>whenToUse</code>	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
<code>instructions</code>	string	No	Markdown text with instructions for the skill
<code>attachedToolIds</code>	array[string]	No	
<code>groups</code>	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"?: string // Optional
  "description"?: string // Optional
  "whenToUse"?: string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"?: string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"?: [ // Optional
    string (uuid)
  ]
  "groups"?: [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
    {
```

```
        "id": string (uuid)
    }
]
}
```

Request Example

```
{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

```
# Call API example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "description": "Example String",
    "whenToUse": "Example String",
    "instructions": "Example String",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
    a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "description": "Example String",
            "whenToUse": "Example String",
            "instructions": "Example String",
            "attachedToolIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
  Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
  Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
  Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
  Agent Skills Spec frontmatter (experimental).
  "source": // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
  package
  "skillPackageSize": integer // Size of the uploaded skill package in
  bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```

```

    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {

```



```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"resources": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "path": "Response String",
    "filePath": "Response String",
    "fileSize": 456,
    "mimeType": "Response String",
    "createdAt": "Response String"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Delete Skill

DELETE

/api/v1/skills/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X DELETE "https://api.maiaagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Export Skill

GET

/api/v1/skills/{id}/export/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	.skill file download

Re-upload Skill

POST

`/api/v1/skills/{id}/reupload/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/", formData, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
    'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "message"?: string // Optional
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "message": "Response String"
}
```

Status Code: **400**

Response Schema Example

```
{
  "detail"?: string // Error message (Optional)
}
```

Response Example

```
{
  "detail": "Response String"
}
```

List Skill Resources

GET**/api/v1/skills/{skillPk}/resources/**

Parameters

Parameter	Required	Type	Description
skillPk	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "path": string // Relative path within the skill directory, e.g.
```

```
scripts/helper.py
    "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
    "fileSize": integer // File size in bytes
    "mimeType": string
    "createdAt": string (timestamp)
}
]
```

Response Example

```
[
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "path": "Response String",
  "filePath": "Response String",
  "fileSize": 456,
  "mimeType": "Response String",
  "createdAt": "Response String"
}
]
```

Delete Skill Resource

DELETE`/api/v1/skills/{skillPk}/resources/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>skillPk</code>	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Download Skill Resource

GET

`/api/v1/skills/{skillPk}/resources/{id}/download/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>skillPk</code>	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e
8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	File download

Call API example (Shell)

...

目錄

1. Create a knowledge base
2. List all knowledge bases
3. Get knowledge base details
4. Get details of a specific knowledge base
5. Update a knowledge base
6. Partially Update Knowledge Base
7. Delete Knowledge Base
8. Check RRF Availability
9. Export labels to Excel
10. Import labels from Excel
11. Export metadata to Excel
12. Import metadata from Excel
13. Create a new label
14. List labels in a knowledge base
15. Get Label Details
16. Update a Label
17. Partially Update a Label
18. Delete a Label
19. Create a Metadata Field
20. List Metadata Fields
21. Get a Specific Metadata Field
22. Update a Metadata Field
23. Upload Files to the Knowledge Base
24. Batch Delete Files
25. Batch reparse files
26. List files in a knowledge base
27. Get details of a specific file
28. Get Details of a Specific File
29. Get the Latest File Failure Reason
30. Get File Transcription
31. Update File Information
32. Partially Update a File
33. Delete a file
34. Create a file translation
35. List file translations

- 36. Retrieve a translation
- 37. Delete translation
- 38. Download translated file
- 39. List knowledge base documents
- 40. Update document
- 41. Partially Update a Document
- 42. Delete a Document
- 43. Create a New FAQ
- 44. Delete FAQs in Bulk
- 45. List All FAQs in a Specific Knowledge Base
- 46. Get FAQ details
- 47. Update FAQ
- 48. Partially update FAQ
- 49. Delete FAQ

Call API example (Shell)

Create a knowledge base

POST

/api/v1/knowledge-bases/

Request body

Request parameters

Field	Type	Required	Description
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	

Field	Type	Required	Description
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve, defaults to 12, minimum is 1
enableHyde	boolean	No	Enable HyDE, defaults to False
similarityCutoff	number (double)	No	Similarity threshold, defaults to 0.0, range is 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking, defaults to True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license. If not provided, uses system default.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. elasticsearch : Elasticsearch ; opensearch : OpenSearch ; oracle : Oracle AI Database 26ai ; synxdb : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.

Field	Type	Required	Description
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request schema example

```
{
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve, defaults to 12, minimum is 1 (optional)
  "enableHyde"?: boolean // Enable HyDE, defaults to False (optional)
  "similarityCutoff"?: number (double) // Similarity threshold, defaults
to 0.0, range is 0.0-1.0 (optional)
  "enableRerank"?: boolean // Whether to enable reranking, defaults to
True (optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. If not provided, uses system default. (optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
```



```

{
}
"vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
"vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
"opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
"opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
"oracleUser"?: string // Oracle database username. (optional)
"oraclePassword"?: string // Oracle database password. (optional)
"synxdbUser"?: string // SynxDB database username. (optional)
"synxdbPassword"?: string // SynxDB database password. (optional)
"chatbots"?: [ // optional
{
    "id": string (uuid)
}
]
"groups"?: [ // optional
{
    "id": string (uuid)
}
]
}

```

Example request values

```

{
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",

```

```
"opensearchPassword": "example string",
"oracleUser": "example string",
"oraclePassword": "example string",
"synxdbUser": "example string",
"synxdbPassword": "example string",
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code examples

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-bases/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example name",
    "description": "example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "example string",
    "vectorDatabaseApiKey": "example string",
    "opensearchUsername": "example string",
    "opensearchPassword": "example string",
    "oracleUser": "example string",
    "oraclePassword": "example string",
    "synxdbUser": "example string",
    "synxdbPassword": "example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "embeddingModel": "550e8400-e29b-41d4-a716-
446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "example name",
            "description": "example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "example string",
            "vectorDatabaseApiKey": "example string",
            "opensearchUsername": "example string",
            "opensearchPassword": "example string",
            "oracleUser": "example string",
            "oraclePassword": "example string",
            "synxdbUser": "example string",
            "synxdbPassword": "example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]
}

```

```

        }
    ]
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 201

Response schema example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // optional
  "numberOfRetrievedChunks?": integer // Number of reference chunks to
retrieve, defaults to 12, minimum is 1 (optional)
  "enableHyde?": boolean // Enable HyDE, defaults to False (optional)
  "similarityCutoff?": number (double) // Similarity threshold, defaults
to 0.0, range is 0.0-1.0 (optional)
  "enableRerank?": boolean // Whether to enable reranking, defaults to

```

```

True (optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
  "oracleUser"?: string // Oracle database username. (optional)
  "oraclePassword"?: string // Oracle database password. (optional)
  "synxdbUser"?: string // SynxDB database username. (optional)
  "synxdbPassword"?: string // SynxDB database password. (optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups with access to the knowledge base
(optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]

```

```

]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "response string",
  "vectorDatabaseApiKey": "response string",
  "opensearchUsername": "response string",
  "opensearchPassword": "response string",
  "oracleUser": "response string",
  "oraclePassword": "response string",
  "synxdbUser": "response string",
  "synxdbPassword": "response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ]
}

```

```
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

List all knowledge bases

GET

</api/v1/knowledge-bases/>

Parameters

Parameter name	Required	Type	Description
<code>embeddingModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.

pageSize	X	integer	Number of results to return per page.
query	X	string	
rerankerModel	X	string	

Code examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/?\nembeddingModel=550e8400-e29b-41d4-a716-\n446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-\ne29b-41d4-a716-446655440000"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "createdBy":
        {
          "id": string (uuid)
          "name": string
        }
      "organization"?: string (uuid) // optional
      "embeddingModel": string (uuid)
      "rerankerModel": string (uuid)
      "name": string
      "description"?: string // optional
      "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve, defaults to 12, minimum is 1 (optional)
      "enableHyde"?: boolean // Enable HyDE, defaults to False (optional)
      "similarityCutoff"?: number (double) // Similarity threshold,
defaults to 0.0, range is 0.0-1.0 (optional)
      "enableRerank"?: boolean // Whether to enable reranking, defaults
to True (optional)
      "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
      "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
      {
      }
      "vectorDatabaseUrl"?: string // Custom vector database endpoint URL
or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
      "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (optional)
      "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (optional)

```

```

    "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (optional)
    "oracleUser"?: string // Oracle database username. (optional)
    "oraclePassword"?: string // Oracle database password. (optional)
    "synxdbUser"?: string // SynxDB database username. (optional)
    "synxdbPassword"?: string // SynxDB database password. (optional)
    "labels": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "chatbots"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "groups"?: [ // List of groups with access to the knowledge base
(optional)
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
    "vectorStorageSize": integer
    "chunksCount": integer
    "canUpdate": boolean // Return whether the current user can update
this resource.
    "canDelete": boolean // Return whether the current user can delete
this resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Example response values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",

```

```
"previous": "http://api.example.org/accounts/?page=2",
"results": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "createdBy": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    },
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string",
    "numberOfRetrievedChunks": 456,
    "enableHyde": false,
    "similarityCutoff": 456,
    "enableRerank": false,
    "enableRrf": false,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "response string",
    "vectorDatabaseApiKey": "response string",
    "opensearchUsername": "response string",
    "opensearchPassword": "response string",
    "oracleUser": "response string",
    "oraclePassword": "response string",
    "synxdbUser": "response string",
    "synxdbPassword": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  },
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "createdBy": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    },
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string",
    "numberOfRetrievedChunks": 456,
    "enableHyde": false,
    "similarityCutoff": 456,
    "enableRerank": false,
    "enableRrf": false,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "response string",
    "vectorDatabaseApiKey": "response string",
    "opensearchUsername": "response string",
    "opensearchPassword": "response string",
    "oracleUser": "response string",
    "oraclePassword": "response string",
    "synxdbUser": "response string",
    "synxdbPassword": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  }
]
```

```
    "filesCount": 456,  
    "vectorStorageSize": 456,  
    "chunksCount": 456,  
    "canUpdate": false,  
    "canDelete": false,  
    "createdAt": "response string",  
    "updatedAt": "response string"  
  }  
]  
}
```

Get knowledge base details

GET

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "createdBy":
```

```

{
  "id": string (uuid)
  "name": string
}
"organization"?: string (uuid) // optional
"embeddingModel": string (uuid)
"rerankerModel": string (uuid)
"name": string
"description"?: string // optional
"numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; default: 12; minimum: 1 (optional)
"enableHyde"?: boolean // Enable HyDE; default: False (optional)
"similarityCutoff"?: number (double) // Similarity threshold; default:
0.0; range: 0.0–1.0 (optional)
"enableRerank"?: boolean // Whether to enable reranking; default: True
(optional)
"enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
"vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
{
}
"vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
"vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
"opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
"opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
"oracleUser"?: string // Oracle database username. (optional)
"oraclePassword"?: string // Oracle database password. (optional)
"synxdbUser"?: string // SynxDB database username. (optional)
"synxdbPassword"?: string // SynxDB database password. (optional)
"labels": [
  {
    "id": string (uuid)

```

```

    "name": string
  }
]
"chatbots"?: [ // optional
  {
    "id": string (uuid)
    "name": string
  }
]
"groups"?: [ // List of groups that can access the knowledge base
(optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,

```

```
"enableRerank": false,
"enableRrf": false,
"vectorDatabaseType": {},
"vectorDatabaseUrl": "response string",
"vectorDatabaseApiKey": "response string",
"opensearchUsername": "response string",
"opensearchPassword": "response string",
"oracleUser": "response string",
"oraclePassword": "response string",
"synxdbUser": "response string",
"synxdbPassword": "response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string",
"updatedAt": "response string"
}
```

Get details of a specific knowledge base

GET

/api/v1/knowledge-bases/move-options/

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/move-
options/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/move-
options/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/move-options/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/move-options/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string
}
```



```
"embeddingModel": string (uuid)
"vectorDatabaseType"?: // Type of vector database: Elasticsearch or
OpenSearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
{
}
"canUpdate": boolean // Return whether the current user can update this
resource.
}
```

Example response

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "vectorDatabaseType": {},
  "canUpdate": false
}
```

Update a knowledge base

PUT

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

id

V

string

A UUID string identifying this Knowledge Base.

Request body

Request parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; default: 12; minimum: 1
enableHyde	boolean	No	Enable Hyde; default: False
similarityCutoff	number (double)	No	Similarity threshold; default: 0.0; range: 0.0–1.0
enableRerank	boolean	No	Whether to enable reranking; default: True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. <code>elasticsearch</code> : Elasticsearch ; <code>opensearch</code> : OpenSearch ; <code>oracle</code> : Oracle AI

Field	Type	Required	Description
			Database 26ai ; synxdb : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access the knowledge base

Request schema example

```
{
  "organization?": string (uuid) // optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // optional
  "numberOfRetrievedChunks?": integer // Number of reference chunks to
```

```

retrieve; default: 12; minimum: 1 (optional)
  "enableHyde"?: boolean // Enable HyDE; default: False (optional)
  "similarityCutoff"?: number (double) // Similarity threshold; default:
0.0; range: 0.0–1.0 (optional)
  "enableRerank"?: boolean // Whether to enable reranking; default: True
(optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
  "oracleUser"?: string // Oracle database username. (optional)
  "oraclePassword"?: string // Oracle database password. (optional)
  "synxdbUser"?: string // SynxDB database username. (optional)
  "synxdbPassword"?: string // SynxDB database password. (optional)
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Example request

```
{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'
```

```
# Replace YOUR_API_KEY and verify the request data before running  
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```



```
]  
};
```

```
axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-  
e29b-41d4-a716-446655440000/", data, config)  
  .then(response => {  
    console.log('Response received:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request failed:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example name",
    "description": "example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "example string",
    "vectorDatabaseApiKey": "example string",
    "opensearchUsername": "example string",
    "opensearchPassword": "example string",
    "oracleUser": "example string",
    "oraclePassword": "example string",
    "synxdbUser": "example string",
    "synxdbPassword": "example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-
446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "example name",
            "description": "example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "example string",
            "vectorDatabaseApiKey": "example string",
            "opensearchUsername": "example string",
            "opensearchPassword": "example string",
            "oracleUser": "example string",
            "oraclePassword": "example string",
            "synxdbUser": "example string",
            "synxdbPassword": "example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

l);

$data = json_decode($response->getBody(), true);
echo "Response received:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Response schema example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // optional
  "numberOfRetrievedChunks?": integer // Number of reference chunks to
retrieve; default: 12; minimum: 1 (optional)
  "enableHyde?": boolean // Enable HyDE; default: False (optional)
  "similarityCutoff?": number (double) // Similarity threshold; default:

```

```
0.0; range: 0.0–1.0 (optional)
  "enableRerank"?: boolean // Whether to enable reranking; default: True
(optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
{
}
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
  "oracleUser"?: string // Oracle database username. (optional)
  "oraclePassword"?: string // Oracle database password. (optional)
  "synxdbUser"?: string // SynxDB database username. (optional)
  "synxdbPassword"?: string // SynxDB database password. (optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(optional)
    {
      "id": string (uuid)
```

```

        "name": string
    }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "response string",
  "vectorDatabaseApiKey": "response string",
  "opensearchUsername": "response string",
  "opensearchPassword": "response string",
  "oracleUser": "response string",
  "oraclePassword": "response string",
  "synxdbUser": "response string",
  "synxdbPassword": "response string",
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string",
"updatedAt": "response string"
}
```

Partially Update Knowledge Base

PATCH

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Request Body

Request Parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	No	
rerankerModel	string (uuid)	No	
name	string	No	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of reference items retrieved; defaults to 12, minimum 1
enableHyde	boolean	No	Enable HyDE; defaults to False
similarityCutoff	number (double)	No	Similarity threshold; defaults to 0.0, range 0.0–1.0
enableRerank	boolean	No	Whether to enable reranking; defaults to True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. <code>elasticsearch</code> : Elasticsearch ; <code>opensearch</code> : OpenSearch ; <code>oracle</code> : Oracle AI Database 26ai ; <code>synxdb</code> : SynxDB;

Field	Type	Required	Description
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access the knowledge base

Request Schema Example

```
{
  "organization"?: string (uuid) // optional
  "embeddingModel"?: string (uuid) // optional
  "rerankerModel"?: string (uuid) // optional
  "name"?: string // optional
  "description"?: string // optional
  "numberOfRetrievedChunks"?: integer // Number of reference items
retrieved; defaults to 12, minimum 1 (optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (optional)
```

```

    "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, range 0.0–1.0 (optional)
    "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (optional)
    "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
    "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
    {
    }
    "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
    "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
    "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
    "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
    "oracleUser"?: string // Oracle database username. (optional)
    "oraclePassword"?: string // Oracle database password. (optional)
    "synxdbUser"?: string // SynxDB database username. (optional)
    "synxdbPassword"?: string // SynxDB database password. (optional)
    "chatbots"?: [ // optional
    {
        "id": string (uuid)
    }
    ]
    "groups"?: [ // List of groups that can access the knowledge base
(optional)
    {
        "id": string (uuid)
    }
    ]
}

```

Example Request Value

```

{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

Code Examples

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'
```

```
# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "description": "example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "example string",
  "vectorDatabaseApiKey": "example string",
  "opensearchUsername": "example string",
  "opensearchPassword": "example string",
  "oracleUser": "example string",
  "oraclePassword": "example string",
  "synxdbUser": "example string",
  "synxdbPassword": "example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
]  
};
```

```
axios.patch("https://api.maiagent.ai/api/v1/knowledge-  
bases/550e8400-e29b-41d4-a716-446655440000/", data, config)  
  .then(response => {  
    console.log('Response received:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request failed:');  
    console.error(error.response?.data || error.message);  
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "description": "example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "example string",
    "vectorDatabaseApiKey": "example string",
    "opensearchUsername": "example string",
    "opensearchPassword": "example string",
    "oracleUser": "example string",
    "oraclePassword": "example string",
    "synxdbUser": "example string",
    "synxdbPassword": "example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
}
```

```
response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example name",
            "description": "example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "example string",
            "vectorDatabaseApiKey": "example string",
            "opensearchUsername": "example string",
            "opensearchPassword": "example string",
            "oracleUser": "example string",
            "oraclePassword": "example string",
            "synxdbUser": "example string",
            "synxdbPassword": "example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

l);

$data = json_decode($response->getBody(), true);
echo "Response received:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // optional
  "numberOfRetrievedChunks?": integer // Number of reference items
retrieved; defaults to 12, minimum 1 (optional)
  "enableHyde?": boolean // Enable HyDE; defaults to False (optional)
  "similarityCutoff?": number (double) // Similarity threshold; defaults

```

```
to 0.0, range 0.0–1.0 (optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
  "oracleUser"?: string // Oracle database username. (optional)
  "oraclePassword"?: string // Oracle database password. (optional)
  "synxdbUser"?: string // SynxDB database username. (optional)
  "synxdbPassword"?: string // SynxDB database password. (optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(optional)
    {
      "id": string (uuid)
```

```

    "name": string
  }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "response string",
  "vectorDatabaseApiKey": "response string",
  "opensearchUsername": "response string",
  "opensearchPassword": "response string",
  "oracleUser": "response string",
  "oraclePassword": "response string",
  "synxdbUser": "response string",
  "synxdbPassword": "response string",
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string",
"updatedAt": "response string"
}
```

Delete Knowledge Base

DELETE

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Check RRF Availability

GET

/api/v1/knowledge-bases/{id}/check-rrf-availability/

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/check-rrf-availability/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "createdBy":
```

```

{
  "id": string (uuid)
  "name": string
}
"organization"?: string (uuid) // optional
"embeddingModel": string (uuid)
"rerankerModel": string (uuid)
"name": string
"description"?: string // optional
"numberOfRetrievedChunks"?: integer // Number of reference items
retrieved; defaults to 12, minimum 1 (optional)
"enableHyde"?: boolean // Enable HyDE; defaults to False (optional)
"similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, range 0.0–1.0 (optional)
"enableRerank"?: boolean // Whether to enable reranking; defaults to
True (optional)
"enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
"vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
{
}
"vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
"vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (optional)
"opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(optional)
"opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(optional)
"oracleUser"?: string // Oracle database username. (optional)
"oraclePassword"?: string // Oracle database password. (optional)
"synxdbUser"?: string // SynxDB database username. (optional)
"synxdbPassword"?: string // SynxDB database password. (optional)
"labels": [
  {
    "id": string (uuid)

```



```

        "name": string
    }
]
"chatbots"?: [ // optional
    {
        "id": string (uuid)
        "name": string
    }
]
"groups"?: [ // List of groups that can access the knowledge base
(optional)
    {
        "id": string (uuid)
        "name": string
    }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response string",
  "description": "Example response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,

```

```
"enableRerank": false,
"enableRrf": false,
"vectorDatabaseType": {},
"vectorDatabaseUrl": "Example response string",
"vectorDatabaseApiKey": "Example response string",
"opensearchUsername": "Example response string",
"opensearchPassword": "Example response string",
"oracleUser": "Example response string",
"oraclePassword": "Example response string",
"synxdbUser": "Example response string",
"synxdbPassword": "Example response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Example response string",
"updatedAt": "Example response string"
}
```

Export labels to Excel

GET**/api/v1/knowledge-bases/{id}/export-labels-excel/**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-labels-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/export-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	Excel file download

Import labels from Excel

POST

`/api/v1/knowledge-bases/{id}/import-labels-excel/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "excelFile=@example_file.txt"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('excelFile',
* actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-labels-excel/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare the file and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
    'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "success"?: boolean // optional
  "updated_count"?: integer // optional
  "total_rows"?: integer // optional
  "errors"?: [ // optional
    {
      "row"?: integer // optional
      "file_id"?: string // optional
      "error"?: string // optional
    }
  ]
}
```

Example response values

```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Example response string",
      "error": "Example response string"
    }
  ]
}
```

Export metadata to Excel

GET

</api/v1/knowledge-bases/{id}/export-metadata-excel/>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-metadata-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/export-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	Excel file download

Import metadata from Excel

POST

/api/v1/knowledge-bases/{id}/import-metadata-excel/

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "excelFile=@example_file.txt"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('excelFile',
* actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-metadata-excel/", formData,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare the file and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
    'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "success"?: boolean // optional
  "updated_count"?: integer // optional
  "total_rows"?: integer // optional
  "errors"?: [ // optional
    {
      "row"?: integer // optional
      "file_id"?: string // optional
      "error"?: string // optional
    }
  ]
}
```

Example response values

```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Example response string",
      "error": "Example response string"
    }
  ]
}
```

Create a new label

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/labels/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	

Example request schema

```
{
  "name": string
}
```

Example request values

```
{
  "name": "Example name"
}
```

Code examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name"
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name"
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "name"?: string // optional
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response string"
}
```

List labels in a knowledge base

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/`

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

Code examples

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/?
page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
  "next?": string (uri) // optional
}
```

```
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "name"?: string // optional
  }
]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ]
}
```

Get Label Details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.

knowledgeBasePk

V

string

Code Examples

Shell/Bash

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"name"?: string // optional
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string"
}
```

Update a Label

PUT

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Request Schema Example

```
{
  "name": string
}
```

Example Request Value

```
{
  "name": "Example Name"
}
```

Code Examples

Shell/Bash

```
# API request example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "name": "Example Name"
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example Name"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example


```
{
  "id": string (uuid)
  "name"?: string // optional
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string"
}
```

Partially Update a Label

PATCH

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	

Request Schema Example

```
{
  "name"?: string // optional
}
```

Example Request Value

```
{
  "name": "Example Name"
}
```

Code Examples

```
# API request example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "name": "Example Name"
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example Name"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: string // optional
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string"
}
```

Delete a Label

DELETE

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API request example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response Body

Status Code	Description
204	No response body

Create a Metadata Field

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
keyName	string	Yes	Metadata key name
description	string	No	Describe the purpose of this field
isRequired	boolean	No	Whether this field is required
defaultValue	string	No	Default value for new files
enabledForSearch	boolean	No	Whether to use this field during retrieval

Request Schema Example

```
{
  "keyName": string // Metadata key name
  "description"?: string // Describe the purpose of this field (optional)
  "isRequired"?: boolean // Whether this field is required (optional)
  "defaultValue"?: string // Default value for new files (optional)
  "enabledForSearch"?: boolean // Whether to use this field during
```

```
retrieval (optional)
}
```

Example Request Value

```
{
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
}
```

Code Examples

```
# API request example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "keyName": "example string",
    "description": "example string",
    "isRequired": true,
    "defaultValue": "example string",
    "enabledForSearch": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "keyName": "example string",
            "description": "example string",
            "isRequired": true,
            "defaultValue": "example string",
            "enabledForSearch": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined from the field name
  (default or custom field)
  "description"?: string // Describe the purpose of this field (optional)
  "isRequired"?: boolean // Whether this field is required (optional)
  "defaultValue"?: string // Default value for new files (optional)
  "enabledForSearch"?: boolean // Whether to use this field during
  retrieval (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "keyName": "response string",
  "keyType": "response string",
  "description": "response string",
  "isRequired": false,
  "defaultValue": "response string",
  "enabledForSearch": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

List Metadata Fields

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	boolean	Set to false to return all keys without pagination (defaults to true).

Code Examples

Shell/Bash

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1&pagination=true"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
}
```

```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "knowledgeBase":
    {
      "id": string (uuid)
      "name": string
    }
    "keyName": string // Metadata key name
    "keyType": string // Automatically determined from the field name
(default or custom field)
    "description"?: string // Description of this field's purpose
(optional)
    "isRequired"?: boolean // Whether this field is required (optional)
    "defaultValue"?: string // Default value for new files (optional)
    "enabledForSearch"?: boolean // Whether to use this field during
retrieval (optional)
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "keyName": "response string",
      "keyType": "response string",
      "description": "response string",
      "isRequired": false,
      "defaultValue": "response string",
      "enabledForSearch": false,
    }
  ]
}

```



```
    "createdAt": "response string",
    "updatedAt": "response string"
  }
]
```

Get a Specific Metadata Field

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Metadata Key.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```

"knowledgeBase":
{
  "id": string (uuid)
  "name": string
}
"keyName": string // Metadata key name
"keyType": string // Automatically determined from the field name
(default or custom field)
"description?": string // Description of this field's purpose
(optional)
"isRequired?": boolean // Whether this field is required (optional)
"defaultValue?": string // Default value for new files (optional)
"enabledForSearch?": boolean // Whether to use this field during
retrieval (optional)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "keyName": "response string",
  "keyType": "response string",
  "description": "response string",
  "isRequired": false,
  "defaultValue": "response string",
  "enabledForSearch": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}

```

Update a Metadata Field

PUT**/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}**

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Metadata Key.
<code>knowledgeBasePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
<code>keyName</code>	string	Yes	Metadata key name
<code>description</code>	string	No	Description of this field's purpose
<code>isRequired</code>	boolean	No	Whether this field is required
<code>defaultValue</code>	string	No	Default value for new files
<code>enabledForSearch</code>	boolean	No	Whether to use this field during retrieval

Request Schema Example

```
{
  "keyName": string // Metadata key name
  "description"?: string // Description of this field's purpose
  (optional)
  "isRequired"?: boolean // Whether this field is required (optional)
  "defaultValue"?: string // Default value for new files (optional)
  "enabledForSearch"?: boolean // Whether to use this field during
```

```
retrieval (optional)
}
```

Example Request Value

```
{
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
}
```

Code Examples


```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "keyName": "example string",
  "description": "example string",
  "isRequired": true,
  "defaultValue": "example string",
  "enabledForSearch": true
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "keyName": "example string",
    "description": "example string",
    "isRequired": true,
    "defaultValue": "example string",
    "enabledForSearch": true
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "keyName": "example string",
            "description": "example string",
            "isRequired": true,
            "defaultValue": "example string",
            "enabledForSearch": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined from the field name
  (default or custom field)
  "description"?: string // Description of this field's purpose
  (optional)
  "isRequired"?: boolean // Whether this field is required (optional)
  "defaultValue"?: string // Default value for new files (optional)
  "enabledForSearch"?: boolean // Whether to use this field during
  retrieval (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "keyName": "response string",
  "keyType": "response string",
  "description": "response string",
  "isRequired": false,
  "defaultValue": "response string",
  "enabledForSearch": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

Upload Files to the Knowledge Base

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/files/

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	
fileType	X	string	Filter by file type, such as pdf, docx, txt, or xlsx
knowledgeBase	X	string	Knowledge base ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search keywords; supports multiple space-separated conditions. Searchable fields: file ID, filename, label name, and metadata value
status	X	string	Filter by file status. Options: initial (initialized), processing (processing), done (completed), deleting (deleting), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Request Body

Request Parameters

Field	Type	Required	Description
files	array[KnowledgeBaseFileCreate]	Yes	Supports batch uploads of up to 50 files per request

Request Schema Example

```
{
  "files": [ // Supports batch uploads of up to 50 files per request
    {
      "filename": string // Filename
      "file": string (uri) // File to upload
      "parser"?: string (uuid) // optional
      "labels"?: [ // optional
        {
          "id": string (uuid)
        }
      ]
      "rawUserDefineMetadata"?: object // optional
      "asrOptions"?: object // Per-file canonical transcription options
      (languages, prompt, diarization). Empty means no per-file options;
      provider defaults apply. Completely decoupled from user-defined metadata;
      never indexed for search. (optional)
    }
  ]
}
```

Example Request Value

```
{
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
```

```
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
}
```

Code Examples


```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
```

```
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```

import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "files": [
        {
            "filename": "my-file.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
            "labels": [
                {
                    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                    "name": "my-label"
                }
            ],
            "rawUserDefineMetadata": {
                "key1": "value1",
                "key2": "value2"
            }
        },
        {
            "filename": "you-can-upload-multiple-files.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")

```

```
print(response.json())  
except Exception as e:  
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "files": [
                {
                    "filename": "my-file.pdf",
                    "file": "string",
                    "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                    "labels": [
                        {
                            "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                            "name": "my-label"
                        }
                    ],
                    "rawUserDefineMetadata": {
                        "key1": "value1",
                        "key2": "value2"
                    }
                },
                {
                    "filename": "you-can-upload-multiple-files.pdf",
                    "file": "string",
                    "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
                }
            ]
        }
    ]);
}

```

```

        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 201

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {

```

```

"parser":
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "isTimestampSttProvider": boolean
}
"labels"?: [ // optional
{
  "id": string (uuid)
  "name": string
}
]
"rawUserDefineMetadata"?: object // optional
"speakerLabels": [
{
  "id": string (uuid)
  "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
  "customName"?: string // User-defined speaker name (e.g., John)
(optional)
  "displayName": string
}
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from
this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the
URL of a previewable PDF derivative (displayed in the frontend PDF
viewer); otherwise None.

Generate the URL with the same CustomizedFileFieldSerializer used by the
`file` field to ensure consistent formatting (including the
storage provider presign/host rules). Do not use get_absolute_url()
because it produces an incorrect URL format.

"createdAt": string (timestamp)
}

```



```
]
}
```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "response string",
          "customName": "response string",
          "displayName": "response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
    }
  ]
}
```

```
    "waitingTime": 456,  
    "processingTime": 456,  
    "processingTimeDetails": null,  
    "previewUrl": "response string",  
    "createdAt": "response string"  
  }  
]  
}
```

Batch Delete Files

POST

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-delete/`

Parameters

Parameter name	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
<code>ids</code>	<code>array[string]</code>	Yes	

Example request schema

```
{  
  "ids": [  
    string (uuid)  
  ]  
}
```

```
]
}
```

Example request value

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
-------------	-------------

204

No response body

Batch reparse files

PATCH

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-reparse/`

Parameters

Parameter name	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>fileType</code>	X	string	Filter by file type, such as pdf, docx, txt, or xlsx
<code>knowledgeBase</code>	X	string	Knowledge base ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	Search keywords. Supports multiple space-separated criteria. Searchable fields: file ID, filename, label name, and metadata value
<code>status</code>	X	string	Filter by file status. Values: initial (initializing), processing (in progress), done (completed), deleting (being deleted), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '[]'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = [];

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = []

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/files/batch-reparse/?
    fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
    446655440000&page=1&pageSize=1&query=example&status=delete_failed",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => []
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
      ]
    }
  ]
}

```

```

    ]
    "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
    "chunksCount": integer // Number of chunks/nodes generated from
this file
    "waitingTime": number (double)
    "processingTime": number (double)
    "processingTimeDetails": object
    "previewUrl": string // For presentations (pptx/ppt), return the
URL of a previewable derived PDF (displayed by the frontend PDF viewer);
otherwise None.

Generate the URL with the same CustomizedFileFieldSerializer used for the
`file` field to keep the format consistent (including each
storage provider's presign/host rules). Do not use get_absolute_url()
because it produces an incorrect URL format.
    "createdAt": string (timestamp)
    }
  ]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "provider": {},
        "isTimestampSttProvider": false
      }
    }
  ]
}

```

```
    },
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "speakerLabels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "originalLabel": "response string",
        "customName": "response string",
        "displayName": "response string"
      }
    ],
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "waitingTime": 456,
    "processingTime": 456,
    "processingTimeDetails": null,
    "previewUrl": "response string",
    "createdAt": "response string"
  }
]
}
```

List files in a knowledge base

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/`

Parameters

Parameter name	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

<code>fileType</code>	X	string	Filter by file type, such as pdf, docx, txt, or xlsx
<code>knowledgeBase</code>	X	string	Knowledge base ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	Search keywords. Supports multiple space-separated criteria. Searchable fields: file ID, filename, label name, and metadata value
<code>status</code>	X	string	Filter by file status. Values: initial (initializing), processing (in progress), done (completed), deleting (being deleted), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
      ]
    }
  ]
}

```

```

    ]
    "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
    "chunksCount": integer // Number of chunks/nodes generated from
this file
    "waitingTime": number (double)
    "processingTime": number (double)
    "processingTimeDetails": object
    "previewUrl": string // For presentations (pptx/ppt), return the
URL of a previewable derived PDF (displayed by the frontend PDF viewer);
otherwise None.

Generate the URL with the same CustomizedFileFieldSerializer used for the
`file` field to keep the format consistent (including each
storage provider's presign/host rules). Do not use get_absolute_url()
because it produces an incorrect URL format.
    "createdAt": string (timestamp)
    }
  ]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "provider": {},
        "isTimestampSttProvider": false
      }
    }
  ]
}

```

```
    },
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "speakerLabels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "originalLabel": "response string",
        "customName": "response string",
        "displayName": "response string"
      }
    ],
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "waitingTime": 456,
    "processingTime": 456,
    "processingTimeDetails": null,
    "previewUrl": "response string",
    "createdAt": "response string"
  }
]
```

Get details of a specific file

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Knowledge Base File.
----	---	--------	---

knowledgeBasePk	V	string
-----------------	---	--------

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```


Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```
"filename": string // Filename
"file": string (uri) // File to upload
"fileType": string
"knowledgeBase"?: // Optional
{
  "id": string (uuid)
  "name": string
}
"size": integer
"status":
{
}
"parser":
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "isTimestampSttProvider": boolean
}
"labels"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Optional
"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
```

```
"previewUrl": string // For presentations (pptx/ppt), return the URL of
a previewable derived PDF (displayed by the frontend PDF viewer);
otherwise None.
```

Generate the URL with the same CustomizedFileFieldSerializer used for the `file` field to keep the format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()` because it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
  ]
}
```

```
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "Response string",
"createdAt": "Response string"
}
```

Get Details of a Specific File

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/move-options/`

Parameters

Parameter Name	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/move-options/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string
}
```



```

    "size": integer
    "status"?: string (enum: initial, processing, done, deleting, deleted,
failed, delete_failed) // * `initial` - Initial
* `processing` - Processing
* `done` - Done
* `deleting` - Deleting
* `deleted` - Deleted
* `failed` - Failed
* `delete_failed` - Delete Failed (optional)
    "chunksCount"?: integer // Number of chunks/nodes generated from this
file (optional)
    "labels": [
        {
            "id": string (uuid)
            "name": string
        }
    ]
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "size": 456,
  "status": "initial",
  "chunksCount": 456,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ]
}

```

Get the Latest File Failure Reason

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/latest-failed-reason/

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base File.
knowledgeBasePk	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/latest-failed-reason/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"errorMessage": string
  "errorCode"?: string // Machine-readable code of a known failure type
  (see FAILED_REASON_LOCALIZED_MESSAGES in whizchat/parsers/exceptions.py);
  the serializer translates it into the request language on read. An empty
  string means no translation, and error_message is shown as is. (optional)
  "createdAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "errorMessage": "Response string",
  "errorCode": "Response string",
  "createdAt": "Response string"
}
```

Get File Transcription

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/transcription/`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/transcription/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/transcription/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"filename": string // File name
"file": string (uri) // File to upload
"fileType": string
"knowledgeBase"?: // optional
{
  "id": string (uuid)
  "name": string
}
"size": integer
"status":
{
}
"parser":
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "isTimestampSttProvider": boolean
}
"labels"?: [ // optional
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // optional
"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John)
(optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
```

```
"previewUrl": string // For presentations (pptx/ppt), returns the URL
of a previewable PDF derivative (displayed in the frontend PDF viewer);
otherwise None.
```

Generate the URL with the same CustomizedFileFieldSerializer used by the `file` field to keep its format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()`—it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
  ]
}
```

```
] ,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "Response string",
  "createdAt": "Response string"
}
```

Update File Information

PUT

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base File.
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
filename	string	Yes	File name
file	string (uri)	Yes	File to upload

Field	Type	Required	Description
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
asrOptions	object	No	Per-file canonical transcription options (languages, prompt, diarization). Empty means no per-file options: provider defaults apply. Completely decoupled from user-defined metadata; never indexed for ...

Request Schema Example

```
{
  "filename": string // File name
  "file": string (uri) // File to upload
  "parser"?: string (uuid) // optional
  "labels"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "asrOptions"?: object // Per-file canonical transcription options
(languages, prompt, diarization). Empty means no per-file options:
provider defaults apply. Completely decoupled from user-defined metadata;
never indexed for search. (optional)
}
```

Example Request Value

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
    }  
  ],  
  "rawUserDefineMetadata": null,  
  "asrOptions": null  
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)  
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-  
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-  
a716-446655440000/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "filename": "document.pdf",  
  "file": "https://example.com/file.jpg",  
  "parser": "550e8400-e29b-41d4-a716-446655440000",  
  "labels": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000"  
    }  
  ],  
  "rawUserDefineMetadata": null,  
  "asrOptions": null  
}'  
  
# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "parser": "550e8400-e29b-41d4-a716-446655440000",
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null,
            "asrOptions": null
        }
    ]
);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
      (e.g., SPEAKER_00)
```

```

    "customName"?: string // User-defined speaker name (e.g., John)
(optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL
of a previewable PDF derivative (displayed in the frontend PDF viewer);
otherwise None.

Generate the URL with the same CustomizedFileFieldSerializer used by the
`file` field to keep its format consistent (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
produces an incorrect URL format.
"createdAt": string (timestamp)
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "Response string",
    "customName": "Response string",
    "displayName": "Response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "Response string",
"createdAt": "Response string"
}
```

Partially Update a File

PATCH

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
filename	string	No	File name
file	string (uri)	No	File to upload
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
asrOptions	object	No	Per-file canonical transcription options (languages, prompt, diarization). Empty means no per-file options: provider defaults apply. Completely decoupled from user-defined metadata; never indexed for ...

Example request schema

```
{
  "filename"?: string // File name (optional)
  "file"?: string (uri) // File to upload (optional)
  "parser"?: string (uuid) // optional
  "labels"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "asrOptions"?: object // Per-file canonical transcription options
(languages, prompt, diarization). Empty means no per-file options:
provider defaults apply. Completely decoupled from user-defined metadata;
never indexed for search. (optional)
}
```

Example request value

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
}
```

Code examples

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
}'

# Replace YOUR_API_KEY and check the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "parser": "550e8400-e29b-41d4-a716-446655440000",
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null,
            "asrOptions": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
      (e.g., SPEAKER_00)
    }
  ]
}
```

```

        "customName"?: string // User-defined speaker name (e.g., John)
(optional)
        "displayName": string
    }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL
of a previewable PDF derivative (displayed with the PDF viewer in the
frontend); otherwise None.

Generate the URL with the same CustomizedFileFieldSerializer used for the
`file` field to keep its format consistent (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
produces an incorrect URL format.
    "createdAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "fileType": "response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "response string",
    "customName": "response string",
    "displayName": "response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "response string",
"createdAt": "response string"
}
```

Delete a file

DELETE

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status code	Description
204	No response body

Create a file translation

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/

Parameters

Parameter name	Required	Type	Description
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
sourceLanguage	string	No	
targetLanguage	string	Yes	
customPrompt	string	No	
largeLanguageModelId	string (uuid)	Yes	

Example request schema

```
{
  "sourceLanguage"?: string // optional
  "targetLanguage": string
  "customPrompt"?: string // optional
  "largeLanguageModelId": string (uuid)
}
```

Example request value

```
{
  "sourceLanguage": "example string",
  "targetLanguage": "example string",
  "customPrompt": "example string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "sourceLanguage": "example string",
  "targetLanguage": "example string",
  "customPrompt": "example string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}'

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "sourceLanguage": "example string",
  "targetLanguage": "example string",
  "customPrompt": "example string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/",
data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceLanguage": "example string",
    "targetLanguage": "example string",
    "customPrompt": "example string",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY',
        'Content-Type' => 'application/json'
    ],
    'json' => {
        "sourceLanguage": "example string",
        "targetLanguage": "example string",
        "customPrompt": "example string",
        "largeLanguageModelId": "550e8400-e29b-41d4-a716-
446655440000"
    }
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```
{
  "sourceLanguage?": string // optional
  "targetLanguage": string
  "customPrompt?": string // optional
  "largeLanguageModelId": string (uuid)
}
```

Example response value

```
{
  "sourceLanguage": "response string",
  "targetLanguage": "response string",
  "customPrompt": "response string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

List file translations

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/`

Parameters

Parameter name	Required	Type	Description
<code>knowledgeBaseFilePk</code>	V	string	
<code>knowledgeBasePk</code>	V	string	

page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?  
page=1&pageSize=1"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
}
```

```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "chatbotFileId": string (uuid)
    "chatbotFileFilename": string
    "organization": string (uuid)
    "largeLanguageModel":
    {
      "id": string (uuid)
      "name": string
    }
    "sourceLanguage": string
    "targetLanguage": string
    "customPrompt": string // User-provided terminology/glossary rules
    "status":
    {
    }
    "errorMessage": string
    "translatedFileUrl": string
    "bilingualFileUrl": string
    "processDuration": string
    "processingDurationSeconds": number (double)
    "pageCount": integer
    "originalWordCount": integer
    "translatedWordCount": integer
    "inputTokensUsed": integer
    "outputTokensUsed": integer
    "totalTokensUsed": integer
    "originalFileSize": integer
    "translatedFileSize": integer
    "bilingualFileSize": integer
    "llmTranslationDuration": string // Total time spent on LLM API
calls
    "llmTranslationDurationSeconds": number (double)
    "llmCallCount": integer // Number of LLM API calls made
    "avgLlmCallDuration": number (double) // Average time per LLM call
in seconds
    "pdfParsingDuration": string // Time spent on PDF parsing and
processing
    "pdfParsingDurationSeconds": number (double)
    "fileSavingDuration": string // Time spent on saving output files
    "fileSavingDurationSeconds": number (double)
    "startedAt": string (timestamp)

```

```
    "completedAt": string (timestamp)
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileFilename": "response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "largeLanguageModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "sourceLanguage": "response string",
      "targetLanguage": "response string",
      "customPrompt": "response string",
      "status": {},
      "errorMessage": "response string",
      "translatedFileUrl": "response string",
      "bilingualFileUrl": "response string",
      "processDuration": "response string",
      "processingDurationSeconds": 456,
      "pageCount": 456,
      "originalWordCount": 456,
      "translatedWordCount": 456,
      "inputTokensUsed": 456,
      "outputTokensUsed": 456,
      "totalTokensUsed": 456,
      "originalFileSize": 456,
      "translatedFileSize": 456,
      "bilingualFileSize": 456,
      "llmTranslationDuration": "response string",
      "llmTranslationDurationSeconds": 456,
      "llmCallCount": 456,
    }
  ]
}
```

```
    "avgLlmCallDuration": 456,
    "pdfParsingDuration": "response string",
    "pdfParsingDurationSeconds": 456,
    "fileSavingDuration": "response string",
    "fileSavingDurationSeconds": 456,
    "startedAt": "response string",
    "completedAt": "response string",
    "createdAt": "response string",
    "updatedAt": "response string"
  }
]
```

Retrieve a translation

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this File Translation.
<code>knowledgeBaseFilePk</code>	V	string	
<code>knowledgeBasePk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```

"chatbotFileId": string (uuid)
"chatbotFileFilename": string
"organization": string (uuid)
"largeLanguageModel":
{
  "id": string (uuid)
  "name": string
}
"sourceLanguage": string
"targetLanguage": string
"customPrompt": string // User-provided terminology/glossary rules
"status":
{
}
"errorMessage": string
"translatedFileUrl": string
"bilingualFileUrl": string
"processDuration": string
"processingDurationSeconds": number (double)
"pageCount": integer
"originalWordCount": integer
"translatedWordCount": integer
"inputTokensUsed": integer
"outputTokensUsed": integer
"totalTokensUsed": integer
"originalFileSize": integer
"translatedFileSize": integer
"bilingualFileSize": integer
"llmTranslationDuration": string // Total time spent on LLM API calls
"llmTranslationDurationSeconds": number (double)
"llmCallCount": integer // Number of LLM API calls made
"avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
"pdfParsingDuration": string // Time spent on PDF parsing and
processing
"pdfParsingDurationSeconds": number (double)
"fileSavingDuration": string // Time spent on saving output files
"fileSavingDurationSeconds": number (double)
"startedAt": string (timestamp)
"completedAt": string (timestamp)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete translation

DELETE

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this File Translation.
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.delete(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

Download translated file

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}/download/

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this File Translation.
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/download/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/download/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```

"chatbotFileId": string (uuid)
"chatbotFileFilename": string
"organization": string (uuid)
"largeLanguageModel":
{
  "id": string (uuid)
  "name": string
}
"sourceLanguage": string
"targetLanguage": string
"customPrompt": string // User-provided terminology/glossary rules
"status":
{
}
"errorMessage": string
"translatedFileUrl": string
"bilingualFileUrl": string
"processDuration": string
"processingDurationSeconds": number (double)
"pageCount": integer
"originalWordCount": integer
"translatedWordCount": integer
"inputTokensUsed": integer
"outputTokensUsed": integer
"totalTokensUsed": integer
"originalFileSize": integer
"translatedFileSize": integer
"bilingualFileSize": integer
"llmTranslationDuration": string // Total time spent on LLM API calls
"llmTranslationDurationSeconds": number (double)
"llmCallCount": integer // Number of LLM API calls made
"avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
"pdfParsingDuration": string // Time spent on PDF parsing and
processing
"pdfParsingDurationSeconds": number (double)
"fileSavingDuration": string // Time spent on saving output files
"fileSavingDurationSeconds": number (double)
"startedAt": string (timestamp)
"completedAt": string (timestamp)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List knowledge base documents

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/

Parameters

Parameter name	Required	Type	Description
knowledgeBasePk	V	string	
chatbotFileId	X	string	[Deprecated] File ID (use knowledge_base_file_id)
knowledgeBaseFileId	X	string	Knowledge base file ID

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/?chatbotFileId=example&knowledgeBaseFileId=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/?chatbotFileId=example&knowledgeBaseFileId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/?chatbotFileId=example&knowledgeBaseFileId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
[
  {
```

```

    "id": string (uuid)
    "fileName": string
    "pageNumber"?: integer // optional
    "chatbotFile": string (uuid)
    "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (optional)
    "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (optional)
    "llamaIndexObjectJson"?: object // optional
    "metadata"?: object // optional
    "text": string
    "updatedAt": string (timestamp)
  }
]

```

Example response value

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "fileName": "Response string",
    "pageNumber": 456,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "Response string",
    "updatedAt": "Response string"
  }
]

```

Status code: 404 - The knowledge base file does not exist or does not belong to the current organization

Update document

PUT**/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}**

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Chatbot Document.
<code>knowledgeBasePk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
fileName	string	Yes	
pageNumber	integer	No	
chatbotFile	string (uuid)	Yes	
excludedEmbedMetadataKeys	object	No	List[str]: metadata keys to exclude from the embedding.
excludedLLmMetadataKeys	object	No	List[str]: metadata keys to exclude from the LLM context.
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	Yes	

Example request schema

```
{
  "fileName": string
  "pageNumber"?: integer // optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (optional)
  "llamaIndexObjectJson"?: object // optional
  "metadata"?: object // optional
  "text": string
}
```

Example request value

```
{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "fileName": "Example string",
            "pageNumber": 123,
            "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
            "excludedEmbedMetadataKeys": null,
            "excludedLlmMetadataKeys": null,
            "llamaIndexObjectJson": null,
            "metadata": null,
            "text": "This is an example message"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (optional)
  "llamaIndexObjectJson"?: object // optional
  "metadata"?: object // optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "response string",
  "updatedAt": "response string"
}
```

Partially Update a Document

PATCH

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Document.
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
fileName	string	No	
pageNumber	integer	No	
chatbotFile	string (uuid)	No	
excludedEmbedMetadataKeys	object	No	List[str]: metadata keys to exclude from the embedding.
excludedLlmMetadataKeys	object	No	List[str]: metadata keys to exclude from the LLM context.
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	No	

Request Schema Example

```
{
  "fileName?": string // optional
  "pageNumber?": integer // optional
  "chatbotFile?": string (uuid) // optional
  "excludedEmbedMetadataKeys?": object // List[str]: metadata keys to
exclude from the embedding. (optional)
  "excludedLlmMetadataKeys?": object // List[str]: metadata keys to
exclude from the LLM context. (optional)
  "llamaIndexObjectJson?": object // optional
  "metadata?": object // optional
  "text?": string // optional
}
```

Example Request Value

```
{
  "fileName": "example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code Examples

```
# Example API Call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "fileName": "example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "fileName": "example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "fileName": "example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "fileName": "example string",
            "pageNumber": 123,
            "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
            "excludedEmbedMetadataKeys": null,
            "excludedLlmMetadataKeys": null,
            "llamaIndexObjectJson": null,
            "metadata": null,
            "text": "This is an example message"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (optional)
  "llamaIndexObjectJson"?: object // optional
  "metadata"?: object // optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "response string",
  "updatedAt": "response string"
}
```

Delete a Document

DELETE

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Document.
knowledgeBasePk	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE "https://api.maiaagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response Body

Status Code	Description
204	No response body

Create a New FAQ

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request Schema Example

```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
```

```
"labels"?: [ // optional
  {
    "id": string (uuid)
  }
]
"rawUserDefineMetadata"?: object // optional
}
```

Example Request Value

```
{
  "question": "example string",
  "answer": "example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples


```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "question": "example string",
  "answer": "example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "example string",
  "answer": "example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "example string",
    "answer": "example string",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "example string",
            "answer": "example string",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
  create this FAQ.
  "labels"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "knowledgeBase": // Always taken from the knowledge base in the URL; a
  value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "response string",
  "answer": "response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
}
```

Delete FAQs in Bulk

POST

</api/v1/knowledge-bases/{knowledgeBasePk}/faqs/batch-delete/>

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
ids	array[string]	No	

Request Schema Example

```
{
  "ids"?: [ // optional
    string (uuid)
  ]
}
```

Example Request Value

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
-------------	-------------

204

No response body

List All FAQs in a Specific Knowledge Base

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/`

Parameters

Parameter Name	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	Search keywords. Supports multiple search terms separated by spaces. Searches FAQ ID, question, answer, label name, and metadata value.

Code Examples

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/?
page=1&pageSize=1&query=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // optional
```



```

"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "question": string
    "answer": string
    "answerMediaUrls"?: object // URLs of media files, such as images
and videos, in the answer. (optional)
    "hitsCount": integer
    "embeddingTokensCount": integer // Number of embedding tokens used
to create this FAQ.
    "labels"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // optional
    "knowledgeBase": // Always taken from the knowledge base in the
URL; a value in the request body is ignored.
    {
      "id": string (uuid)
      "name": string
    }
  }
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "Response string",
      "answer": "Response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {

```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
]
```

Get FAQ details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
```

```
"question": string
"answer": string
"answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (optional)
"hitsCount": integer
"embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
"labels"?: [ // optional
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // optional
"knowledgeBase": // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
{
  "id": string (uuid)
  "name": string
}
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

```
}  
}
```

Update FAQ

PUT`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request schema example


```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
  "labels"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // optional
}
```

Example request value

```
{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "Example string",
            "answer": "Example string",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null
        }
    ]
);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
  create this FAQ.
  "labels"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "knowledgeBase": // Always taken from the knowledge base in the URL; a
  value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"knowledgeBase": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
}
```

Partially update FAQ

PATCH

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
question	string	No	
answer	string	No	

Field	Type	Required	Description
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request schema example

```
{
  "question"?: string // optional
  "answer"?: string // optional
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
  "labels"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // optional
}
```

Example request value

```
{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```


Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "Example string",
            "answer": "Example string",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
  videos, in the answer. (optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
  create this FAQ.
  "labels"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // optional
  "knowledgeBase": // Always taken from the knowledge base in the URL; a
  value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

Delete FAQ

DELETE

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status code	Description
204	No response body

Example API Call (Shell)

...

目錄

- 1. List Knowledge Base Files for an AI Assistant
- 2. List Knowledge Base FAQs for an AI Assistant
- 3. List Text Nodes for All AI Assistant Files
- 4. Get Text Nodes for a Specific AI Assistant File
- 5. Test Search
- 6. List Parsers Supported by File Type

Example API Call (Shell)

List Knowledge Base Files for an AI Assistant

GET `/api/v1/chatbots/{id}/knowledge-bases-files/`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	
<code>largeLanguageModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>replyMode</code>	X	string	
<code>source</code>	X	string	<code>`self_built`: self_built; `built_in`: built_in; `all`: all;</code>

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```



```

{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
      "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from
this file
  "waitingTime": number (double)

```

```
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the
URL of a previewable PDF derivative (displayed by the frontend PDF
viewer); otherwise None.
```

Generate the URL with the same CustomizedFileFieldSerializer used for the `file` field to keep the format consistent (including each storage provider's presign/host rules). Do not use `get_absolute_url()`; it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
]
}
```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ]
    }
  ]
}
```

```
    ],
    "rawUserDefineMetadata": null,
    "speakerLabels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "originalLabel": "Response string",
        "customName": "Response string",
        "displayName": "Response string"
      }
    ],
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "waitingTime": 456,
    "processingTime": 456,
    "processingTimeDetails": null,
    "previewUrl": "Response string",
    "createdAt": "Response string"
  }
]
```

List Knowledge Base FAQs for an AI Assistant

GET

`/api/v1/chatbots/{id}/knowledge-bases-faqs/`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	
<code>largeLanguageModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.

pageSize	X	integer	Number of results to return per page.
query	X	string	
replyMode	X	string	
source	X	string	`self_built`:self_built;`built_in`:built_in;`all`:all;

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```

{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images
and videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used
to create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase": // Always taken from the knowledge base in the
URL; a value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "Response string",
      "answer": "Response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ]
    }
  ]
}

```



```
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
]
```

List Text Nodes for All AI Assistant Files

GET

</api/v1/chatbot-text-nodes/>

Parameters

Parameter Name	Required	Type	Description
<code>chatbotFile</code>	X	string	[Deprecated] Chatbot file ID (use knowledge_base_file)
<code>knowledgeBaseFile</code>	X	string	Knowledge base file ID (recommended)
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?\nchatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next?": string (uri) // Optional
}
```

```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "charactersCount": integer
    "hitsCount": integer
    "text": string
    "updatedAt": string (timestamp)
    "filename": string
    "chatbotFile": object // Returns complete ChatbotFile information,
    including the file URL and type, to support image previews in the
    frontend
    "knowledgeBaseFile": object // Same as get_chatbot_file, for
    backward compatibility
    "pageNumber": integer // Backward-compatible alias of
    ``page_start``.
    "pageStart": integer
    "pageEnd": integer
    "citationTitle": string // Source title shown on the inline
    citation hover card.
    "citationDescription": string // Source description shown on the
    inline citation hover card.
    "citationQuote": string // Quoted text snippet shown on the inline
    citation hover card.
    "hasImage": boolean // Whether an image is present (checks the file
    type or Markdown images in text)
    "imageUrl": string // Extracted image URL (prefers the file URL;
    otherwise extracts it from Markdown)
    "displayText": string // Returns the full text with Markdown image
    markup removed
    "displayTitle": string // Returns the display title (fallback:
    citation_title -> filename)
    "highlightedText": string // Return text with matched terms wrapped
    in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```

"labels": [
  {
    "id": string (uuid)
    "name": string
  }
]

```

```
"rawUserDefineMetadata"?: object // Metadata keys and values
defined by the user. (Optional)
```

```
"metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and*

the node carries a value for it. Since this maps over

```
``raw_user_define_metadata``
```

(the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's

knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
    "startCharIdx": integer
    "endCharIdx": integer
  }
]
```

```
}
```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
    },
  ],
}
```

```
"pageNumber": 456,
"pageStart": 456,
"pageEnd": 456,
"citationTitle": "Response string",
"citationDescription": "Response string",
"citationQuote": "Response string",
"hasImage": false,
"imageUrl": "Response string",
"displayText": "Response string",
"displayTitle": "Response string",
"highlightedText": "Response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"metadataEnabledForSearch": {
  "key1": "value1",
  "key2": "value2",
  "createdAt": "2025-01-01T00:00:00.000Z"
},
"startCharIdx": 456,
"endCharIdx": 456
}
]
}
```

Status Code: 403 - Not authorized to read this file's text nodes: knowledge base files require chatbot_knowledge_base_access and group authorization for the knowledge base; conversation attachments require chatbot_chatbot_access for the assistant; inbox files require group authorization for the inbox

Status Code: 404 - The knowledge base file does not exist or does not belong to the current organization

Get Text Nodes for a Specific AI Assistant File

GET /api/v1/chatbot-text-nodes/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this ChatbotTextNode.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-  
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "charactersCount": integer
}
```

```
"hitsCount": integer
"text": string
"updatedAt": string (timestamp)
"filename": string
"chatbotFile": object // Returns complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
"knowledgeBaseFile": object // Same as get_chatbot_file, for backward
compatibility
"pageNumber": integer // Backward-compatible alias of ``page_start``.
"pageStart": integer
"pageEnd": integer
"citationTitle": string // Source title shown on the inline citation
hover card.
"citationDescription": string // Source description shown on the inline
citation hover card.
"citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
"hasImage": boolean // Whether an image is present (checks the file
type or Markdown images in text)
"imageUrl": string // Extracted image URL (prefers the file URL;
otherwise extracts it from Markdown)
"displayText": string // Returns the full text with Markdown image
markup removed
"displayTitle": string // Returns the display title (fallback:
citation_title -> filename)
"highlightedText": string // Return text with matched terms wrapped in
``<mark>`` tags.
```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via
``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Metadata keys and values defined by
the user. (optional)
"metadataEnabledForSearch": object // Map each user-defined metadata
key on this node to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is

```
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.
  "startCharIdx": integer
  "endCharIdx": integer
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "charactersCount": 456,
  "hitsCount": 456,
  "text": "Response string",
  "updatedAt": "Response string",
  "filename": "Response string",
  "chatbotFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "knowledgeBaseFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "pageNumber": 456,
  "pageStart": 456,
  "pageEnd": 456,
  "citationTitle": "Response string",
  "citationDescription": "Response string",
  "citationQuote": "Response string",
  "hasImage": false,
  "imageUrl": "Response string",
  "displayText": "Response string",
  "displayTitle": "Response string",
  "highlightedText": "Response string",
  "labels": [
    {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"metadataEnabledForSearch": {
  "key1": "value1",
  "key2": "value2",
  "createdAt": "2025-01-01T00:00:00.000Z"
},
"startCharIdx": 456,
"endCharIdx": 456
}
```

Test Search

POST

/api/v1/chatbots/{id}/search/

Parameters

Parameter Name	Required	Type	Description
id	V	string	
largeLanguageModel	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
replyMode	X	string	

source

X

string

```
`self_built`: self_built; `built_in`:  
built_in; `all`: all;
```

Request Body

Request Parameters

Field	Type	Required	Description
queryMetadata	object	No	Not supported for chatbot-wide search; providing a value returns 400.

Example Request Schema

```
{  
  "queryMetadata"?: object // Not supported for chatbot-wide search;  
  providing a value returns 400. (optional)  
}
```

Example Request Value

```
{  
  "queryMetadata": null  
}
```

Code Examples


```
# Example API call (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "query": "Example string",
    "queryMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "query": "Example string",
  "queryMetadata": null
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "query": "Example string",
    "queryMetadata": null
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "query": "Example string",
            "queryMetadata": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example Response Schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Return complete ChatbotFile details,
including the file URL and type, for frontend image previews
      "knowledgeBaseFile": object // Same as get_chatbot_file, provided
for backward compatibility
      "pageNumber": integer // Backward-compatible alias of
``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title shown on the inline
citation hover card.
      "citationDescription": string // Source description shown on the
inline citation hover card.
      "citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
      "hasImage": boolean // Whether an image is present (checks the file
type or Markdown images in text)
      "imageUrl": string // Extract the image URL (prefer the file URL;
otherwise extract it from Markdown)
      "displayText": string // Return the full text with Markdown image
markup removed
      "displayTitle": string // Return the display title (fallback:
citation_title -> filename)
      "highlightedText": string // Return text with matched terms wrapped
in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```

  "labels": [
    {
      "id": string (uuid)

```

```

        "name": string
    }
]
"rawUserDefineMetadata"?: object // Metadata keys and values
defined by the user. (optional)
    "metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.

    "startCharIdx": integer
    "endCharIdx": integer
}
]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",

```

```

    "name": "Example response name",
    "description": "Example response description"
  },
  "pageNumber": 456,
  "pageStart": 456,
  "pageEnd": 456,
  "citationTitle": "Response string",
  "citationDescription": "Response string",
  "citationQuote": "Response string",
  "hasImage": false,
  "imageUrl": "Response string",
  "displayText": "Response string",
  "displayTitle": "Response string",
  "highlightedText": "Response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "metadataEnabledForSearch": {
    "key1": "value1",
    "key2": "value2",
    "createdAt": "2025-01-01T00:00:00.000Z"
  },
  "startCharIdx": 456,
  "endCharIdx": 456
}
]
}

```

List Parsers Supported by File Type

GET

</api/v1/parsers/supported-file-types/>

Parameters

Parameter Name	Required	Type	Description
<code>knowledgeBaseId</code>	X	string	Knowledge base ID. When provided and the knowledge base does not support multimodal embeddings, image file types will be filtered out.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example Response Schema

```
[
  {
    "fileType": string
```

```

    "parsers": [
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "order"?: integer // optional
        "isTimestampSttProvider": boolean
        "asrProvider": object // The provider spec, built once per
        provider row per response.

```

The spec is platform-level, not per parser, and it is large – a whisper-family row carries a 100-entry language catalog in its ``languages`` spec, as codes plus display names. This serializer is nested per row in the knowledge-base file listing and in message citations, so without memoizing, one response rebuilds the same catalog for every row. ``self.context`` is the root serializer's shared dict, which is also what keeps ``get\_active\_diarization\_rows`` at one query per response.

```

    }
  ]
  "autoDetect": boolean // Whether files of this type get parser auto-
  detection after upload
  (the system auto-detects a parser when the user did not choose one).
}
]

```

Example Response Value

```

[
  [
    {
      "fileType": ".pdf",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
          "order": 0,
          "isDefault": true
        },
        {
          "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",

```

```
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
    }
]
},
{
    "fileType": ".doc",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiaagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
{
    "fileType": ".docx",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiaagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
```

```
{
  "fileType": ".ppt",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    },
    {
      "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
      "provider": "llama",
      "order": 2,
      "isDefault": false
    }
  ]
},
{
  "fileType": ".pptx",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    },
    {
      "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
      "provider": "llama",
      "order": 2,
      "isDefault": false
    }
  ]
},
{
  "fileType": ".xls",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
```

```
        "order": 0,
        "isDefault": true
    }
]
},
{
    "fileType": ".xlsx",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".csv",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".txt",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
}
```

```
    }
  ]
},
{
  "fileType": ".md",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    }
  ]
},
{
  "fileType": ".json",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    }
  ]
},
{
  "fileType": ".jsonl",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    }
  ]
},
{
  "fileType": ".html",
  "parsers": [
    {
      "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
```



```
        "provider": "llama",
        "order": 2,
        "isDefault": false
    }
]
},
{
    "fileType": ".mp3",
    "parsers": [
        {
            "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
            "name": "Azure Speech",
            "provider": "azure",
            "order": 1,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
{
    "fileType": ".wav",
    "parsers": [
        {
            "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
            "name": "Azure Speech",
            "provider": "azure",
            "order": 1,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
{
```

```
"fileType": ".mp4",
"parsers": [
  {
    "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
    "name": "MaiAgent Parser (Online)",
    "provider": "llama",
    "order": 2,
    "isDefault": false
  }
]
}
```

Example API Call (Shell)

...

目錄

1. Account Registration
2. Change Password
3. Create a New Organization
4. Add a Member to a Specified Organization
5. List Organizations
6. Get information about a specific organization
7. Get details of the current user
8. Get permissions of the current user
9. List members of a specified organization
10. Get details of a specific member
11. Get details of a specific member
12. Get details of a specific member
13. Get details of a specific member
14. Get details of a specific member
15. Update organization information
16. Update current user details
17. Update member roles
18. Delete Organization
19. Delete a Specific Member
20. Get Member Usage Statistics
21. Get Member Usage Statistics Summary
22. Get AI Assistant Usage Statistics
23. Get AI Assistant Usage Statistics Summary

Example API Call (Shell)

Account Registration

POST

`/api/v1/auth/registration/`

Request Body

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
password1	string	Yes	
password2	string	Yes	
name	string	Yes	
company	string	Yes	
referralCode	string	No	
authSource	object (with 2 properties: id, name)	No	
authSource.id	string (uuid)	Yes	

Example Request Schema

```
{
  "email": string (email)
  "password1": string
  "password2": string
  "name": string
  "company": string
  "referralCode?": string // optional
  "authSource?": // optional
  {
    "id": string (uuid)
  }
}
```

Example Request Value

```
{
  "email": "user@example.com",
  "password1": "example string",
  "password2": "example string",
```

```
"name": "Example Name",
"company": "example string",
"referralCode": "example string",
"authSource": {
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/registration/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "email": "user@example.com",
  "password1": "example string",
  "password2": "example string",
  "name": "Example Name",
  "company": "example string",
  "referralCode": "example string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "email": "user@example.com",
  "password1": "example string",
  "password2": "example string",
  "name": "Example Name",
  "company": "example string",
  "referralCode": "example string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
};

axios.post("https://api.maiagent.ai/api/v1/auth/registration/",
data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/auth/registration/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "email": "user@example.com",
    "password1": "example string",
    "password2": "example string",
    "name": "Example Name",
    "company": "example string",
    "referralCode": "example string",
    "authSource": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/auth/registration/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "email": "user@example.com",
            "password1": "example string",
            "password2": "example string",
            "name": "Example Name",
            "company": "example string",
            "referralCode": "example string",
            "authSource": {
                "id": "550e8400-e29b-41d4-a716-446655440000"
            }
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "email": string (email)
  "password1": string
  "password2": string
  "name": string
  "company": string
  "referralCode?": string // optional
  "authSource?": // optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Value

```
{
  "email": "response@example.com",
  "password1": "response string",
  "password2": "response string",
  "name": "response string",
  "company": "response string",
  "referralCode": "response string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
}
```

Change Password

POST

/api/v1/auth/password/change/

Request Body

Request Parameters

Field	Type	Required	Description
oldPassword	string	Yes	
newPassword1	string	Yes	
newPassword2	string	Yes	

Example Request Schema

```
{
  "oldPassword": string
  "newPassword1": string
  "newPassword2": string
}
```

Example Request Value

```
{
  "oldPassword": "example string",
  "newPassword1": "example string",
  "newPassword2": "example string"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/password/change/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "oldPassword": "example string",
  "newPassword1": "example string",
  "newPassword2": "example string"
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "oldPassword": "example string",
  "newPassword1": "example string",
  "newPassword2": "example string"
};

axios.post("https://api.maiagent.ai/api/v1/auth/password/change/",
data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/auth/password/change/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "oldPassword": "example string",
    "newPassword1": "example string",
    "newPassword2": "example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/auth/password/change/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "oldPassword": "example string",
            "newPassword1": "example string",
            "newPassword2": "example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "detail": string
}
```

Example Response Value

```
{
  "detail": "response string"
}
```

Create a New Organization

POST

[/api/v1/organizations/](#)

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Organization name
compact_logo	string (binary)	No	Compact logo (optional; Enterprise plan only)
full_logo	string (binary)	No	Full logo (optional; Enterprise plan only)

Example Request Schema

```
{
  "name": string // Organization name
  "compact_logo?": string (binary) // Compact logo (Enterprise plan only;
```



```
optional)
  "full_logo"?: string (binary) // Full logo (Enterprise plan only;
optional)
}
```

Example Request Value

```
{
  "name": "My Organization"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "My Organization"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "My Organization"
};

axios.post("https://api.maiagent.ai/api/v1/organizations/", data,
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "My Organization"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "My Organization"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```

{
  "id": string (uuid)
  "name": string
  "createdAt": string (timestamp)
  "compactLogo"?: string (uri) // optional
  "fullLogo"?: string (uri) // optional
  "themePrimaryColor"?: // may have different types (optional)
    string // UI theme color for buttons, links, and other elements (HEX
format, e.g. #0960bd); leave empty to use the system brand primary color
as the frontend fallback (optional)
  "usageStatistics": object
  "organizationPlan": object
  "creditWallet": object // Return credit wallet with ID only.
  "maigptInboxId": string (uuid)
  "canUseCustomElasticsearch": boolean // Allow this organization to
configure custom Elasticsearch endpoints for knowledge bases.
  "canUseVoiceAgent": boolean // Allow this organization to use voice
agent features.
  "canUseMeetingRecords": boolean // Allow this organization to use
meeting records features.
  "canUseCallCenter": boolean // Allow this organization to use call
center features.
  "canUseHookSystem": boolean // Manual override to grant this
organization the hook system regardless of plan. Enterprise-plan
organizations get access automatically without this flag. Layered under
the global Config.hook_system_enabled switch: it must also be enabled.
  "canUseAiGateway": boolean // Allow this organization to call the
OpenAI-compatible `/api/v1/chat/completions` Gateway. Disabled by
default; must be explicitly enabled by platform admin.
  "canUseAgentTeams": boolean // Read-only projection of the platform-
wide Agent Teams switch (Config.enable_agent_teams). Not a per-
organization flag; member-level access is still governed by RBAC.
  "canUseFileLibrary": boolean // Allow this organization to use the file
library. Off by default for gradual rollout.
  "canUseFileOnlineEditing": boolean // Enable online editing in the file
library (P7): the text editor and upload-new-version entries. Requires
can_use_file_library to also be on.
  "enableAgentMemory": boolean // Enable persistent cross-conversation
memory for agents in this organization. When off, agents use only in-
session memory (existing behavior). Off by default for gradual rollout.
  "gatewayServiceFeePercent": string (decimal) // Markup applied on top
of the token price when billing MaiAgent-managed models called through
the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway
markup only; the organization's own models (BYOK / self-hosted) are

```

billed via the AI Gateway traffic fee percent instead.

"gatewayTrafficFeePercent": string (decimal) // Pass-through fee for calls that run on this organization's own AI Gateway models (BYOK / self-hosted): charged as the call's total tokens (input, output, and prompt-cache) x this percent in credits, with no LLM token pricing. Adjustable per organization; changes apply to new calls only.

"platformProcessingFeePercent": string (decimal) // Surcharge on every credit consumption of this organization, billed as a separate "Platform Processing Fee" line equal to the consumed credits x this percent. 0 disables the fee. Applies to new consumption only; the percent is shown read-only on the customer pricing page.

"platformProcessingFeeEnabled": boolean // Whether new consumption of this organization accrues the platform processing fee. The percent is exposed read-only as platform_processing_fee_percent.

"isTrial": boolean // Whether the organization is in a trial period. True when it has exactly one trial plan and that plan is currently active

"expirationDays": integer // Days since expiration. A positive number indicates elapsed days; null means the plan has not expired or is permanent

"showTrialBanner": boolean // Controls whether the frontend displays the trial/free plan banner

"noActivePlanDays": integer // Consecutive days without an active paid plan. null when cleanup is disabled.

"cleanupRemainingDays": integer // Days remaining before organization data is permanently deleted. null when cleanup is disabled, org has an active paid plan, or no_active_plan_days is 0.

"estimatedCleanupDate": string // ISO date when organization data will be permanently deleted. null when cleanup_remaining_days is null.

"approvalStatus": // Trial-eligibility review status. Only approved organizations can receive a trial plan when organization approval is enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review

* `approved` - Approved

* `rejected` - Rejected

{
}

"companyName?": string // Applicant-reported company name collected at organization creation for approval review. (optional)

"taxId?": // may have different types (optional)

string // Applicant-reported business tax ID (8 digits, optional).

Stored as-is; not verified. (optional)

"contactPhone?": string // Applicant-reported contact phone collected at organization creation for approval review. (optional)

"applicationMetadata?": object // Extra applicant-reported key-value

fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax ID, contact phone) stay as named columns. (optional)

```
"invitationCode?": string // optional
}
```

Example Response Value

```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "My Organization",
  "createdAt": "1748336288000",
  "compactLogo": null,
  "fullLogo": null,
  "usageStatistics": {
    "chatbotsCount": 0,
    "countableChatbotsCount": 0,
    "exemptChatbotsByMode": {},
    "canCreateChatbot": true,
    "createChatbotLimit": 1,
    "currentMonthWordsCountTotal": 1000000,
    "currentMonthUsedWordsCountTotal": 0,
    "currentMonthUsedConversationsCountTotal": 0,
    "availableUploadFileSizeTotal": 104857600,
    "usedUploadFileSizeTotal": 0
  },
  "organizationPlan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "planName": "Free Plan",
    "planType": "free",
    "expiredAt": null
  },
  "maigptInboxId": null,
  "canUseCustomElasticsearch": false,
  "isTrial": true,
  "expirationDays": null
}
```

Add a Member to a Specified Organization

POST

/api/v1/organizations/{organizationPk}/members/

Parameters

Parameter Name	Required	Type	Description
organizationPk	V	string	A UUID identifying the organization

Request Body

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
organization	string (uuid)	No	

Example Request Schema

```
{
  "email": string (email)
  "organization?": string (uuid) // optional
}
```

Example Request Value

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
```



```
"is_owner": false
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
    "is_owner": false
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
            "is_owner": false
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "organization": string (uuid) // organization ID
  "is_owner": boolean // Whether the member is an organization owner
}
```

Example Response Value

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}
```

List Organizations

GET

</api/v1/organizations/>

Parameters

Parameter name	Required	Type	Description
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Enable pagination (true/false)

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/?page=1&pageSize=1&pagination=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // optional
}
```



```
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "memberCreatedAt": string (timestamp)
    "approvalStatus": string
  }
]
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "createdAt": "response string",
      "memberCreatedAt": "response string",
      "approvalStatus": "response string"
    }
  ]
}
```

Get information about a specific organization

GET

[/api/v1/organizations/{id}](#)

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this organization.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"createdAt": string (timestamp)
"compactLogo"?: string (uri) // optional
"fullLogo"?: string (uri) // optional
"themePrimaryColor"?: // May have different types (optional)
string // UI Theme color for buttons, links, and other elements (HEX
format, such as #0960bd). If empty, the frontend falls back to the system
brand primary color (optional)
"usageStatistics": object
"organizationPlan": object
"creditWallet": object // Return credit wallet with ID only.
"maigptInboxId": string (uuid)
"canUseCustomElasticsearch": boolean // Allow this organization to
configure custom Elasticsearch endpoints for knowledge bases.
"canUseVoiceAgent": boolean // Allow this organization to use voice
agent features.
"canUseMeetingRecords": boolean // Allow this organization to use
meeting records features.
"canUseCallCenter": boolean // Allow this organization to use call
center features.
"canUseHookSystem": boolean // Manual override to grant this
organization the hook system regardless of plan. Enterprise-plan
organizations get access automatically without this flag. Layered under
the global Config.hook_system_enabled switch: it must also be enabled.
"canUseAiGateway": boolean // Allow this organization to call the
OpenAI-compatible `/api/v1/chat/completions` Gateway. Disabled by
default; must be explicitly enabled by platform admin.
"canUseAgentTeams": boolean // Read-only projection of the platform-
wide Agent Teams switch (Config.enable_agent_teams). Not a per-
organization flag; member-level access is still governed by RBAC.
"canUseFileLibrary": boolean // Allow this organization to use the file
library. Off by default for gradual rollout.
"canUseFileOnlineEditing": boolean // Enable online editing in the file
library (P7): the text editor and upload-new-version entries. Requires
can_use_file_library to also be on.
"enableAgentMemory": boolean // Enable persistent cross-conversation
memory for agents in this organization. When off, agents use only in-
session memory (existing behavior). Off by default for gradual rollout.
"gatewayServiceFeePercent": string (decimal) // Markup applied on top
of the token price when billing MaiAgent-managed models called through
the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway
markup only; the organization's own models (BYOK / self-hosted) are
billed via the AI Gateway traffic fee percent instead.
"gatewayTrafficFeePercent": string (decimal) // Pass-through fee for
calls that run on this organization's own AI Gateway models (BYOK / self-
hosted): charged as the call's total tokens (input, output, and prompt-
```

cache) x this percent in credits, with no LLM token pricing. Adjustable per organization; changes apply to new calls only.

"platformProcessingFeePercent": string (decimal) // Surcharge on every credit consumption of this organization, billed as a separate "Platform Processing Fee" line equal to the consumed credits x this percent. 0 disables the fee. Applies to new consumption only; the percent is shown read-only on the customer pricing page.

"platformProcessingFeeEnabled": boolean // Whether new consumption of this organization accrues the platform processing fee. The percent is exposed read-only as platform_processing_fee_percent.

"isTrial": boolean // Whether the organization is in a trial period. True when it has exactly one trial plan and that plan is currently active

"expirationDays": integer // Days since expiration. Positive value = days expired; null = not yet expired or a permanent plan

"showTrialBanner": boolean // Controls whether the frontend displays the trial/free-plan banner

"noActivePlanDays": integer // Consecutive days without an active paid plan. null when cleanup is disabled.

"cleanupRemainingDays": integer // Days remaining before organization data is permanently deleted. null when cleanup is disabled, org has an active paid plan, or no_active_plan_days is 0.

"estimatedCleanupDate": string // ISO date when organization data will be permanently deleted. null when cleanup_remaining_days is null.

"approvalStatus": // Trial-eligibility review status. Only approved organizations can receive a trial plan when organization approval is enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review

* `approved` - Approved

* `rejected` - Rejected

{
}

"companyName"?: string // Applicant-reported company name collected at organization creation for approval review. (optional)

"taxId"?: // May have different types (optional)

string // Applicant-reported business tax ID (8 digits, optional).

Stored as-is; not verified. (optional)

"contactPhone"?: string // Applicant-reported contact phone collected at organization creation for approval review. (optional)

"applicationMetadata"?: object // Extra applicant-reported key-value fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax ID, contact phone) stay as named columns. (optional)

```
"invitationCode"?: string // optional
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "createdAt": "response string",
  "compactLogo": "https://example.com/file.jpg",
  "fullLogo": "https://example.com/file.jpg",
  "themePrimaryColor": "response string",
  "usageStatistics": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "organizationPlan": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "creditWallet": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "maigptInboxId": "550e8400-e29b-41d4-a716-446655440000",
  "canUseCustomElasticsearch": false,
  "canUseVoiceAgent": false,
  "canUseMeetingRecords": false,
  "canUseCallCenter": false,
  "canUseHookSystem": false,
  "canUseAiGateway": false,
  "canUseAgentTeams": false,
  "canUseFileLibrary": false,
  "canUseFileOnlineEditing": false,
  "enableAgentMemory": false,
  "gatewayServiceFeePercent": "response string",
  "gatewayTrafficFeePercent": "response string",
  "platformProcessingFeePercent": "response string",
  "platformProcessingFeeEnabled": false,
  "isTrial": false,
  "expirationDays": 456,
}
```



```
"showTrialBanner": false,
"noActivePlanDays": 456,
"cleanupRemainingDays": 456,
"estimatedCleanupDate": "response string",
"approvalStatus": {},
"companyName": "response string",
"taxId": "response string",
"contactPhone": "response string",
"applicationMetadata": null,
"invitationCode": "response string"
}
```

Get details of the current user

GET

</api/v1/users/current/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/users/current/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/users/current/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // optional
  "originalAvatar"?: string (uri) // Original avatar uploaded by the user
```

(longest side limited to 2048px). Each edit crops this original again to avoid accumulating distortion from repeated cropping. Existing data from before this feature was released is empty and is not backfilled (optional)

"avatarCropBox"?: object // Position and size of the last crop box on the original image (x/y/width/height, optionally rotate/scale_x/scale_y). Coordinates are relative to the saved original image (optional)

"name": string

"email": string (email)

"authSource":

{

"id": string (uuid)

"name": string

}

"company"?: string // optional

"invitationCode"?: // May have different types (optional)

string // optional

"apiKeys": [// Get the list of API keys for the user in the specified organization.

Process:

1. Get the current organization instance from the request
2. If there is no organization instance (usually during third-party SSO login), use the user's authentication source to find its default organization (is_auth_source_default_organization=True)

Note:

The request does not contain organization_instance during third-party SSO login

Locate the organization using auth_source and the is_auth_source_default_organization flag

object

]

"permissions": object // Get the user's permissions in the current organization (nested parent-child structure).

"resourcePermissions": object // Get the user's resource creation permissions in the current organization.

"isOwner": boolean // Determine whether the user is an Owner in the current organization.

"maigptCapabilities": object // Effective built-in MaiGPT capabilities for the member (resolved from role permissions), used by the frontend to show or hide features.

```
Without member context (no linked organization), all capabilities resolve
as enabled, consistent with the resolver rule that missing data does not
block access
; frontend hiding only affects the user experience, while the gate in the
response path enforces access.
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "company": "response string",
  "invitationCode": "response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "isOwner": false,
  "maigptCapabilities": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
```

```
"description": "Example response description"
}
```

Get permissions of the current user

GET

/api/v1/permissions/

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/permissions/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
[
  {
    "id": string (uuid)
    "name": string
```

```
"value": string
"description": string
"children": [
  object
]
}
```

Example response value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "value": "response string",
    "description": "response string",
    "children": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    ]
  }
]
```

List members of a specified organization

GET

</api/v1/organizations/{organizationPk}/members/>

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

organizationPk	V	string	A UUID string identifying this organization ID
group	X	string	
ids	X	string	
inbox	X	string	
isActive	X	boolean	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Enable pagination (true/false)
query	X	string	
search	X	string	

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&pagination=example&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example


```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "email": string
      "isActive": boolean
      "isOwner": boolean // Check whether the member is the organization
owner (via the OWNER Group)

```

Use caching to avoid duplicate queries within the same request
 Cache lifetime: valid for the lifetime of the member instance

Prefer annotation results, when available, to avoid duplicate queries

```

  "permissions": [
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
  "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes,
injected into LLM system prompt
  "createdAt": string (timestamp)
  }
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
"name": "Response string",
"email": "Response string",
"isActive": false,
"isOwner": false,
"permissions": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"createdAt": "Response string"
}
]
```

Get details of a specific member

GET

</api/v1/organizations/{organizationPk}/members/{id}>

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this member.
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
```

```
"name": string
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is the organization
owner (via the OWNER Group)
```

Use caching to avoid duplicate queries within the same request

Cache lifetime: valid for the lifetime of the member instance

Prefer annotation results, when available, to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
```

```
    "description": "Example response description"
  },
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Get details of a specific member

GET

`/api/v1/organizations/{organizationPk}/members/current/`

Parameters

Parameter name	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/current/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/current/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is the organization
owner (via the OWNER Group)
```

Use caching to avoid duplicate queries within the same request

Cache lifetime: valid for the lifetime of the member instance

Prefer annotation results, when available, to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ]
}
```

```
    }
  ],
  "creditQuotaRequired": false,
  "activeQuota": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "createdAt": "Response string"
}
```

Get details of a specific member

GET

</api/v1/organizations/{organizationPk}/members/export/>

Parameters

Parameter name	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
string (binary)
```

Example response value

"Response string"

Get details of a specific member

GET

/api/v1/organizations/{organizationPk}/members/export-template/

Parameters

Parameter name	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/export-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
string (binary)
```

Example response value

"Response string"

Get details of a specific member

GET

/api/v1/organizations/{organizationPk}/members/metadata-template/

Parameters

Parameter name	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/metadata-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/metadata-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/metadata-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/metadata-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is the organization
owner (via the OWNER Group)
```

Use caching to avoid duplicate queries within the same request

Cache lifetime: valid for the lifetime of the member instance

Prefer annotation results, when available, to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ]
}
```

```
    }
  ],
  "creditQuotaRequired": false,
  "activeQuota": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "createdAt": "Response string"
}
```

Update organization information

PUT

/api/v1/organizations/{id}

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this organization.

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	
compactLogo	string (uri)	No	

Field	Type	Required	Description
fullLogo	string (uri)	No	
themePrimaryColor	object	No	UI theme color for buttons, links, and other elements (HEX format, e.g. #0960bd). If empty, the frontend falls back to the system brand primary color
companyName	string	No	Applicant-reported company name collected at organization creation for approval review.
taxId	object	No	Applicant-reported business tax ID (8 digits, optional). Stored as-is; not verified.
contactPhone	string	No	Applicant-reported contact phone collected at organization creation for approval review.
applicationMetadata	object	No	Extra applicant-reported key-value fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax I...
invitationCode	string	No	

Example request schema

```
{
  "name": string
  "compactLogo"?: string (uri) // optional
  "fullLogo"?: string (uri) // optional
  "themePrimaryColor"?: // May have different types (optional)
    string // UI theme color for buttons, links, and other elements (HEX
format, e.g. #0960bd). If empty, the frontend falls back to the system
brand primary color (optional)
  "companyName"?: string // Applicant-reported company name collected at
organization creation for approval review. (optional)
  "taxId"?: // May have different types (optional)
    string // Applicant-reported business tax ID (8 digits, optional).
Stored as-is; not verified. (optional)
  "contactPhone"?: string // Applicant-reported contact phone collected
at organization creation for approval review. (optional)
```

```
"applicationMetadata"?: object // Extra applicant-reported key-value
fields collected at organization creation. Lets the frontend collect
additional review fields without a schema migration; the core review
fields (company name, tax ID, contact phone) stay as named columns.
(optional)
"invitationCode"?: string // optional
}
```

Example request value

```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}
```

Code examples


```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}'

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
};

axios.put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
```

```
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
    "name": "Updated Org Name",
    "created_at": "1748336288000",
    "usage_statistics": {
        "chatbots_count": 0,
        "countable_chatbots_count": 0,
        "exempt_chatbots_by_mode": {},
        "allowed_chatbot_reply_modes": [],
        "can_create_chatbot": true,
        "create_chatbot_limit": 1,
        "current_month_words_count_total": 1000000,
        "current_month_used_words_count_total": 0,
        "current_month_used_conversations_count_total": 0,
        "available_upload_files_size_total": 104857600,
        "used_upload_file_size_total": 0
    },
    "organization_plan": {
        "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
        "plan_name": "Free plan",
        "expired_at": null
    },
    "is_trial": false,
    "expiration_days": null
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:  
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
            "name": "Updated Org Name",
            "created_at": "1748336288000",
            "usage_statistics": {
                "chatbots_count": 0,
                "countable_chatbots_count": 0,
                "exempt_chatbots_by_mode": {},
                "allowed_chatbot_reply_modes": [],
                "can_create_chatbot": true,
                "create_chatbot_limit": 1,
                "current_month_words_count_total": 1000000,
                "current_month_used_words_count_total": 0,
                "current_month_used_conversations_count_total": 0,
                "available_upload_files_size_total": 104857600,
                "used_upload_file_size_total": 0
            },
            "organization_plan": {
                "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
                "plan_name": "Free plan",
                "expired_at": null
            },
            "is_trial": false,
            "expiration_days": null
        }
    ]);

```

```
$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid) // organization ID
  "name": string // Organization name
  "created_at": string (timestamp) // Creation timestamp
  "usage_statistics": { // Usage statistics
    {
      "chatbots_count"? : integer // Number of chatbots (optional)
      "countable_chatbots_count"? : integer // Number of chatbots counted
        against the creation quota (modes outside the plan allowlist are
        excluded) (optional)
      "exempt_chatbots_by_mode"? : object // Number of chatbots exempt from
        the quota, grouped by the mode that makes them exempt (optional)
      "allowed_chatbot_reply_modes"? : [ // Allowlist of reply modes the
        plan permits creating (an empty array means no restriction) (optional)
        string
      ]
      "can_create_chatbot"? : boolean // Whether chatbots can be created
        (optional)
      "create_chatbot_limit"? : integer // Maximum number of chatbots that
        can be created (-1 means unlimited; 0 means creation is prohibited)
        (optional)
      "current_month_words_count_total"? : integer // Monthly total word
        limit (optional)
    }
  }
}
```

```

    "current_month_used_words_count_total"?: integer // Words used this
month (optional)
    "current_month_used_conversations_count_total"?: integer //
Conversations used this month (optional)
    "available_upload_files_size_total"?: integer // Total upload
capacity (bytes) (optional)
    "used_upload_file_size_total"?: integer // Used upload capacity
(bytes) (optional)
}
}
"organization_plan": { // Organization plan information
{
    "id"?: string (uuid) // Plan ID (optional)
    "plan_name"?: string // Plan name (optional)
    "expired_at"?: string (date-time) // Expiration time (optional)
}
}
"is_trial"?: boolean // Whether the organization is in a trial period:
it has exactly one trial plan and that plan is currently active
(optional)
    "expiration_days"?: integer // Days since expiration: a positive number
is the days elapsed; null means not yet expired or a permanent plan
(optional)
}

```

Example response value

```

{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },

```



```
"organization_plan": {
  "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
  "plan_name": "Free plan",
  "expired_at": null
},
"is_trial": false,
"expiration_days": null
}
```

Update current user details

PUT**/api/v1/users/current/**

Request body

Request parameters

Field	Type	Required	Description
avatar	string (uri)	No	
originalAvatar	string (uri)	No	Original avatar uploaded by the user (longest side limited to 2048px). Each edit crops from this original to avoid cumulative quality loss. Older records from before this feature was released remain empty and are not backfilled
avatarCropBox	object	No	Position and size of the last crop box on the original image (x/y/width/height; rotate/scale_x/scale_y optional). Coordinates refer to the saved original image
name	string	Yes	
email	string (email)	Yes	

Field	Type	Required	Description
company	string	No	
invitationCode	object	No	

Example request schema

```
{
  "avatar"?: string (uri) // optional
  "originalAvatar"?: string (uri) // Original avatar uploaded by the user
  (longest side limited to 2048px). Each edit crops from this original to
  avoid cumulative quality loss. Older records from before this feature was
  released remain empty and are not backfilled (optional)
  "avatarCropBox"?: object // Position and size of the last crop box on
  the original image (x/y/width/height; rotate/scale_x/scale_y optional).
  Coordinates refer to the saved original image (optional)
  "name": string
  "email": string (email)
  "company"?: string // optional
  "invitationCode"?: // May have different types (optional)
  string // optional
}
```

Example request value

```
{
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Example name",
  "email": "user@example.com",
  "company": "Example string",
  "invitationCode": null
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/users/current/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Add form fields

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.put("https://api.maiagent.ai/api/v1/users/current/", formData,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example.pdf', open('example.pdf', 'rb'),
    'application/pdf')
}

response = requests.put(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example.pdf', 'r'),
                'filename' => 'example.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // optional
  "originalAvatar"?: string (uri) // Original avatar uploaded by the user
(longest side limited to 2048px). Each edit crops from this original to
avoid cumulative quality loss. Older records from before this feature was
released remain empty and are not backfilled (optional)
  "avatarCropBox"?: object // Position and size of the last crop box on
the original image (x/y/width/height; rotate/scale_x/scale_y optional).
Coordinates refer to the saved original image (optional)
  "name": string
  "email": string (email)
  "authSource":
  {
    "id": string (uuid)
    "name": string
  }
  "company"?: string // optional
  "invitationCode"?: // May have different types (optional)
string // optional
  "apiKeys": [ // Get the user's API key list for the specified
organization.
```

Process:

1. Get the current organization instance from the request
2. If no organization instance is available (usually with third-party SSO login), use the user's authentication source to find the default organization for that authentication source (is_auth_source_default_organization=True)

Note:

For third-party SSO login, the request does not contain organization_instance

Use auth_source and the is_auth_source_default_organization flag to locate the organization


```
    object
  ]
  "permissions": object // Get the user's permissions in the current
organization (nested parent-child structure).
  "resourcePermissions": object // Get the user's resource creation
permissions in the current organization.
```

```
"isOwner": boolean // Determine whether the user is an Owner in the
current organization.

"maigptCapabilities": object // Effective built-in MaiGPT capabilities
for the member (resolved role permissions), for frontend show/hide.

Without member context (no organization bound), all capabilities resolve
as enabled, consistent with the resolver rule "missing data does not lock
access".

Frontend hiding only affects the user experience; the gate in the
response path enforces authorization.

}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "company": "Response string",
  "invitationCode": "Response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
}
```



```
"isOwner": false,
"maigptCapabilities": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Update member roles

POST

/api/v1/organizations/{organizationPk}/members/{id}/update-groups/

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this member.
organizationPk	V	string	A UUID string identifying this organization ID

Request body

Request parameters

Field	Type	Required	Description
groups	array[string]	Yes	

Example request schema

```
{
  "groups": [
    string (uuid)
  ]
}
```

Example request value

```
{
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
}
```

Code examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
}'

# Replace YOUR_API_KEY and check the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "groups": [
        "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
        "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "groups": [
                "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
                "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name": string
  "email": string
  "isActive": boolean
  "isOwner": boolean // Check whether the member is an organization owner
  (via the OWNER Group)
```

Use the cache to avoid repeated queries within the same request

Cache lifetime: valid while the member instance exists

Prefer the annotation result, if available, to avoid repeated queries

```
  "permissions": [
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired"? : boolean // When True, member must have an
  active quota to consume credits (optional)
  "activeQuota": object // Return the most recent quota summary (active
  first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes, injected
  into LLM system prompt
  "createdAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
```

```
    "description": "Example response description"
  },
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Status code: **400**

Response Schema Example

object

Delete Organization

DELETE

/api/v1/organizations/{id}

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this organization.
----	---	--------	--

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

Delete a Specific Member

DELETE

/api/v1/organizations/{organizationPk}/members/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this member.
organizationPk	V	string	A UUID string identifying this organization.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status code	Description
204	No response body

Get Member Usage Statistics

GET

/api/v1/organizations/{organizationPk}/usage-statistics/members/

Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization.
chatbot	X	string	Filter by a specific AI assistant ID
chatbotId	X	string	
endDate	X	string	End date, format: YYYY-MM-DD
includeTime	X	boolean	Whether to return time series data aggregated only by time for trend charts (true/false). When true, the response contains only time fields and statistical values, with no entity information or pagination.
memberId	X	string	
page	X	integer	Page number
pageSize	X	integer	Items per page
search	X	string	Search by member ID, name, or email
sortBy	X	string	Sort field
sortOrder	X	string	Sort order (asc/desc)
startDate	V	string	Start date, format: YYYY-MM-DD

timeGranularity

X

string

Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/?chatbot=example&chatbotId=550e8400-e29b-41d4-a716-446655440000&endDate=example&includeTime=true&memberId=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&search=example&sortBy=conversations_count&sortOrder=asc&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "count"?: integer // Result count (optional)
      "results"?: [ // optional
        {
          "member"?: { // optional
            {
              "id"?: string (uuid) // Member ID (optional)
              "name"?: string // Member name (optional)
              "email"?: string // Member email (optional)
            }
          }
          "datetime"?: string (date-time) // Timestamp (present only when
include_time=true and time_granularity=hour) (optional)
          "period"?: string (date) // Date (present only when
include_time=true and time_granularity=day/month) (optional)
          "input_characters_count"?: integer // Input character count
(optional)
          "output_characters_count"?: integer // Output character count
(optional)
          "input_token_usage"?: integer // Input token count (optional)
          "output_token_usage"?: integer // Output token count (optional)
          "messages_count"?: integer // Message count (optional)
          "conversations_count"?: integer // Conversation count
(optional)
          "total_characters_count"?: integer // Total character count
(optional)
          "total_token_usage"?: integer // Total token count (optional)
        }
      ]
    }
  ]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",

```

```
"previous": "http://api.example.org/accounts/?page=2",
"results": [
  {
    "count": 2,
    "results": [
      {
        "member": {
          "id": "3a245511-d5e0-4e06-9dc6-1106915b9a8a",
          "name": "John Smith",
          "email": "wang@example.com"
        },
        "input_characters_count": 10000,
        "output_characters_count": 15000,
        "input_token_usage": 2500,
        "output_token_usage": 3750,
        "messages_count": 100,
        "conversations_count": 50,
        "total_characters_count": 25000,
        "total_token_usage": 6250
      }
    ]
  }
]
```

Get Member Usage Statistics Summary

GET

</api/v1/organizations/{organizationPk}/usage-statistics/members/summary/>

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this organization.
<code>endDate</code>	X	string	End date, format: YYYY-MM-DD

startDate

V

string

Start date, format: YYYY-MM-DD

timeGranularity

X

string

Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/summary/?endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/summary/?endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "total_input_characters": integer // Total input character count
```

```
(optional)
  "total_output_characters"?: integer // Total output character count
(optional)
  "total_characters"?: integer // Total character count (optional)
  "total_input_tokens"?: integer // Total input token count (optional)
  "total_output_tokens"?: integer // Total output token count (optional)
  "total_tokens"?: integer // Total token count (optional)
  "total_messages"?: integer // Total message count (optional)
  "total_conversations"?: integer // Total conversation count (optional)
  "total_members"?: integer // Total member count (optional)
}
```

Example Response Value

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_members": 456
}
```

Get AI Assistant Usage Statistics

GET

</api/v1/organizations/{organizationPk}/usage-statistics/chatbots/>

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

organizationPk	V	string	A UUID string identifying this organization.
chatbotId	X	string	
endDate	X	string	End date, format: YYYY-MM-DD
includeTime	X	boolean	Whether to return time series data aggregated only by time for trend charts (true/false). When true, the response contains only time fields and statistical values, with no entity information or pagination.
page	X	integer	Page number
pageSize	X	integer	Items per page
search	X	string	Search by AI assistant ID or name
sortBy	X	string	Sort field
sortOrder	X	string	Sort order (asc/desc)
startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/?chatbotId=550e8400-e29b-41d4-a716-446655440000&endDate=example&includeTime=true&page=1&pageSize=1&search=example&sortBy=conversations_count&sortOrder=asc&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Number of results (optional)
      "results"?: [ // Optional
        {
          "chatbot"?: { // Optional
            {
              "id"?: string (uuid) // AI assistant ID (optional)
              "name"?: string // AI assistant name (optional)
            }
          }
          "datetime"?: string (date-time) // Timestamp (present only when
include_time=true and time_granularity=hour) (optional)
          "period"?: string (date) // Date (present only when
include_time=true and time_granularity=day/month) (optional)
          "input_characters_count"?: integer // Number of input
characters (optional)
          "output_characters_count"?: integer // Number of output
characters (optional)
          "input_token_usage"?: integer // Number of input tokens
(optional)
          "output_token_usage"?: integer // Number of output tokens
(optional)
          "messages_count"?: integer // Number of messages (optional)
          "conversations_count"?: integer // Number of conversations
(optional)
          "total_characters_count"?: integer // Total number of
characters (optional)
          "total_token_usage"?: integer // Total number of tokens
(optional)
        }
      ]
    }
  ]
}

```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "count": 2,
      "results": [
        {
          "chatbot": {
            "id": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
            "name": "Customer Support Assistant"
          },
          "input_characters_count": 10000,
          "output_characters_count": 15000,
          "input_token_usage": 2500,
          "output_token_usage": 3750,
          "messages_count": 100,
          "conversations_count": 50,
          "total_characters_count": 25000,
          "total_token_usage": 6250
        }
      ]
    }
  ]
}
```

Get AI Assistant Usage Statistics Summary

GET

</api/v1/organizations/{organizationPk}/usage-statistics/chatbots/summary/>

Parameters

Parameter Name	Required	Type	Description
----------------	----------	------	-------------

organizationPk	V	string	A UUID string identifying this organization
endDate	X	string	End date, format: YYYY-MM-DD
startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?\nendDate=example&startDate=example&timeGranularity=day"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example Response Schema

```
{
  "total_input_characters": integer // Total number of input characters
```

```
(optional)
  "total_output_characters"?: integer // Total number of output
characters (optional)
  "total_characters"?: integer // Total number of characters (optional)
  "total_input_tokens"?: integer // Total number of input tokens
(optional)
  "total_output_tokens"?: integer // Total number of output tokens
(optional)
  "total_tokens"?: integer // Total number of tokens (optional)
  "total_messages"?: integer // Total number of messages (optional)
  "total_conversations"?: integer // Total number of conversations
(optional)
  "total_chatbots"?: integer // Total number of AI assistants (optional)
}
```

Example Response Value

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_chatbots": 456
}
```


API request example (Shell)

...

目錄

1. List contacts
2. Create a contact
3. Get contact details
4. Get contact details
5. Get contact details
6. Update contact
7. Partially update contact
8. Delete contact
9. List contact conversations
10. Get the latest conversation for a contact
11. Broadcast message

API request example (Shell)

List contacts

GET

/api/v1/contacts/

Parameters

Parameter	Required	Type	Description
channels	X	array	Multiple values may be separated by commas.
inbox	X	string	Deprecated: Use inboxes instead
inboxes	X	string	Conversation platform ID (filters contacts in a specific inbox)
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

query	X	string	Search by name or source ID
search	X	string	Full-text search
tags	X	array	Multiple values may be separated by commas.

Code examples

Shell/Bash

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/?  
channels=example&inbox=550e8400-e29b-41d4-a716-  
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa  
mple&tags=example"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
```

```

"results": [
  {
    "id": string (uuid)
    "inboxes"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "inbox"?: // Backward-compatible field: a single inbox; ignored when
inboxes is also provided (optional)
    {
      "id": string (uuid)
      "name": string
    }
    "name": string
    "avatar"?: // May have different types (optional)
string (uri) // Contact avatar URL; accepts an absolute URL or a
relative path (optional)
    "phoneNumber"?: string // optional
    "email"?: // May have different types (optional)
string (email) // optional
    "sourceId"?: string // optional
    "queryMetadata"?: { // Query metadata controlling the scope of
searchable knowledge; see the endpoint descriptions (optional)
      {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null means none can be queried (no permission); omitted means
unrestricted (optional)
          {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence if both are provided) (optional)
              string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or selected according
to hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
          }
        ]
      }
    ]
  }
]

```

```

    "labelRelations"?: { // Label filters; an empty conditions array
means no label filtering (not a lack of permission). Provide with
knowledgeBases; alone, the filters do not apply to knowledge-base retrieval
(equivalent to unrestricted) (optional)
      {
        "operator": string (enum: AND, OR) // How label conditions are
combined
        "conditions"?: [ // List of label conditions; supports nested
operator/conditions groups (optional)
          object
        ]
      }
    }
    "metadata"?: object // Custom customer information shown in the
customer panel of the conversation page (optional)
    "mcpCredentials": [ // List of MCP credentials for the contact
      object
    ]
    "apiCredentials": [ // List of API credentials for the contact
      object
    ]
    "contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding tokens)
      object
    ]
    "tags": [
      {
        "id": string (uuid)
        "name": string
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
    ]
    "preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes; mapping {field_id: {label: {locale: text}}} lets the
frontend resolve metadata UUID keys to display names
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Example response value


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    [
      {
        "id": "user1_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user1",
        "avatar": "",
        "sourceId": null,
        "queryMetadata": null,
        "createdAt": "1751875361000",
        "updatedAt": "1751875361000"
      },
      {
        "id": "user2_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user2",
        "avatar": "avatar_url",
        "sourceId": "self defined source id",
        "queryMetadata": {
          "knowledgeBases": [
            {
              "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
              "hasUserSelectedAll": true
            }
          ]
        },
        "labelRelations": {
          "operator": "OR",
          "conditions": [
            {
              "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
            }
          ]
        }
      }
    ]
  ]
}
```

```
    }
  },
  "createdAt": "1751875400000",
  "updatedAt": "1751875400000"
}
]
}
```

Create a contact

POST

/api/v1/contacts/

Code examples

```
# API request example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "inbox_uuid"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": "self defined source id",
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "OR",
      "conditions": [
        {
          "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
        }
      ]
    }
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VIP"
    },
    {
      "key": "Preferred language",
      "value": "Traditional Chinese"
    }
  ]
}
```

```
    }  
  ]  
}'
```

Replace YOUR_API_KEY and verify the request data before running this example.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "inbox_uuid"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": "self defined source id",
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "OR",
      "conditions": [
        {
          "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
        }
      ]
    }
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VIP"
    }
  ]
}
```

```
    },  
    {  
      "key": "Preferred language",  
      "value": "Traditional Chinese"  
    }  
  ]  
};
```

```
axios.post("https://api.maiagent.ai/api/v1/contacts/", data, config)  
  .then(response => {  
    console.log('Response received:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "inbox_uuid"
        }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": "self defined source id",
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "OR",
            "conditions": [
                {
                    "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
                }
            ]
        }
    },
    "metadata": [
        {
            "key": "Customer tier",
            "value": "VIP"
        },
        {
```

```
        "key": "Preferred language",  
        "value": "Traditional Chinese"  
    }  
]  
}
```

```
response = requests.post(url, json=data, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "inbox_uuid"
                }
            ],
            "name": "user1",
            "avatar": "",
            "phoneNumber": 912345678,
            "email": "user1@example.com",
            "sourceId": "self defined source id",
            "queryMetadata": {
                "knowledgeBases": [
                    {
                        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-
fdeea327a3a2",
                        "hasUserSelectedAll": true
                    }
                ],
                "labelRelations": {
                    "operator": "OR",
                    "conditions": [
                        {
                            "labelId": "957998f9-1824-4872-becd-
b7f8a7962896"
                        }
                    ]
                }
            },
            "metadata": [

```

```

        {
            "key": "Customer tier",
            "value": "VIP"
        },
        {
            "key": "Preferred language",
            "value": "Traditional Chinese"
        }
    ]
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status code: 201

Example response schema

```

{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: a single inbox; ignored when
inboxes is also provided (optional)
  {
    "id": string (uuid)

```

```

    "name": string
  }
  "name": string
  "avatar"?: // May have different types (optional)
    string (uri) // Contact avatar URL; accepts an absolute URL or a relative
path (optional)
  "phoneNumber"?: string // optional
  "email"?: // May have different types (optional)
    string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata controlling the scope of searchable
knowledge; see the endpoint descriptions (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null means none can be queried (no permission); omitted means
unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence if both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filters; an empty conditions array means no
label filtering (not a lack of permission). Provide with knowledgeBases;
alone, the filters do not apply to knowledge-base retrieval (equivalent to
unrestricted) (optional)
        {
          "operator": string (enum: AND, OR) // How label conditions are combined
          "conditions"?: [ // List of label conditions; supports nested
operator/conditions groups (optional)
            object
          ]
        }
      }
    }
  }
}

```

```

    }
    "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
    "mcpCredentials": [ // List of MCP credentials for the contact
        object
    ]
    "apiCredentials": [ // List of API credentials for the contact
        object
    ]
    "contactConnectors": [ // List of Connector OAuth authorizations for the
contact (excluding tokens)
        object
    ]
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "createdAt": string (timestamp)
            "updatedAt": string (timestamp)
        }
    ]
    "preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes; mapping {field_id: {label: {locale: text}}} lets the
frontend resolve metadata UUID keys to display names
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",

```

```
"updatedAt": "1751875361000"
}
```

Get contact details

GET

`/api/v1/contacts/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.

Code examples

Shell/Bash

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
```



```

        "name": string
    }
]
"inbox"?: // Backward-compatible field: a single inbox; ignored when
inboxes is also provided (optional)
{
    "id": string (uuid)
    "name": string
}
"name": string
"avatar"?: // May have different types (optional)
string (uri) // Contact avatar URL; accepts an absolute URL or a relative
path (optional)
"phoneNumber"?: string // optional
"email"?: // May have different types (optional)
string (email) // optional
"sourceId"?: string // optional
"queryMetadata"?: { // Query metadata controlling the scope of searchable
knowledge; see the endpoint descriptions (optional)
    {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null means none can be queried (no permission); omitted means
unrestricted (optional)
            {
                "knowledgeBaseId": string (uuid) // Knowledge base ID
                "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (optional)
                    string (uuid)
                ]
                "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence if both are provided) (optional)
                    string (uuid)
                ]
                "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (optional)
                    string (uuid)
                ]
                "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
            }
        ]
        "labelRelations"?: { // Label filters; an empty conditions array means no
label filtering (not a lack of permission). Provide with knowledgeBases;
alone, the filters do not apply to knowledge-base retrieval (equivalent to
unrestricted) (optional)
            {
                "operator": string (enum: AND, OR) // How label conditions are combined

```

```

    "conditions"?: [ // List of label conditions; supports nested
operator/conditions groups (optional)
      object
      ]
    }
  }
}
"metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
"mcpCredentials": [ // List of MCP credentials for the contact
  object
]
"apiCredentials": [ // List of API credentials for the contact
  object
]
"contactConnectors": [ // List of Connector OAuth authorizations for the
contact (excluding tokens)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes; mapping {field_id: {label: {locale: text}}} lets the
frontend resolve metadata UUID keys to display names
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],

```

```
"name": "user1",
"avatar": "",
"phoneNumber": 912345678,
"email": "user1@example.com",
"sourceId": null,
"queryMetadata": null,
"createdAt": "1751875361000",
"updatedAt": "1751875361000"
}
```

Get contact details

GET

</api/v1/contacts/bulk-import-template/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/bulk-import-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/bulk-import-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/bulk-import-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/bulk-import-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
```

```

    }
  ]
  "inbox"?: // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // May have different types (optional)
  string (uri) // Contact avatar URL; supports an absolute URL or a relative
path (optional)
  "phoneNumber"?: string // optional
  "email"?: // May have different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata controlling the searchable knowledge
scope; see each endpoint for details (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null denies access to all, while omission means unrestricted
(optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // File ID list (exclusions or selected items
according to hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // File ID list (new name for
chatbotFileIds; this field takes precedence if both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // FAQ ID list (exclusions or selected items according
to hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
        }
      ]
    }
    "labelRelations"?: { // Label filter; an empty conditions array disables
label filtering (it does not deny access). Provide it together with
knowledgeBases; if provided alone, label filtering is not applied to
knowledge-base searches (equivalent to unrestricted). (optional)
      {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested

```

```

operator/conditions combinations (optional)
  object
    ]
  }
}
}
}
"metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
"mcpCredentials": [ // List of the contact's MCP credentials
  object
]
"apiCredentials": [ // List of the contact's API credentials
  object
]
"contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding token values)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes, mapping {field_id: {label: {locale: text}}}; the frontend
uses these to resolve UUID keys in metadata to display names
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {

```



```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
```

```
},  
  "createdAt": "Response string",  
  "updatedAt": "Response string"  
}
```

Get contact details

GET

[/api/v1/contacts/connector-management/](#)

Code examples

Shell/Bash

```
# Example API call (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/contacts/connector-  
management/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/connector-
management/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/connector-management/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/connector-management/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	No response body

Update contact

PUT

`/api/v1/contacts/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.

Request body

Request parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	List of inboxes (new format)
inbox	object	No	Single inbox (legacy format, backward compatible)
inbox.id	string (uuid)	Yes	Inbox ID
avatar	string	No	Avatar URL
phoneNumber	string	No	Contact phone number
email	string (email)	No	Contact email address
sourceId	string	No	Source ID
queryMetadata	object	No	Query metadata controls the scope of searchable knowledge. Omitted or null means not configured (unrestricted); an empty object {} is treated as unrestricted (all

Field	Type	Required	Description
			selected), not as no permission; an empty array or null for knowledgeBases denies access to all knowledge bases; an empty array for labelRelations.conditions disables label filtering, not access. See this endpoint for details.
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null denies access to all, while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filter; an empty conditions array disables label filtering (it does not deny access). Provide it together with knowledgeBases; if provided alone, label filtering is not applied to knowledge-base searches (equivalent to unrestricted).
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	array[object]	No	List of custom contact attributes

Example request schema

```

{
  "name"?: string // Contact name (optional)
  "inboxes"?: [ // List of inboxes (new format) (optional)
    {
      "id": string (uuid) // Inbox ID
    }
  ]
  "inbox"?: { // Single inbox (legacy format, backward compatible) (optional)
    {
      "id": string (uuid) // Inbox ID
    }
  }
  "avatar"?: string // Avatar URL (optional)
  "phoneNumber"?: string // Contact phone number (optional)
  "email"?: string (email) // Contact email address (optional)
  "sourceId"?: string // Source ID (optional)
  "queryMetadata"?: { // Query metadata controls the scope of searchable
knowledge. Omitted or null means not configured (unrestricted); an empty
object {} is treated as unrestricted (all selected), not as no permission; an
empty array or null for knowledgeBases denies access to all knowledge bases;
an empty array for labelRelations.conditions disables label filtering, not
access. See this endpoint for details. (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null denies access to all, while omission means unrestricted
(optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // File ID list (exclusions or selected items
according to hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // File ID list (new name for
chatbotFileIds; this field takes precedence if both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // FAQ ID list (exclusions or selected items according
to hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filter; an empty conditions array disables
label filtering (it does not deny access). Provide it together with

```



```

knowledgeBases; if provided alone, label filtering is not applied to
knowledge-base searches (equivalent to unrestricted). (optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
            object
        ]
    }
}
}
"metadata"?: [ // List of custom contact attributes (optional)
    {
        "key": string // Attribute name
        "value": string // Attribute value
        "updatedAt"?: string // Update time (ISO format; optional, populated
automatically if omitted) (optional)
    }
]
}

```

Example request value

```

{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",

```

```
      "value": "Japanese"  
    }  
  ]  
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",
      "value": "Japanese"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",
      "value": "Japanese"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  });
```

```
})  
.catch(error => {  
  console.error('Request failed:');  
  console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
        "knowledgeBases": []
    },
    "metadata": [
        {
            "key": "Customer tier",
            "value": "VVIP"
        },
        {
            "key": "Preferred language",
            "value": "Japanese"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:  
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "new_inbox_uuid"
                }
            ],
            "name": "updated_user_name",
            "avatar": "updated_avatar_url",
            "phoneNumber": 987654321,
            "email": "updated@example.com",
            "sourceId": "updated_source_id",
            "queryMetadata": {
                "knowledgeBases": []
            },
            "metadata": [
                {
                    "key": "Customer tier",
                    "value": "VVIP"
                },
                {
                    "key": "Preferred language",
                    "value": "Japanese"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```



```
        print_r($data);
    } catch (Exception $e) {
        echo 'Request failed: ' . $e->getMessage();
    }
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // May have different types (optional)
  string (uri) // Contact avatar URL; supports an absolute URL or a relative
path (optional)
  "phoneNumber"?: string // optional
  "email"?: // May have different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata controlling the searchable knowledge
scope; see each endpoint for details (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
```

```

array or null denies access to all, while omission means unrestricted
(optional)
    {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // File ID list (exclusions or selected items
according to hasUserSelectedAll) (optional)
            string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // File ID list (new name for
chatbotFileIds; this field takes precedence if both are provided) (optional)
            string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (exclusions or selected items according
to hasUserSelectedAll) (optional)
            string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (optional)
    }
]
    "labelRelations"?: { // Label filter; an empty conditions array disables
label filtering (it does not deny access). Provide it together with
knowledgeBases; if provided alone, label filtering is not applied to
knowledge-base searches (equivalent to unrestricted). (optional)
        {
            "operator": string (enum: AND, OR) // How label conditions are combined
            "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
                object
            ]
        }
    }
}
    "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
    "mcpCredentials": [ // List of the contact's MCP credentials
        object
    ]
    "apiCredentials": [ // List of the contact's API credentials
        object
    ]
    "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding token values)
        object
    ]
    "tags": [

```

```

    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes, mapping {field_id: {label: {locale: text}}}; the frontend
uses these to resolve UUID keys in metadata to display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}

```

Partially update contact

PATCH

/api/v1/contacts/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.

Request body

Request parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	List of inboxes (new format)
inbox	object	No	Single inbox (legacy format, backward compatible)
inbox.id	string (uuid)	Yes	Inbox ID
avatar	string	No	Avatar URL
phoneNumber	string	No	Contact phone number
email	string (email)	No	Contact email address
sourceId	string	No	Source ID
queryMetadata	object	No	Query metadata controls which knowledge can be queried. Omitted or null means unset (unrestricted); an empty object {} is also unrestricted (select all), not no access. An empty or null knowledgeBases means no knowledge can be queried (no access). Empty

Field	Type	Required	Description
			labelRelations.conditions means no label filtering, not no access. See this endpoint description.
queryMetadata.knowledgeBases	array[object]	No	List of queryable knowledge bases; an empty array or null means none can be queried (no access), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array means no label filtering, not no access. Must be supplied with knowledgeBases; if supplied alone, the label filter is not applied to knowledge-base search (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	array[object]	No	List of custom contact attributes

Request schema example

```
{
  "name"?: string // Contact name (Optional)
  "inboxes"?: [ // List of inboxes (new format) (Optional)
    {
      "id": string (uuid) // Inbox ID
    }
  ]
}
```

```

]
"inbox"?: { // Single inbox (legacy format, backward compatible) (Optional)
{
  "id": string (uuid) // Inbox ID
}
}
"avatar"?: string // Avatar URL (Optional)
"phoneNumber"?: string // Contact phone number (Optional)
"email"?: string (email) // Contact email address (Optional)
"sourceId"?: string // Source ID (Optional)
"queryMetadata"?: { // Query metadata controls which knowledge can be
queried. Omitted or null means unset (unrestricted); an empty object {} is
also unrestricted (select all), not no access. An empty or null
knowledgeBases means no knowledge can be queried (no access). Empty
labelRelations.conditions means no label filtering, not no access. See this
endpoint description. (Optional)
{
  "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array
or null means none can be queried (no access), while omission means
unrestricted (Optional)
    {
      "knowledgeBaseId": string (uuid) // Knowledge-base ID
      "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (Optional)
        string (uuid)
      ]
      "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence if both are supplied) (Optional)
        string (uuid)
      ]
      "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
        string (uuid)
      ]
      "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (Optional)
    }
  ]
  "labelRelations"?: { // Label filter conditions; an empty conditions
array means no label filtering, not no access. Must be supplied with
knowledgeBases; if supplied alone, the label filter is not applied to
knowledge-base search (equivalent to unrestricted) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (Optional)
        object

```

```

    ]
  }
}
}
"metadata"?: [ // List of custom contact attributes (Optional)
  {
    "key": string // Attribute name
    "value": string // Attribute value
    "updatedAt"?: string // Update time (ISO format; optional, filled in
automatically if omitted) (Optional)
  }
]
}

```

Example request value

```

{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",
      "value": "Japanese"
    }
  ]
}

```

Code examples


```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",
      "value": "Japanese"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred language",
      "value": "Japanese"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  });
```

```
})  
.catch(error => {  
  console.error('Request error:');  
  console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
        "knowledgeBases": []
    },
    "metadata": [
        {
            "key": "Customer tier",
            "value": "VVIP"
        },
        {
            "key": "Preferred language",
            "value": "Japanese"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
```

```
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "new_inbox_uuid"
                }
            ],
            "name": "updated_user_name",
            "avatar": "updated_avatar_url",
            "phoneNumber": 987654321,
            "email": "updated@example.com",
            "sourceId": "updated_source_id",
            "queryMetadata": {
                "knowledgeBases": []
            },
            "metadata": [
                {
                    "key": "Customer tier",
                    "value": "VVIP"
                },
                {
                    "key": "Preferred language",
                    "value": "Japanese"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
```

```
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "inboxes"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: one inbox; ignored if inboxes is
also supplied (Optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL; supports a full URL or relative path
(Optional)
  "phoneNumber"?: string // Optional
  "email"?: // May have different types (Optional)
string (email) // Optional
  "sourceId"?: string // Optional
  "queryMetadata"?: { // Query metadata controlling the scope of queryable
knowledge; see each endpoint description (Optional)
  {
    "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array
```

or null means none can be queried (no access), while omission means unrestricted (Optional)

```
{
  "knowledgeBaseId": string (uuid) // Knowledge-base ID
  "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (Optional)
    string (uuid)
  ]
  "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence if both are supplied) (Optional)
    string (uuid)
  ]
  "faqIds"?: [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
    string (uuid)
  ]
  "hasUserSelectedAll"?: boolean // true selects all knowledge-base
content except listed items; false selects only listed items (Optional)
}
]
"labelRelations"?: { // Label filter conditions; an empty conditions
array means no label filtering, not no access. Must be supplied with
knowledgeBases; if supplied alone, the label filter is not applied to
knowledge-base search (equivalent to unrestricted) (Optional)
  {
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (Optional)
      object
    ]
  }
}
}
"metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (Optional)
"mcpCredentials": [ // List of contact MCP credentials
  object
]
"apiCredentials": [ // List of contact API credentials
  object
]
"contactConnectors": [ // List of contact Connector OAuth authorizations
(excluding the tokens)
  object
]
"tags": [
```



```

    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from this
Contact's inboxes, mapped as {field_id: {label: {locale: text}}}; the
frontend uses them to resolve metadata UUID keys to display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}

```

Delete contact

DELETE

`/api/v1/contacts/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body
400	The contact has associated conversations and cannot be deleted

List contact conversations

GET

/api/v1/contacts/{id}/conversations/

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this Contact.
channels	X	array	Multiple values may be separated by commas.
inbox	X	string	Filter by one inbox ID (UUID format)
inboxes	X	string	Filter by multiple inbox IDs (comma-separated, e.g. uuid1,uuid2, or repeated inboxes parameters)
keyword	X	string	Search message content in conversations (case-insensitive)
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
tags	X	array	Multiple values may be separated by commas.

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=1&page
Size=1&query=example&tags=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running this
example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?channels=example&inbox=example&inboxes=example&keyword=example&page=1&pageSize=1&query=example&tags=example", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?channels=example&inbox=example&inboxes=example&keyword=example&page=1&pageSize=1&query=example&tags=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=1&p
ageSize=1&query=example&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "contact": { // Lightweight Contact serializer for the conversation
list view

```

Includes only fields needed by the list view; omits inboxes, mcp_credentials, and api_credentials

to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact

needed by the frontend conversation sidebar, with no N+1 cost.

```

  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key,
or an external CDN URL as-is. (Optional)
    "email"?: // May have different types (Optional)
string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the
customer panel of the conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
}
"inbox": { // Inbox summary embedded in a conversation payload.

```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```

  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger,
instagram, email, whatsapp, slack, team_plus, line_works, viber) // * `line`
- LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web

```

```

* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?: // Optional
    {
        "id": string (uuid)
        "signSource": string (uuid)
        "signParams": {
            {
                "keycloak":
                {
                    "clientId": string
                    "url": string
                    "realm": string
                }
                "line":
                {
                    "liffId": string
                }
                "ad":
                {
                    "saml": string
                }
                "google"?: // Optional
                {
                    "enabled": boolean
                }
            }
        }
        "sourceIdAccessEnabled"?: boolean // When enabled, embedded
        visitors presenting a Source ID (verified per the verify mode below) may chat
        without going through the login flow. Disabled keeps every existing behavior
        unchanged. (Optional)
        "sourceIdVerifyMode"?: // "Require signature verification": the
        embedding site must send an HMAC signature computed on its backend. "Source
        ID only": anyone knowing a Source ID can act as that contact; suitable only
        for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {

```

```

    }
    "sourceIdSignatureTtlMinutes"?: // How long a signature can be
used to obtain the contact identity. Not a session lifetime: once exchanged,
the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak
access token a visitor logged in with is forwarded as the Authorization
header when the AI calls MCP and API tools on their behalf, so tools do not
require a separate per-tool login. Only for the Keycloak sign source. Never
overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the
conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or
"*.*.example.com" wildcards) of the tools the visitor Keycloak token may be
sent to (Optional)
        string
    ]
    }
    "enableAnalysis"?: boolean // Optional
}
}
"chatbot": object // Chatbot (Agent) summary from the conversation's
inbox, already eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon,
feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
    "title": string
    "lastMessage"?: { // Optional
    {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
    }
    }
    "callRecord":
    {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed,
failed, busy, no-answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing

```

```

* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` -
Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
}
    "lastMessageCreatedAt": string (timestamp)
    "unreadMessagesCount": integer
    "autoReplyEnabled": boolean
    "isAutoReplyNow": boolean
    "lastReadAt": string (timestamp)
    "createdAt": string (timestamp)
    "isGroupChat": boolean
    "enableGroupMention"?: boolean // Optional
    "queryMetadata"?: { // Query metadata that controls the scope of
searchable knowledge; see the endpoint descriptions (Optional)
    {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty
array or null means none are accessible (no permission); omitted means
unrestricted (Optional)
        {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (Optional)
            string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; this field takes precedence if both are provided) (Optional)
            string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or selected according
to hasUserSelectedAll) (Optional)
            string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true selects all content in the
knowledge base except listed items; false selects only listed items
(Optional)
        }
    ]
    "labelRelations"?: { // Label filters; an empty conditions array
applies no label filter (it does not deny access). Provide this with

```

```

knowledgeBases; on its own, label filtering is not applied to knowledge base
retrieval (equivalent to unrestricted) (Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are
combined
        "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (Optional)
            object
        ]
    }
}
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool}
(Optional)
"connectorMask"?: object // Enabled/disabled state per connector:
{connector_id: bool} (Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot
setting), {} = explicitly off, {...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking
config after applying the priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

"llm": string (uuid)
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
    {
    }
    "deepResearchOutputFormat": // May have different types
    string (enum: canvas, chat, file) // Chosen delivery form for the
finished report. Set only when the user clicks a format in the pre-research
choice card; reset to NULL each time Deep Research is (re-)armed. NULL means
no explicit choice (or a plan-skipping request) and falls back to Canvas at
render time, but stays distinct from an explicit canvas pick so the frontend

```

can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force this to NULL/canvas – no redeploy needed.

```
* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"? : string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"? : [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"? : string (timestamp) // Optional
  "queuedAt"? : string (timestamp) // Optional
  "assignedAt"? : string (timestamp) // Optional
  "transferReason"? : string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a
  5-point scale (1=very dissatisfied, 5=very satisfied)
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis
  summary (its active summary revision).
```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```
  "clientPlatform": // Client platform that created this conversation
  (web, desktop)
```

```
* `web` - Web
```



```
* `desktop` - Desktop
{
  "workingDirectory": string // Absolute path of the local folder bound
to a desktop conversation. Only set for desktop conversations; null for web
conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance:
  ``get_matched_message_id`` / ``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched
message, with ellipses.
```

Reads the ``matched\_message\_content`` annotation added by
 ``ConversationViewSet.\_annotate\_keyword\_match``; no extra query.

```
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned.
Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this
conversation is grouped into. Null means unclassified (shown under the time
groups). Deleting the folder resets this to null without deleting the
conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session
snapshot exists to resume.
```

Lets the frontend know it should proactively re-request the browser
 panel (emit ``browser:start``) after a page reload, instead of only
 ever showing the panel when a live ``browser:frame`` push happens to
 arrive first.

```
  "callerOrganizationId": string // Bound caller organization (stamped at
conversation start); null when unbound
  "handoff": object // Handoff details: enabled is the platform switch; a
continuation conversation carries source_conversation_id/source_title; the
original conversation carries continued_to_conversation_id/continued_to_title
and latest_summary_message_id
  }
]
}
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
```

```
"previous": "http://api.example.org/accounts/?page=2",
"results": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "contact": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "avatar": "Response string",
      "email": "response@example.com",
      "phoneNumber": "Response string",
      "metadata": null,
      "sourceId": "Response string",
      "tags": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ]
    },
    "inbox": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "channelType": "line",
      "unreadConversationsCount": 456,
      "signAuth": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Response string",
            "url": "Response string",
            "realm": "Response string"
          },
          "line": {
            "liffId": "Response string"
          },
          "ad": {
            "saml": "Response string"
          },
          "google": {
            "enabled": false
          }
        },
        "sourceIdAccessEnabled": false,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": false,
```

```
    "keycloakTokenPassthroughAllowedHosts": [
      "Response string"
    ]
  },
  "enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"effectiveThinkingConfig": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
```

```
      "description": "Example response description"
    }
  }
]
```

Get the latest conversation for a contact

GET

`/api/v1/contacts/{id}/conversations/latest/`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.
<code>inbox</code>	X	string	Filter by one inbox ID (UUID format)
<code>inboxes</code>	X	string	Filter by multiple inbox IDs (comma-separated, such as uuid1,uuid2, or repeat the inboxes parameter)
<code>keyword</code>	X	string	Search message content in conversations (case-insensitive)

Code examples

```
# API request example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?inbox=example&inboxes=example&keyword=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest?inbox=example&inboxes=example&keyword=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Response structure example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact serializer for the conversation list
    view
```

Includes only the fields needed for the list view; omits inboxes, mcp_credentials, and api_credentials to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact that the conversation sidebar needs to display, with no N+1 cost.

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string // Stores our own media as a relative storage key, or
an external CDN URL as-is. (Optional)
  "email"?: // May have different types (Optional)
string (email) // Optional
  "phoneNumber"?: string // Optional
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (Optional)
  "sourceId"?: string // Optional
  "tags": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
}
"inbox": { // Inbox summary embedded in a conversation payload.
```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger,
instagram, email, whatsapp, slack, team_plus, line_works, viber) // * `line`
- LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
```

```

* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
        "google"?: // Optional
        {
          "enabled": boolean
        }
      }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors
    presenting a Source ID (verified per the verify mode below) may chat without
    going through the login flow. Disabled keeps every existing behavior
    unchanged. (Optional)
    "sourceIdVerifyMode"?: // "Require signature verification": the
    embedding site must send an HMAC signature computed on its backend. "Source
    ID only": anyone knowing a Source ID can act as that contact; suitable only
    for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
  {
  }
  "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to
  obtain the contact identity. Not a session lifetime: once exchanged, the
  session follows the existing WebChat mechanism.

* `1` - 1

```

```

* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak
access token a visitor logged in with is forwarded as the Authorization
header when the AI calls MCP and API tools on their behalf, so tools do not
require a separate per-tool login. Only for the Keycloak sign source. Never
overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the
conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or
"*example.com" wildcards) of the tools the visitor Keycloak token may be
sent to (Optional)
        string
    ]
    }
    "enableAnalysis"?: boolean // Optional
}
}
    "chatbot": object // Chatbot (Agent) summary from the conversation's inbox,
already eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon,
feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
    "title": string
    "lastMessage"?: { // Optional
    {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
    }
    }
    "callRecord":
    {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed,
failed, busy, no-answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)

```

```

    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
    * `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"? : boolean // Optional
  "queryMetadata"? : { // Query metadata that controls the scope of searchable
knowledge; see the endpoint descriptions (Optional)
    {
      "knowledgeBases"? : [ // List of searchable knowledge bases; an empty
array or null means none are accessible (no permission); omitted means
unrestricted (Optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"? : [ // List of file IDs (excluded or selected
according to hasUserSelectedAll) (Optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"? : [ // List of file IDs (new name for
chatbotFileIds; this field takes precedence if both are provided) (Optional)
            string (uuid)
          ]
          "faqIds"? : [ // List of FAQ IDs (excluded or selected according to
hasUserSelectedAll) (Optional)
            string (uuid)
          ]
          "hasUserSelectedAll"? : boolean // true selects all content in the
knowledge base except listed items; false selects only listed items
(Optional)
        }
      ]
      "labelRelations"? : { // Label filters; an empty conditions array applies
no label filter (it does not deny access). Provide this with knowledgeBases;
on its own, label filtering is not applied to knowledge base retrieval
(equivalent to unrestricted) (Optional)
        {
          "operator": string (enum: AND, OR) // How label conditions are combined
          "conditions"? : [ // List of label conditions; supports nested
operator/conditions combinations (Optional)
            object

```

```

    ]
  }
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool}
(Optional)
"connectorMask"?: object // Enabled/disabled state per connector:
{connector_id: bool} (Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {}
= explicitly off, {...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config
after applying the priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

"llm": string (uuid)
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished
report. Set only when the user clicks a format in the pre-research choice
card; reset to NULL each time Deep Research is (re-)armed. NULL means no
explicit choice (or a plan-skipping request) and falls back to Canvas at
render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas
behavior, force this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"ipAddress": string
"ipCountryCode": string

```

```

    "ipRecordedAt": string (timestamp)
    "assignee":
    {
        "id": string (uuid)
        "name": string
        "userId": string (uuid)
    }
    "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "memberIds"?: [ // optional
                string (uuid)
            ]
        }
    ]
    "progressStatus": string
    "firstResponseAt"?: string (timestamp) // optional
    "queuedAt"?: string (timestamp) // optional
    "assignedAt"?: string (timestamp) // optional
    "transferReason"?: string // optional
    "satisfactionScore": integer // Conversation satisfaction rating on a 5-
point scale (1=very dissatisfied, 5=very satisfied)
    "satisfactionComment": string
    "satisfactionSubmittedAt": string (timestamp)
    "latestAnalysisSummary": string // The conversation's current analysis
summary (its active summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```

    "clientPlatform": // Client platform that created this conversation (web,
desktop)

```

```

* `web` - Web

```

```

* `desktop` - Desktop

```

```

    {
    }
    "workingDirectory": string // Absolute path of the local folder bound to a
desktop conversation. Only set for desktop conversations; null for web
conversations.

```

```

    "slideProjectId": string

```

```

    "matchedField": string // Cached per instance: ``get_matched_message_id`` /

```

```get_matched_snippet```

both read this, so resolving it once avoids recomputing the keyword and title-match logic three times per serialized conversation.

"matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ```matched_message_content``` annotation added by

```ConversationViewSet._annotate_keyword_match```; no extra query.

"matchedMessageId": string

"pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.

"folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into. Null means unclassified (shown under the time groups). Deleting the folder resets this to null without deleting the conversation.

"hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to resume.

Lets the frontend know it should proactively re-request the browser panel (emit ```browser:start```) after a page reload, instead of only ever showing the panel when a live ```browser:frame``` push happens to arrive first.

"callerOrganizationId": string // Bound caller organization (stamped at conversation start); null when unbound

"handoff": object // Handoff information: enabled=platform switch; the continued conversation includes source_conversation_id/source_title; the original conversation includes continued_to_conversation_id/continued_to_title and latest_summary_message_id
}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string",
    "avatar": "Example response string",
    "email": "response@example.com",
    "phoneNumber": "Example response string",
    "metadata": null,
    "sourceId": "Example response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
```



```
    "name": "Example response string"
  }
]
},
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response string",
  "channelType": "line",
  "unreadConversationsCount": 456,
  "signAuth": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "signSource": "550e8400-e29b-41d4-a716-446655440000",
    "signParams": {
      "keycloak": {
        "clientId": "Example response string",
        "url": "Example response string",
        "realm": "Example response string"
      },
      "line": {
        "liffId": "Example response string"
      },
      "ad": {
        "saml": "Example response string"
      },
      "google": {
        "enabled": false
      }
    },
    "sourceIdAccessEnabled": false,
    "sourceIdVerifyMode": {},
    "sourceIdSignatureTtlMinutes": {},
    "keycloakTokenPassthrough": false,
    "keycloakTokenPassthroughAllowedHosts": [
      "Example response string"
    ]
  },
  "enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Example response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Example response string",
```

```
    "content": "Example response string",
    "createdAt": "Example response string"
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Example response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Example response string",
  "createdAt": "Example response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Example response string",
  "ipCountryCode": "Example response string",
  "ipRecordedAt": "Example response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  },
```

```
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "Example response string",
"firstResponseAt": "Example response string",
"queuedAt": "Example response string",
"assignedAt": "Example response string",
"transferReason": "Example response string",
"satisfactionScore": 456,
"satisfactionComment": "Example response string",
"satisfactionSubmittedAt": "Example response string",
"latestAnalysisSummary": "Example response string",
"clientPlatform": {},
"workingDirectory": "Example response string",
"slideProjectId": "Example response string",
"matchedField": "Example response string",
"matchedSnippet": "Example response string",
"matchedMessageId": "Example response string",
"pinnedAt": "Example response string",
"folder": "550e8400-e29b-41d4-a716-446655440000",
"hasActiveBrowserSession": false,
"callerOrganizationId": "Example response string",
"handoff": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Status code: **404** - No response body

Broadcast message

POST**/api/v1/contacts/broadcast-message/**

Request body

Request parameters

Field	Type	Required	Description
contactIds	array[string]	No	[Explicit selection mode] List of contact IDs to message
inboxId	string (uuid)	No	[Exclusion/send-to-all mode] Inbox ID; sends to all contacts in that inbox
excludeContactIds	array[string]	No	[Exclusion mode] List of contact IDs to exclude; use with inbox_id
message	string	Yes	Message content to send

Example request schema

```
{
  "contactIds"?: [ // [Explicit selection mode] List of contact IDs to
    message (optional)
    string (uuid)
  ]
  "inboxId"?: string (uuid) // [Exclusion/send-to-all mode] Inbox ID; sends
to all contacts in that inbox (optional)
  "excludeContactIds"?: [ // [Exclusion mode] List of contact IDs to exclude;
use with inbox_id (optional)
    string (uuid)
  ]
  "message": string // Message content to send
}
```

Example request value

```
{
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
```

```
"inboxId": "550e8400-e29b-41d4-a716-446655440000",
"excludeContactIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"message": "This is an example message"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/broadcast-
message/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "excludeContactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "message": "This is an example message"
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "excludeContactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "message": "This is an example message"
};

axios.post("https://api.maiagent.ai/api/v1/contacts/broadcast-
message/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/broadcast-message/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "inboxId": "550e8400-e29b-41d4-a716-446655440000",
    "excludeContactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "message": "This is an example message"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiaagent.ai/api/v1/contacts/broadcast-message/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "contactIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "inboxId": "550e8400-e29b-41d4-a716-446655440000",
            "excludeContactIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "message": "This is an example message"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "results"?: [ // optional
    {
      "contact_id"?: string (uuid) // optional
      "conversation_id"?: string (uuid) // optional
      "message_id"?: string (uuid) // optional
      "status"?: string // optional
    }
  ]
  "errors"?: [ // optional
    {
      "contact_id"?: string (uuid) // optional
      "error"?: string // optional
    }
  ]
  "total_contacts"?: integer // optional
  "success_count"?: integer // optional
  "error_count"?: integer // optional
}
```

Example response value

```
{
  "results": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "conversation_id": "550e8400-e29b-41d4-a716-446655440000",
      "message_id": "550e8400-e29b-41d4-a716-446655440000",
      "status": "success"
    }
  ],
  "errors": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "error": "Example response string"
    }
  ],
  "total_contacts": 456,
  "success_count": 456,
}
```

```
"error_count": 456  
}
```

Example API call (Shell)

...

目錄

1. Create a role
2. Bulk add role members
3. Bulk assign AI assistants to a role
4. Bulk assign inboxes to a role
5. Bulk assign knowledge bases to a role
6. Bulk Assign Databases to a Role
7. List Permissions
8. List Roles
9. Get role details
10. Get role details
11. Get role details
12. Get role details
13. List role members
14. Get a specific role member
15. List AI assistants for a role
16. List inboxes for a role
17. List knowledge bases for a role
18. List databases accessible to the role
19. Update role permissions
20. Partially update role permissions
21. Delete role
22. Remove a Role Member
23. Remove an AI Assistant from a Role
24. Remove an Inbox from a Role
25. Remove a Knowledge Base from a Role
26. Remove a Database from a Role

Example API call (Shell)

Create a role

POST**/api/v1/organizations/{organizationPk}/groups/**

Parameters

Parameter name	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.

Field	Type	Required	Description
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLLms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.
allowedLLms	array[string]	No	

Field	Type	Required	Description
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Example request schema

```
{
  "name": string
  "organization?": string (uuid) // optional
  "permissions": [
    string (uuid)
  ]
  "canCreateChatbot?": boolean // optional
  "canCreateKnowledgeBase?": boolean // optional
  "canCreateInbox?": boolean // optional
  "canCreateDatabase?": boolean // optional
  "canCreateTool?": boolean // optional
  "canCreateAgentUiTool?": boolean // optional
  "canCreateSkill?": boolean // optional
  "maigptCanCodeInterpreter?": boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag?": boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch?": boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool?": boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLLMs?": boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools?": boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors?": boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills?": boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
```

```

"allowedLlms"?: [ // optional
  string (uuid)
]
"allowedTools"?: [ // optional
  string (uuid)
]
"allowedConnectors"?: [ // optional
  string (uuid)
]
"allowedSkills"?: [ // optional
  string (uuid)
]
}

```

Example request value

```

{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [

```



```
    "550e8400-e29b-41d4-a716-446655440000"  
  ],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
}
```

Code examples

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

```
]
}'
```

```
# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};
```

```
],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
};
```

```
axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-  
e29b-41d4-a716-446655440000/groups/", data, config)  
  .then(response => {  
    console.log('Response received:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request failed:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
}
```

```
        "allowedSkills": [  
            "550e8400-e29b-41d4-a716-446655440000"  
        ]  
    }
```

```
response = requests.post(url, json=data, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "permissions": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "canCreateChatbot": true,
            "canCreateKnowledgeBase": true,
            "canCreateInbox": true,
            "canCreateDatabase": true,
            "canCreateTool": true,
            "canCreateAgentUiTool": true,
            "canCreateSkill": true,
            "maigptCanCodeInterpreter": true,
            "maigptCanRag": true,
            "maigptCanDeepResearch": true,
            "maigptCanBrowserTool": true,
            "maigptAllLlms": true,
            "maigptAllTools": true,
            "maigptAllConnectors": true,
            "maigptAllSkills": true,
            "allowedLlms": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedTools": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
        }
    ],

```



```

        "allowedConnectors": [
            "550e8400-e29b-41d4-a716-446655440000"
        ],
        "allowedSkills": [
            "550e8400-e29b-41d4-a716-446655440000"
        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 201

Example response schema

```

{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // List of permissions returned for the group

```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, load the missing parent from the database
4. Serialize parent permissions with their children and sort by order

```
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Number of remaining members (excluding the
three shown in the list), used for the +N badge in the UI
```

Unlike resource counts:

members_count: Returns total - 3 (for the +N badge); None means the total is <= 3

chatbots_count/knowledge_bases_count/inboxes_count: Returns the total count


```
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
```

```

search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLLMs"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
  "createdAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "Response string"
  ],
  "inboxesCount": 456,

```

```
"inboxesPreview": [
  "Response string"
],
"databasesCount": 456,
"databasesPreview": [
  "Response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanCodeInterpreter": false,
"maigptCanRag": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"maigptAllLlms": false,
"maigptAllTools": false,
"maigptAllConnectors": false,
"maigptAllSkills": false,
"createdAt": "Response string"
}
```

Bulk add role members

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/bulk-create/>

Parameters

Parameter name	Required	Type	Description
groupPk	V	string	

organizationPk

V

string

A UUID string identifying this organization ID

Request body

Request parameters

Field	Type	Required	Description
members	array[string]	Yes	

Example request schema

```
{
  "members": [
    string (uuid)
  ]
}
```

Example request value

```
{
  "members": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "members": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "members": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "members": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "members": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
[
  {
    "id"?: string (uuid) // Role member ID (optional)
    "member"?: { // optional
      {
        "id"?: string (uuid) // Member ID (optional)
        "name"?: string // Member name (optional)
        "email"?: string (email) // Member email (optional)
        "organization"?: { // optional
          {
            "id"?: string (uuid) // Organization ID (optional)
            "name"?: string // Organization name (optional)
            "createdAt"?: string // Organization creation timestamp
(optional)
            "usageStatistics"?: { // optional
              {
                "chatbotsCount"?: integer // Number of chatbots (optional)
                "countableChatbotsCount"?: integer // Number of chatbots
counted toward the creation limit (modes outside the plan allowlist are
excluded) (optional)
                "exemptChatbotsByMode"?: object // Number of chatbots exempt
from the limit, grouped by the mode that grants the exemption (optional)
                "canCreateChatbot"?: boolean // Whether chatbot creation is
allowed (optional)
                "createChatbotLimit"?: integer // Maximum number of chatbots
that can be created (-1 means unlimited; 0 means creation is disabled)
(optional)
                "currentMonthWordsCountTotal"?: integer // Monthly word limit
(optional)
                "currentMonthUsedWordsCountTotal"?: integer // Words used this
month (optional)
                "currentMonthUsedConversationsCountTotal"?: integer //
Conversations used this month (optional)
                "availableUploadFileSizeTotal"?: integer // Total upload size
allowed (optional)
                "usedUploadFileSizeTotal"?: integer // Upload size used
(optional)
              }
            }
          }
        "organizationPlan"?: { // optional
          {
            "id"?: string (uuid) // Plan ID (optional)
            "planName"?: string // Plan name (optional)
```

```

        "expiredAt"?: string // Plan expiration time (optional)
    }
}
}
}
"isOwner"?: boolean // Whether this member owns the organization
(optional)
"permissions"?: { // optional
{
    "hasOrganizationAccessPermission"?: boolean // Whether
organization access is allowed (optional)
    "hasChatAccessPermission"?: boolean // Whether chat access is
allowed (optional)
    "hasConversationAccessPermission"?: boolean // Whether
conversation access is allowed (optional)
    "hasChatbotAccessPermission"?: boolean // Whether chatbot access
is allowed (optional)
}
}
    "createdAt"?: string // Member creation timestamp (optional)
}
}
    "createdAt"?: string // Role member creation timestamp (optional)
}
]

```

Example response value

```

[
{
    "id": "c1e54f04-01fc-45e9-b40c-9d8b37bd0ac0",
    "member": {
        "id": "af4e472d-3f9b-446b-bab8-05de4112d086",
        "name": "h",
        "email": "h@gmail.com",
        "organization": {
            "id": "670e3d30-9d24-47b4-b2cc-6bf224b2f3bb",
            "name": "Updated Org Name",
            "createdAt": "1745288290000",
            "usageStatistics": {
                "chatbotsCount": 1,
                "countableChatbotsCount": 1,
                "exemptChatbotsByMode": {},
                "canCreateChatbot": false,
            }
        }
    }
}
]

```

```
    "createChatbotLimit": 1,  
    "currentMonthWordsCountTotal": 1000000,  
    "currentMonthUsedWordsCountTotal": 0,  
    "currentMonthUsedConversationsCountTotal": 0,  
    "availableUploadFilesSizeTotal": 104857600,  
    "usedUploadFileSizeTotal": 0  
  },  
  "organizationPlan": {  
    "id": "16136181-ae0d-4b3f-aaeb-9a6ed251459f",  
    "planName": "Free plan",  
    "expiredAt": null  
  }  
},  
"isOwner": false,  
"permissions": {  
  "hasMaigptAccessPermission": false,  
  "hasChatbotAccessPermission": false,  
  "hasAgentopsAccessPermission": false,  
  "hasConversationAccessPermission": false,  
  "hasChatAccessPermission": false,  
  "hasDeveloperAccessPermission": false,  
  "hasOrganizationAccessPermission": true  
},  
"createdAt": "1748335331000"  
},  
"createdAt": "1748341252000"  
}  
]
```

Bulk assign AI assistants to a role

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/bulk-create/>

Parameters

Parameter name	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this organization
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Request body

Request parameters

Field	Type	Required	Description
chatbots	array[string]	Yes	

Example request schema

```
{
  "chatbots": [
    string (uuid)
  ]
}
```

Example request value

```
{
  "chatbots": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

```
]
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "chatbots": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "chatbots": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbots": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "chatbots": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "chatbot":
        {
          "id": string (uuid)
          "name": string // Chatbot name. In Agent mode it carries semantic
            meaning; in other modes it only distinguishes chatbots.
          "largeLanguageModel": string (uuid) // Large language model used
            by the chatbot to generate responses, including organization-created AI
            Gateway models
          "llm": // ID and name of the bound model, read directly from the
            foreign key; returned even if the organization has disabled the model
          {
            "id": string (uuid)
            "name": string
          }
          "embeddingModel"?: string (uuid) // Embedding model used to
            vectorize text; optional (optional)
          "rerankerModel"?: string (uuid) // Model used for reranking;
            optional (optional)
          "instructions"?: string // Role instructions describing the
            chatbot's role and behavior. (optional)
          "knowledgeBases"?: [ // List of knowledge bases accessible to the
            chatbot (optional)
            {
              "id": string (uuid)
              "name": string
            }
          ]
          "databases"?: [ // List of SQL databases accessible to the
            chatbot (for Text-to-SQL) (optional)
            {
              "id": string (uuid)
              "name": string
            }
          ]
        }
      ]
    }
  ]
}
```

```

    ]
    "updatedAt": string (timestamp)
    "organization"?: string (uuid) // Organization that owns the
chatbot; an empty value indicates a personal chatbot (optional)
    "replyMode"?: // Reply mode: normal or streaming

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
    {
    }
    "template"?: string // Template used by Template and Hybrid
modes. (optional)
    "unanswerableTemplate"?: string // Template used by Template and
Hybrid modes when the question cannot be answered. (optional)
    "totalWordsCount"?: integer (int64) // Accumulated total word
count used. (optional)
    "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
    {
    }
    "rawOutputFormat"?: object // JSON schema defining the custom
output format (optional)
    "groups"?: [ // List of groups accessible to the chatbot
(optional)
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "tools"?: [ // List of tools available to the chatbot (optional)
        {
            "id": string (uuid)
            "name": string
            "description": string
            "displayName": string
            "toolType": string
        }
    ]
    "skills"?: [ // List of skills associated with the chatbot
(optional)
        {

```

```

        "id": string (uuid)
        "name": string
        "description": string
    }
]
"connectors"?: [ // Connectors associated with this chatbot,
gating per-contact OAuth authorization (optional)
    {
        "id": string (uuid)
        "name": string
    }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
    {
    }
    "numberOfRetrievedChunks"?: integer // Number of reference chunks
retrieved; default 12, minimum 1 (optional)
    "defaultLabelRelations"?: object // Default knowledge base label
filter at the assistant level, in the same format as
query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller omits query_metadata.
(optional)
    "useFullRetrievalRange"?: boolean // When enabled, ignore the LLM
top_k cutoff and return all reranker results (optional)
    "enableEvaluation"?: boolean // optional
    "enableInlineCitations"?: boolean // Enables inline citations,
inserting citation markers such as [1][2] into responses. (optional)
    "enableCodeInterpreter"?: boolean // When enabled, provides the
Code Interpreter tool to run code in an isolated container. (optional)
    "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (optional)
    "enableImageSearch"?: boolean // Provide the image_search tool
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (optional)
    "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
    "customMaxLlmOutputTokens"?: integer // Custom maximum number of
LLM output tokens; minimum 512 (optional)

```

```
"voiceConfig"?: { // optional
{
    "enabled"?: boolean // Whether this chatbot can be used as a
voice agent. Checked by every conversation platform (WebChat, SIP phone,
admin/marketplace voice test) instead of a per-platform toggle.
(optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type.
(optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service
provider. (optional)
    "sttConfig"?: object // STT configuration parameters (JSON).
(optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service
provider. (optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON).
(optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice
model provider. (optional)
    "realtimeConfig"?: object // Realtime configuration parameters
(JSON). (optional)
    "realtimeInstructions"?: string // Prompt dedicated to the
realtime voice session; falls back to chatbot instructions when empty
(optional)
    "turnHandling"?: object // Turn-taking and interruption
settings for the voice agent. (optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent
replies in text only and does not speak (optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON
format) (optional)
"enableToolSearching"?: boolean // Enable dynamic tool search so
the agent can discover and load tools through tool_searching_tool
(optional)
"maxAgentIterations"?: integer // Maximum number of iterations
for AgentWorkflow (1-100, default: 20) (optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for
AgentWorkflow execution (30-1200, default: 600) (optional)
"agentTimeoutMessage"?: string // Custom user-facing message
shown when AgentWorkflow times out (max 500 chars). Leave empty to use
the system default. (optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports
multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled,
tool results exceeding the character threshold are truncated using
```

head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (optional)

"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)

"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (optional)

"longTermMemoryTopK"?: integer // Number of long-term memory nodes retrieved per search (similarity_top_k for vector search); default 10, range 1-50 (optional)

"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (optional)

"crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (optional)

"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (optional)

"gatewayFallbackGroup"?: string (uuid) // optional

"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (optional)

"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (optional)

"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (optional)

"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (optional)

"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (optional)

"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (optional)

"isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified

"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental

"canUpdate": boolean // Return whether the current user can

update this resource.

```
    "canDelete": boolean // Return whether the current user can
delete this resource.
```

```
    }
    "canRead"?: boolean // optional
    "canUpdate"?: boolean // optional
    "canDelete"?: boolean // optional
    "createdAt": string (timestamp)
  }
]
}
```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
        "llm": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        },
      },
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
      "instructions": "response string",
      "knowledgeBases": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ],
      "databases": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ]
    }
  ]
}
```

```
],
"updatedAt": "response string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"replyMode": {},
"template": "response string",
"unanswerableTemplate": "response string",
"totalWordsCount": 456,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string",
    "displayName": "response string",
    "toolType": "response string"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
```



```
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
```

```
}  
]  
}
```

Bulk assign inboxes to a role

POST

`/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/bulk-create/`

Parameters

Parameter name	Required	Type	Description
<code>groupPk</code>	V	string	
<code>organizationPk</code>	V	string	A UUID string identifying this organization

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/bulk-create/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.post(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/bulk-create/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
200	No response body

Bulk assign knowledge bases to a role

POST

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/bulk-create/

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Request Body

Request Parameters

Field	Type	Required	Description
knowledgeBases	array[string]	Yes	

Request Schema Example

```
{
  "knowledgeBases": [
    string (uuid)
  ]
}
```

Request Example Value

```
{
  "knowledgeBases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples


```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "knowledgeBases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "knowledgeBases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "knowledgeBases": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "knowledgeBases": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "knowledgeBase":
        {
          "id": string (uuid)
          "createdBy":
            {
              "id": string (uuid)
              "name": string
            }
          "organization"?: string (uuid) // optional
          "embeddingModel": string (uuid)
          "rerankerModel": string (uuid)
          "name": string
          "description"?: string // optional
          "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks; default: 12, minimum: 1 (optional)
          "enableHyde"?: boolean // Enable HyDE; default: False (optional)
          "similarityCutoff"?: number (double) // Similarity cutoff;
default: 0.0, range: 0.0-1.0 (optional)
          "enableRerank"?: boolean // Whether to enable reranking; default:
True (optional)
          "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
          "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

          * `elasticsearch` - Elasticsearch
          * `opensearch` - OpenSearch
          * `oracle` - Oracle AI Database 26ai
          * `synxdb` - SynxDB (optional)
          {
          }
          "vectorDatabaseUrl"?: string // Custom vector database endpoint
        }
    }
  ]
}
```

```
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
    "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (optional)
    "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (optional)
    "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (optional)
    "oracleUser"?: string // Oracle database username. (optional)
    "oraclePassword"?: string // Oracle database password. (optional)
    "synxdbUser"?: string // SynxDB database username. (optional)
    "synxdbPassword"?: string // SynxDB database password. (optional)
    "labels": [
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "chatbots"?: [ // optional
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "groups"?: [ // List of groups with access to the knowledge base
(optional)
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
    "vectorStorageSize": integer
    "chunksCount": integer
    "canUpdate": boolean // Return whether the current user can
update this resource.
    "canDelete": boolean // Return whether the current user can
delete this resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
"canRead"?: boolean // optional
"canUpdate"?: boolean // optional
```

```
    "canDelete"?: boolean // optional
    "createdAt": string (timestamp)
  }
]
}
```

Response Example Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "createdBy": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        },
      },
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "description": "response string",
      "numberOfRetrievedChunks": 456,
      "enableHyde": false,
      "similarityCutoff": 456,
      "enableRerank": false,
      "enableRrf": false,
      "vectorDatabaseType": {},
      "vectorDatabaseUrl": "response string",
      "vectorDatabaseApiKey": "response string",
      "opensearchUsername": "response string",
      "opensearchPassword": "response string",
      "oracleUser": "response string",
      "oraclePassword": "response string",
      "synxdbUser": "response string",
      "synxdbPassword": "response string",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
```

```

        "name": "response string"
      }
    ],
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "filesCount": 456,
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "canUpdate": false,
    "canDelete": false,
    "createdAt": "response string",
    "updatedAt": "response string"
  },
  "canRead": false,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "response string"
}
]
}

```

Bulk Assign Databases to a Role

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/bulk-create/>

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Request Body

Request Parameters

Field	Type	Required	Description
databases	array[string]	Yes	

Request Schema Example

```
{
  "databases": [
    string (uuid)
  ]
}
```

Request Example Value

```
{
  "databases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

```
]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "databases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "databases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "databases": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "databases": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "database":
        {
          "id": string (uuid)
          "name": string // Display name for this database connection
          "description": string // Description of the database and its
purpose
          "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
          {
          }
          "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
          "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
          "status":
            {
            }
          "deletedAt": string (timestamp)
          "tables": [
            {
              "id": string (uuid)
              "tableName": string // Name of the table in the database
              "sheetName": string // Name of the source worksheet/sheet
this table was created from. Empty for tables created before this field
was added.
```

```
    "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (optional)
    "status":
    {
    }
    "errorMessage": string // Error details if table creation or
deletion failed
    "sourceFile":
    {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.
```

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.

```
    }
    "columns": [
    {
        "id": string (uuid)
        "columnName": string
        "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
        "description"?: string // Description of the column for
better Text-to-SQL understanding (optional)
        "isPrimaryKey"?: boolean // optional
        "isNullable"?: boolean // optional
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
    }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
]
"chatbots": [
{
    "id": string (uuid)
    "name": string
}
]
"groups": [
{
    "id": string (uuid)
    "name": string
```

```

    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
"canRead"?: boolean // optional
"canUpdate"?: boolean // optional
"canDelete"?: boolean // optional
"createdAt": string (timestamp)
}
]
}

```

Response Example Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "database": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "description": "response string",
        "databaseType": {},
        "includeTables": null,
        "isActive": false,
        "status": {},
        "deletedAt": "response string",
        "tables": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "response string",
            "sheetName": "response string",
            "description": "response string",
            "status": {},
            "errorMessage": "response string",
            "sourceFile": {
              "id": "550e8400-e29b-41d4-a716-446655440000",
              "name": "response string",
              "isFromKnowledgeBase": false
            }
          }
        ]
      }
    }
  ]
}

```



```
    },
    "columns": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "columnName": "response string",
        "columnType": "response string",
        "description": "response string",
        "isPrimaryKey": false,
        "isNullable": false,
        "createdAt": "response string",
        "updatedAt": "response string"
      }
    ],
    "createdAt": "response string",
    "updatedAt": "response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"createdAt": "response string",
"updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
```

List Permissions

GET

/api/v1/permissions/

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/permissions/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string
```

```
"value": string
"description": string
"children": [
  object
]
}
```

Response Example Value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "value": "response string",
    "description": "response string",
    "children": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example Response Name",
        "description": "Example Response Description"
      }
    ]
  }
]
```

List Roles

GET

</api/v1/organizations/{organizationPk}/groups/>

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

<code>organizationPk</code>	V	string	A UUID string identifying this organization ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>search</code>	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/?page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/?page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
}
```

```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "name": string
    "type":
    {
    }
    "permissions": [ // Returns the list of group permissions

```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, automatically load the missing parent permission from the database
4. Serialize parent permissions and their corresponding child permissions, then sort by order

```

    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the number of remaining members
(excluding the three shown in the list) for the +N badge in the UI

```

Unlike resource counts:

```

<ul>
  <li>members_count: Returns total - 3 (for the +N badge); None means the
total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Return the
total count</li>
</ul>

```

```

  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]

```

```

    "databasesCount": integer
    "databasesPreview": [
        string
    ]
    "canCreateChatbot"?: boolean // optional
    "canCreateKnowledgeBase"?: boolean // optional
    "canCreateInbox"?: boolean // optional
    "canCreateDatabase"?: boolean // optional
    "canCreateTool"?: boolean // optional
    "canCreateAgentUiTool"?: boolean // optional
    "canCreateSkill"?: boolean // optional
    "maigptCanCodeInterpreter"?: boolean // Whether members of this
role may use the code interpreter built-in tool in MaiGPT. (optional)
    "maigptCanRag"?: boolean // Whether members of this role may use
the knowledge base RAG built-in tool in MaiGPT. (optional)
    "maigptCanDeepResearch"?: boolean // Whether members of this role
may use Deep Research in MaiGPT. Deep Research reaches the public
internet through its own built-in search, so a role that revokes the
official web-search tools needs this switch too. (optional)
    "maigptCanBrowserTool"?: boolean // Whether members of this role
may use the browser built-in tool in MaiGPT. (optional)
    "maigptAllLLMs"?: boolean // Whether members of this role may use
every organization-available LLM in MaiGPT. Off = only the bound LLMs; an
empty binding then means none. (optional)
    "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
    "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
    "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
    "createdAt": string (timestamp)
}
]
}

```

Example response value

```

{
    "count": 123,
    "next": "http://api.example.org/accounts/?page=4",
    "previous": "http://api.example.org/accounts/?page=2",
}

```

```
"results": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "type": {},
    "permissions": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    ],
    "membersPreview": [
      "response string"
    ],
    "membersCount": 456,
    "chatbotsCount": 456,
    "chatbotsPreview": [
      "response string"
    ],
    "knowledgeBasesCount": 456,
    "knowledgeBasesPreview": [
      "response string"
    ],
    "inboxesCount": 456,
    "inboxesPreview": [
      "response string"
    ],
    "databasesCount": 456,
    "databasesPreview": [
      "response string"
    ],
    "canCreateChatbot": false,
    "canCreateKnowledgeBase": false,
    "canCreateInbox": false,
    "canCreateDatabase": false,
    "canCreateTool": false,
    "canCreateAgentUiTool": false,
    "canCreateSkill": false,
    "maigptCanCodeInterpreter": false,
    "maigptCanRag": false,
    "maigptCanDeepResearch": false,
    "maigptCanBrowserTool": false,
    "maigptAllLlms": false,
    "maigptAllTools": false,
```

```
    "maigptAllConnectors": false,
    "maigptAllSkills": false,
    "createdAt": "response string"
  }
]
}
```

Get role details

GET

`/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Group.
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}

?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```
"name": string
"type":
{
}
"permissions": [ // Returns the list of group permissions
```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, automatically load the missing parent permission from the database
4. Serialize parent permissions and their corresponding child permissions, then sort by order

```
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the number of remaining members
(excluding the three shown in the list) for the +N badge in the UI
```

Unlike resource counts:

```
<ul>
  <li>members_count: Returns total - 3 (for the +N badge); None means the
total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Return the
total count</li>
</ul>
```

```
"chatbotsCount": integer
"chatbotsPreview": [
  string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
  string
]
"inboxesCount": integer
"inboxesPreview": [
  string
]
"databasesCount": integer
"databasesPreview": [
  string
]
"canCreateChatbot"?: boolean // optional
```

```

"canCreateKnowledgeBase"?: boolean // optional
"canCreateInbox"?: boolean // optional
"canCreateDatabase"?: boolean // optional
"canCreateTool"?: boolean // optional
"canCreateAgentUiTool"?: boolean // optional
"canCreateSkill"?: boolean // optional
"maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
"maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"maigptAllLLMs"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
"maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
"maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
"maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
"createdAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
}

```

```
"membersPreview": [
  "response string"
],
"membersCount": 456,
"chatbotsCount": 456,
"chatbotsPreview": [
  "response string"
],
"knowledgeBasesCount": 456,
"knowledgeBasesPreview": [
  "response string"
],
"inboxesCount": 456,
"inboxesPreview": [
  "response string"
],
"databasesCount": 456,
"databasesPreview": [
  "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanCodeInterpreter": false,
"maigptCanRag": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"maigptAllLLMs": false,
"maigptAllTools": false,
"maigptAllConnectors": false,
"maigptAllSkills": false,
"createdAt": "response string"
}
```

Get role details

GET**/api/v1/organizations/{organizationPk}/groups/export/**

Parameters

Parameter name	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
string (binary)
```

Example response value

```
"response string"
```

Get role details

GET

/api/v1/organizations/{organizationPk}/groups/export-template/

Parameters

Parameter name	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export-template/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/export-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/export-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
string (binary)
```

Example response value

```
"response string"
```

Get role details

GET

/api/v1/organizations/{organizationPk}/groups/lite/

Parameters

Parameter name	Required	Type	Description
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/lite/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/lite/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/lite/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/lite/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name": string
```

```
"type":  
{  
}  
"permissions": [ // Returns the list of group permissions
```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, automatically load the missing parent permission from the database
4. Serialize parent permissions and their corresponding child permissions, then sort by order

```
    object  
  ]  
  "membersPreview": [  
    string  
  ]  
  "membersCount": integer // Returns the number of remaining members  
                        (excluding the three shown in the list) for the +N badge in the UI
```

Unlike resource counts:

```
<ul>  
  <li>members_count: Returns total - 3 (for the +N badge); None means the  
total is <= 3</li>  
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Returns the  
total count</li>  
</ul>
```

```
"chatbotsCount": integer  
"chatbotsPreview": [  
  string  
]  
"knowledgeBasesCount": integer  
"knowledgeBasesPreview": [  
  string  
]  
"inboxesCount": integer  
"inboxesPreview": [  
  string  
]  
"databasesCount": integer  
"databasesPreview": [  
  string  
]  
"canCreateChatbot"?: boolean // optional  
"canCreateKnowledgeBase"?: boolean // optional
```

```

"canCreateInbox"?: boolean // optional
"canCreateDatabase"?: boolean // optional
"canCreateTool"?: boolean // optional
"canCreateAgentUiTool"?: boolean // optional
"canCreateSkill"?: boolean // optional
"maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
"maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
"maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
"maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
"maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
"createdAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "example response name",
      "description": "example response description"
    }
  ],
  "membersPreview": [

```

```
    "response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLLMs": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "response string"
}
```

List role members

GET**/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/**

Parameters

Parameter name	Required	Type	Description
<code>groupPk</code>	V	string	
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>search</code>	X	string	

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "member": {
        {
          "id": string (uuid)
          "name": string
          "email": string
          "isActive": boolean
          "isOwner": boolean // Checks whether the member is an
organization owner (through the OWNER Group)

```

Use caching to avoid repeated queries within the same request
Cache lifetime: valid while the member instance exists

Prefer annotation results, when available, to avoid repeated queries

```

      "permissions": [
        object
      ]
      "groups": [
        object
      ]
      "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
      "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
      "metadata": object // Organization-scoped member attributes,
injected into LLM system prompt
      "createdAt": string (timestamp)
    }
  }
  "createdAt": string (timestamp)
}
]
}

```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "member": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "email": "response string",
        "isActive": false,
        "isOwner": false,
        "permissions": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "example response name",
            "description": "example response description"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "example response name",
            "description": "example response description"
          }
        ],
        "creditQuotaRequired": false,
        "activeQuota": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "example response name",
          "description": "example response description"
        },
        "metadata": null,
        "createdAt": "response string"
      },
      "createdAt": "response string"
    }
  ]
}
```

Get a specific role member

GET

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}

Parameters

Parameter name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Member.
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```

"member": {
  {
    "id": string (uuid)
    "name": string
    "email": string
    "isActive": boolean
    "isOwner": boolean // Checks whether the member is an organization
owner (through the OWNER Group)

Use caching to avoid repeated queries within the same request
Cache lifetime: valid while the member instance exists

Prefer annotation results, when available, to avoid repeated queries
    "permissions": [
      object
    ]
    "groups": [
      object
    ]
    "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (optional)
    "activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
    "metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
    "createdAt": string (timestamp)
  }
}
"createdAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "email": "response string",
    "isActive": false,
    "isOwner": false,
    "permissions": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
    "name": "example response name",
    "description": "example response description"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example response name",
    "description": "example response description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example response name",
  "description": "example response description"
},
"metadata": null,
"createdAt": "response string"
},
"createdAt": "response string"
}
```

List AI assistants for a role

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/>

Parameters

Parameter name	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this organization ID

page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema


```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "chatbot":
      {
        "id": string (uuid)
        "name": string // Chatbot name. In Agent mode it carries semantic
        meaning; in other modes it only distinguishes chatbots.
        "largeLanguageModel": string (uuid) // The large language model
        used by the chatbot to generate responses (including organization-created
        AI Gateway models)
        "llm": // ID and name of the bound model, read directly from the
        FK; returned even if the organization has disabled the model
        {
          "id": string (uuid)
          "name": string
        }
        "embeddingModel"?: string (uuid) // Embedding model used to
        vectorize text (optional)
        "rerankerModel"?: string (uuid) // Model used for reranking
        (optional)
        "instructions"?: string // Role instructions describing the
        chatbot's role and behavior. (optional)
        "knowledgeBases"?: [ // List of knowledge bases accessible to the
        chatbot (optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "databases"?: [ // List of SQL databases accessible to the
        chatbot (for Text-to-SQL) (optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "updatedAt": string (timestamp)
        "organization"?: string (uuid) // Organization the chatbot

```

```
belongs to; if empty, it is a personal chatbot (optional)
    "replyMode"?: // Reply mode: regular or streaming replies

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
    {
    }
    "template"?: string // Template used by Template and Hybrid
modes. (optional)
    "unanswerableTemplate"?: string // Template used by Template and
Hybrid modes when the question cannot be answered. (optional)
    "totalWordsCount"?: integer (int64) // Accumulated total word
count used. (optional)
    "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
    {
    }
    "rawOutputFormat"?: object // JSON schema for the custom output
format (optional)
    "groups"?: [ // List of groups accessible to the chatbot
(optional)
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "tools"?: [ // List of tools available to the chatbot (optional)
        {
            "id": string (uuid)
            "name": string
            "description": string
            "displayName": string
            "toolType": string
        }
    ]
    "skills"?: [ // List of skills associated with the chatbot
(optional)
        {
            "id": string (uuid)
            "name": string
            "description": string
```

```

    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot,
gating per-contact OAuth authorization (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?: // Agent mode: standard, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve; defaults to 12, minimum 1 (optional)
  "defaultLabelRelations"?: object // Default knowledge base label
filters at the assistant level, using the same format as
query_metadata.label_relations; null disables filtering. Applied as the
lowest-priority fallback only when the caller omits query_metadata.
(optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the
LLM top_k cutoff and returns all reranker results (optional)
  "enableEvaluation"?: boolean // optional
  "enableInlineCitations"?: boolean // Enables inline citations,
inserting citation markers such as [1][2] into responses. (optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the
Code Interpreter tool to run code in an isolated container. (optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (optional)
  "enableImageSearch"?: boolean // Provide the image_search tool
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of
LLM output tokens; minimum 512 (optional)
  "voiceConfig"?: { // optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a

```

voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.

(optional)

 "voiceAgentType"?: string (uuid) // Voice agent mode type.

(optional)

 "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)

 "sttConfig"?: object // STT configuration parameters (JSON). (optional)

 "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)

 "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)

 "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)

 "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)

 "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (optional)

 "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (optional)

 "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (optional)

 }

 }

 "thinkingConfig"?: object // Thinking effort settings (JSON format) (optional)

 "enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to discover and load tools through tool_searching_tool (optional)

 "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (optional)

 "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (optional)

 "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (optional)

 "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)

 "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (optional)

"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)

"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (optional)

"longTermMemoryTopK"?: integer // Number of long-term memory nodes retrieved per search (similarity_top_k for vector search); defaults to 10, range 1-50 (optional)

"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (optional)

"crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (optional)

"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (optional)

"gatewayFallbackGroup"?: string (uuid) // optional

"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (optional)

"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (optional)

"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (optional)

"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (optional)

"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (optional)

"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (optional)

"isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified

"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental

"canUpdate": boolean // Return whether the current user can update this resource.

"canDelete": boolean // Return whether the current user can delete this resource.

```

    }
    "canRead"?: boolean // optional
    "canUpdate"?: boolean // optional
    "canDelete"?: boolean // optional
    "createdAt": string (timestamp)
  }
]
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
        "llm": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        },
      },
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
      "instructions": "response string",
      "knowledgeBases": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ],
      "databases": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ],
      "updatedAt": "response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
    }
  ]
}

```

```
"replyMode": {},
"template": "response string",
"unanswerableTemplate": "response string",
"totalWordsCount": 456,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string",
    "displayName": "response string",
    "toolType": "response string"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "description": "response string"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"defaultLabelRelations": null,
"useFullRetrievalRange": false,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
```

```
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "response string",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "response string",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"longTermMemoryTopK": 456,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
```



```
]
}
```

List inboxes for a role

GET`/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/`

Parameters

Parameter name	Required	Type	Description
<code>groupPk</code>	V	string	
<code>organizationPk</code>	V	string	A UUID string identifying this organization
<code>channelType</code>	X	string	<code>`line`</code> : LINE ; <code>`telegram`</code> : Telegram ; <code>`teams`</code> : Teams ; <code>`web`</code> : Web ; <code>`messenger`</code> : Messenger ; <code>`instagram`</code> : Instagram ; <code>`email`</code> : Email ; <code>`whatsapp`</code> : WhatsApp ; <code>`slack`</code> : Slack ; <code>`team_plus`</code> : Team+ ; <code>`lin...</code>
<code>chatbot</code>	X	string	
<code>isActive</code>	X	boolean	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>search</code>	X	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "inbox":
      {
        "id": string (uuid)
        "channelType": string (enum: line, telegram, teams, web,
messenger, instagram, email, whatsapp, slack, team_plus, line_works,
viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
        "name": string
        "unreadConversationsCount": integer
        "accessType": string
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "canViewConversations"?: boolean // Whether group members can
browse all conversations of this inbox in the conversations page.
Independent from can_read, which controls using the inbox for chatting.
(optional)
      "createdAt": string (timestamp)
    }
  ]
}

```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "inbox": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "channelType": "line",
        "name": "response string",
        "unreadConversationsCount": 456,
        "accessType": "response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "canViewConversations": false,
      "createdAt": "response string"
    }
  ]
}
```

List knowledge bases for a role

GET

`/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/`

Parameters

Parameter name	Required	Type	Description
<code>groupPk</code>	V	string	

organizationPk	V	string	A UUID string identifying this organization
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/?page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/?page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "knowledgeBase":
      {
        "id": string (uuid)
        "createdBy":
        {
          "id": string (uuid)
          "name": string
        }
        "organization"?: string (uuid) // optional
        "embeddingModel": string (uuid)
        "rerankerModel": string (uuid)
        "name": string
        "description"?: string // optional
        "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve; defaults to 12, minimum 1 (optional)
        "enableHyde"?: boolean // Enable Hyde; defaults to False
(optional)
        "similarityCutoff"?: number (double) // Similarity threshold;
defaults to 0.0, range 0.0–1.0 (optional)
        "enableRerank"?: boolean // Enable reranking; defaults to True
(optional)
        "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
        "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
      {
      }
      "vectorDatabaseUrl"?: string // Custom vector database endpoint
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,

```

```
"host:1521/service_name"). (optional)
    "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (optional)
    "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (optional)
    "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (optional)
    "oracleUser"?: string // Oracle database username. (optional)
    "oraclePassword"?: string // Oracle database password. (optional)
    "synxdbUser"?: string // SynxDB database username. (optional)
    "synxdbPassword"?: string // SynxDB database password. (optional)
    "labels": [
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "chatbots"?: [ // optional
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "groups"?: [ // List of groups with access to the knowledge base
(optional)
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
    "vectorStorageSize": integer
    "chunksCount": integer
    "canUpdate": boolean // Return whether the current user can
update this resource.
    "canDelete": boolean // Return whether the current user can
delete this resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
"canRead"?: boolean // optional
"canUpdate"?: boolean // optional
"canDelete"?: boolean // optional
"createdAt": string (timestamp)
```

```
}  
]  
}
```

Example response

```
{  
  "count": 123,  
  "next": "http://api.example.org/accounts/?page=4",  
  "previous": "http://api.example.org/accounts/?page=2",  
  "results": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000",  
      "group": "550e8400-e29b-41d4-a716-446655440000",  
      "knowledgeBase": {  
        "id": "550e8400-e29b-41d4-a716-446655440000",  
        "createdBy": {  
          "id": "550e8400-e29b-41d4-a716-446655440000",  
          "name": "response string"  
        },  
      },  
      "organization": "550e8400-e29b-41d4-a716-446655440000",  
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",  
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",  
      "name": "response string",  
      "description": "response string",  
      "numberOfRetrievedChunks": 456,  
      "enableHyde": false,  
      "similarityCutoff": 456,  
      "enableRerank": false,  
      "enableRrf": false,  
      "vectorDatabaseType": {},  
      "vectorDatabaseUrl": "response string",  
      "vectorDatabaseApiKey": "response string",  
      "opensearchUsername": "response string",  
      "opensearchPassword": "response string",  
      "oracleUser": "response string",  
      "oraclePassword": "response string",  
      "synxdbUser": "response string",  
      "synxdbPassword": "response string",  
      "labels": [  
        {  
          "id": "550e8400-e29b-41d4-a716-446655440000",  
          "name": "response string"  
        }  
      ]  
    }  
  ]  
}
```

```
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string",
"updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}
```

List databases accessible to the role

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/>

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

groupPk	V	string	
organizationPk	V	string	A UUID string identifying this organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/?page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/?page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "database":
        {
          "id": string (uuid)
          "name": string // Display name for this database connection
          "description": string // Description of the database and its
purpose
          "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
          {
          }
          "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
          "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
          "status":
            {
            }
          "deletedAt": string (timestamp)
          "tables": [
            {
              "id": string (uuid)
              "tableName": string // Name of the table in the database
              "sheetName": string // Name of the source worksheet/sheet
this table was created from. Empty for tables created before this field
was added.
              "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (optional)
              "status":

```

```

    {
    }
    "errorMessage": string // Error details if table creation or
deletion failed
    "sourceFile":
    {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.

```

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.

```

    }
    "columns": [
    {
        "id": string (uuid)
        "columnName": string
        "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
        "description"?: string // Description of the column for
better Text-to-SQL understanding (optional)
        "isPrimaryKey"?: boolean // optional
        "isNullable"?: boolean // optional
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
    }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
]
"chatbots": [
{
    "id": string (uuid)
    "name": string
}
]
"groups": [
{
    "id": string (uuid)
    "name": string
}
]
"createdAt": string (timestamp)

```

```

    "updatedAt": string (timestamp)
  }
  "canRead"?: boolean // optional
  "canUpdate"?: boolean // optional
  "canDelete"?: boolean // optional
  "createdAt": string (timestamp)
}
]
}

```

Example response

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "database": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "description": "response string",
        "databaseType": {},
        "includeTables": null,
        "isActive": false,
        "status": {},
        "deletedAt": "response string",
        "tables": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "response string",
            "sheetName": "response string",
            "description": "response string",
            "status": {},
            "errorMessage": "response string",
            "sourceFile": {
              "id": "550e8400-e29b-41d4-a716-446655440000",
              "name": "response string",
              "isFromKnowledgeBase": false
            },
            "columns": [
              {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000",
        "columnName": "response string",
        "columnType": "response string",
        "description": "response string",
        "isPrimaryKey": false,
        "isNullable": false,
        "createdAt": "response string",
        "updatedAt": "response string"
      }
    ],
    "createdAt": "response string",
    "updatedAt": "response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"createdAt": "response string",
"updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}

```

Update role permissions

PUT**/api/v1/organizations/{organizationPk}/groups/{id}**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Group.
<code>organizationPk</code>	V	string	A UUID string identifying this organization ID

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.

Field	Type	Required	Description
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLLms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.
allowedLLms	array[string]	No	

Field	Type	Required	Description
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Example request schema

```
{
  "name": string
  "organization?": string (uuid) // optional
  "permissions": [
    string (uuid)
  ]
  "canCreateChatbot?": boolean // optional
  "canCreateKnowledgeBase?": boolean // optional
  "canCreateInbox?": boolean // optional
  "canCreateDatabase?": boolean // optional
  "canCreateTool?": boolean // optional
  "canCreateAgentUiTool?": boolean // optional
  "canCreateSkill?": boolean // optional
  "maigptCanCodeInterpreter?": boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag?": boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch?": boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool?": boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLLMs?": boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools?": boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors?": boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills?": boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
```

```

"allowedLlms"?: [ // optional
  string (uuid)
]
"allowedTools"?: [ // optional
  string (uuid)
]
"allowedConnectors"?: [ // optional
  string (uuid)
]
"allowedSkills"?: [ // optional
  string (uuid)
]
}

```

Example request

```

{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [

```

```
    "550e8400-e29b-41d4-a716-446655440000"  
  ],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
```

```
        "550e8400-e29b-41d4-a716-446655440000"  
    ]  
}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};
```

```
],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
};
```

```
axios.put("https://api.maiagent.ai/api/v1/organizations/550e8400-  
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-  
446655440000/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request failed:');  
    console.error(error.response?.data || error.message);  
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
}
```

```
    "allowedSkills": [  
        "550e8400-e29b-41d4-a716-446655440000"  
    ]  
}
```

```
response = requests.put(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "permissions": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "canCreateChatbot": true,
            "canCreateKnowledgeBase": true,
            "canCreateInbox": true,
            "canCreateDatabase": true,
            "canCreateTool": true,
            "canCreateAgentUiTool": true,
            "canCreateSkill": true,
            "maigptCanCodeInterpreter": true,
            "maigptCanRag": true,
            "maigptCanDeepResearch": true,
            "maigptCanBrowserTool": true,
            "maigptAllLlms": true,
            "maigptAllTools": true,
            "maigptAllConnectors": true,
            "maigptAllSkills": true,
            "allowedLlms": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedTools": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]
} catch (\Exception $e) {
    // Handle exception
}

```

```

        ],
        "allowedConnectors": [
            "550e8400-e29b-41d4-a716-446655440000"
        ],
        "allowedSkills": [
            "550e8400-e29b-41d4-a716-446655440000"
        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Response schema example

```

{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Return the list of group permissions

```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, load the missing parent permission from the database

4. Serialize parent permissions and their corresponding child permissions, then sort by order

```
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Return the remaining member count (excluding
the 3 shown in the list) for the frontend +N badge
```

Unlike resource counts:

members_count: Return total - 3 (for the +N badge); None means the total is <= 3

chatbots_count/knowledge_bases_count/inboxes_count: Return the total count


```
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
```

use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (optional)

"maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (optional)

"maigptAllLlms"?: boolean // Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none. (optional)

"maigptAllTools"?: boolean // Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none. (optional)

"maigptAllConnectors"?: boolean // Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none. (optional)

"maigptAllSkills"?: boolean // Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none. (optional)

"createdAt": string (timestamp)

}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "Response string"
  ]
}
```

```
],
"inboxesCount": 456,
"inboxesPreview": [
  "Response string"
],
"databasesCount": 456,
"databasesPreview": [
  "Response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanCodeInterpreter": false,
"maigptCanRag": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"maigptAllLlms": false,
"maigptAllTools": false,
"maigptAllConnectors": false,
"maigptAllSkills": false,
"createdAt": "Response string"
}
```

Partially update role permissions

PATCH

`/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Group.
organizationPk	V	string	A UUID string identifying this organization ID

Request body

Request parameters

Field	Type	Required	Description
name	string	No	
organization	string (uuid)	No	
permissions	array[string]	No	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes

Field	Type	Required	Description
			the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLlms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.
allowedLlms	array[string]	No	
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Request schema example

```
{
  "name"?: string // optional
```

```
"organization"?: string (uuid) // optional
"permissions"?: [ // optional
  string (uuid)
]
"canCreateChatbot"?: boolean // optional
"canCreateKnowledgeBase"?: boolean // optional
"canCreateInbox"?: boolean // optional
"canCreateDatabase"?: boolean // optional
"canCreateTool"?: boolean // optional
"canCreateAgentUiTool"?: boolean // optional
"canCreateSkill"?: boolean // optional
"maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
"maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"maigptAllLLMs"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
"maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
"maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
"maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
"allowedLlms"?: [ // optional
  string (uuid)
]
"allowedTools"?: [ // optional
  string (uuid)
]
"allowedConnectors"?: [ // optional
  string (uuid)
]
"allowedSkills"?: [ // optional
  string (uuid)
```

```
]
}
```

Example request value

```
{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

```
# Example API call (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
```

```
        "550e8400-e29b-41d4-a716-446655440000"  
    ]  
}'
```

```
# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};
```

```
],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
};
```

```
axios.patch("https://api.maiagent.ai/api/v1/organizations/550e8400-  
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-  
446655440000/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
}
```

```
    "allowedSkills": [  
        "550e8400-e29b-41d4-a716-446655440000"  
    ]  
}
```

```
response = requests.patch(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "permissions": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "canCreateChatbot": true,
            "canCreateKnowledgeBase": true,
            "canCreateInbox": true,
            "canCreateDatabase": true,
            "canCreateTool": true,
            "canCreateAgentUiTool": true,
            "canCreateSkill": true,
            "maigptCanCodeInterpreter": true,
            "maigptCanRag": true,
            "maigptCanDeepResearch": true,
            "maigptCanBrowserTool": true,
            "maigptAllLlms": true,
            "maigptAllTools": true,
            "maigptAllConnectors": true,
            "maigptAllSkills": true,
            "allowedLlms": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedTools": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);
}
```

```

        ],
        "allowedConnectors": [
            "550e8400-e29b-41d4-a716-446655440000"
        ],
        "allowedSkills": [
            "550e8400-e29b-41d4-a716-446655440000"
        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Response schema example

```

{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Return the list of group permissions

```

Processing steps:

1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission lacks its parent, load the missing parent permission from the database

4. Serialize parent permissions and their corresponding child permissions, then sort by order

```
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Return the remaining member count (excluding
the 3 shown in the list) for the frontend +N badge
```

Unlike resource counts:

members_count: Return total - 3 (for the +N badge); None means the total is <= 3

chatbots_count/knowledge_bases_count/inboxes_count: Return the total count


```
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
```

use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (optional)

"maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (optional)

"maigptAllLLMs"?: boolean // Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none. (optional)

"maigptAllTools"?: boolean // Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none. (optional)

"maigptAllConnectors"?: boolean // Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none. (optional)

"maigptAllSkills"?: boolean // Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none. (optional)

"createdAt": string (timestamp)
}

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "Response string"
  ]
}
```

```
],
"inboxesCount": 456,
"inboxesPreview": [
  "Response string"
],
"databasesCount": 456,
"databasesPreview": [
  "Response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanCodeInterpreter": false,
"maigptCanRag": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"maigptAllLlms": false,
"maigptAllTools": false,
"maigptAllConnectors": false,
"maigptAllSkills": false,
"createdAt": "Response string"
}
```

Delete role

DELETE

`/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Group.
organizationPk	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Remove a Role Member

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Member.
organizationPk	V	string	A UUID string identifying this organization ID

Code Examples

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Remove an AI Assistant from a Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/{id}

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Chatbot.
organizationPk	V	string	A UUID string identifying this organization ID

Code Examples

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
chatbots/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove an Inbox from a Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/{id}

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Inbox.
organizationPk	V	string	A UUID string identifying this organization ID

Code Examples

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
inboxes/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Remove a Knowledge Base from a Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/{id}

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group KnowledgeBase.
organizationPk	V	string	A UUID string identifying this organization ID

Code Examples

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiaagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove a Database from a Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/{id}

Parameters

Parameter Name	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Database.
organizationPk	V	string	A UUID string identifying this organization ID

Code Examples


```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Example API call (Shell)

...

目錄

1. Create a SQL database connection
2. List SQL databases
3. Retrieve a specific SQL database
4. Update an SQL database
5. Partially update an SQL database
6. Delete an SQL database
7. Test database connection
8. Upload files to a database
9. Batch delete SQL databases
10. List database tables
11. Create a database table
12. Get a database table
13. Update a database table
14. Partially update a database table
15. Delete a database table
16. Batch Delete Database Tables
17. List Database Table Columns
18. Get a Database Table Column
19. Update a Database Table Column
20. Partially Update a Database Table Column
21. Test Database Connection (General)
22. List database connection string examples
23. Retrieve a specific connection string example
24. List database configurations for an AI assistant
25. Add a database configuration for an AI assistant
26. Retrieve a specific database configuration
27. Update Database Configuration
28. Partially Update Database Configuration
29. Delete Database Configuration

Example API call (Shell)

Create a SQL database connection

POST

/api/v1/sql-databases/

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseType	object	Yes	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: <code>{"tables": ["table1", "table2"]}</code> . For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request schema example


```

{
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(optional)
  "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (optional)
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
}

```

Request example value

```

{
  "name": "Example name",
  "description": "Example string",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,

```

```
"chatbots": [  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
],  
"groups": [  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
]  
}
```

Code examples

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "description": "Example string",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and check the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "description": "Example string",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
```

```
console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "description": "Example string",
            "databaseType": {},
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true,
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description?": string // Description of the database and its purpose
  (optional)
  "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
  SynxDB, or MaiAgent

  * `postgresql` - PostgreSQL
  * `mysql` - MySQL
  * `maiagent` - MaiAgent
  * `oracle` - Oracle
  * `mssql` - MSSQL
  * `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables?": object // Optional list of tables to include. Format:
  {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
  populated from uploaded files. (optional)
  "isActive?": boolean // When disabled, this database will not be used
  in Text-to-SQL queries (optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```


table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{

```

        "id": string (uuid)
        "name": string
    }
]
"createdBy":
{
    "id": string (uuid)
    "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response example value

```

{
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "description": "Response string",
    "databaseType": {},
    "databaseUrlMasked": "Response string",
    "includeTables": null,
    "isActive": false,
    "status": {},
    "deletedAt": "Response string",
    "tables": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "Response string",
            "sheetName": "Response string",
            "description": "Response string",
            "status": {},
            "errorMessage": "Response string",
            "sourceFile": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "Response string",
                "isFromKnowledgeBase": false
            },
            "columns": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000",
        "columnName": "Response string",
        "columnType": "Response string",
        "description": "Response string",
        "isPrimaryKey": false,
        "isNullable": false,
        "createdAt": "Response string",
        "updatedAt": "Response string"
    }
],
    "createdAt": "Response string",
    "updatedAt": "Response string"
}
],
"chatbots": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"groups": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}

```

List SQL databases

GET

</api/v1/sql-databases/>

Parameters

Parameter name	Required	Type	Description
<code>databaseType</code>	X	string	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent `postgresql`: PostgreSQL; `mysql`: MySQL; `maiaagent`: MaiAgent; `oracle`: Oracle; `mssql`: MSSQL; `synxdb`: SynxDB;
<code>isActive</code>	X	boolean	
<code>isExternal</code>	X	boolean	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	

Code examples

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/?\ndatabaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=\n1&query=example"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "count": integer
}
```



```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "name": string // Display name for this database connection
    "description"?: string // Description of the database and its
purpose (optional)
    "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
    {
    }
    "databaseUrlMasked": string
    "includeTables"?: object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files. (optional)
    "isActive"?: boolean // When disabled, this database will not be
used in Text-to-SQL queries (optional)
    "status":
    {
    }
    "deletedAt": string (timestamp)
    "tables": [
      {
        "id": string (uuid)
        "tableName": string // Name of the table in the database
      }
    ]
    "tablesCount": integer
    "chatbots": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "groups": [
      {
        "id": string (uuid)

```

```

        "name": string
      }
    ]
    "createdBy":
    {
      "id": string (uuid)
      "name": string
    }
    "canUpdate": boolean // Return whether the current user can update
this resource.
    "canDelete": boolean // Return whether the current user can delete
this resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Response example value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "databaseType": {},
      "databaseUrlMasked": "Response string",
      "includeTables": null,
      "isActive": false,
      "status": {},
      "deletedAt": "Response string",
      "tables": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "tableName": "Response string"
        }
      ],
      "tablesCount": 456,
      "chatbots": [
        {

```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
]
```

Retrieve a specific SQL database

GET

`/api/v1/sql-databases/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description?": string // Description of the database and its purpose
```

```

(optional)
  "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

```

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.


```

    }
    "columns": [
      {
        "id": string (uuid)
        "columnName": string
        "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
        "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
        "isPrimarykey"?: boolean // optional
        "isNullable"?: boolean // optional
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"chatbots": [
  {
    "id": string (uuid)
    "name": string
  }
]
"groups": [
  {
    "id": string (uuid)
    "name": string
  }
]
"createdBy":
{
  "id": string (uuid)
  "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response example value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "databaseType": {},
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "status": {},
  "deletedAt": "Response string",
  "tables": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ]
}
```

```
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"createdBy": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Update an SQL database

PUT

/api/v1/sql-databases/{id}

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request schema example

```
{
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
  (optional)
  "databaseUrl"?: string // Connection string for external databases. Not
  required for MaiAgent type as it uses the system default. (optional)
  "includeTables"?: object // Optional list of tables to include. Format:
  {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
  populated from uploaded files. (optional)
  "isActive"?: boolean // When disabled, this database will not be used
  in Text-to-SQL queries (optional)
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
}
```

```
]
}
```

Example request value

```
{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```




```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "description": "Example string",
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true,
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description?": string // Description of the database and its purpose
  (optional)
  "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
  SynxDB, or MaiAgent

  * `postgresql` - PostgreSQL
  * `mysql` - MySQL
  * `maiagent` - MaiAgent
  * `oracle` - Oracle
  * `mssql` - MSSQL
  * `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables?": object // Optional list of tables to include. Format:
  {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
  populated from uploaded files. (optional)
  "isActive?": boolean // When disabled, this database will not be used
  in Text-to-SQL queries (optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```

table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{

```

        "id": string (uuid)
        "name": string
    }
]
"createdBy":
{
    "id": string (uuid)
    "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "description": "Response string",
    "databaseType": {},
    "databaseUrlMasked": "Response string",
    "includeTables": null,
    "isActive": false,
    "status": {},
    "deletedAt": "Response string",
    "tables": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "Response string",
            "sheetName": "Response string",
            "description": "Response string",
            "status": {},
            "errorMessage": "Response string",
            "sourceFile": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "Response string",
                "isFromKnowledgeBase": false
            },
            "columns": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000",
        "columnName": "Response string",
        "columnType": "Response string",
        "description": "Response string",
        "isPrimaryKey": false,
        "isNullable": false,
        "createdAt": "Response string",
        "updatedAt": "Response string"
    }
],
    "createdAt": "Response string",
    "updatedAt": "Response string"
}
],
"chatbots": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"groups": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}

```

Partially update an SQL database

PATCH**/api/v1/sql-databases/{id}**

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
name	string	No	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request schema example

```

{
  "name"?: string // Display name for this database connection (optional)
  "description"?: string // Description of the database and its purpose
(optional)
  "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (optional)
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
  "chatbots"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // optional
    {
      "id": string (uuid)
    }
  ]
}

```

Example request value

```

{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```



```
]
}
```

Code examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "description": "Example string",
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true,
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>

```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description?": string // Description of the database and its purpose
  (optional)
  "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
  SynxDB, or MaiAgent

  * `postgresql` - PostgreSQL
  * `mysql` - MySQL
  * `maiagent` - MaiAgent
  * `oracle` - Oracle
  * `mssql` - MSSQL
  * `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables?": object // Optional list of tables to include. Format:
  {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
  populated from uploaded files. (optional)
  "isActive?": boolean // When disabled, this database will not be used
  in Text-to-SQL queries (optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```

table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{


```

        "id": string (uuid)
        "name": string
    }
]
"createdBy":
{
    "id": string (uuid)
    "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "description": "Response string",
    "databaseType": {},
    "databaseUrlMasked": "Response string",
    "includeTables": null,
    "isActive": false,
    "status": {},
    "deletedAt": "Response string",
    "tables": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "Response string",
            "sheetName": "Response string",
            "description": "Response string",
            "status": {},
            "errorMessage": "Response string",
            "sourceFile": {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "name": "Response string",
                "isFromKnowledgeBase": false
            },
            "columns": [
                {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000",
        "columnName": "Response string",
        "columnType": "Response string",
        "description": "Response string",
        "isPrimaryKey": false,
        "isNullable": false,
        "createdAt": "Response string",
        "updatedAt": "Response string"
    }
],
    "createdAt": "Response string",
    "updatedAt": "Response string"
}
],
"chatbots": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"groups": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
    }
],
"createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}

```

Delete an SQL database

DELETE**/api/v1/sql-databases/{id}**

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

Test database connection

POST**/api/v1/sql-databases/{id}/test-connection/**

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
databaseType	string (enum: postgresql, mysql, maiagent, oracle, mssql, synxdb)	Yes	<code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	Yes	

Example request schema

```
{
  "databaseType": string (enum: postgresql, mysql, maiagent, oracle,
mssql, synxdb) // * `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  "databaseUrl": string
}
```

Example request value

```
{  
  "databaseType": "postgresql",  
  "databaseUrl": "Example string"  
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)  
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-  
e29b-41d4-a716-446655440000/test-connection/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "databaseType": "postgresql",  
  "databaseUrl": "Example string"  
}'  
  
# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "databaseType": "postgresql",
  "databaseUrl": "Example string"
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/test-connection/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/test-connection/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "databaseType": "postgresql",
    "databaseUrl": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/test-connection/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "databaseType": "postgresql",
            "databaseUrl": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "success"?: boolean // optional
  "message"?: string // optional
}
```

Example response value

```
{
  "success": false,
  "message": "Response string"
}
```

Upload files to a database

POST

</api/v1/sql-databases/{id}/upload-files/>

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
files	array[object]	Yes	

Example request schema

```
{
  "files": [
    {
      "file": string // File path from presigned URL upload (e.g.,
media/chatbotfiles/xxx/file.csv)
      "filename": string // Original filename
      "size": integer // File size in bytes
    }
  ]
}
```

Example request value

```
{
  "files": [
    {
      "file": "media/chatbotfiles/xxx/data.csv",
      "filename": "data.csv",
      "size": 1024
    },
    {
      "file": "media/chatbotfiles/xxx/report.xlsx",
      "filename": "report.xlsx",
      "size": 2048
    }
  ]
}
```

Code examples

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "files": [
    {
      "file": "media/chatbotfiles/xxx/data.csv",
      "filename": "data.csv",
      "size": 1024
    },
    {
      "file": "media/chatbotfiles/xxx/report.xlsx",
      "filename": "report.xlsx",
      "size": 2048
    }
  ]
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "files": [
    {
      "file": "media/chatbotfiles/xxx/data.csv",
      "filename": "data.csv",
      "size": 1024
    },
    {
      "file": "media/chatbotfiles/xxx/report.xlsx",
      "filename": "report.xlsx",
      "size": 2048
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/upload-files/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "files": [
        {
            "file": "media/chatbotfiles/xxx/data.csv",
            "filename": "data.csv",
            "size": 1024
        },
        {
            "file": "media/chatbotfiles/xxx/report.xlsx",
            "filename": "report.xlsx",
            "size": 2048
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/upload-files/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "files": [
                {
                    "file": "media/chatbotfiles/xxx/data.csv",
                    "filename": "data.csv",
                    "size": 1024
                },
                {
                    "file": "media/chatbotfiles/xxx/report.xlsx",
                    "filename": "report.xlsx",
                    "size": 2048
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```
{
  "message"? : string // optional
  "filesCount"? : integer // optional
  "fileIds"? : [ // optional
    string (uuid)
  ]
}
```

Example response value

```
{
  "message": "Response string",
  "filesCount": 456,
  "fileIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Batch delete SQL databases

POST

</api/v1/sql-databases/batch-delete/>

Request body

Request parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Example request schema

```
{
  "ids": [
    string (uuid)
  ]
}
```

Example request value

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/batch-delete/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
-------------	-------------

204

No response body

List database tables

GET

`/api/v1/sql-databases/{sqlDatabasePk}/tables/`

Parameters

Parameter name	Required	Type	Description
<code>sqlDatabasePk</code>	V	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>status</code>	X	string	`pending`: Pending; `processing`: Processing; `success`: Success; `failed`: Failed; `dropping`: Dropping; `drop_failed`: Drop Failed;

Code examples


```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/?  
page=1&pageSize=1&query=example&status=drop_failed"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "count": integer
  "next?": string (uri) // optional
}
```

```

"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "tableName": string // Name of the table in the database
    "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
    "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
    "status":
    {
    }
    "errorMessage": string // Error details if table creation or
deletion failed
    "sourceFile":
    {
      "id": string (uuid)
      "name": string
      "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
    }
    "columns": [
      {
        "id": string (uuid)
        "columnName": string
        "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
        "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
        "isPrimaryKey"?: boolean // optional
        "isNullable"?: boolean // optional
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
    ]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Example response value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```

Create a database table

POST

/api/v1/sql-databases/{sqlDatabasePk}/tables/

Parameters

Parameter name	Required	Type	Description
sqlDatabasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Example request schema

```
{
  "tableName": string // Name of the table in the database
  "description?": string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

Example request value

```
{
  "tableName": "Example string",
```

```
"description": "Example string"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "tableName": "Example string",
  "description": "Example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "tableName": "Example string",
  "description": "Example string"
};

axios.post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "tableName": "Example string",
            "description": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Example response schema

```

{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
      "isPrimaryKey"?: boolean // optional
      "isNullable"?: boolean // optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Get a database table

GET

</api/v1/sql-databases/{sqlDatabasePk}/tables/{id}>

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Table.
<code>sqlDatabasePk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-  
446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Successfully retrieved response:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
```

```

"sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
"description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
"status":
{
}
"errorMessage": string // Error details if table creation or deletion
failed
"sourceFile":
{
  "id": string (uuid)
  "name": string
  "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
}
"columns": [
  {
    "id": string (uuid)
    "columnName": string
    "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
    "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
    "isPrimaryKey"?: boolean // optional
    "isNullable"?: boolean // optional
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",

```

```
"status": {},
"errorMessage": "Response string",
"sourceFile": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "isFromKnowledgeBase": false
},
"columns": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "columnName": "Response string",
    "columnType": "Response string",
    "description": "Response string",
    "isPrimaryKey": false,
    "isNullable": false,
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
],
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Update a database table

PUT

`/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Table.
<code>sqlDatabasePk</code>	V	string	

Request body

Request parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Example request schema

```
{
  "tableName": string // Name of the table in the database
  "description?": string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

Example request value

```
{
  "tableName": "Example string",
  "description": "Example string"
}
```

Code examples

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "tableName": "Example string",
  "description": "Example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "tableName": "Example string",
  "description": "Example string"
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-  
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "tableName": "Example string",
            "description": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema


```

{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
      "isPrimaryKey"?: boolean // optional
      "isNullable"?: boolean // optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially update a database table

PATCH

</api/v1/sql-databases/{sqlDatabasePk}/tables/{id}>

Parameters

Parameter name	Required	Type	Description
id	V	string	A UUID string identifying this Database Table.
sqlDatabasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
tableName	string	No	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Example request schema

```
{
  "tableName?": string // Name of the table in the database (optional)
  "description?": string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

Example request value

```
{
  "tableName": "Example string",
  "description": "Example string"
}
```

Code examples

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "tableName": "Example string",
  "description": "Example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "tableName": "Example string",
  "description": "Example string"
};

axios.patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-  
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "tableName": "Example string",
            "description": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Example response schema

```

{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
      "isPrimaryKey"?: boolean // optional
      "isNullable"?: boolean // optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Example response value


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete a database table

DELETE

`/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Table.
<code>sqlDatabasePk</code>	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-  
446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.delete(url, headers=headers)  
try:  
    print("Successfully retrieved response:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Batch Delete Database Tables

POST

/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/

Parameters

Parameter Name	Required	Type	Description
sqlDatabasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Request Schema Example

```
{
  "ids": [
    string (uuid)
  ]
}
```

Example Request Value

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

```
]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
-------------	-------------

204

No response body

List Database Table Columns

GET

/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/

Parameters

Parameter Name	Required	Type	Description
sqlDatabasePk	V	string	
tablePk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/{tablePk}/columns/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "columnName": string
```

```
"columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
"description"?: string // Description of the column for better Text-
to-SQL understanding (optional)
"isPrimaryKey"?: boolean // optional
"isNullable"?: boolean // optional
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
]
```

Example Response Value

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "columnName": "response string",
    "columnType": "response string",
    "description": "response string",
    "isPrimaryKey": false,
    "isNullable": false,
    "createdAt": "response string",
    "updatedAt": "response string"
  }
]
```

Get a Database Table Column

GET

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

Parameters

Parameter Name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Database Table Column.
sqlDatabasePk	V	string	
tablePk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this command.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-  
41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "columnName": string
```

```
"columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
"description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
"isPrimaryKey"?: boolean // optional
"isNullable"?: boolean // optional
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "response string",
  "columnType": "response string",
  "description": "response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

Update a Database Table Column

PUT

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Table Column.

sqlDatabasePk V string

tablePk V string

Request Body

Request Parameters

Field	Type	Required	Description
columnName	string	Yes	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

Request Schema Example

```
{
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
  "isPrimaryKey"?: boolean // optional
  "isNullable"?: boolean // optional
}
```

Example Request Value

```
{
  "columnName": "example string",
  "columnType": "example string",
  "description": "example string",
```

```
"isPrimaryKey": true,  
"isNullable": true  
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X PUT "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-  
41d4-a716-446655440000/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "columnName": "example string",  
  "columnType": "example string",  
  "description": "example string",  
  "isPrimaryKey": true,  
  "isNullable": true  
'  
  
# Replace YOUR_API_KEY and verify the request data before running  
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "columnName": "example string",
  "columnType": "example string",
  "description": "example string",
  "isPrimaryKey": true,
  "isNullable": true
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-  
41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY",  
    "Content-Type": "application/json"  
}  
  
# Request body (payload)  
data = {  
    "columnName": "example string",  
    "columnType": "example string",  
    "description": "example string",  
    "isPrimaryKey": true,  
    "isNullable": true  
}  
  
response = requests.put(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "columnName": "example string",
            "columnType": "example string",
            "description": "example string",
            "isPrimaryKey": true,
            "isNullable": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
  "isPrimaryKey"?: boolean // optional
  "isNullable"?: boolean // optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "response string",
  "columnType": "response string",
  "description": "response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

Partially Update a Database Table Column

PATCH

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Database Table Column.
sqlDatabasePk	V	string	
tablePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
columnName	string	No	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

Request Schema Example

```
{
  "columnName"?: string // optional
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
  INTEGER, TIMESTAMP) (optional)
  "description"?: string // Description of the column for better Text-to-
  SQL understanding (optional)
  "isPrimaryKey"?: boolean // optional
```

```
"isNullable"?: boolean // optional  
}
```

Example Request Value

```
{  
  "columnName": "example string",  
  "columnType": "example string",  
  "description": "example string",  
  "isPrimaryKey": true,  
  "isNullable": true  
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "columnName": "example string",
  "columnType": "example string",
  "description": "example string",
  "isPrimaryKey": true,
  "isNullable": true
}'

# Replace YOUR_API_KEY and verify the request data before running
this command.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "columnName": "example string",
  "columnType": "example string",
  "description": "example string",
  "isPrimaryKey": true,
  "isNullable": true
};

axios.patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-  
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-  
41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY",  
    "Content-Type": "application/json"  
}  
  
# Request body (payload)  
data = {  
    "columnName": "example string",  
    "columnType": "example string",  
    "description": "example string",  
    "isPrimaryKey": true,  
    "isNullable": true  
}  
  
response = requests.patch(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "columnName": "example string",
            "columnType": "example string",
            "description": "example string",
            "isPrimaryKey": true,
            "isNullable": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
  "isPrimaryKey"?: boolean // optional
  "isNullable"?: boolean // optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "response string",
  "columnType": "response string",
  "description": "response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

Test Database Connection (General)

POST

</api/v1/database-connection/test/>

Request body

Request parameters

Field	Type	Required	Description
databaseType	object	Yes	Database type (excluding types used internally by MaiAgent) <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	Yes	Database connection string

Request schema example

```
{
  "databaseType": // Database type (excluding types used internally by
MaiAgent)

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string
}
```

Example request value

```
{
  "database_type": "postgresql",
  "database_url": "postgresql://user:password@localhost:5432/dbname"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/database-connection/test/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "database_type": "postgresql",
  "database_url":
"postgresql://user:password@localhost:5432/dbname"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "database_type": "postgresql",
  "database_url":
"postgresql://user:password@localhost:5432/dbname"
};

axios.post("https://api.maiagent.ai/api/v1/database-
connection/test/", data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/database-connection/test/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "database_type": "postgresql",
    "database_url":
"postgresql://user:password@localhost:5432/dbname"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/database-connection/test/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "database_type": "postgresql",
            "database_url":
"postgresql://user:password@localhost:5432/dbname"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "status": // Connection status

  * `success` - success
  * `failed` - failed
  {
  }
  "responseTime"?: number (double) // Response time (seconds) (optional)
  "message"?: string // Success message (optional)
  "errorMessage"?: string // Error message (optional)
}
```

Example response value

```
{
  "status": "success",
  "response_time": 0.123,
  "message": "Successfully connected to the PostgreSQL database"
}
```

Status code: **400**

Response schema example

```
{
  "detail"?: string // optional
}
```

Example response value

```
{
  "status": "success",
  "response_time": 0.123,
  "message": "Successfully connected to the PostgreSQL database"
}
```

List database connection string examples

GET

</api/v1/database-examples/>

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/database-examples/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/database-examples/",
config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/database-examples/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
[
  {
    "databaseType"?: // Database type options
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (optional)
  {
  }
  "databaseTypeExample": string // For example:
postgresql://db_user:db_password@db_ip:db_port/db_name
}
```

Example response value

```
[
  {
    "databaseType": {},
    "databaseTypeExample": "Response string"
  }
]
```

Status code: **401**

Response schema example

```
{
  "detail"?: string // optional
}
```

Example response value

```
{
  "detail": "Response string"
}
```

Retrieve a specific connection string example

GET

`/api/v1/database-examples/{id}`

Parameters

Parameter name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this database connection example.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/database-examples/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/database-examples/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/550e8400-  
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/database-examples/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "databaseType?": // Database type options
```



```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (optional)
{
}
"databaseTypeExample": string // For example:
postgresql://db_user:db_password@db_ip:db_port/db_name
}
```

Example response value

```
{
  "databaseType": {},
  "databaseTypeExample": "Response string"
}
```

List database configurations for an AI assistant

GET

</api/v1/chatbots/{chatbotPk}/databases/>

Parameters

Parameter name	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>page</code>	X	integer	A page number within the paginated result set.

pageSize

X

integer

Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
```

```

{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL,
Oracle, SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as
{"tables": ["table1", "table2"]}. The MaiAgent type reads them from
uploaded files automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used
for Text-to-SQL queries. (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  }
}
}

```

Example response value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "databaseType": {},
      "databaseUrl": "Response string",
      "databaseUrlMasked": "Response string",
      "includeTables": null,
      "isActive": false,
      "createdAt": "Response string",

```

```
    "updatedAt": "Response string"
  }
]
}
```

Add a database configuration for an AI assistant

POST

`/api/v1/chatbots/{chatbotPk}/databases/`

Parameters

Parameter name	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	Yes	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiaagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	Yes	Database connection string.

Field	Type	Required	Description
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Request schema example

```
{
  "name": string // Display name, for example: Main Business DB,
  Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
  SynxDB, or MaiAgent.

  * `postgresql` - PostgreSQL
  * `mysql` - MySQL
  * `maiagent` - MaiAgent
  * `oracle` - Oracle
  * `mssql` - MSSQL
  * `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "includeTables?": object // Tables to query, formatted as {"tables":
  ["table1", "table2"]}. The MaiAgent type reads them from uploaded files
  automatically, so no manual setup is needed. (optional)
  "isActive?": boolean // When inactive, this database is not used for
  Text-to-SQL queries. (optional)
}
```

Example request value

```
{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
```



```
"isActive": true
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  }'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "databaseType": {},
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example response value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Retrieve a specific database configuration

GET

/api/v1/chatbots/{chatbotPk}/databases/{id}

Parameters

Parameter name	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
id	V	string	A UUID string identifying this Chatbot Database Configuration.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
```

SynxDB, or MaiAgent.

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
{
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response example value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update Database Configuration

PUT

`/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter name	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>id</code>	V	string	A UUID string identifying this Chatbot Database Configuration.

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	Yes	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiaagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	Yes	Database connection string.
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Request schema example

```

{
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
}

```

Request example value

```

{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}

```

Code examples

```
# API Request Example (Shell)
curl -X PUT "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}'

# Replace YOUR_API_KEY and verify the request data before running
the example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
};

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "databaseType": {},
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response example value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update Database Configuration

PATCH

/api/v1/chatbots/{chatbotPk}/databases/{id}

Parameters

Parameter name	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
id	V	string	A UUID string identifying this Chatbot Database Configuration.

Request body

Request parameters

Field	Type	Required	Description
name	string	No	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	No	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. postgresql : PostgreSQL ; mysql : MySQL ; maiagent : MaiAgent ; oracle : Oracle ; mssql : MSSQL ; synxdb : SynxDB;
databaseUrl	string	No	Database connection string.
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.

Field	Type	Required	Description
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Request schema example

```
{
  "name"?: string // Display name, for example: Main Business DB,
Analytics DB. (optional)
  "databaseType"?: // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (optional)
  {
  }
  "databaseUrl"?: string // Database connection string. (optional)
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
}
```

Request example value

```
{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}
```

Code examples

Shell/Bash

```
# API Request Example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}'

# Replace YOUR_API_KEY and verify the request data before running
the example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
};

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example name",
            "databaseType": {},
            "databaseUrl": "Example string",
            "includeTables": null,
            "isActive": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType": // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response example value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Delete Database Configuration

DELETE`/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter name	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>id</code>	V	string	A UUID string identifying this Chatbot Database Configuration.

Code examples

Shell/Bash

```
# API Request Example (Shell)
curl -X DELETE "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
the example.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

API call example (Shell)

...

目錄

1. Create New Attachment (Integration)
2. Get Presigned URL for File Upload
3. Presigned Upload Attachment
4. Create Conversation Attachment
5. Get Allowed File Specifications

API call example (Shell)

Create New Attachment (Integration)

POST

/api/v1/attachments-upload/

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments-upload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/attachments-upload/",
formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments-upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
    'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/attachments-upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
-------------	-------------

200

No response body

Get Presigned URL for File Upload

POST

/api/v1/upload-presigned-url/

Parameters

Parameter Name	Required	Type	Description
fieldName	X	string	Field name
filename	X	string	File name
modelName	X	string	Model name

Request

Request Parameters

Field	Type	Required	Description
fileSize	integer	Yes	

Request Structure Example

```
{
  "fileSize": integer
}
```

Request Example Values

```
{  
  "fileSize": 123  
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X POST "https://api.maiagent.ai/api/v1/upload-presigned-url/?  
fieldName=example&filename=example&modelName=example"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "modelName": "example string",  
  "fieldName": "example string",  
  "filename": "document.pdf",  
  "fileSize": 123  
}'  
  
# Please make sure to replace YOUR_API_KEY and verify the request  
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "modelName": "example string",
  "fieldName": "example string",
  "filename": "document.pdf",
  "fileSize": 123
};

axios.post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "modelName": "example string",
    "fieldName": "example string",
    "filename": "document.pdf",
    "fileSize": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "modelName": "example string",
            "fieldName": "example string",
            "filename": "document.pdf",
            "fileSize": 123
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Structure Example

```
{
  "url": string(uri) // S3 bucket URL
  "fields": object // Form fields for POST request
}
```

Response Example Values

```
{
  "url": "https://s3.example.com/bucket-name",
  "fields": {
    "key": "media/chat/attachment/filename.pdf",
    "x-amz-algorithm": "AWS4-HMAC-SHA256",
    "x-amz-credential":
      "AKIAXXXXXXXXXXXXXXXX/YYMMDD/region/s3/aws4_request",
    "x-amz-date": "YYMMDDTHHMMSSZ",
    "policy": "base64-encoded-policy-document",
    "x-amz-signature": "generated-signature-hash"
  }
}
```

Presigned Upload Attachment

POST

/api/v1/attachments/

Request

Request Parameters

Field	Type	Required	Description
type	object	No	

Field	Type	Required	Description
filename	string	Yes	
file	string (uri)	Yes	

Request Structure Example

```
{
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
}
```

Request Example Values

```
{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg"
}
```

Code Examples


```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg"
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg"
};

axios.post("https://api.maiagent.ai/api/v1/attachments/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/attachments/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "type": {},
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Structure Example

```
{
  "id": string (uuid)
  "type"?: // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

  For conversation-bound attachments: conversation.last_message_created_at
  + retention_days.
  For attachments without a conversation: created_at + retention_days.
  Returns None when cleanup is disabled (effective_from not set).
  "downloadStatus": string
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "downloadStatus": "response string"
}
```

Create Conversation Attachment

POST

/api/v1/conversations/{conversationPk}/attachments/

Parameters

Parameter Name	Required	Type	Description
conversationPk	V	string	

Request

Request Parameters

Field	Type	Required	Description
type	object	No	
filename	string	Yes	
file	string (uri)	Yes	
conversation	string (uuid)	No	

Request Structure Example

```
{
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "conversation"?: string (uuid) // Optional
}
```

Request Example Values

```
{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
```

```
"conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/attachments/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/attachments/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/attachments/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "conversation": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-
41d4-a716-446655440000/attachments/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "type": {},
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "conversation": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Structure Example

```
{
  "id": string (uuid)
  "type"?: // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-
deleted.

For conversation-bound attachments: conversation.last_message_created_at
+ retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
  "conversation"?: string (uuid) // Optional
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

Get Allowed File Specifications

GET

</api/v1/allowed-file-specs/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/allowed-file-specs/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/allowed-file-specs/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/allowed-file-specs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/allowed-file-specs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Structure Example

```
{
  "webChat": object
  "organization": object
  "attachment": object
}
```

```
"chatbotFile": object
"knowledgeBaseFile": object
"batchQa": object
"meetingRecord": object
}
```

Response Example Values

```
{
  "web_chat": {
    "logo": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 5242880
      }
    ],
    "avatar": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 2097152
      }
    ],
    "cover": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 10485760
      }
    ],
    "powered-by-logo": [
```



```
{
  "content_type": {
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".png": "image/png",
    ".gif": "image/gif"
  },
  "max_size": 5242880
}
],
},
"organization": {
  "compact_logo": [
    {
      "content_type": {
        ".jpg": "image/jpeg",
        ".jpeg": "image/jpeg",
        ".png": "image/png",
        ".gif": "image/gif"
      },
      "max_size": 2097152
    }
  ],
  "full_logo": [
    {
      "content_type": {
        ".jpg": "image/jpeg",
        ".jpeg": "image/jpeg",
        ".png": "image/png",
        ".gif": "image/gif"
      },
      "max_size": 2097152
    }
  ]
},
"attachment": {
  "file": [
    {
      "content_type": {
        ".pdf": "application/pdf",
        ".doc": "application/msword",
        ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        ".txt": "text/plain"
      },

```

```
    "max_size": 20971520
  },
  {
    "content_type": {
      ".jpg": "image/jpeg",
      ".jpeg": "image/jpeg",
      ".png": "image/png",
      ".gif": "image/gif"
    },
    "max_size": 5242880
  }
]
},
"chatbot_file": {
  "file": [
    {
      "content_type": {
        ".pdf": "application/pdf",
        ".doc": "application/msword",
        ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        ".ppt": "application/vnd.ms-powerpoint",
        ".pptx": "application/vnd.openxmlformats-officedocument.presentationml.presentation",
        ".txt": "text/plain"
      },
      "max_size": 104857600
    }
  ]
},
"batch_qa": {
  "file": [
    {
      "content_type": null,
      "max_size": 104857600
    }
  ]
},
"faq": {
  "answer": [
    {
      "content_type": {
        ".jpg": "image/jpeg",
```

```
        ".jpeg": "image/jpeg",
        ".png": "image/png",
        ".gif": "image/gif",
        ".webp": "image/webp",
        ".tiff": "image/tiff",
        ".mp4": "video/mp4"
    },
    "max_size": 104857600
}
]
}
}
```